Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 1
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Titulari de Curs

- Prof. Emil-Ioan Sluşanschi – Seriile CA & CB
  emil.slusanschi@cs.pub.ro

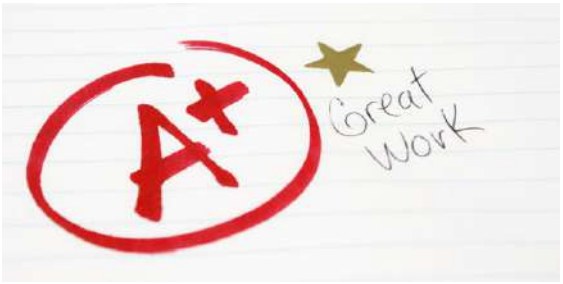
- Șl. Voichița Iancu – Seria CC
  voichita.iancu@cs.pub.ro

Cuprins

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

Notarea la ASC

- Laborator: 5/5.5
  - Trei teme: 4.5:
    - Python: 1.5
    - Optimizare C: 1.5
    - GP-GPU: 1.5
  - Activitate la laborator: 0.5
  - Prezentare iff act == 0.5
  - Bonusuri:
    - Max 20% @teme
- Examen: 5 (teorie)
  - Min > 2.5 lab
  - CA/CB: Oral || Scris



Colaborarea in **Negru** si Alb

- **Colaborarea Buna**
  - Discutii, brainstorming, dezbateri on/off-line
  - Participarea activa la prezentarile temelor in laboratoarele dedicate
- **Colaborarea Rea**
  - Lucrul in comun/revizie pe cod
  - Reluare/oferire /partajare/copiere de (segmente de) cod
- Ar trebui sa fie de la sine inteles, dar…
  - Nu cumparati teme!
  - Nu rugati pe cineva din afara cursului sa va ajute
  - Nu cautati pe internet / GitHub / ChatGPT solutii pentru teme
  - Nu puneti teme rezolvate pe site-uri nesecurizate sau pe GitHub / GitLab
  - Doar din propriile teme veti invata cu adevarat

Sumarul Cursului

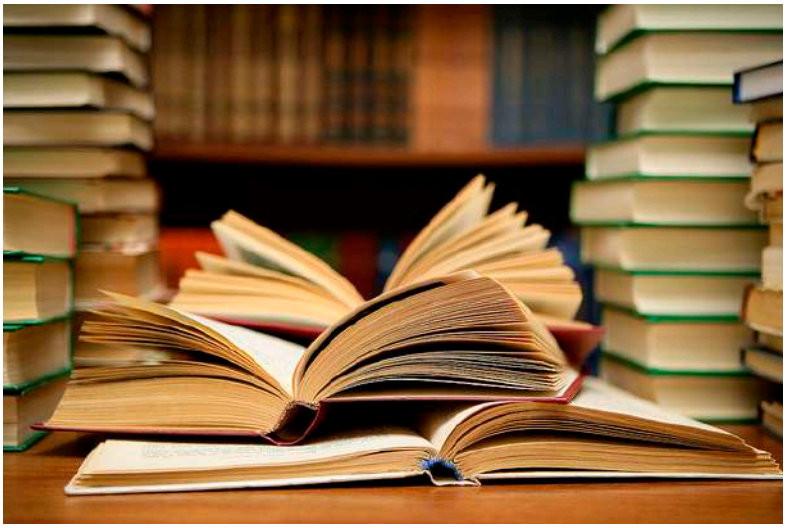
Un Scurt Sumar al Cursului

- Intro & Recapitulare
- Nivelul structural de descriere a sitemelor numerice - PMS
- Ierarhia de memorii, localitatea datelor, cache
- Comutatoare ierarhice & neierarhice
- Structuri de calcul cu prelucrare paralela – taxonomii
- Structuri SIMD
- Intercalarea perfecta si conectarea inversa
- Arhitecturi omogene/eterogene: CPU / GP-GPU

Un Scurt Sumar al Cursului (2)

- Arhitecturi MIMD – Caracteristici
- Arhitectura Cm* - comunicare intra- si inter-cluster
- Interconectarea in sisteme cu resurse multiple
- Retele de comutare (Ierarhice & Delta)
- Debugging, profiling, analiza de performanta
- Top 500, Benchmarking & the current Top 10

  Sumarul cursului se poate modifica pe parcursul semestrului



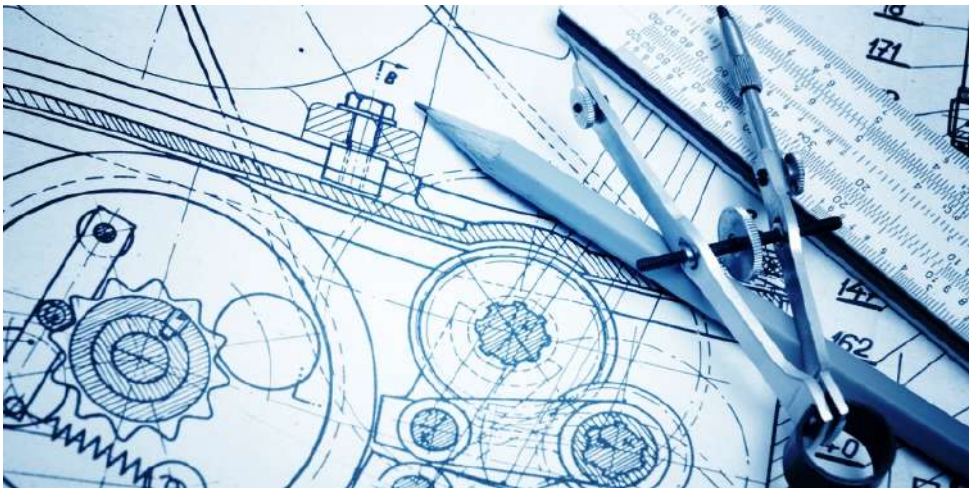
Bibliografie

Bibliografie

- Structura si Arhitectura Sistemelor Numerice; T. Moisa, N. Tapus, C. Morarescu 1999
- Introduction to Parallel Computing: Design & Analysis of Algorithms; V. Kumar, A. Grama, A. Gupta, G. Karypis; Addison Wesley; 2nd Edition 2003
- The Sourcebook of Parallel Computing; J. Dongarra, I. Foster, W. Grapp, K. Kennedy; Morgan Kaufmann 2002
- Computer Organization & Design – The Hardware Software Interface; D.A. Patterson, J. Hennesy; Morgan Kaufmann; 5th Edition 2014
- Computer Architecture: A Quantitative Approach; J. Hennesy, D.A. Patterson; Morgan Kaufmann; 6th Edition 2017
- Introduction to Parallel Processing; Algorithms and Architectures; Behrooz Parhami; Kluwer 2002
- Techniques for Optimizing Applications: High Performance Computing; Rajat P. Garg, Ilya Sharapov; Sun 2001.
- Inside the Machine; Jon Stokes; No Starch Press 2006.
- www.top500.org

Bibliografie vs. Cursuri

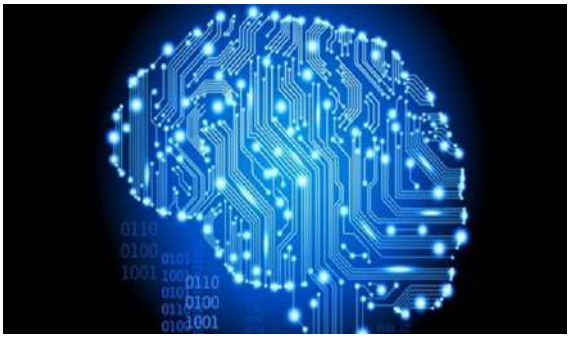
- Structura si Arhitectura Sistemelor Numerice
  - Cursurile 2, 3, 6, 7, 8, 9, 10, 11, 12
- Introduction to Parallel Computing: Design & Analysis of Algorithms
  - Cursurile 6, 8, 11
- The Sourcebook of Parallel Computing, www.top500.org, netlib.org, icl.cs.utk.edu/hpcc/
  - Cursurile 13, 14
- Computer Organization & Design – The Hardware Software Interface
  - Cursurile 2-14
- Computer Architecture: A Quantitative Approach
  - Cursurile 3, 4, 7, 9, 10, 11
- Introduction to Parallel Processing; Algorithms and Architectures
  - Cursurile 6-11
- Techniques for Optimizing Applications
  - Cursul 4
- Inside the Machine; Jon Stokes
  - Cursurile 1, 4, 5, 7, 8



Educatie Inginereasca

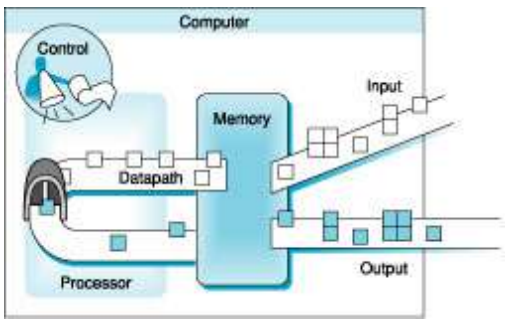
Ce este un Computer?

- Un instrument pentru a rezolva probleme
  - Prelucrarea datelor/numerelor
  - Prelucrarea algoritmilor
- Un procesor ce controleaza un sistem (avion, fabrica, monitor de inima, trafic, robot, etc.)
  - Senzori / Intrari
  - Actuator / Iesiri
  - Functii
  - Stari



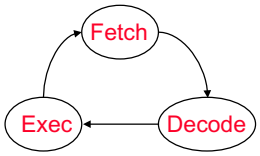
@Yale Patt - University of Texas at Austin

Ce este un Computer? (2)



- Components:
  - Processor (Datapath & Control)
  - Input (mouse, keyboard)
  - Output (display, printer)
  - Memory (cache, main memory, disks, CD/DVD)
  - Network
- Our primary focus: the Processor (**Datapath** & **Control**)
  - Implemented using billions of transistors
  - Impossible to understand by looking at each transistor
  - We need abstraction!

Processor Organisation



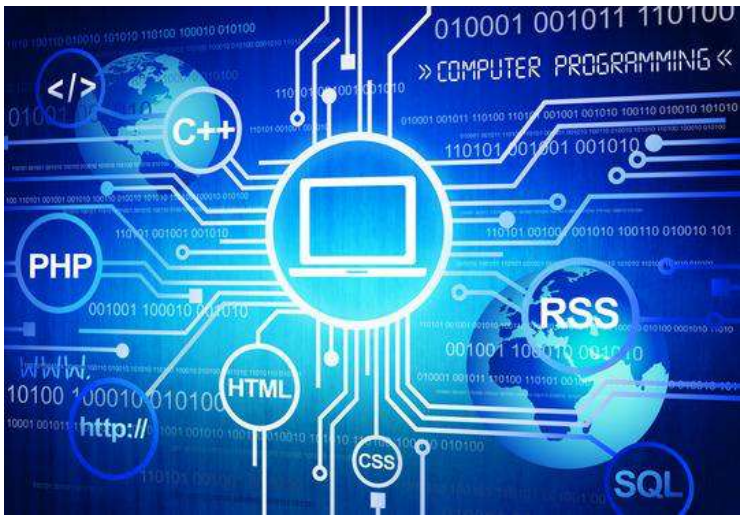
- **Control** needs to have circuitry to
  - Decide which is the next instruction and input it from memory
  - Decode the instruction
  - Issue signals that control the way information flows between datapath components
  - Control what operations the datapath's functional units perform
- **Datapath** needs to have circuitry to
  - Execute instructions - functional units (e.g. adder, multiplier, etc.) and storage locations (e.g. register file)
  - Interconnect the functional units so that the instructions can be executed as required
  - Load data from and store data to memory

Typical Design Goals for Processors

- Maximize performance (runtime, throughput, etc.)
- Minimize memory space (embedded systems)
- Minimize power consumption (mobile systems)
- Reduce design time (time-to-market)
- Minimize cost ($)

The Four Design Principles (CN)

- Simplicity favours regularity
- Smaller is faster
- Make the common case fast
- Good design demands good compromises



De ce ASC?

Why Computer Organization?

- Embarrassing if you are a BS in CSE and can't make sense of the following terms: DRAM, pipelining, cache hierarchies, I/O, virtual memory, …
- Embarrassing if you are a BS in CSE and can't decide which processor to buy wrt different metrics: 2.2 GHz Xeon or 3.6 GHz A8
  - E.g. Performance vs. Power
- Obvious first step for chip designers, compiler & OS writers
- **Will knowledge of the hardware help you write better programs?**

Must a Programmer **care** about Hardware?

- Must know how to reason about program performance and energy
- Memory management: if we understand how/where data is placed, we can help ensure that **relevant data** is **nearby**
- Thread management: if we understand how threads **interact**, we can write **smarter** multi-threaded programs
- Why do we care about multi-threaded programs?
  - **Because most HW is now multi/many-core**

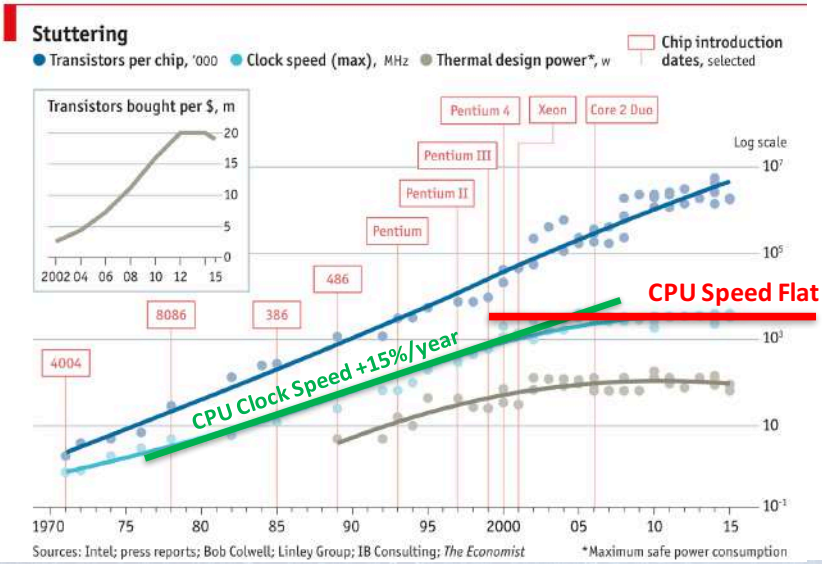
Ce vom discuta la ASC?

- Discussions about the hardware-software interface
  - What do you need to know to achieve best possible performance
- Introduction to the design of major components of a computer system, how they function together in an executing program
- But what do we mean by a computer?
  - Different types: mobile, embedded, laptop, desktop, server,…
  - Different uses: AI, auto, graphics, finance, genomics,…
- Best way to learn:
  - Focus on a specific instance and learn how it works
  - While learning general principles and historical perspectives

De ce sa discutam de HW/SW?

- You want to call yourself a “computer scientist/engineer”
- You want to build HW/SW people use
  - So you need to deliver **performance** at **low cost**
- You need to make a purchasing decision or offer “expert” advice
- Both hardware and software affect performance
  - The algorithm determines number of source-level statements
  - The language/compiler/architecture determines the number of machine instructions
  - The processor/memory determines how fast machine-level instructions are executed

Why is Architecture Exciting Today?



ACM Communications, February 2019

Important Trends

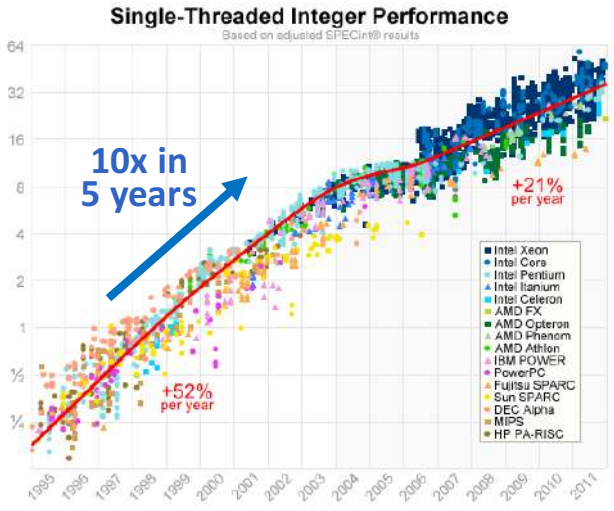
- Running out of ideas to improve single thread performance
- Power wall makes it harder to add complex features
- Power wall makes it harder to increase frequency
- Historical contributions to performance:
  - Better processes (faster devices) ~**20%** (eventually disappearing)
  - Better circuits/pipelines ~**15%** (trending lower)
  - Better organization/architecture ~**15%**

Practical Example

- +**300x** speedup for matrix vector multiplication
  - Data level parallelism: **3.8x**
  - Loop unrolling and out-of-order execution: **2.3x**
  - Cache blocking: **2.5x**
  - Thread level parallelism: **14x**
- Further, one can use accelerators to get an additional **100x**

Predicting the Future

- Predictable performance improvements
- Predictable market
  - Everybody buys a new CPU every X years
- Predictable expenses
  - Cost of new integrated circuit fab
- CEOs and Wall Street love this



Opportunities

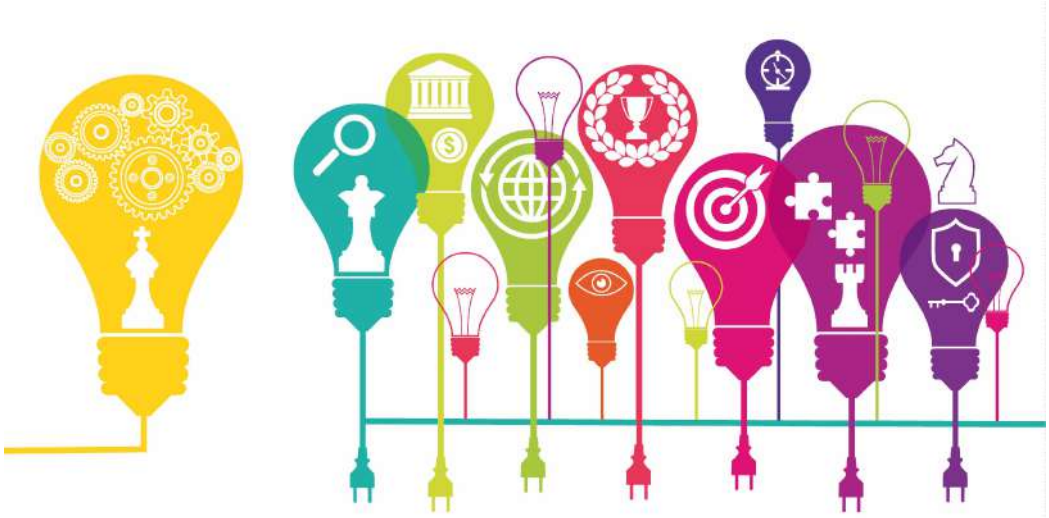
- Computer architecture today
  - General purpose: optimized for speed
  - Few players: Intel, AMD, ARM, NVidia…
- Future directions?
  - Applications have different requirements
    - Battery life / energy
    - Security & privacy
- Best solutions depend on application
  - Many players: HW / SW
  - **Innovation matters more than Moore's Law**

What Does This Mean to a Programmer?

- Today, one can expect at most a 20% annual improvement / even lower if the program is not multi-threaded
  - A program needs many threads
  - The threads need efficient synchronization and communication (Tema 1)
  - Data placement in the memory hierarchy is important (Tema 2)
  - Accelerators should be used whenever possible (Tema 3)

Challenges for Hardware Designers

- Find efficient ways to
  - **Boost single-thread performance** (Tema 2)
  - **Improve data sharing & synch techniques** (Tema 1)
  - **Boost programmer productivity** (Temele 1-3)
  - **Manage the memory system** (Temele 2-3)
  - **Build accelerators for important kernels** (Tema 3)
  - Reduce system energy per instruction



Idei Importante in Arhitectura Sistemelor de Calcul

Sisteme de Calcul

Atunci…

Sisteme de Calcul

Acum.

Arhitectura Sistemelor de Calcul

Software

- Parallel Requests
  Assigned to computer
  e.g., Search "cats"
- Parallel Threads
  Assigned to core
  e.g., Lookup, Ads
- Parallel Instructions
  >1 instruction @ one time
  e.g., 5 pipelined instructions
- Parallel Data
  >1 data item @ one time
  e.g., Add of 4 pairs of words
- Hardware descriptions
  All gates functioning in parallel at same time

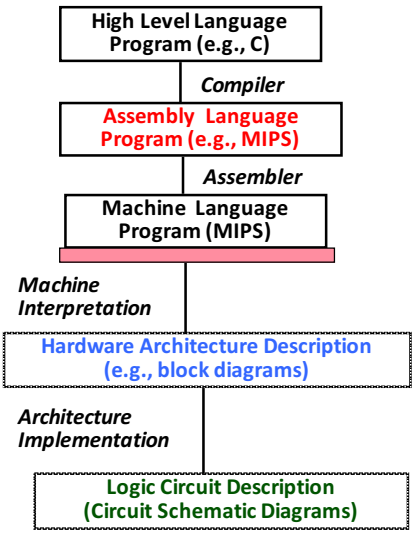
Harness Parallelism & Achieve High Performance

Hardware

Cinci Idei Importante in Arhitectura Sistemelor de Calcul

1. Abstraction - Layers of Representation / Interpretation
2. Moore's Law - Designing through trends
3. Principle of Locality - Memory Hierarchy
4. Parallelism
5. Dependability via Redundancy

Ideea 1: Abstractizarea

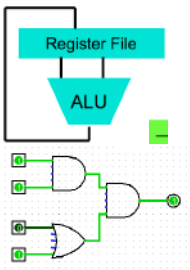



```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

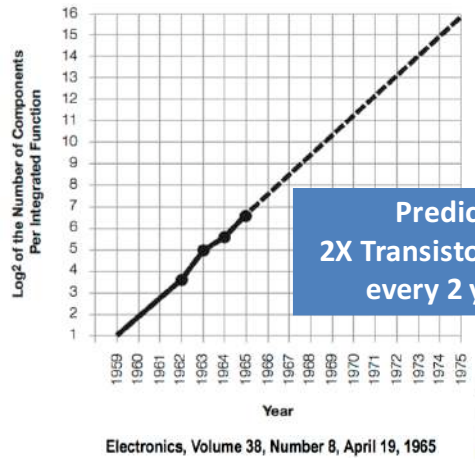
```
lw   $t0, 0($2)
lw   $t1, 4($2)
sw   $t1, 0($2)
sw   $t0, 4($2)
```

Anything can be represented as a *number*, i.e., data or instructions

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 [illegible]  1
0101 1000 0000 1001 1100 [illegible]  1
```



Ideea 2: Legea lui Moore



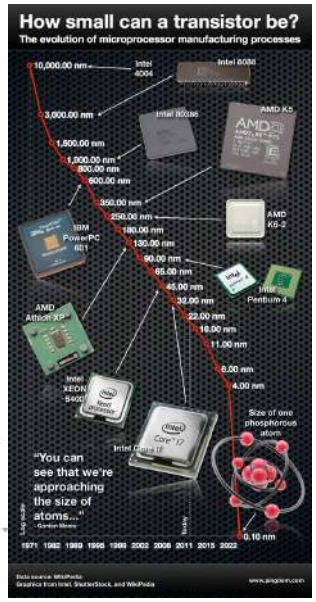
Predicts:
2X Transistors / chip every 2 years

Electronics, Volume 38, Number 8, April 19, 1965

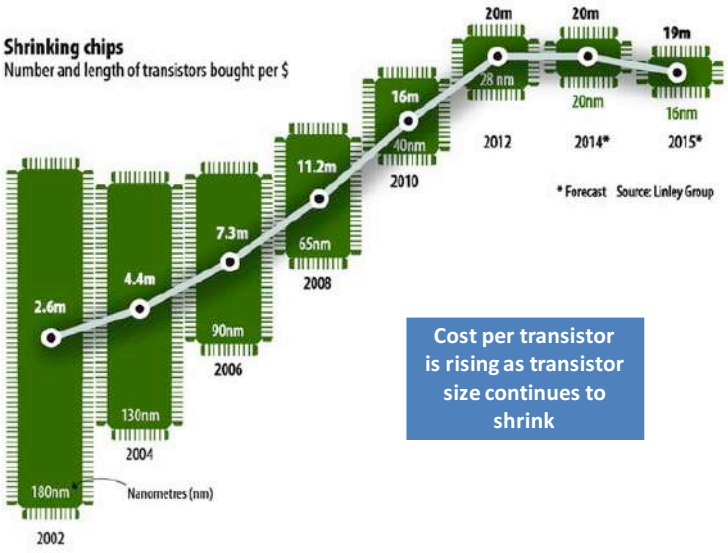
Gordon Moore
Intel Cofounder
B.S. Cal 1950!

Moore's law has **died** at the age of 51 after an extended illness. In 1965, Intel co-founder Gordon **Moore** made an observation that the number of components in integrated circuits was doubling every 12 months or so. Feb 11, 2016

Moore's law really is dead this time | Ars Technica
arstechnica.com/information.../2016/.../**moores-law**-really-is-dead-this-time... Ars Technica



Numarul de tranzistori si latenta

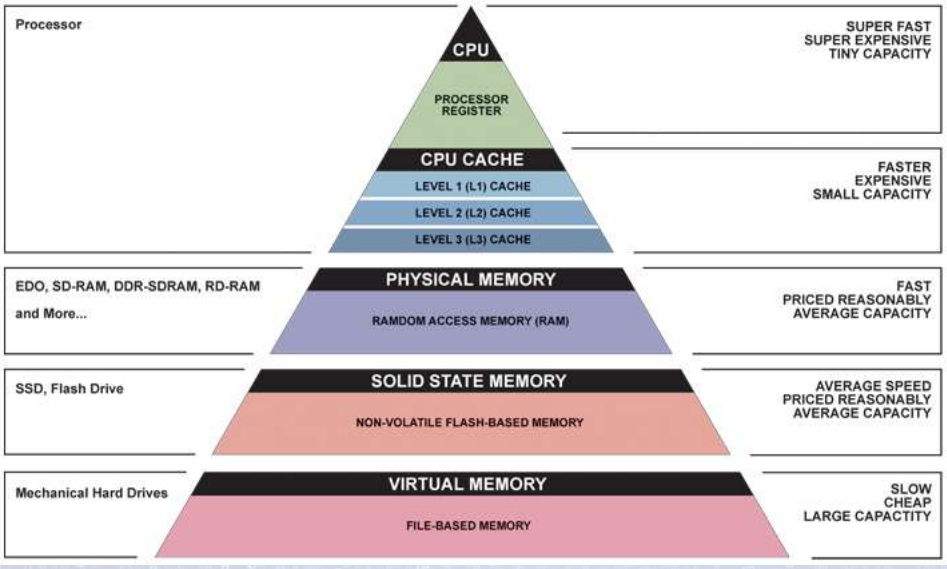


Cost per transistor is rising as transistor size continues to shrink

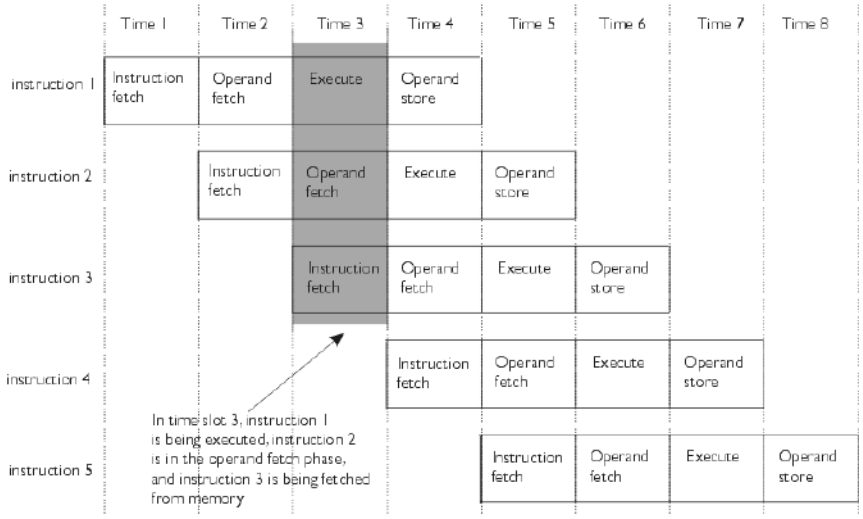
Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal 1969!

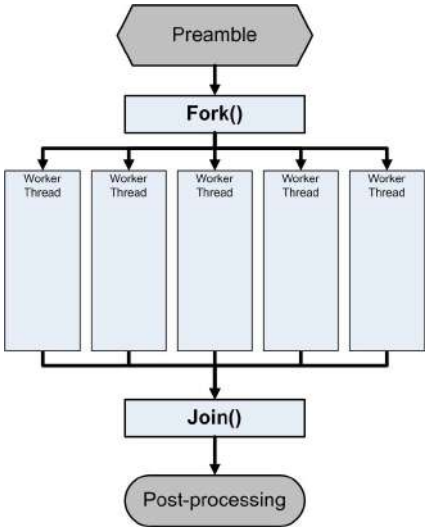
10^9	Banda Robot Optic	Andromeda	2,000 Ani
10^6	Disk	Pluto	2 Ani
		Brasov	1.5 h
100	Memorie Principala		
10	On Board Cache	Politehnica	10 min
2	On Chip Cache	Sala de curs	
1	Registri	Capul meu	1 min
(ns)			

Ideea 3: Localitate - Ierarhia de Memorii



Ideea 4: Paralelism





Ideea 5: Fiabilitate prin Redundanta

- Applies to everything from datacenters to storage to memory to instructors
  - Redundant **datacenters**: can lose 1 datacenter but Internet service stays online
  - Redundant **disks**: can lose 1 disk but not lose data (Redundant Arrays of Independent Disks - RAID)
  - Redundant **memory** bits: can loose 1-2 bits but no data is lost (Error Correcting Code - ECC Memory)

Ce am aflat in aces curs?

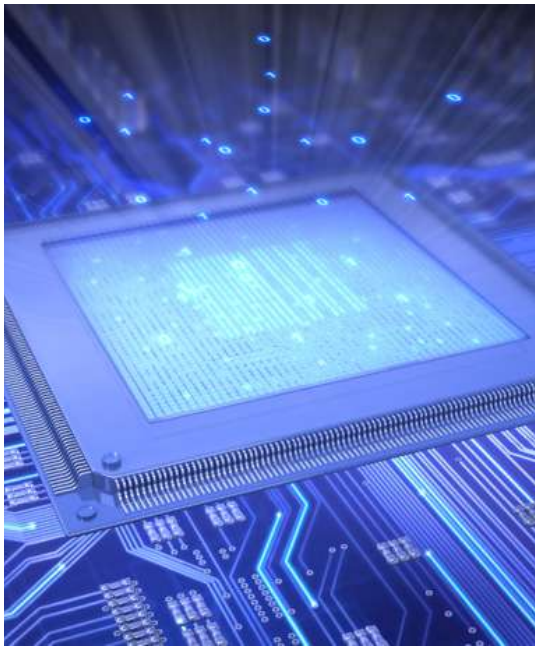
- Despre notarea de la ASC
- Sumarul cursului de ASC
- Bibliografia cursului de ASC
- Despre educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

In cursul urmator

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)

Slides & Support Material Acknowledgements

Yale Patt - University of Texas at Austin 2007
CSE 331 - Penn State University 2007
CS61C - University of Berkeley 2016
CS/ECE 3810 - University of Colorado 2017
Ryan J. Leng



Thank you for your attention

Computer Science & Engineering Department

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 2
cs.pub.ro
cs.curs.pub.ro

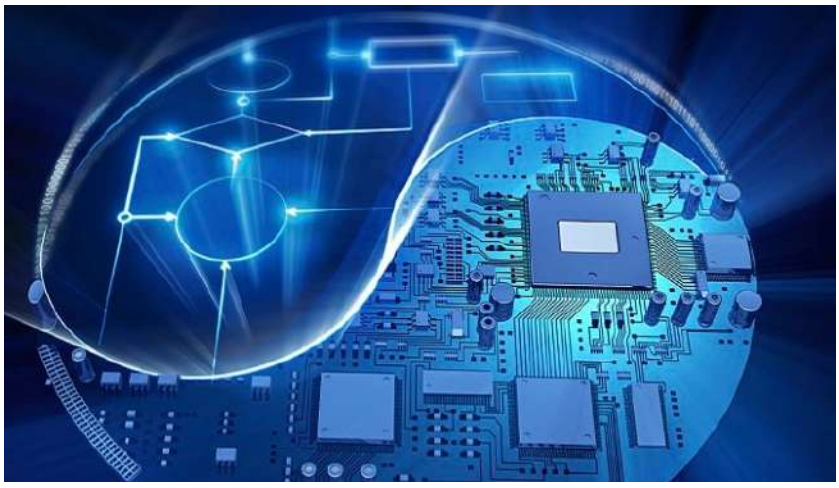
University Politehnica of Bucharest

In cursul anterior

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

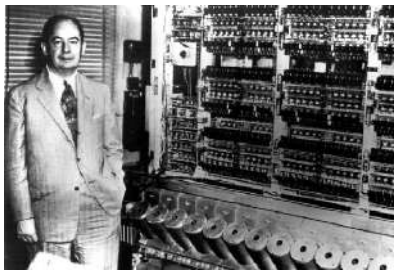
Cuprins

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)



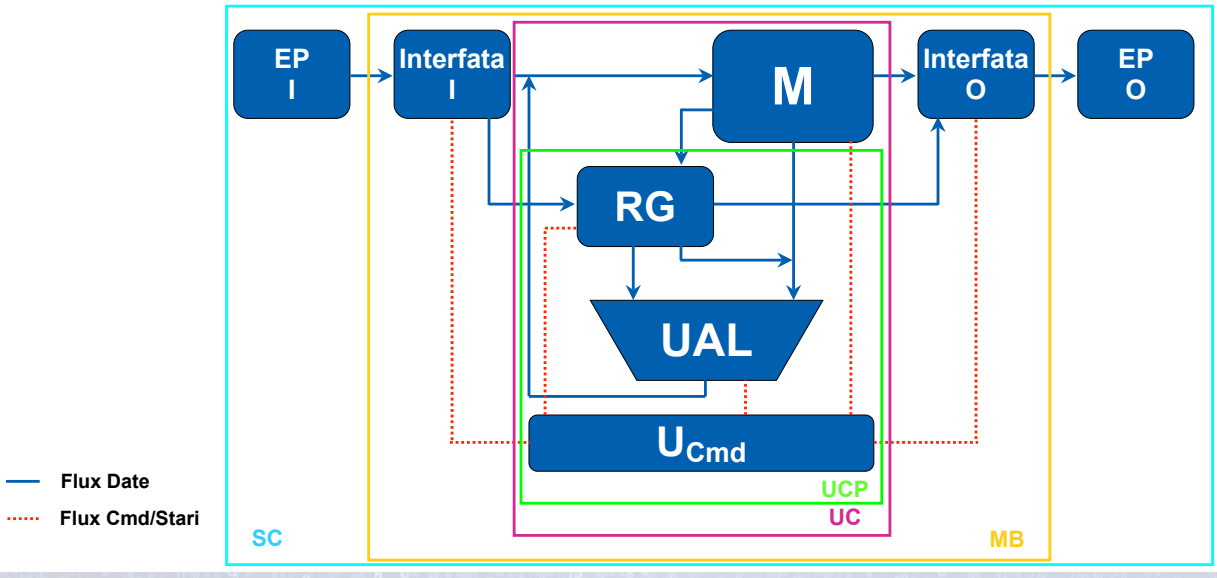
Componentele Sistemelor de Calcul

Masina von Neumann



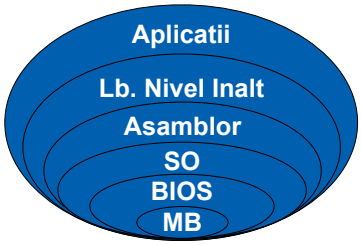
- John von Neumann (1903 – 1957)
- Structura cu acces secvential
  - CPU
  - Memorie (Instructiuni & Date)
  - Ex: EDVAC (1945) – binar
- *“There's no sense in being precise when you don't even know what you're talking about.”*
- *“In mathematics you don't understand things. You just get used to them.”*
- *“It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in 5 years.” (Said in 1949)*

Arhitectura masinii von Neumann

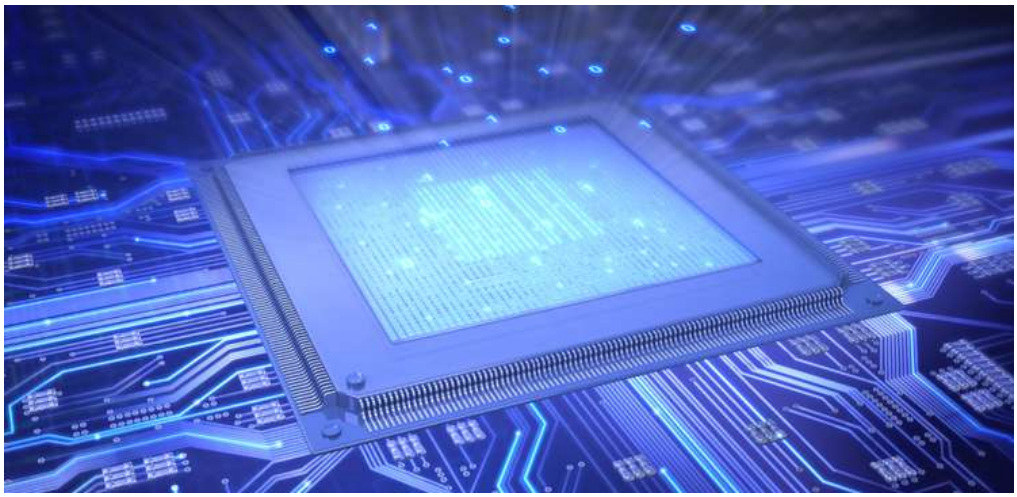


Componentele Arhitecturii

- UCP = {RG, UAL, U_{Cmd}} = Unitatea Centrala de Prelucrare
- UC = {UCP, M} = Unitatea Centrala
- MB = {UC, I/E} = Masina de Baza
- SC = {MB, EP, Software de Baza} = Sistem de Calcul



Software de Baza



Directive PMS

Nivelul Structural de Descriere al Sistemelor Numerice

- PMS (Processor Memory Switches)
- La nivel structural sistemele numerice (SN) sunt specificate si analizate considerand urmatoarele elementele primare:
  - PC = procesorul central
  - M = memoria
  - S = comutatorul
  - L = linia de legatura
  - K = unitatea de comanda
  - PI/E = procesorul de intrare/iesire sau interfata
  - D = operatorul de date
  - T = terminalul

Primitivele PMS

- Fiecare primitiva
  - Este caracterizata printr-un atribut:
    - a_i = atributul asociat primitivei
- Fiecare atribut
  - Are o anumita valoare:
    - v_i = valoarea corespunzatoare atributului
  - Astfel un sistem de calcul devine:
    - SN (a_1:v_1; a_2:v_2; … a_n:v_n)

Memoria – M

- Rolul memoriei:
  - Pastreaza informatia
  - Actualizeaza informatia prin operatii de citire/scriere
  - Necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute:
  - Functie: primara | auxiliara
  - Tehnologie: bipolara | MOS | statica | dinamica
  - Operatii: citire | citire/scriere
  - Mod acces: aleator | secvential | FIFO | LIFO | asociativ
  - Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
  - Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb | 8Gb | 16Gb | …
  - Ciclu de lucru: 4ms | … | 400ns | 200ns | 100ns | 50ns | …

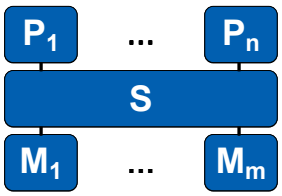
Procesorul Central – P_c

- Rolul PC:
  - Citeste, interpreteaza si executa instructiuni masina
  - In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute:
  - Functie: universal | specializat (de semnale, etc)
  - Implementarea: conventionala | μprogramata | μprogramata specializata
  - Formatul instructiunilor: fix | variabil
  - Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
  - Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
  - Tehnologia: LSI | VLSI | ULSI |…

Switch/Comutatorul – S

- Rolul S:
  - Asigura conexiuni intre componentele sistemului
  - Poate evolua
    - De la structuri foarte simple = buffer 3 state de acces pe magistrala
    - La structuri complexe cu unitati de comanda proprii = comutatoare cu discipline de servire a cererilor de stabilire a legaturilor
- Atribute:
  - Structura: ierarhica | neierarhica
  - Tip: simplex | semiduplex | duplex (e vorba de sensuri)
  - Numar de legaturi realizate:
    - m intrari | n iesiri din S
    - n intrari | m iesiri din S
  - Concurenta: c

Maxim de paralelism: min (n, m)



Legatura – L

- Rolul L:
  - Asigura legatura fizica intre diverse componente ale sistemului
  - Nu prelucreaza informatia ci doar asigura traseul transferului spatial al datelor
  - E fie o magistrala, fie o interfata seriala/paralela
- Atribute:
  - Functia: legatura seriala | legatura paralela | legatura de tip magistrala
  - Lungimea cuvantului: 5 | 6 | 7 | 8 | 16 | 32 | 64 + comanda
  - Mod de dialog: sincron | asincron | cu Q&A: cu/fara interblocare sau cu interblocare completa | fara Ack
  - Mod de control al accesului: inlantuire seriala (token) | interogare | cereri independente
  - Standard: paralel (SCSI) | serial (RS232; 485; 482) | magistrala (MultiBus; MCI(IBM); HPIBus; etc.)

Unitatea de Comanda – K

- Rolul K:
  - Componenta care exprima controlul in diverse subansamble ale sistemului
  - Are o functie de comanda a resurselor asociate unitatilor functionale
    - Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar **comanda** & **control**
- Atribute:
  - Functia: comanda resursa I
  - Implementare: conventionala | μprogramata
  - Numar de stari: n
    - Daca are 2 stari este un bistabil
    - Daca are 64 de stari este un automat complex

Unitatea de Comanda – K

- Citireste, interpreteaza si executa instructiuni masina de **comanda si control**
- Legata de:
  - Setul de instructiuni
    - Intructiuni Universale; Ciclu variabil
    - RISC cu ciclu fix
    - Complex CISC – emularea unei masini intermediare
- Codificarea instructiunilor
  - 0 adrese

  - 1 adresa

  - 2 adrese

  - 3 adrese

- Moduri de adresare multiple la ora actuala

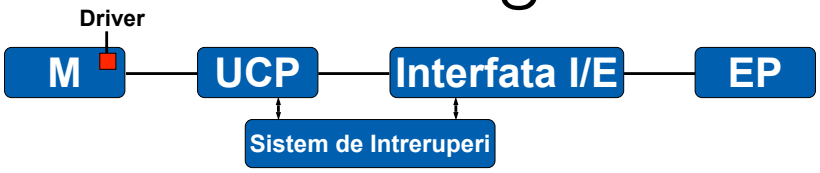
Procesorul de I/E – $P_{I/E}$

- Rolul $P_{I/E}$:
  - Poate fi incorporat in PC-ul in care are functie de I/E sau poate fi privit independent
  - Daca este o primitiva independenta are urmatoarele atribute
- Atribute:
  - Functie: DMA | canal I/E | procesor specializat I/E
  - Tip implementare: conventional | μprogramat | (co)procesor specializat I/E
  - Rata de transfer: Mb/s | Gb/s | Tb/s
  - Mod de transfer: cum rezolva conflictul de acces la UC
    - Prin furt de ciclu
    - Prin rafala
  - Lungimea cuvantului: 8 | 16 biti (in general caractere)

Subsistemul de I/E

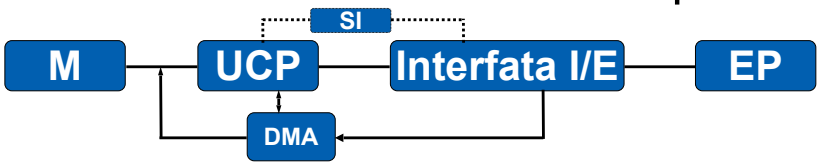
- Asigura transferul datelor intre UC si mediul extern
- Interfetele I/E asigura
  - O adaptare electrica intre EP si UCP
  - Sincronizarea temporala intre EP si UCP printr-un cuvant de stare
    - Diferenta este de cateva ordine de marime
  - Transfer de date intre EP si UC:
    - Transfer programat
    - Acces direct la memorie
    - Canal de intrare/iesire
    - Calculator de intrare/iesire (Front End Computer/Processor - FEP)

Transfer Programat



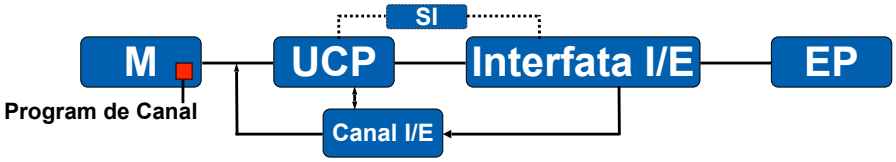
- UCP-ul se ocupa de fiecare caracter in parte
- Driver-ul se ocupa de transferul datelor I/O
- Sincronizarea se poate face
  - Prin citirea ciclica a starii (busy waiting)
    - UCP asteapta dupa EP si ajunge sa lucreze la viteza EP-ului
    - Driver-ul citeste si intrerupe cuvintele de date/stare/comanda
  - Prin intreruperi
    - Se dau ordine EP si se continua prelucrarile
    - Cand se termina se genereaza o intrerupere
    - UCP-ul nu se adapteaza la viteza EP-ului

Acces Direct la Memorie prin DMA



- Asigura transferul de date intre EP si M fara interventia UCP
- Dialogul intre UCP si DMA are loc doar la initiere & incheiere
- Dialogul se face prin transfer programat (Intreruperi)
- Initiere: UCP → DMA: adrese date, sensul (R/W), adresa EP
- La orice eroare DMA cheama UCP-ul
- UCP-ul nu poate initia un alt transfer pana nu s-a incheiat cel curent
- Arbitrarea conflictului de acces la M al UCP/DMA se face prin
  - Furt de ciclu
  - Rafala

Canal de I/E



- Canalul I/E e un procesor specializat ce asigura transferul intre EP si M fara interventia UCP
- Dialogul UCP – EP se face printr-un program de canal (din M – se da doar adresa de inceput a programului)
- UCP poate inlantui mai multe programe de canal = mai multe transferuri de blocuri de date (mai bine ca la DMA)
- Canalul aduce informatia necesara transferului in registrele proprii prin DMA (acces direct)
- Canalul reincearca transferul in caz de eroare

Calculator de I/E

- In cazul calculatorului de I/E, SC poate avea toate facilitatile de mai sus:
  - DMA
  - Canal I/E
  - Transfer programat
- Mecanism util pentru a nu irosi timpul unui procesor / calculator puternic
- Ofera datele rapid si eficient pentru prelucrare
- Este folosit des in sistemele multiprocesor

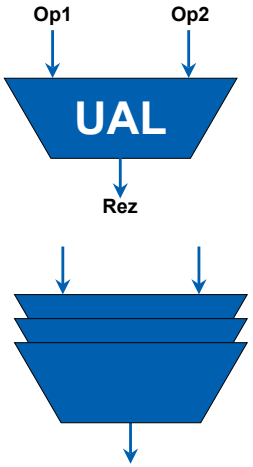
Operatorul de Date – D/UAL

- Rolul D:
  - “Produce” unitati de informatie cu semnificatie noua
  - Efectueaza operatii aritmetice & logice + prelucrari primare de compactare, expandare si asociere asupra datelor
  - Este unitate de prelucrare: in virgula mobila si zecimala
  - Sunt module optionale, de sine statatoare, independente de PC
- Atribute:
  - Functie: prelucrare in virgula mobila | zecimala(BCD) | vectoriala | matriceala | cu liste
  - Tip implementare: conventionala | procesor specializat (coprocesor matematic)
  - Structuri de date asupra carora opereaza: scalari | vectori | matrice | liste
  - Operatii: + | - | * | / | cautare atomica | memorare/extragere atomica

Evolutia Operatorului de Date D/UAL

- UAL: in virgula fixa si virgula mobila
- Formatul general al instructiunilor

- Evolutia spre SIMD (masini vectoriale)
  - UAL multiple
  - Fiecare element are o prelucrare dedicata
  - Rezultatele sunt predate etajului adiacent
- Op UAL: +, -, *, /, ^, %, Shiftari, etc
- In timp UAL-ul s-a extins catre coprocesorul matematic



Terminalul – T

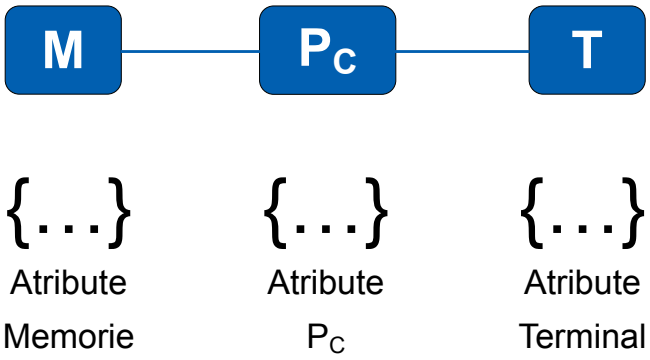
- Rolul T:
  - Asigura conversia din punct de vedere fizic a datelor
    - Adaptare electrica
    - Asigura sincronizarea intre terminal si UC de prelucrare sau PI/E care se ocupa de acel terminal
  - Terminalul este format fie
    - Dintr-o singura componenta
    - Din 3 componente
      - Terminal
      - Unitate de comanda a T: interpreteaza comenzi/stari de la UC
      - Switch T

- Atribute:
  - Functie: cupleaza terminalul i
  - Tip cuplare: seriala | paralela
  - Caracteristici: viteza de transfer | capacitate | pagini/min

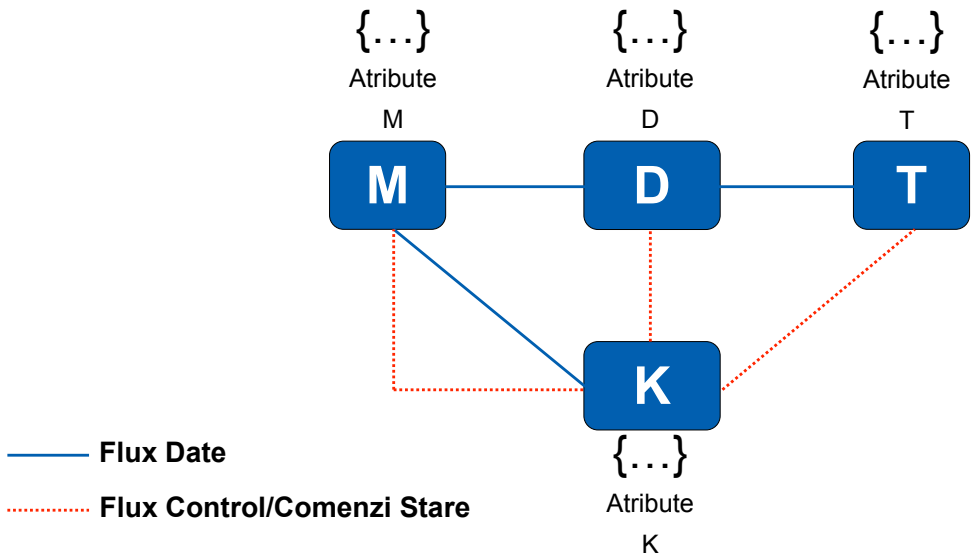
Exemple de Descrieri PMS

Exemplu de Folosire ale Primitivelor PMS

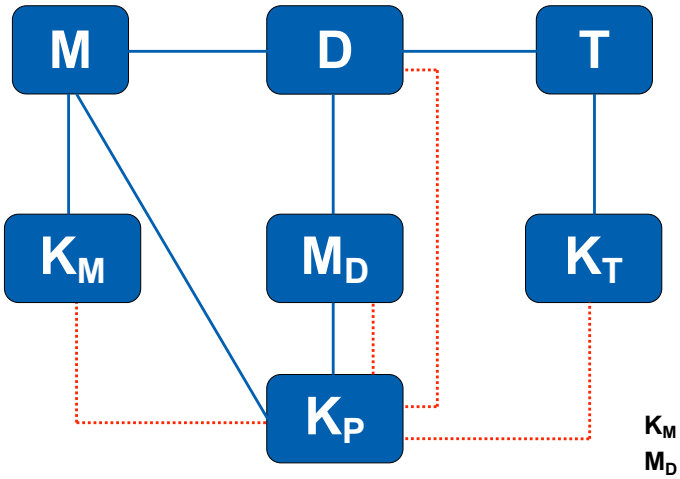
- Sa incercam o structura von Neumann



Detaliem P_C



Detaliem M & T



Flux Date

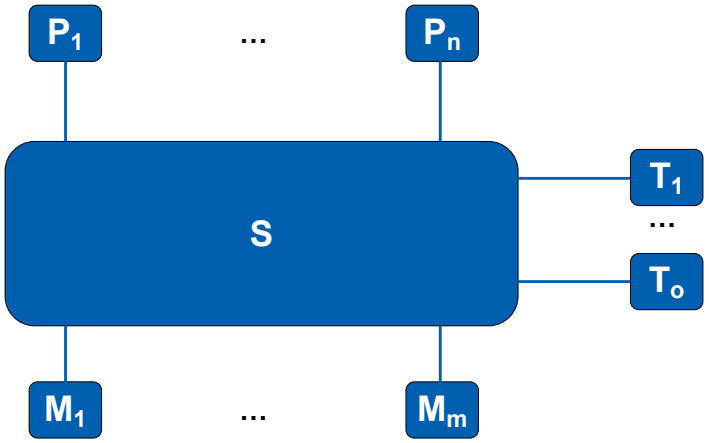
Flux Control/Comenzi Stare

K_M – Unitate de comanda a memoriei

M_D – set de registrii generali

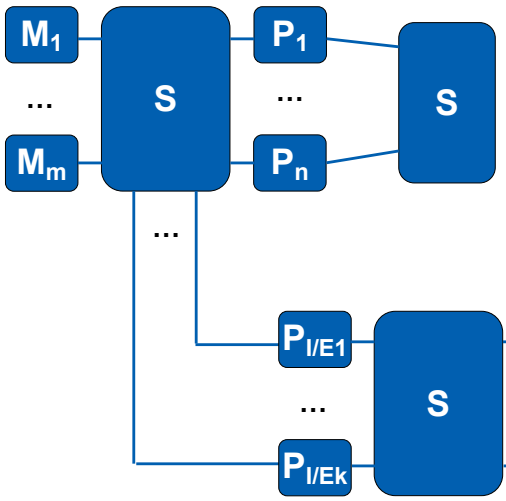
K_P – procesorul ce interpreteaza si executa

Sistem cu mai multe Procesoare si Memorii

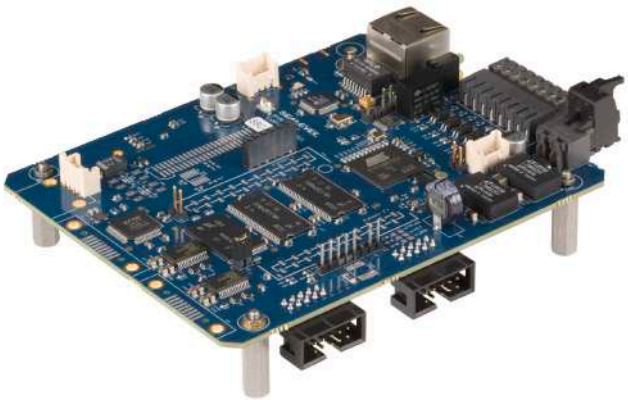


S este cel mai important element pentru ca asigura conexiunile intre $P_i \leftrightarrow M_j$ sau $P_k \leftrightarrow T_l$

Sisteme Multiprocesor



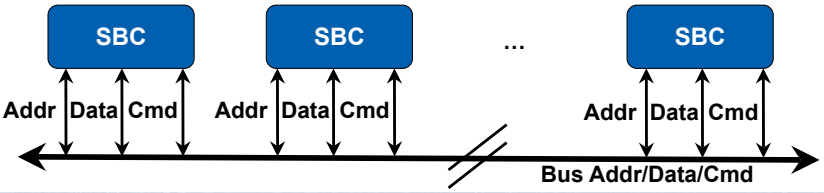
- Memoriile sunt partajate de procesoare
- $P_{I/E}$ pot conecta orice T la orice P



Single Board Computers (SBC)

Single Board Computer

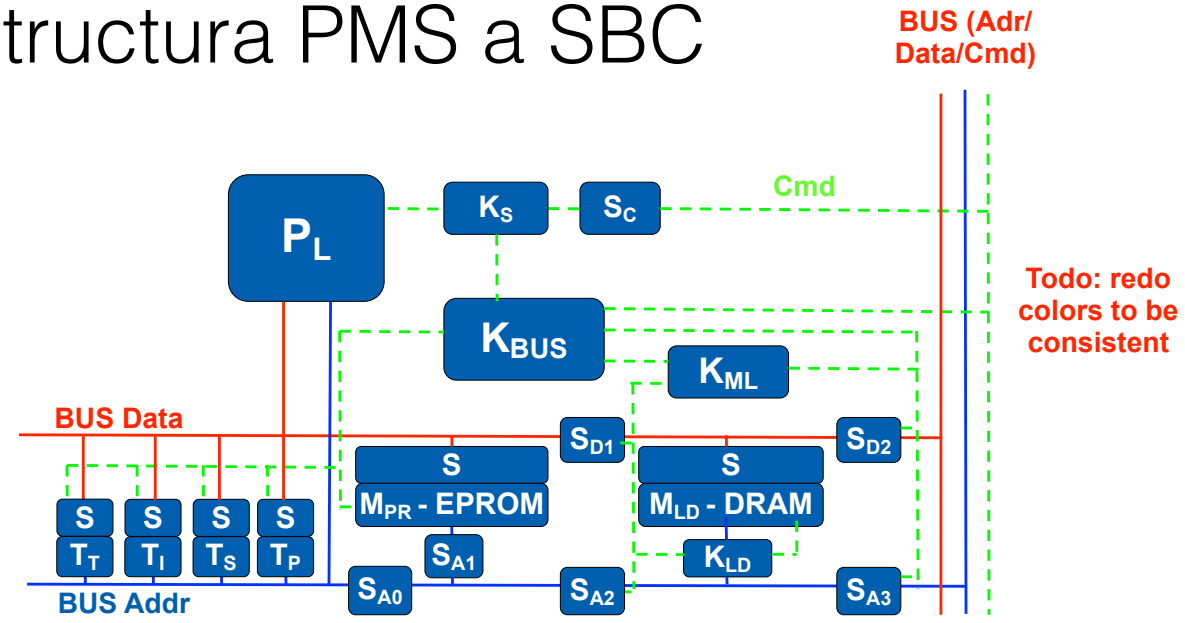
- Are procesor, memorie + interfete I/O
- Conecteaza pe o magistrala comenzi, date si adrese bidirectional
- Resurse SBC:
  - Private: procesor, EPROM, Interfete (S/P), RT Clock, Sistem de Intreruperi
  - Locale: Memoria locala
  - Globale: RAM (in spatiul de adresare)



SBC – Mecanisme Acces Magistrala

- Pentru a gestiona accesul la magistrala (exclusiv) sunt necesare:
  - K_{BUS}: unitate de comanda pt accesul la MAG
  - K_{ML}: unitate de control al accesului la memoria locala
  - K_{LD}: unitate de comanda a memoriei locale (refresh/etc)
  - K_s: unitate de comanda a intregului sistem – asigura controlul procesoarelor la resursele sistemului
- Fiecare procesor al SBC-ului vede memoria locala incapand de la 0 si memoriile celorlalte SBC-uri in spatii adiacente superioare

Structura PMS a SBC



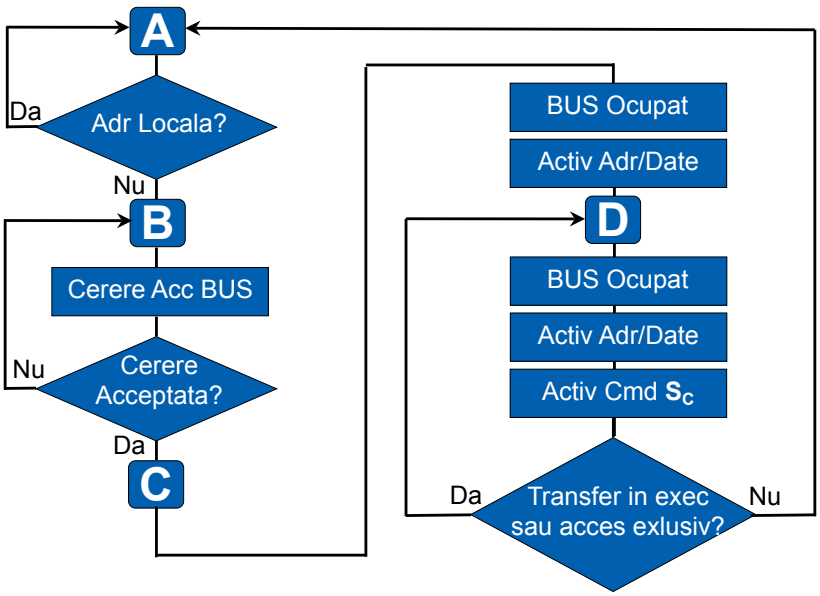
Elementele Structurii PMS a SBC

- P_L: procesor local – furnizeaza date si adrese
- Interfete:
  - T_T – de ceas de timp real
  - T_I – sistemului de intreruperi
  - T_S – seriala & T_P – paralela
  - Fiecare interfata are adresele portului sau de I/O
  - Daca isi recunoaste adresa isi pune datele pe magistrala
- M_{LD} – DRAM: memoria poate fi accesata de P_L si de procesoare din exterior
  - Exista niste S-uri care acorda accesul fie procesorului local fie celor externe
- S_{D1} si S_{D2} S-uri locale de date – separa procesoarele intre ele
- S_{A0} si S_{A1} – S-uri interne de adrese
- S_{A2} si S_{A3} – S-uri externe de adrese
- S_C – S prin care se transmit comenzi catre exterior
- Obs: structura PMS poate fi detaliata pe componente cu directive PMS

SBC – Mecanisme Acces Magistrala

- K_s e specifica fiecarui procesor
  - In procesoarele moderne e inclusa in chip
  - K_s genereaza semnalele de control ale resurselor locale
  - Generarea se face prin interpretarea semnalelor de stare date de microprocesor
- Trebuie sa existe o unitate de comanda K_{BUS}:
  - Fiecare procesor trebuie sa intre intr-o stare de asteptare pana primeste dreptul de acces la magistrala
  - Procesorul face doar un ciclu care apoi se poate prelungi prin stari de T_{WAIT}
- K_{ML} controleaza S-urile de adrese si de date
  - Accesul local sau extern la memorie
  - Daca deschide toate S-urile se ajunge la accesul direct al procesorului local in exterior

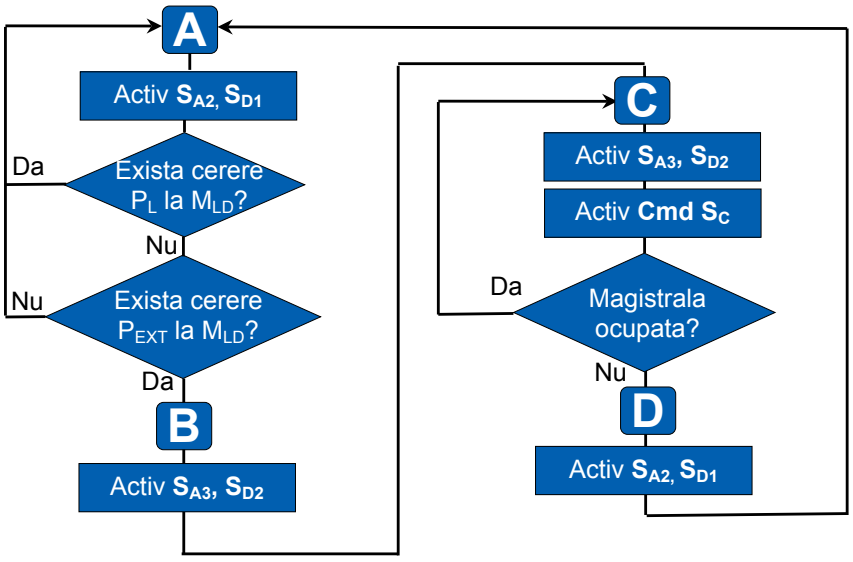
K_{BUS}



Descriere K_{BUS}

- K_{BUS} – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un Δ_t acolo = T_{Wait} datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: K_{BUS} trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ

K_{ML}



Descriere K_{ML}

- K_{ML} – asigura accesul la Mem locala a PL prin intermediul S_{D1} si S_{A2} & a proc externe prin S_{D2} si S_{A3}
- PL are prioritate fata de proc externe (P_{EXT}) == daca vrea P_L la M_{LD} NU ma intereseaza ce vor celelalte P_{EXT}
- Starea A: P_L poate avea comenzi locale
- Starea B: activez accesul ext la M_{LD} – semnalele asociate starii
- Obs: Mem locala e vazuta de fiecare procesor acelasi spatiu de memorie, iar de procesoarele ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un P_{EXT} a terminat ciclul masina curent → abordare top-down

Ce am invatat in aces curs?

- Care sunt componentele sistemelor de calcul
- Processor Memory Switches - PMS
- Exemple de descrieri PMS
- Single Board Computers - Unitati de command

In cursul urmator

- Ierarhia de memorii – Bottleneck-ul SC
- Localitatea datelor
- Cache – design, implementari si exemple
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie

Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 3
cs.pub.ro
cs.curs.pub.ro

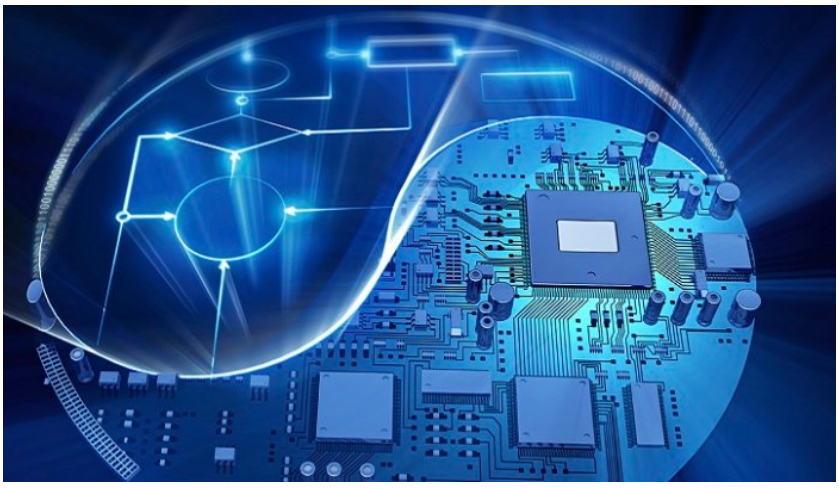
University Politehnica of Bucharest

In cursul anterior

- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)

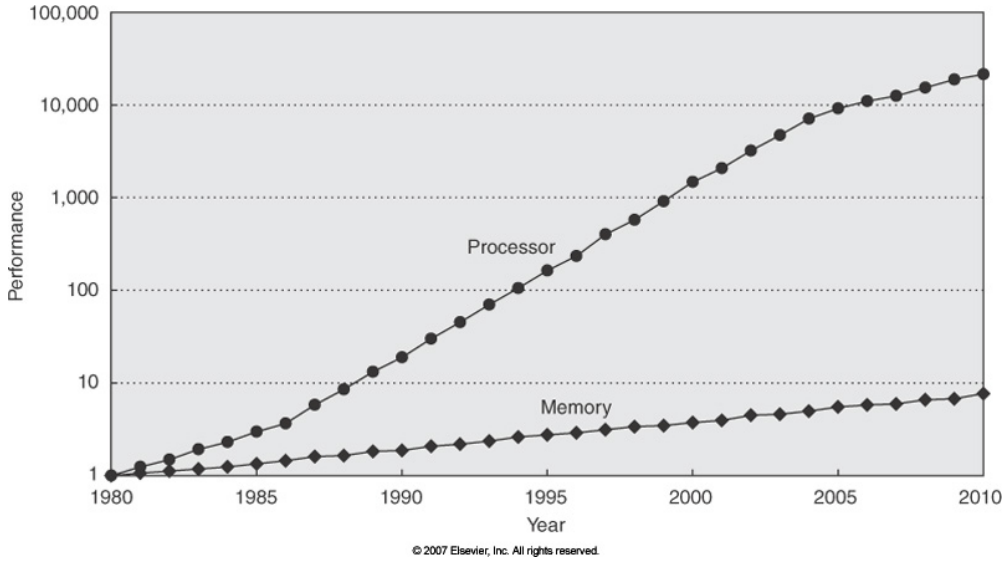
Cuprins

- Ierarhia de memorii – Bottleneck-ul SC
- Localitatea datelor
- Cache – design, implementari si exemple
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie



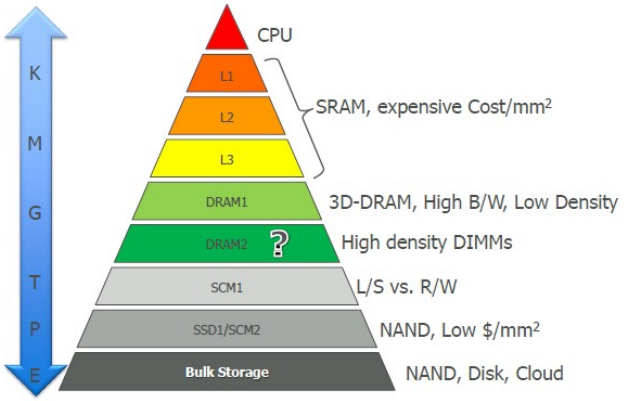
Ierarhia de memorii – Bottleneck-ul SC

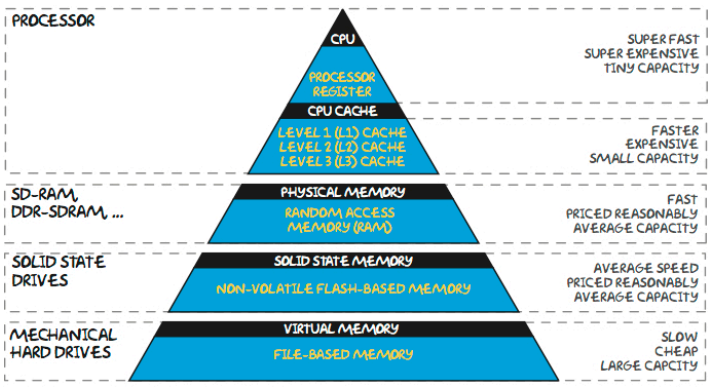
Este Memoria o Problema?



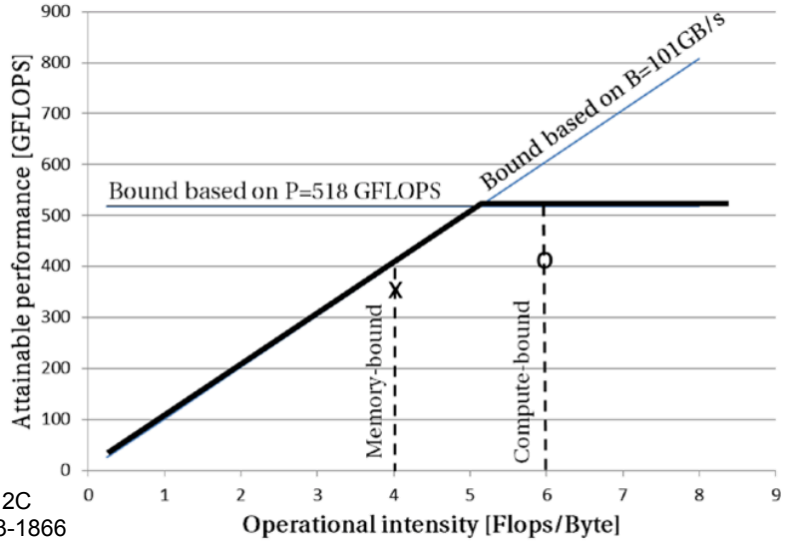
© 2007 Elsevier, Inc. All rights reserved.

Ierarhia de Memorii





Modelul Roofline

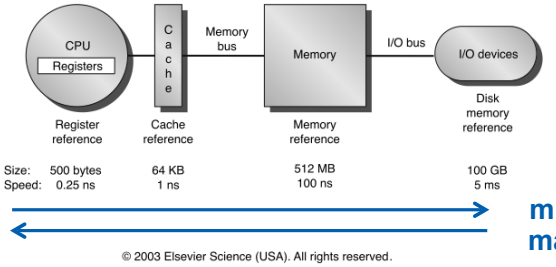


Intel XeonE5-2697v2 12C
2.7GHz cu memorie DDR3-1866

Ierarhia de Memorii

- Deoarece nu putem realiza memorii suficient de rapide(ieftin) – am dezvoltat **ierarhia de memorii**:
  - L1 Cache (pe chip)
  - L2 Cache
  - L3 Cache (uneori)
  - Memoria Principala
  - Discurile fixe

Ierarhia de Memorii

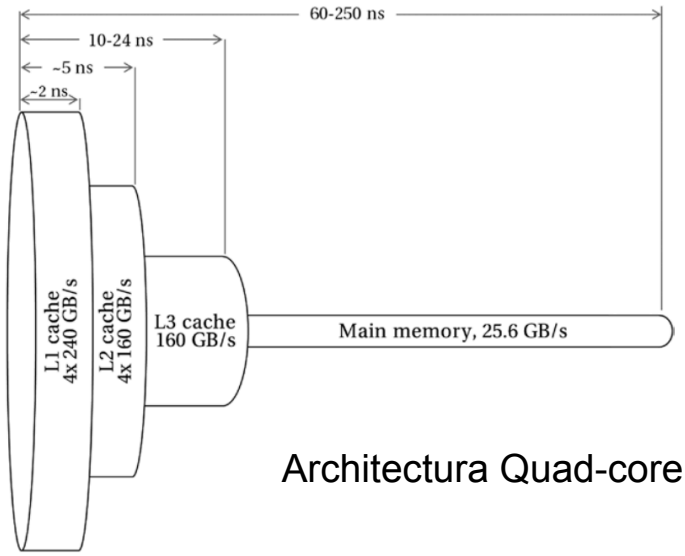


- Numerele sunt orientative si reprezentative pentru sisteme din ultimii ani
- Pentru valori reale, mergeti online si verificati specificatiile
- Pentru a intelege acest desen, trebuie lamurit conceptul de “localitate”
  - Localitate spatiala – datele adiacente unor date utilizate sunt refolosite
  - Localitate temporala – datele sunt refolosite des
- Pentru un cost dat, un hardware mai mic este in general mai rapid
- Datorita acestor aspecte functioneaza ierarhia de memorii

Memorii: Latenta/Bandwidth

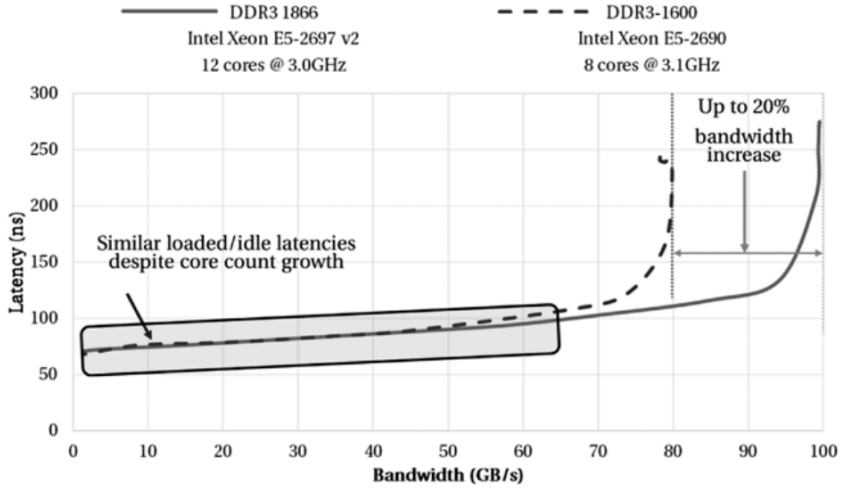
Component	Typical Latency	Typical Throughput
Local SATA HDD	~1 ms	100 MB/s
Local SATA SSD	~1 ms	500 MB/s
1GB Ethernet	~15 us	100 MB/s
10GB Ethernet	~4 us	1 GB/s
Infiniband FDR	1.5 us	~6.5 GB/s
Local memory (loaded)	~250 ns	~100 GB/s
Local memory (idle)	~60 ns	0 GB/s
Remote memory (idle)	~100 ns	0 GB/s
QPI (intersocket)	~100 ns*	~64 GB/s
L3 cache access	10-25 ns	~160 GB/s
L2 cache access	~5 ns	~160 GB/s
L1 cache access	2-3 ns	~240 GB/s
XOR instruction	~6 ns	~2.5 Ginstruction/s
Branch misprediction	~7 ns	-
SIMD Division instruction	~16 ns	~1 Ginstruction/s

Latenta & Bandwidth



Architectura Quad-core Intel Haswell

Latenta & Bandwidth



- Comparatie intre Intel Sandy Bridge 8C (2011) si Intel Ivy Bridge – 12C (2013)
  - Trafic 66% citiri si 33% scrieri

Localitatea datelor

Ce este Localitatea?

- Programele acceseaza date din memorie apropriate intre ele prin instructiuni apropiate intre ele in **Instruction stream**
  - Instruction stream = secventa de instructiuni de la inceputul pana la sfarsitul programului
- **Localitatea spatiala**: cand se acceseaza o locatie anume de memorie, este foarte probabil ca urmatorul acces sa fie in apropierea acestei locatii in memorie
  - Ar fi astfel ideala aducerea “aproape” de procesor, a blocurilor contigue de date pentru ca instructiunile urmatoare sa le gaseasca disponibile
- **Localitatea temporala**: cand se acceseaza o locatie anume de memorie, este foarte probabil ca aceasta locatie sa mai fie accesata din nou
  - Tinerea datelor accesate de curand, “aproape” de procesor, pentru ca instructiunile urmatoare sa le gaseasca disponibile

Exemplu de Localitate

- Iata o secventa de adrese accesate de catre procesor:
  - 1,2,1200,1,1200,3,4,5,6,1200,7,8,9,10
- Aceasta secventa are o **localitate temporala buna**, deoarece locatia @1200 este accesata de trei ori din 14 referinte
  - Poate ca @1200 este un contor ce este actualizat
- Aceasta secventa are si o **localitate spatiala buna**, deoarece locatiile [1,2], [3,4,5,6] si [7,8,9,10] sunt accesate in ordine
  - Poate ca in aceste locatii se afla elementele unui vector ce este accesat in ordine
- Secventa prezentata poate exploata ierarhia de memorii prezentata

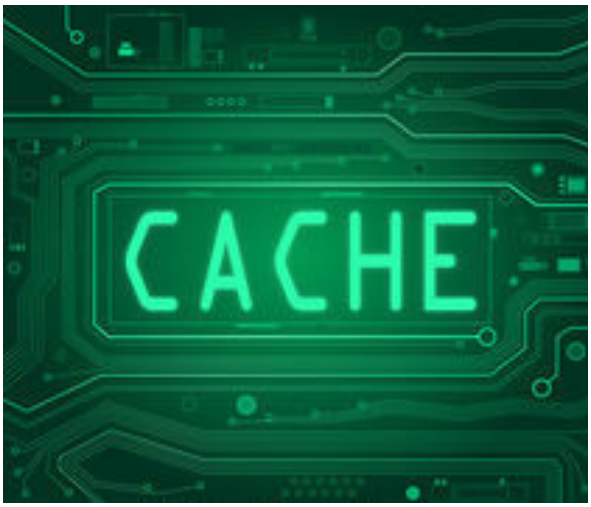
Exemplu de Localitate

```
sum = 0;
for (i = 0; i < n; i++)
  sum += a[i];
*v = sum;
```

- Localitate de Date
  - Consideram secventa de locatii de memorie: &sum, &i, &i, &n, &a[0], &sum, &sum, &i, &i, &i, &n, &a[1], &sum, &sum, &i, &i, &i, &n, &a[2], &sum, &sum, ..., &a[n-1], &sum, &sum, &sum, v
  - sum, i, si n pot fi localizati in registrii deoarece au o **localitate temporala foarte buna**
  - Accesul la vectorul a are insa o **localitate spatiala foarte buna** → cache
- Localitate de Instructiuni
  - Toate instructiunile din bucla sunt accesate in mod repetat si in secventa, si astfel au atat **localitate spatiala** cat si **temporala buna**

Localitate & Ierarhia de Memorii

- Localitatea este exploatata de catre caching
- Cand o locatie de memorie este accesata de catre procesor, se verifica urmatoarele:
  - Se afla in cache-ul L1?
  - Daca nu, se afla in cache-ul L2?
  - (Daca nu, se afla in cache-ul L3?)
  - Daca nu, se afla in memoria principala?
  - Daca nu, se afla pe memoria virtuala (discuri fizice)?
- Acest proces are loc deoarece nu putem construi memorii suficient de mari si rapide la un pret rezonabil
- Daca vom identifica multe din locatiile de memorie cautate in cache-ul L1, vom avea “iluzia” unei memorii mari si rapide, pentru o fractiune din cost



Cache – Design, Implementari si Exemple

Terminologie

- Cache-ul reprezinta locatia in care procesorul poate gasi date cautate mai aproape de el decat in memoria principala a sistemului
- Un cache hit este momentul in care datele au fost gasite intr-un nivel de Cache
- Un cache miss este momentul in care datele nu au fost gasite
- Un exemplu de functionare:
  - L1 cache miss, L2 cache miss, L3 cache hit
- Cand se inregistreaza un miss, un bloc este adus in Cache
  - Un bloc este un set de date de dimensiune fixa
  - Blocul contine celulele cerute, si aditional alte celule ce speram ca vor fi folosite de alte instructiuni (localitate spatiala)
- Un cache miss determina un overhead seminificativ:
  - Definim
    - lat = latenta memoriei (in secunde)
    - bw = latimea de banda (in bytes/sec)
    - S = dimensiunea blocului in bytes
  - Un cache miss duce la un overhead de lat + S/bw secunde

Terminologie

- Cache-ul este compus din seturi de blocuri: Block Frames
  - Fiecare set poate contine unul sau mai multe blocuri (depinde de implementare)
- Hit time este timpul de accesare al Cache-ului
- Miss penalty este timpul necesar mutarii datelor pe diferite nivele de Cache catre procesor
- Hit Ratio este procentul din timp in care datele sunt gasite in Cache
- Miss Ratio este 1 – Hit Ratio
- Instruction Cache este Cache-ul ce contine instructiuni (cod)
- Data Cache este Cache-ul ce contine date
- Unified Cache este Cache-ul ce contine atat date cat si instructiuni

Memorie Virtuala

- Daca sistemul de calcul permite memoriei principale sa fie utilizata ca un cache pentru discurile fixe – memoria virtuala
- In acest caz, blocurile memoriei sunt numite pagini
- Cand procesorul acceseaza o adresa dintr-o pagina ce nu se afla in memorie, se genereaza un page fault similar unui cache miss
- Un page fault este extrem de costisitor (latenta lat e mare si largimea de banda bw este mica) si procesorul executa in mod normal alte taskuri in acest timp
  - Aceste page fault-uri sunt realizate de catre sistemele de operare

Caracteristici Topic ale Memoriilor

	Dimensiune	Latenta (ns)	Largime de banda (MB/sec)	Gestionat de
Registre	< 32KB	0.25 - 0.5	25,000 - 100,000	Compilator
Cache	< 32MB	0.5 (on-chip) 25 (off-chip)	7500 -25,000	Hardware (Software)
Memoria Principala	< 32GB	25 - 250	1000 - 10,000	O/S
Discurile fixe	> 100GB	1,000 - 5,000,000	20 - 150	O/S

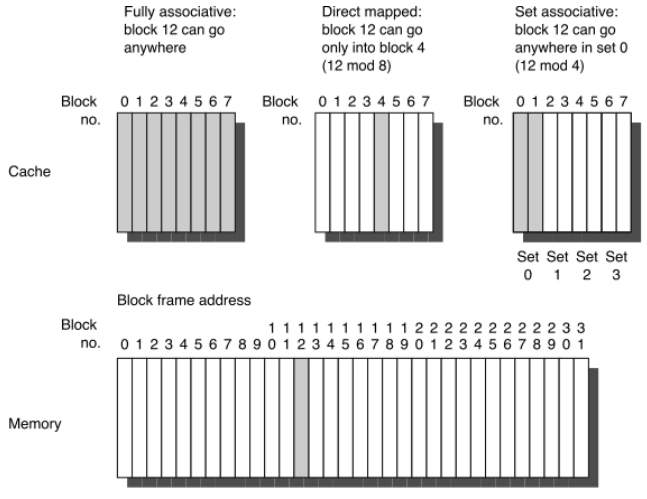
Probleme in Designul Cache-ului

- Ce dimensiune trebuie sa aiba un bloc de Cache?
  - Mare: miss penalty ridicat
  - Mica: oportunitate scazuta de exploatare a localitatii spatiale
  - Trebuie determinat dupa un benchmarking riguros si numeroase simulari
  - Trebuie sa fie eficient pentru aplicatii “tipice”
- Patru intrebari necesare in designul unui cache (L1)
  - Unde trebuie sa fie scrisa un bloc in cache? – block placement
  - Cum se gaseste o linie in cache? – block identification
  - Ce linie trebuie sa fie inlocuita in cazul unui cache miss? – block replacement
  - Ce se intampla cand se scrie in cache? – write strategy
- Limitari – design-ul trebuie sa fie simplu, astfel incat cache-ul sa fie rapid!

Q1 - Block Placement

- Daca un bloc poate fi pozitionat doar intr-o singura pozitie in cache: mapare directa
- Daca un bloc poate fi asezat oriunde: mapare total asociativa
- Daca un bloc poate fi asezat intr-un subset de pozitii: mapare set asociativa
  - Varianta pentru n blocuri in fiecare subset: mapare n set asociativa
  - Astfel, maparea directa este in fapt o mapare 1 set asociativa

Block Placement



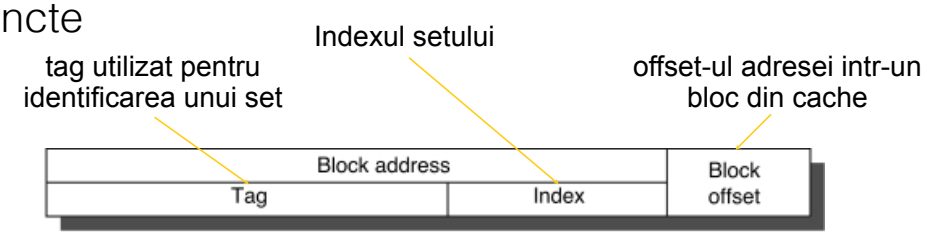
© 2003 Elsevier Science (USA). All rights reserved.

Trade-Offs

- Maparea n-way set associativa devine din ce in ce mai dificila si costisitoare de implementat pentru un n mare
  - Majoritatea cache-urilor sunt 1, 2 sau cel mult 4 set asociative
- Cu cat n-ul este mai mare, cu atat este mai mica probabilitatea de aparitie a fenomenului de thrashing
  - Momentul in care doua (sau mai multe) regiuni de memorie sunt accesate in mod repetat si pot incapea impreuna in acelasi bloc din cache

Q2 - Block Identification

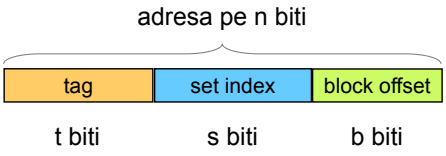
- Avand la dispozitie o adresa, cum putem identifica locatia acesteia in cache?
- Acest lucru se realizeaza impartind adresa in trei parti distincte



© 2003 Elsevier Science (USA). All rights reserved.

Cache Mapat Direct

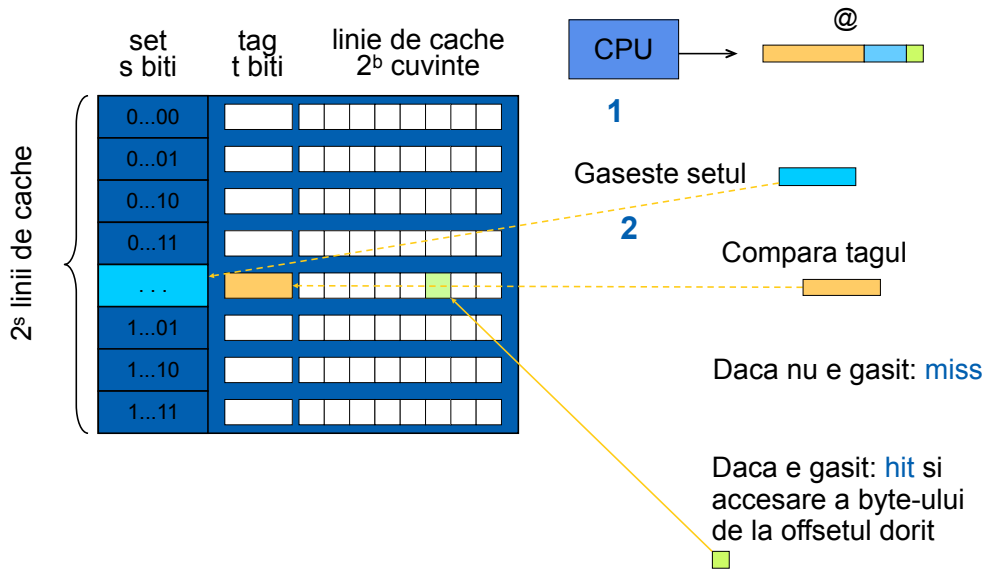
- Cea mai simpla solutie
  - O linie de memorie poate fi plasata intr-un singur loc in Cache



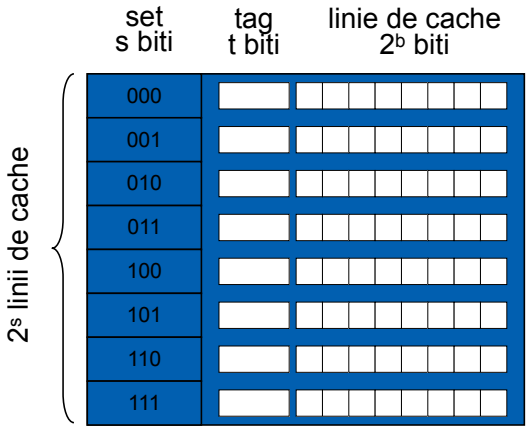
Fiecare adresa se refera la locatia unui cuvant

- Fiecare bloc de memorie/cache contine 2^b cuvinte
- Memoria Cache are contine 2^s linii
- Dimensiunea cache-ului este de 2^{s+b} cuvinte
- Fiecare linie din cache e identificata de tag

Cum Functioneaza?



Cache Mapat Direct



Cache

s=3, t=4, b=3

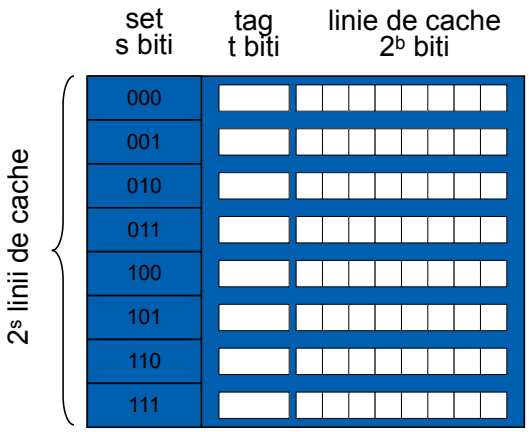
Liniile de cache contin date

Tag-urile contin o parte din adresa liniei de memorie

Seturile corespund unei parti a adresei liniei de memorie

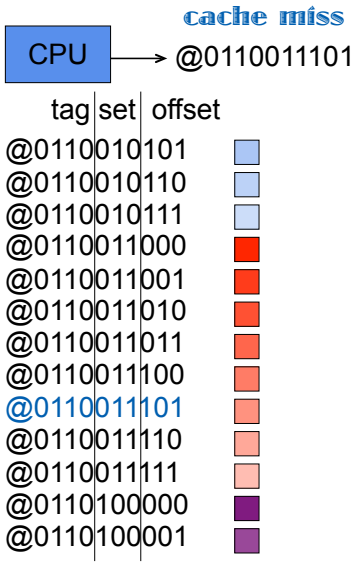
Locatiile de memorie intr-o linie de memorie sunt adrese prefixate cu <tag><set>

Cache Mapat Direct

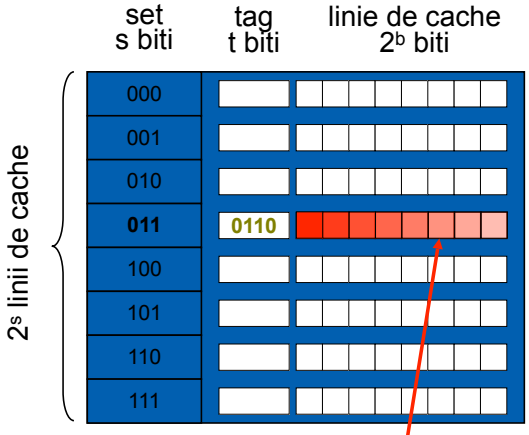


Cache

s=3, t=4, b=3



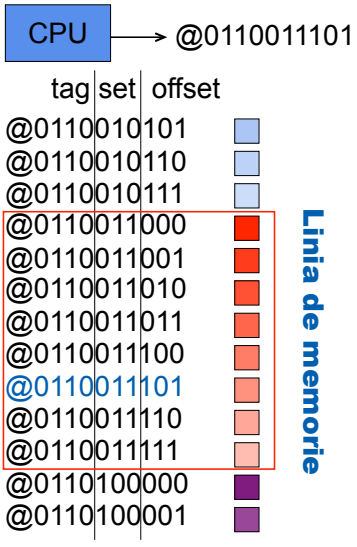
Cache Mapat Direct



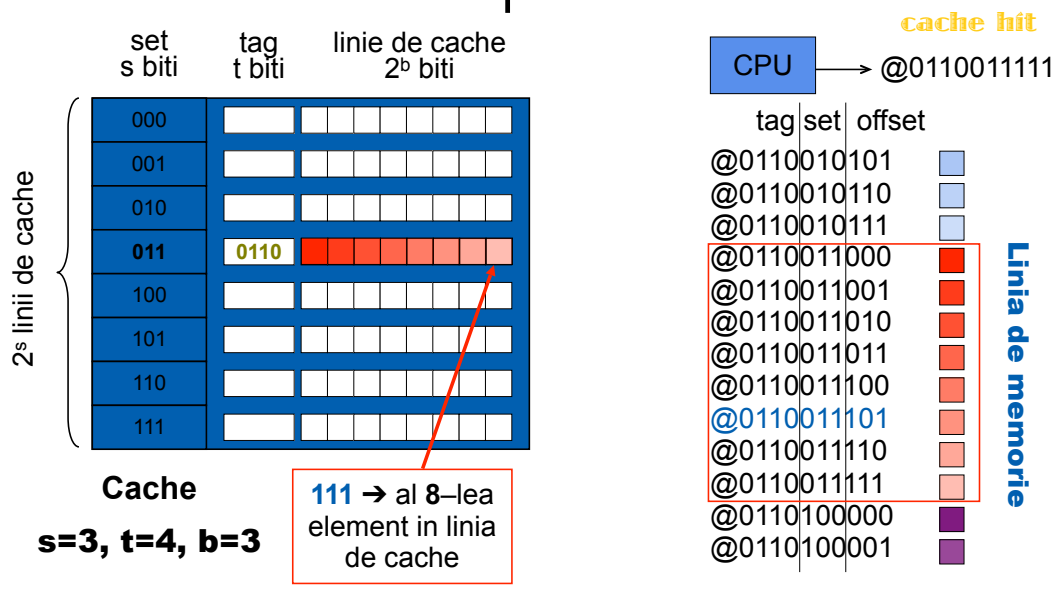
Cache

s=3, t=4, b=3

101 ➔ al **6**–lea element in linia de cache



Cache Mapat Direct

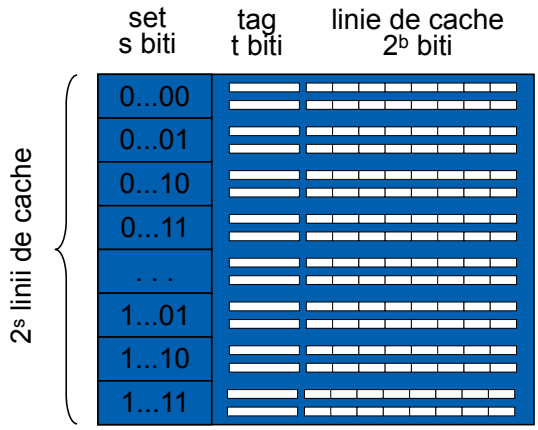


Cache Mapat Direct — Concluzii

- Avantaje
  - Design simplu
    - Doar cateva comparatii intre parti din adrese de memorie
      - In plus, fiecare linie de cache are un bit de validare asociat
  - Astfel, Cache-ul mapat direct este:
    - Rapid
    - Necesita putin hardware
- Dezavantaj major
  - Este vulnerabil la Thrashing (murdarire)

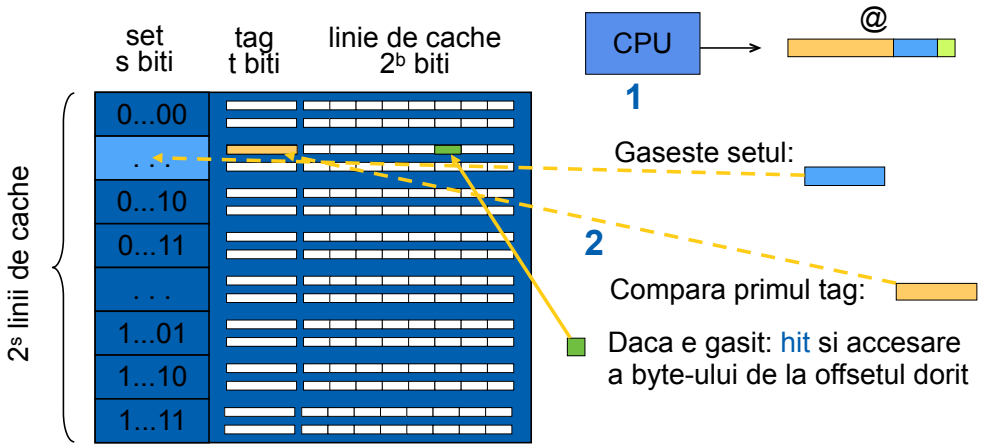
Mapare Set Asociativa

- Fiecare “set” din cache poate pastra mai multe linii din memorie: 2 - 8
- O linie de memorie poate fi mapata pe oricare dintre aceste doua pozitii

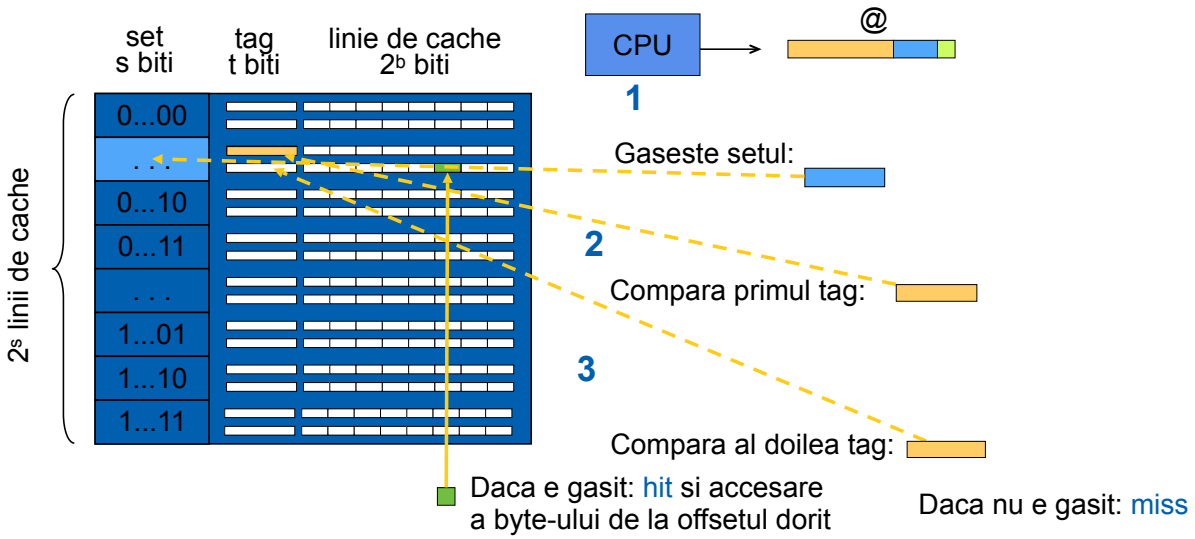


Toate adresele cu un set index similar “concureaza” pentru doua frame bloc-uri posibile

Cum Functioneaza?

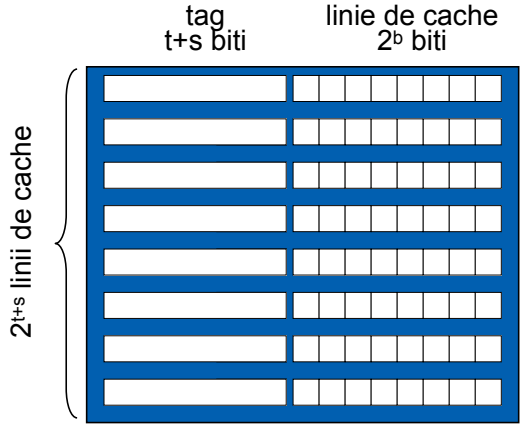


Cum Functioneaza?



Mapare Total Asociativa

- O linie de memorie poate fi plasata oriunde in Cache!
  - Se obtine considerabil mai putin thrashing
  - Complexitate crescuta a unitatii de comanda



Identificarea liniilor de Cache se va face cu bitii t+s din adresa liniei de memorie

Q3 - Block Replacement

- Cand are loc un miss, controlerul de cache trebuie sa “faca ceva”
- Pe un cache mapat direct, este foarte simplu – se va suprascrie continutul unui block-frame cu date noi aduse din memorie
- Intr-o implementare n-set-asociativa insa, trebuie sa facem o alegere: care dintre cele n block-frame-uri trebuie suprascrise?
- In practica, exista trei strategii implementate pentru acest lucru
  - Random: foarte usor de implementat
  - First-In-First-Out (FIFO):
    - Un pic mai dificil de implementat
    - Aduce beneficii la cache-uri de dimensiuni mai mici
  - Least-Recently Used (LRU):
    - Si mai dificil de implementat, mai ales pentru cache-uri de dimensiuni mari
    - Aduce beneficii la cache-uri de dimensiuni mai mici
    - Atentie, trebuie sa avem mereu in vedere costul cache-ul vs latenta oferita de acesta: overhead-ul computational si hardware-ul aditional necesar calcularii valorilor LRU

Q4 - Write Strategy

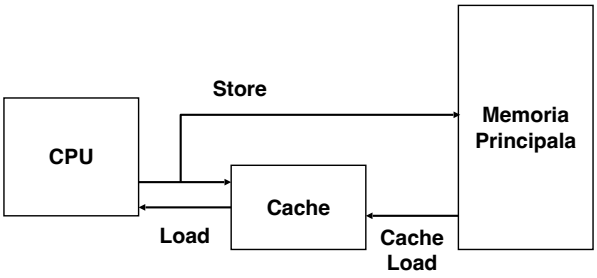
- Sunt mult mai multe citiri decat scrieri in cache!
  - In general, toate instructiunile trebuiesc citite din memorie
  - Astfel, avem in medie:
    - 37% instructiuni load
    - 10% instructiuni store
  - Astfel, procentul de accese de tip scriere este: .10 / (1.0 + .37 + .10) ~ 7%
    - Procentul de accese la date in memorie pentru scriere este astfel: .10 / (.37 + .10) ~ 21%
- Amintiti-va principiul fundamental: Make the common case fast!
- Ca urmare, designul cache-urilor a fost optimizat pentru a face citirile rapide, si nu scrierile
  - De retinut:procesorul trebuie sa astepte mereu la citire, si aproape niciodata la scriere!
- Pe de alta parte, daca scrierile sunt extrem de incete, legea lui Amdahl ne spune ca performantele globale ale sistemului vor fi scazute
- Astfel, trebuie sa investim “ceva” efort si in imbunatatirea scrierilor

Politici de Scriere in Cache

- Dupa cum ne amintim de la CN
- Write-through: datele sunt scrise in acelasi timp in blocul din cache si in blocul din memoria principala
- Write-back: datele sunt scrise in memoria principala doar cand un bloc frame este eliberat/inlocuit din cache
  - Se utilizeaza un bit “dirty” pentru a indica daca un bloc de inlocuit a fost modificat (s-a scris in el), pentru a salva scrieri inutile in memoria principala, cand un bloc este de fapt “curat”

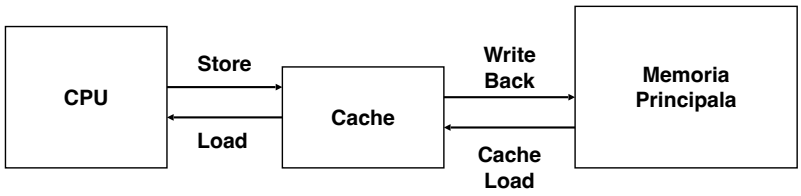
Write-Through

- Ce se intampla cand procesorul modifica o locatie de memorie care se afla in Cache?
- Solutia 1: Write-Through
  - Se scrie in acelasi timp in cache si in memoria principala
  - Memoria si cache-ul sunt astfel mereu consistente



Write-Back

- Solutia 2 – Write-Back
  - Se scrie doar in cache
  - Liniile de cache sunt scrise in memoria principala doar cand sunt evacuate/ inlocuite
  - Necesita un bit “dirty” pentru a indica daca o linie de cache a fost modificata sau nu
  - Memoria si cache-ul nu mai sunt totdeauna consistente!

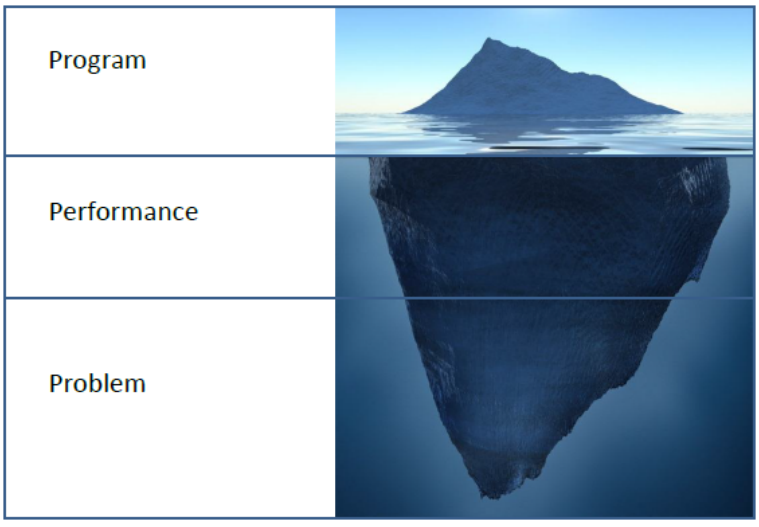


Cache Trade-Offs

- Write-Back
  - Rapid – scrierile se petrec la viteza cache-ului si NU a memorie
  - Rapid – actualizari multiple ale aceluiasi bloc de cache vor fi scrise inapoi in memoria principala in seturi de dimensiuni mai mari, utilizand mai putin din latimea de banda a memoriei
    - Solutie buna pentru servere cu multe procesoare/core-uri
- Write-Through
  - Mult mai usor de implementat ca Write-Back
  - Nivelul urmator de cache va avea intotdeauna o copie actualizata a datelor utilizate
    - Foarte important pentru sisteme multiprocesor – simplifica problemele de coerenta a datelor

Cache Trade-Offs

- Cand doua linii de memorie sunt in cache si a treia linie vine, una din primele doua linii trebuie sa fie inlocuita: care din ele trebuie inlocuita?
- Deoarece nu cunoastem viitorul, utilizam euristici:
  - LRU: Least Recently Used
    - Este greu de implementat, oferă hit ratio bun
  - FIFO
    - Este usor de implementat, hit ratio rezonabil
  - Random
    - Si mai usor de implementat, hit ratio random in functie de pattern-ul de acces
- In general, cache-urile asociative:
  - Pot preveni fenomenul de thrashing
  - Necesita hardware mai complex decat cache-ul mapat direct
  - Sunt mai scumpe decat cache-ul mapat direct
  - Sunt mai incete decat cache-ul mapat direct



Imbunatatirea Performantelor prin Exploatarea Ierarhiei de Memorii

Imbunatatirea Performantelor Memoriei

- Exista mai multe moduri de a imbunatatii performanta cache-urilor si anume
  - Reducerea penalitatilor unui Cache Miss
  - Reducerea ratei de Cache Miss-uri
  - Reducerea timpului de rezolvare a unui Hit
  - Imbunatatirea memoriei
    - Performanta crescuta
    - Cost redus

Optimizari de Compilator

- Sunt mai multe moduri in care un cod poate fi modificat pentru a genera mai putine miss-uri
  - Compilatorul
  - Utilizatorul
- Vom analiza un exemplu simplu – initializarea unui vector bidimensional (matrice)
- Vom presupune ca avem un compilator neperformant si vom optimiza codul in mod direct
  - Un compilator bun trebuie (in principiu) sa poata face aceste lucruri
  - Cateodata insa, compilatorul nu poate face tot ce este necesar

Exemplu: Initializarea unui Vector 2D

```
int a[100][100];
for (i=0;i<100;i++) {
  for (j=0;j<100;j++) {
    a[i][j] = 11;
  }
}
```

```
int a[100][100];
for (j=0;j<100;j++) {
  for (i=0;i<100;i++) {
    a[i][j] = 11;
  }
}
```

- Care varianta este “mai buna”?
  - i,j?
  - j,i?
- Pentru a raspunde la aceasta intrebare, trebuie sa intelegem modul de asezare in memorie al unui vector 2D

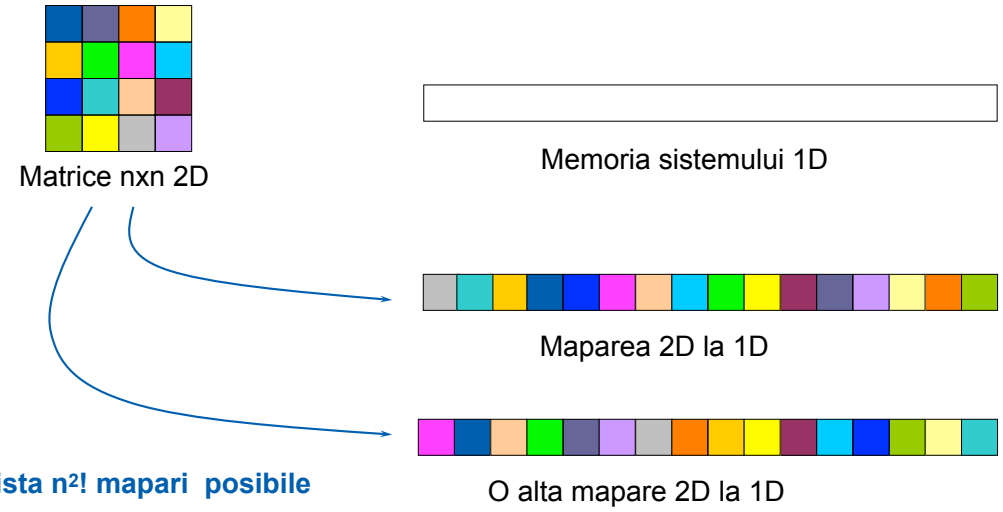
Vectori 2D in Memorie

- Un vector 2D static se declara:

```
<type> <name>[<size>][<size>]
int array2D[10][10];
```

- Elementele unui vector 2D sunt salvate in celule contigue de memorie
- Problema este ca:
  - Matricele sunt conceptual 2D
  - Memoria unui sistem de calcul este 1D
- Memoria 1D este descrisa de un singur numar: adresa de memorie
  - Similar cu numerele de pe axa reala
- Astfel, este necesara o mapare de la 2D la 1D
  - De la abstractizarea 2D din programare, la implementarea fizica in 1D

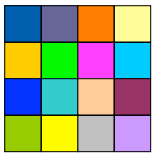
Maparea de la 2D la 1D



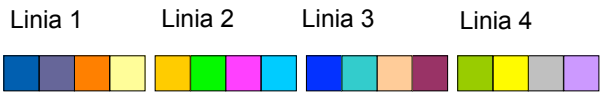
Exista $n^2!$ mapari posibile

Row-Major vs. Column-Major

- Din fericire, in orice limbaj sunt implementate maxim 2 din cele $n^2!$ mapari posibile, si anume:



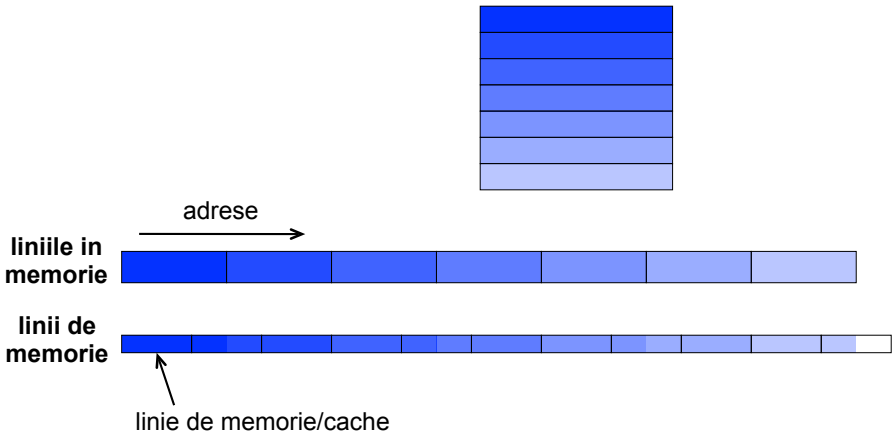
- Row-Major:
  - Liniile sunt stocate continuu



- Column-Major:
  - Coloanele sunt stocate continuu

Row-Major

- Row-Major este utilizat de C/C++

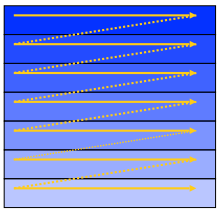


- Elementele matricii sunt stocate contiguu in memorie

Row-Major

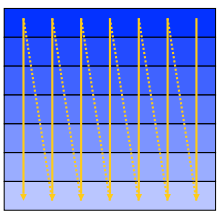
- C/C++ utilizeaza Row-Major
- Prima implementare (i, j)

```
int a[100][100];
for (i=0;i<100;i++)
  for (j=0;j<100;j++)
    a[i][j] = 11;
```



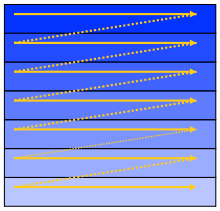
- A doua implementare (j, i)

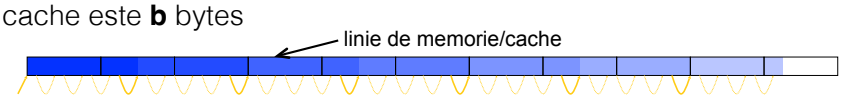
```
int a[100][100];
for (j=0;j<100;j++)
  for (i=0;i<100;i++)
    a[i][j] = 11;
```



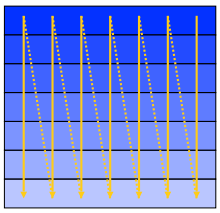
Numarul de Miss-uri

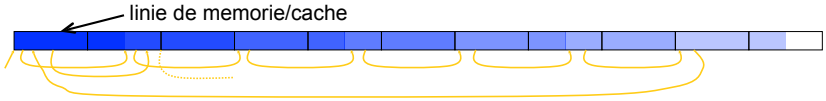
- Definim:
  - Matricea **n** x **n** 2D array,
  - Fiecare element are **e** bytes,
  - Dimensiunea liniei de cache este **b** bytes





- Se obtine un miss la fiecare linie de cache: $\mathbf{n^2}$ x **e/b**
- Daca avem $\mathbf{n^2}$ accese la memorie (toata matricea)
- Rata de miss-uri este: **e/b**
- Exemplu: **Miss rate** = 4 bytes / 64 bytes = **6.25%**
  - In afara cazului in care matricea este foarte mica



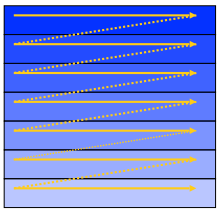


- Avem un miss la fiecare acces
- Exemplu: **Miss rate** = **100%**
  - In afara cazului in care matricea este foarte mica

Initializarea Matricelor in C/C++

- C/C++ utilizeaza Row-Major
- Prima implementare (i, j)

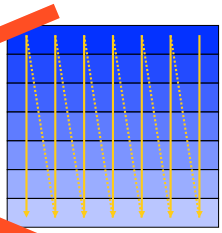
```
int a[100][100];
for (i=0;i<100;i++)
  for (j=0;j<100;j++)
    a[i][j] = 11;
```



Localitate Buna a Datelor

- A doua implementare (j, i)

```
int a[100][100];
for (j=0;j<100;j++)
  for (i=0;i<100;i++)
    a[i][j] = 11;
```



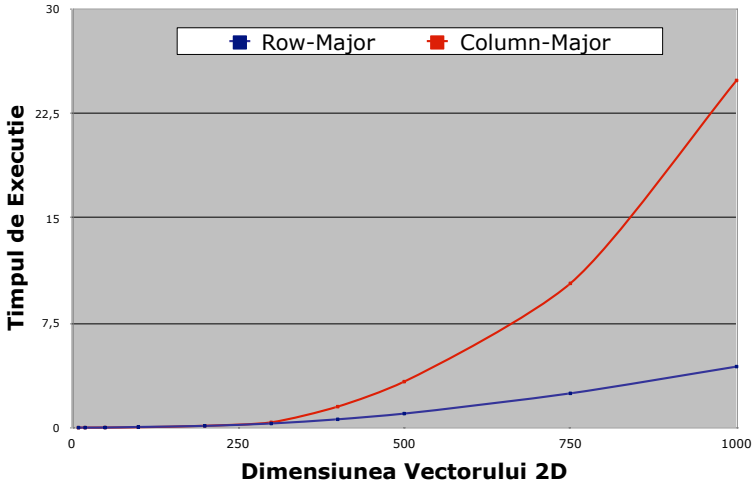
Masurarea Performantelor

- Prima implementare (i, j)

```
int a[100][100];
for (i=0;i<100;i++)
  for (j=0;j<100;j++)
    a[i][j] = 11;
```

- A doua implementare (j, i)

```
int a[100][100];
for (j=0;j<100;j++)
  for (i=0;i<100;i++)
    a[i][j] = 11;
```



- Alte limbare de programare utilizeaza column major
  - FORTRAN de exemplu

Experimente pe un PC normal

Vectori alocați dinamic

- In unele limbaje, putem declara vectori cu dimensiuni variabile
  - FORTRAN:

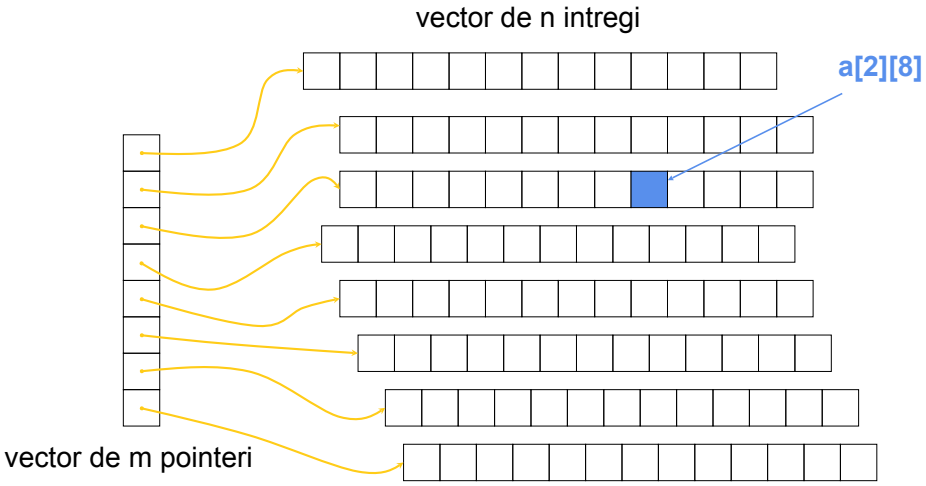
```
INTEGER A(M,N)
```

- C-ul de exemplu nu permite acest lucru
- In C, trebuie sa alocam explicit memoria ca un vector de vectori:

```
int **a;
a = (int **)malloc(m*sizeof(int *));
for (i=0;i<m;i++)
  a[i] = (int *)malloc(n*sizeof(int));
```

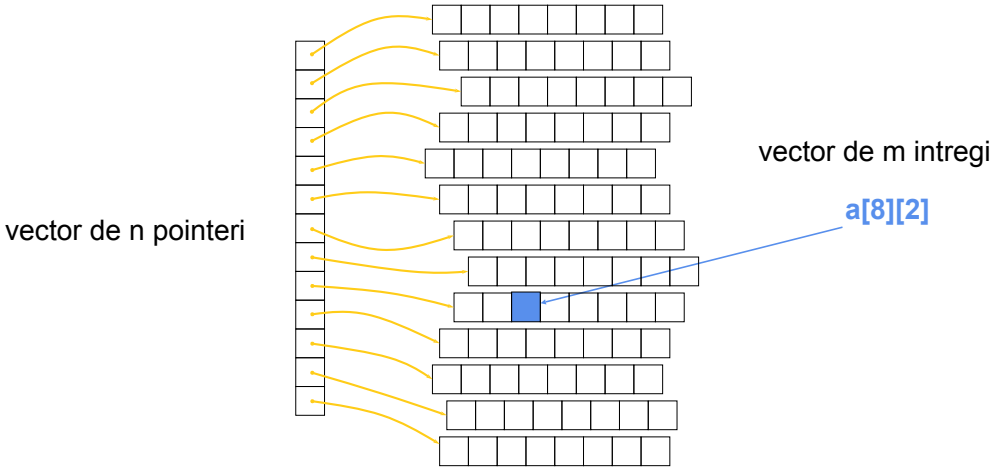
Asezarea Memoriei

- O solutie “non contiguous”de tip row major



Asezarea Memoriei

- Se poate insa face si intr-o implementare column-major

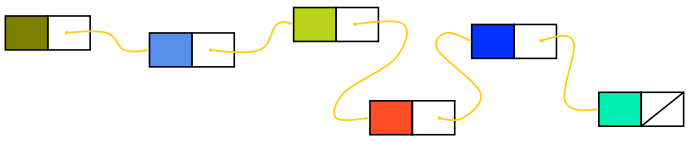


Vectori Dinamici

- Programatorul trebuie sa aleaga
  - Asezarea row-major sau column-major
  - Ordinea in care face initializarea vectorilor
- In Java, toti vectorii sunt alocati dimanic
- Vectorii de dimensiuni mari
  - Dinamici – de dimensiuni ND sunt vectori (N-1)D de vectori 1D
  - Statici – o generalizare a solutiei row/column-major

Alte Exemple: Liste Inlantuite

- Sa luam in considerare exemplul unei liste inlantuite



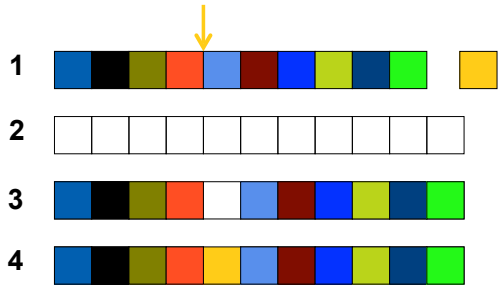
- Intr-o implementare tipica, fiecare element al listei va fi alocat dinamic la momentul inserarii
- Elementele listei nu vor fi contigue
- Parcurgerea listei in ordine va genera in “general” cate un cache-miss pentru fiecare element!
- Acest lucru poate pune probleme majore de performanta daca lista este suficient de lunga si ea este parcursa des...

Liste Implementate ca Vectori

- Cum putem avea o localitate mai buna a datelor in liste?
- Le implementam ca vectori
- Tipul de date lista, poate fi implementat sub forma unui vector unidimensional
- Sunt trei operatii fundamentale cu liste:
  - Insert (list, current, next)
  - Remove (list, current)
  - Next (list, current)

Liste - Inserarea

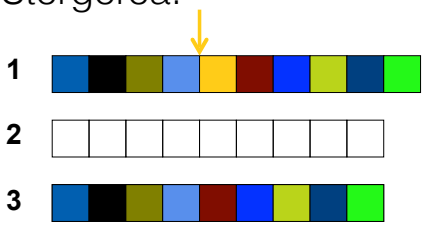
- Vom construi o lista de intregi:
- Tipul de date “lista” arata:
  - int *array: un vector de intregi
  - int array_size: dimensiunea vectorului/listei
- Inserarea:



1. Avem 10 elemente si o inserare
2. Alocam un nou vector de 11 elemente
3. Copiem elementele vechiului vector
4. Copiem noul element
5. Eliberam vechiul vector
6. Facem lista sa “pointeze” catre noul vector

Liste - Stergerea & Next

- Stergerea:



1. Avem o lista de 10 elemente si o stergere
2. Alocam un nou vector de 9 elemenete
3. Copiem elementele din vechiul vector
4. Eliberam vechiul vector
5. Facem lista sa “pointeze” catre noul vector

- Next
  - E returnarea unui simplu index

Liste - Concluzii

- O optimizare simpla:
  - In C, puteti utiliza “realloc” pentru a minimiza copierea elementelor
- Trade-off
  - Inserarea poate dura considerabil mai mult decat in implementarea traditionala
    - Au loc mai multe copieri de date
  - Lucrurile stau la fel si la stergere
  - Operatia Next este insa aproape instantanee
- Utilizatorul trebuie astfel sa identifice “**the common case**” si sa selecteze implementarea corespunzatoare
- Alte optimizari posibile
  - La inserare: dublati dimensiunea vectorului daca acesta e “prea mic”
  - La stergere: permiteti utilizarea unui vector “mai mare”

Ierarhia de Memorii vs. Programatori

- Toate lucrurile prezentate aici pot fi foarte frumoase (poate)…
- Dar de ce ar fi importante pentru programatori?
- Am vazut ca putem determina ordinea de executie a buclelor
  - Poate ca acest lucru il poate face compilatorul pentru noi
- Totusi, un programator ce doreste performante de la codul sau, trebuie sa stie cum arata ierarhia de memorii pe care programeaza!
  - Daca stim **dimensiunea cache-ului L2** (256KB), poate putem descompune problema in subprobleme de aceasta dimensiune pentru a exploata localitatea datelor
  - Daca stim ca o **linie de cache** are 64 bytes, putem calcula precis numarul de cache-miss-uri cu o formula si astfel seta un parametru optim pentru programul nostru
- Pentru a avea o experienta activa cu ierarhia cache-urilor, e util sa vedem cum putem scrie programe care sa masoare in mod automat aceste caracteristici

Masurarea Numarului de Cache-Miss-uri

- Sa presupunem urmatorul fragment de cod

```
char a[N];
for (i=0;i<N;i++)
    a[i]++;
```

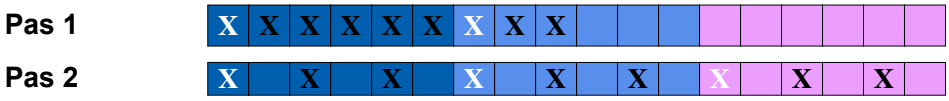
- Presupunem ca L este dimensiunea liniei de cache in bytes
- Numaram numarul de cache-miss-uri:
  - a[0]: miss (incarcam o noua linie de cache)
  - a[1]: hit (in linie de cache)
  - ...
  - a[L-1]: hit (in linie de cache)
  - a[L]: miss (incarcam o noua linie de cache)
  - ...
- Numarul de miss-uri este astfel: ~ N / L

Masurarea L-ului

- Codul anterior acceseaza elementele vectorului cu pasul 1
  - Pasul este diferenta in bytes intre doua adrese accesate in doua accesuri succesive la memorie
- Ce se intampla cand avem un pas 2?

```
char a[N];
for (i=0;i<N;i+=2)
    a[i]++;
```

- Avem un numar considerabil mai mare de miss-uri pentru acelasi nr de accese fata de cazul anterior!
  - O modificare minora (aparent) are un impact major asupra performantelor



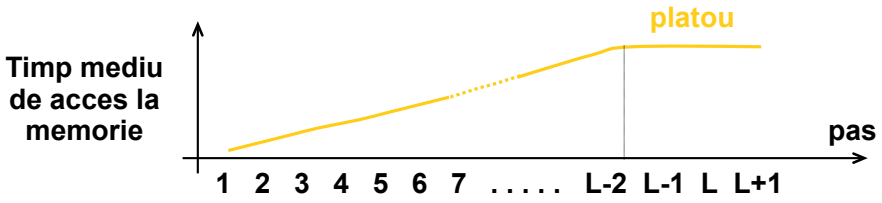
Masurarea L-ului

- Putem astfel creste pasul pana cand…
- Pasul ajunge sa fie egal cu L:
  - Fiecare acces are nevoie de o linie proprie de cache
  - N accesuri la memorie & N cache-miss-uri

- Ce se intampla daca pasul este L+1?
  - Fiecare acces are nevoie de o linie proprie de cache
  - N accesuri la memorie & N cache-miss-uri

Masurarea L-ului

- Cea mai buna performanta: pas=1
- Cea mai proasta performanta: pas ≥ L
- Daca masuram performanta codului pentru diverse valori ale pasului, obtinem un grafic de genul:



- Gaseste inceputul platoului (pasul in bytes) pentru care performanta codului nu se mai inrautateste odata cu cresterea pasului
- Acest pas (in bytes) este dimensiunea liniei de cache!

Masurarea L-ului

- Cum scriem un program pentru masurarea performantei pentru mai multe valori pentru pas?
- Performanta – timpul mediu pentru accesul la memorie
- Alocati vectori mari de caractere
- Creati bucle cu valori pentru pas intre 1 si 256
- Pentru fiecare pas ales, parcurgeti in mod repetat vectorul
  - Faceti operatii cu fiecare element al vectorului (etc)
  - Masurati timpul cat dureaza aceste operatii
  - Masurati cate operatii ati operat in total
  - Impartiti numarul de operatii la timpul masurat

Masurarea Dimensiunii Cache-ului

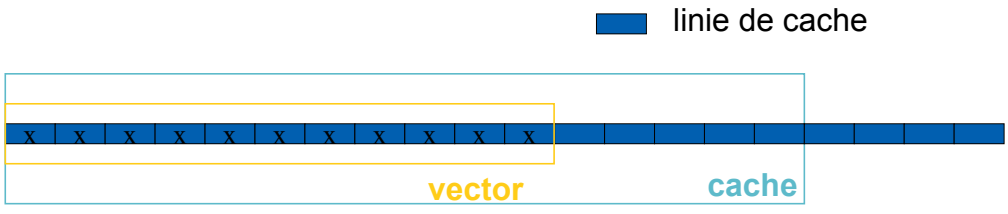
- Daca L este dimensiunea liniei de cache
- Sa consideram urmatorul cod

```
char x[1024];
for (step=0;step<1000;step++)
    for (i=0;i<1024;i+=L)
        x[i]++;
```

- Daca cache-ul este mai mare de 1024 de bytes, dupa prima iteratie a buclei “step”, tot vectorul x e in cache si nu vom mai avea miss-uri deloc
- Daca cache-ul este mai mic decat 1024 de bytes, vom avea mereu un numar de miss-uri

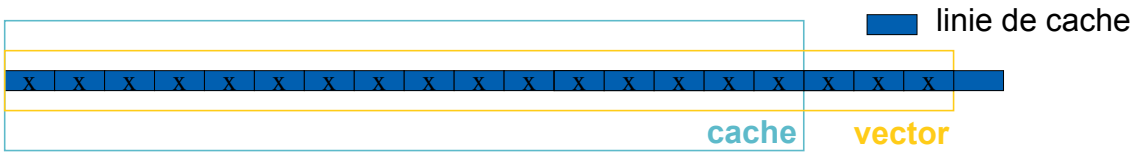
Masurarea Dimensiunii Cache-ului

- Exemplificare:



- Cache-ul este suficient de mare pentru a contine 16 linii
- Vectorul intra in 11 linii de cache
- Astfel, vor fi 11 miss-uri si apoi vor exista doar hit-uri
- Pentru un numar mare de iteratii, hit-rate-ul va fi aproape de 100%

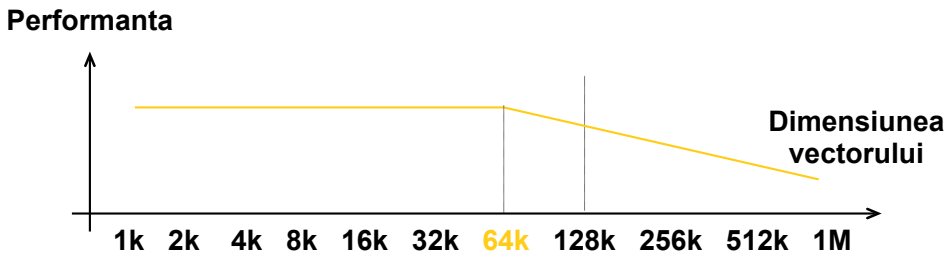
Masurarea Dimensiunii Cache-ului



- Cache-ul poate contine 16 linii
- Vectorul intra insa doar in 19 linii de cache
- Initial vor fi 19 miss-uri
- Apoi va trebui sa citim aceleasi 19 linii de cache
- Doar 16 intra insa in cache
- Vom avea astfel 13 hit-uri si 3 miss-uri
- Acest comportament va fi repetat pentru fiecare iteratie
- Pentru un numar mare de iteratii, se ajunge la un hit-rate de aproximativ 81%

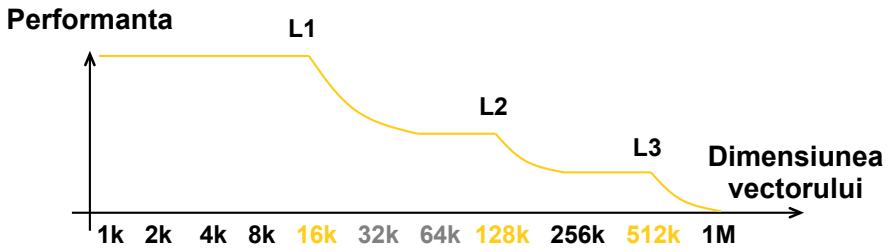
Masurarea Dimensiunii Cache-ului

- Astfel:
  - Daca vectorii sunt mici si intra in cache hit-rate = 100%
  - Daca vectorii sunt mai mari decat cache-ul hit-rate < 100%

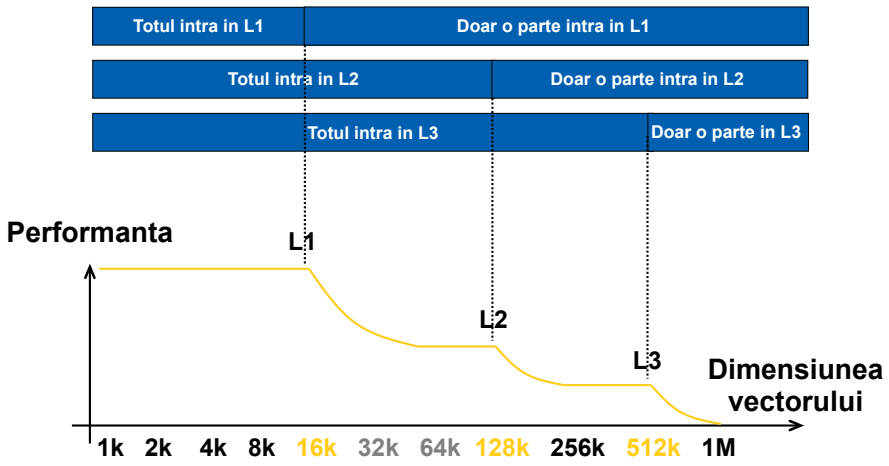


Mai multe Nivele de Cache?

- In general insa, exista mai multe nivele de cache: L1, L2, L3
- Avand in vedere acest fapt, graficul anterior devine



Mai multe Nivele de Cache?



Cache-uri in Procesoare Reale

- Exista 2 sau 3 nivele de cache
- Cache-urile aproape de procesor sunt in general mapate direct, si cele mai departate sunt asociative
- Cache-uri diferite de date/instructiuni aproape de procesor, si unificate in rest
- Write-through si write-back sunt la fel de des intalnite, dar nu va exista nicidodata o implementare write-through pana la memoria principala
- Liniile de cache aveau in mod normal 32-byte, dar acum exista foarte des linii de 64- si 128-bytes
- Cache-uri neblocante
  - La un miss, nu bloca procesorul ci executa instructiuni de dupa load/store-ul curent
  - Blocheaza procesorul doar daca aceste operatii utilizeaza date din load
  - Cache-urile trebuie sa poata sa serveasca multiple accese “invechite”

Performanta Cache-urilor

- Performanta cache-ului este data de accesul mediu la memorie
  - Programatorii sunt interesati in general doar de timpul de excutie – insa timpul de acces la memorie este o componenta extrem de importanta
- Formula e simpla:
  - Timpul de acces la memorie = hit time + miss rate * miss penalty
  - Miss-rate este procentul de miss-uri pe acces la date si NU la instructiuni!
- La fel ca in cazul timpului de executie procesor, termenii ecuatiei NU sunt independenti
  - Nu putem spune: reduc numarul de instructiuni, fara a creste numarul de instructiuni pe ciclu sau viteza ceasului sistemului
  - Similar, nu putem spune: reduc miss-rate-ul fara a creste hit-time

Impactul Miss-urilor

- Miss penalty depinde de tehnologia de implementare a memoriei
- Procesorul masoara numarul de cicluri pierdute
- Un procesor mai rapid e “lovit” mai tare de memorii lente si miss-rate-uri crescute
- Astfel, cand incercati sa estimati performantele unui sistem de calcul, comportamentul cache-urile trebuie luat in considerare (Amdahl: CPU vs. Memorie)
- De ce ne intereseaza?
  - **Pentru ca putem rescrie/rearanja codul pentru a exploata mai bine localitatea datelor**

De ce ne Intereseaza?

- Putem citi dimensiunile cache-urilor din specificatiile masini pe care rulam
  - Dar… ceva poate lipsi
  - Se poate sa nu avem acces la specificatii
- E util sa avem o optimizare automatizata
  - Ce trebuie sa fac pentru a scrie un program ce ia in considerare Cache-ul?
    - Utilizeaza constanta CACHE_SIZE pentru a seta dimensiunea cache-ului
    - Utilizeaza apoi CACHE_SIZE cand aloci memoria

```
char x[CACHE_SIZE]
for (i=0;i < CACHE_SIZE; i++){ … }
```

  - Inainte de a rula un program, ruleaza un utilitar ce afla dimensiunea activa cache-ului
  - Seteaza apoi constanta CACHE_SIZE la valoarea astfel determinata, e.g.:

```
#define CACHE_SIZE 1024*32
```

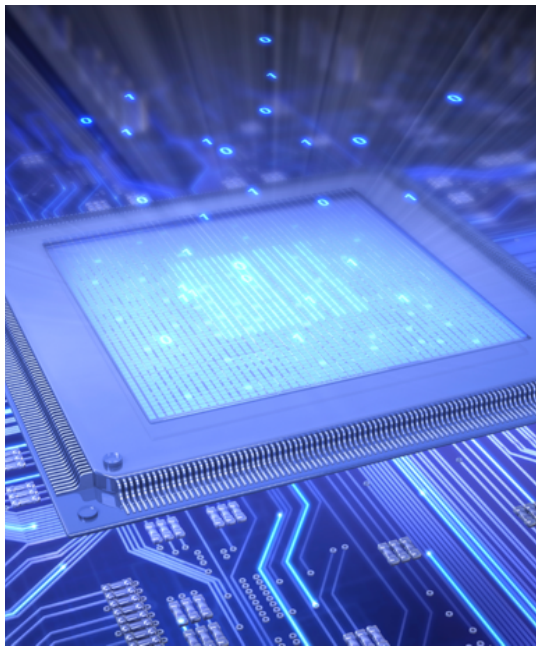
- Un programator (**bun**) nu poate ignora cache-ul
- Desi unele structuri de date par naturale, ele pot fi ineficiente in cache (liste, pointeri, etc.) si de aceea ele ar trebui evitate/adaptate cand se doreste o performanta crescuta a codului

Ce am invatat in aces curs?

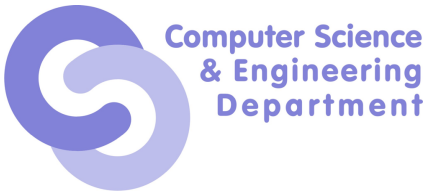
- Ce este ierarhia de memorii?
- Care sunt tipurile de localitate a datelor
- Design si implementare a Cache-urilor
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie

Slides & Support Material Acknowledgements

Henry Casanova - University of Hawaii 2008
Optimizing HPC Applications 2014
Victor Ciurel 2016



Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 4
cs.pub.ro
cs.curs.pub.ro

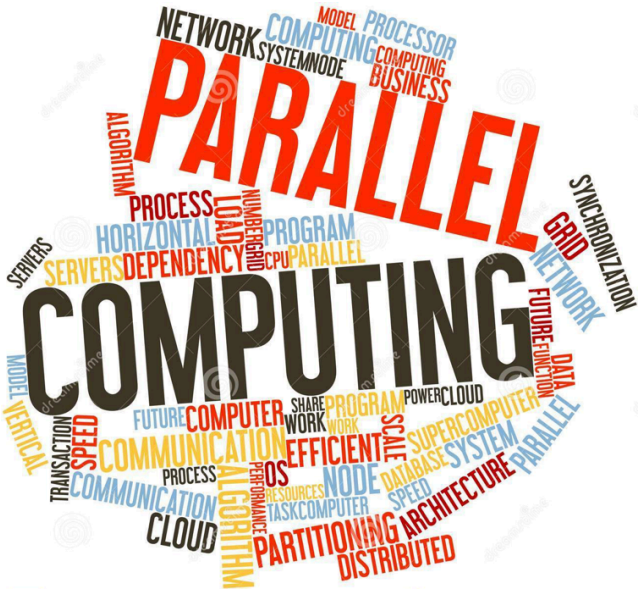
University Politehnica of Bucharest

In cursul anterior

- Ierarhia de memorii – Bottleneck-ul SC
- Localitatea datelor
- Cache – design, implementari si exemple
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie

Cuprins

- De ce avem nevoie de paralelism?
- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD

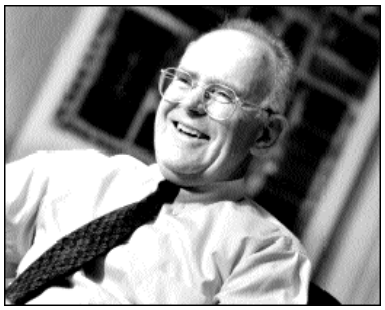


De ce avem nevoie de paralelism?

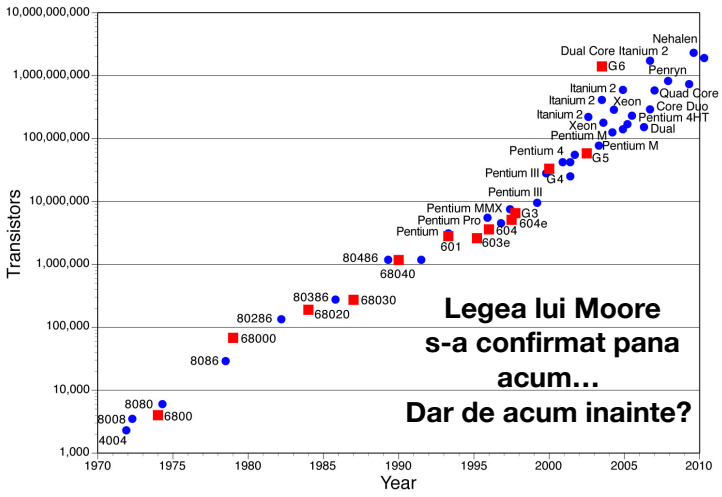
Words of Wisdom

- *"I think there is a world market for maybe five computers."*
  - Thomas Watson, chairman of IBM, 1943.
- *"There is no reason for any individual to have a computer in their home"*
  - Ken Olson, President and founder of Digital Equipment Corporation, 1977.
- *"640KB [of main memory] ought to be enough for anybody."*
  - Bill Gates, Chairman of Microsoft / IBM System Limitation,1981.

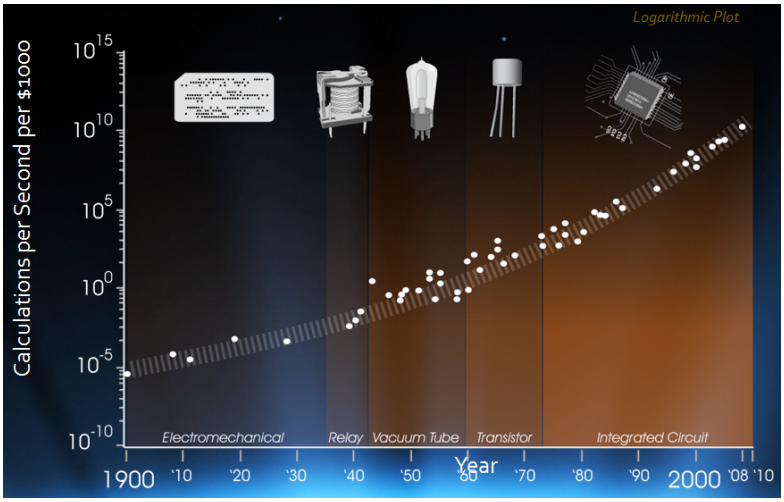
Evolutia Microprocesoarelor

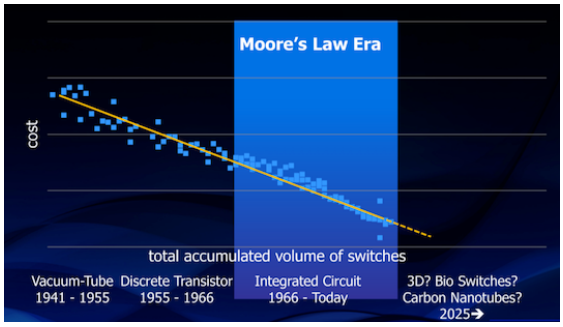


Gordon Moore (cofondator Intel) a prezis in 1965 ca densitatea tranzistoarelor in cipurile cu semiconductori se va **dubla** intr-un interval aproximativ de **18 luni**



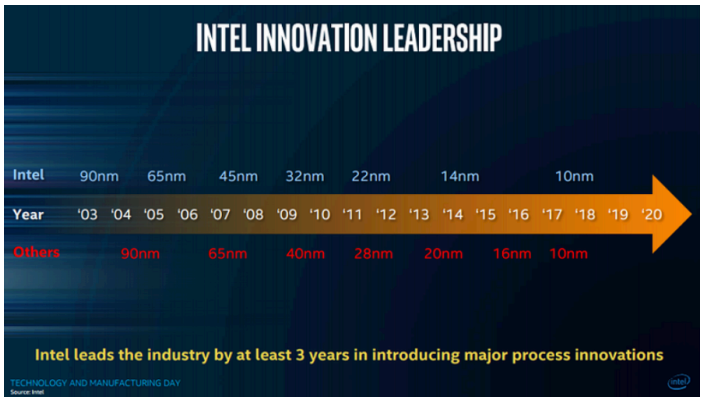
Putere de Calcul vs. Cost



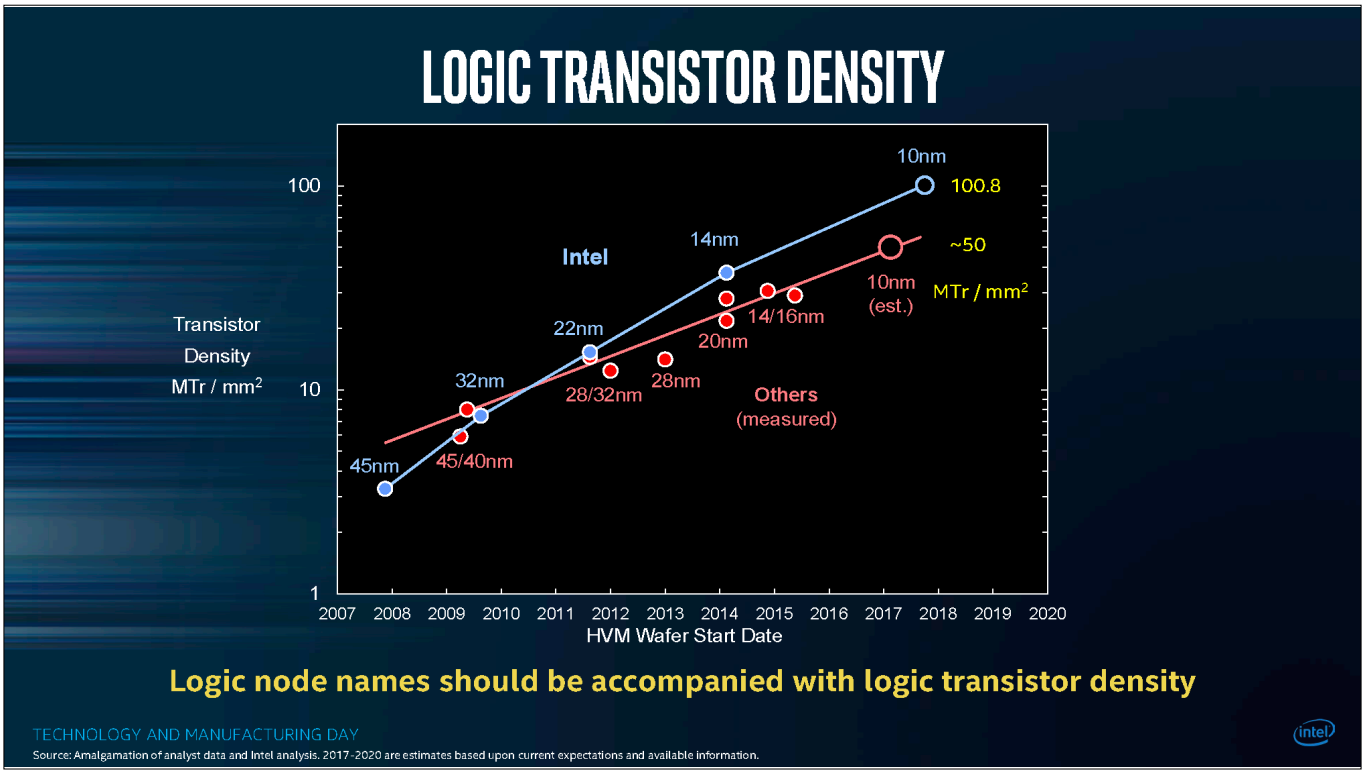


Va putea tehnologia sa continue sa ofere putere de calcul in crestere, la preturi din ce in ce mai mici?

CPU Roadmap vs. Moore's Law

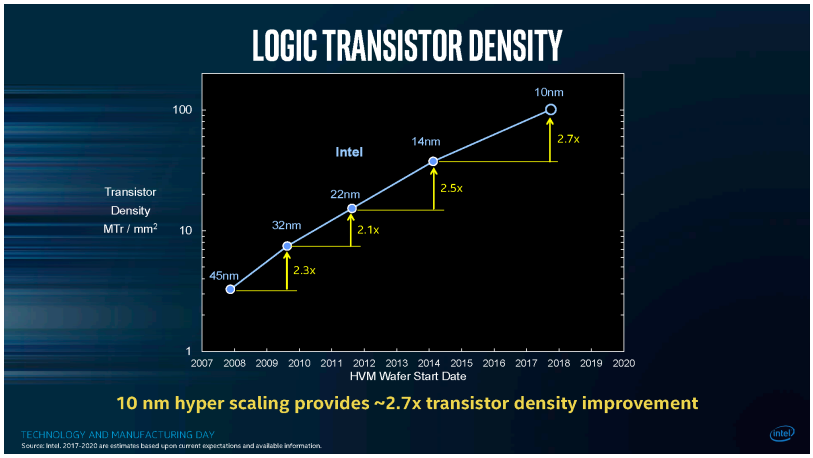


- Up until 1965: 2x / year
- 1965 - 201x: 2x / 2 years
- 201x - 202x: 2x / 3-4 years
- The 2x intervals keep getting longer…

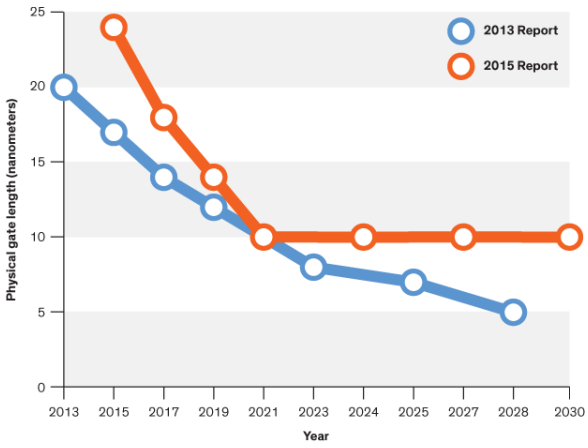


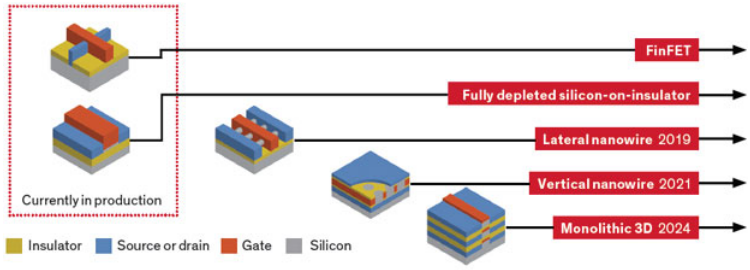
Better Density Improvements than 2x

- Offers possible variants:
  - 14+: better performance
  - 14++: better power usage
- 5nm in production
- 3nm in development
- 1-0.x nm is possible
- Measuring transistor size is not “universal”
  - Can IEEE help?
- What lies after 5nm?

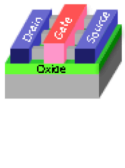
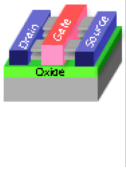
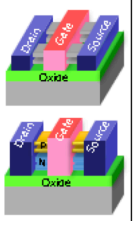
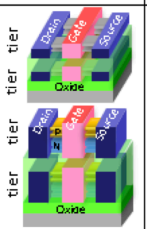
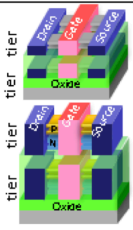
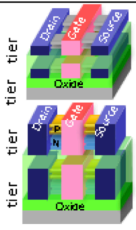


ITRS/IRDS Roadmap 201x

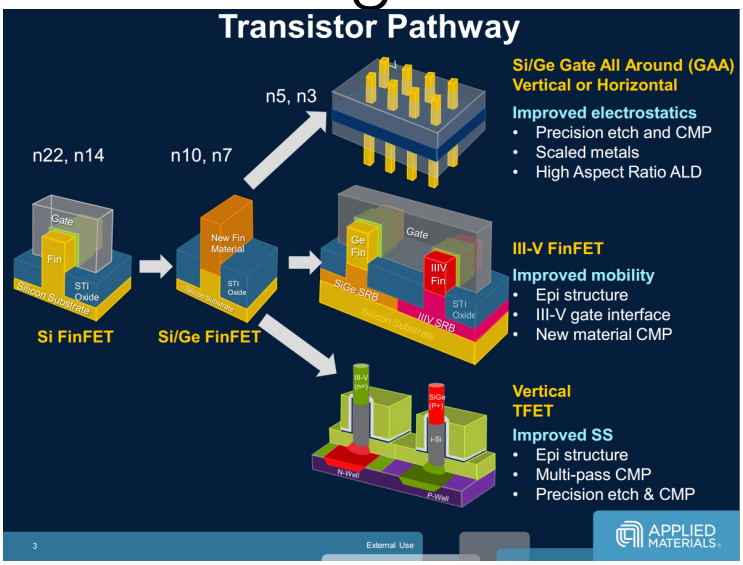




ITRS/IRDS Roadmap 202x - 203x

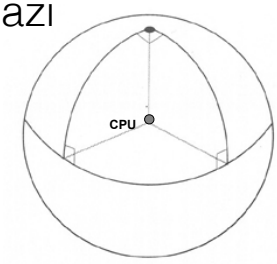
YEAR OF PRODUCTION	2022	2025	2028	2031	2034	2037
	G48M24	G45M20	G42M16	G40M16/T2	G38M16/T4	G38M16/T6
Logic industry "Node Range" Labeling	"3nm"	"2nm"	"1.5nm"	"1.0nm eq"	"0.7nm eq"	"0.5nm eq"
Fine-pitch 3D integration scheme	Stacking	Stacking	Stacking	3DVLSI	3DVLSI	3DVLSI
Logic device structure options	finFET LGAA	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM
Platform device for logic	finFET	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D
						

Posibile Tehnologii la nivel de nm



Un Procesor de 10 TFlops?

- Putem construi un CPU neconcurent (serial) care
  - Ofera 10,000 de miliarde de operatii in virgula mobila pe secunda (10 TFlops)?
  - Opereaza pe 10,000 miliarde de bytes (10 TByte)?
- Este reprezentativ pentru necesitatile din ziua de azi
- Ceasul procesorului trebuie sa fie de 10,000 GHz
- Presupunem ca datele circula cu viteza luminii
- Presupunem ca procesorul este o sfera “ideala”

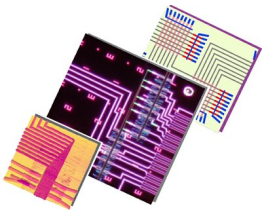


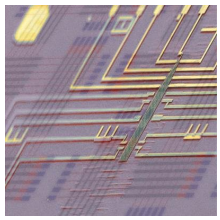
Un Procesor de 10 TFlops?

- Masina genereaza o instructiune pe ciclu, si de aceeas ceasul trebuie sa fie de aproximativ 10,000 GHz ~ 10^{13} Hz
- Datele au de parcurs o distanta intre memorie si CPU
- Fiecare instructiune necesita cel putin 8 bytes de memorie
- Presupunem ca datele circula cu viteza luminii c = $3e^{8}$ m/s
- Astfel, distanta intre memorie si CPU trebuie sa fie r < c / 10^{13} ~ $3e^{-6}$ m
- Apoi, trebuie sa avem 10^{13} bytes de memorie intr-un volum de $4/3\pi r^3 = 3.7e^{-17}$ m^3
- Astfel, fiecare cuvant de memorie trebuie sa ocupe maxim $3.7e^{-30}m^3$ = 3.7 $Angstrom^3$
  - O molecula mica, formata din cativa atomi…
- Densitatea curenta a memoriei este 10GB/cm^3: factor de ordinul 10^{20} sub necesitati!
- Concluzie: Acest procesor nu se poate realiza cu tehnologia actuala.

Nanoprocesoare

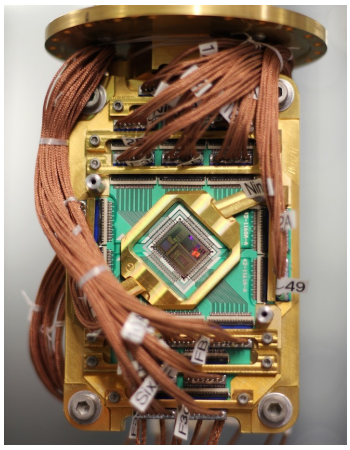
- Harvard University & MITRE Corporation au dezvoltat si demonstrat primul nanoprocesor programabil din lume
  - Circuite la scara nano aranjate ca niste sabloane de nanoprocesoare
  - Firele de interconectare sunt din germaniu-silicon
  - Diametrul total este de doar 30 de nanometri





Quantum Computing

- Quantum bits = Qubits: 0, 1, & 0/1 combinations
  - Superposition - multiple states at the same time
  - Entanglement - particle correlation or “spooky action at a distance”
  - Observation disturbs the system
- Typical examples of applications
  - Factorization: cryptography
  - Simulate quantum systems: new materials & drugs
  - Simultaneous exploration of solution space: optimisation
- Currently machines with just 100s of Qubits:
  - DWave, Google, Microsoft, IBM, NASA, NSA, …

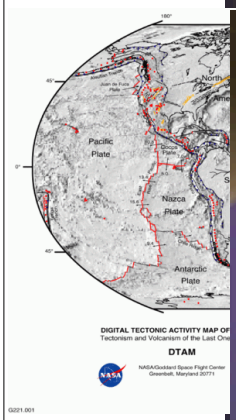


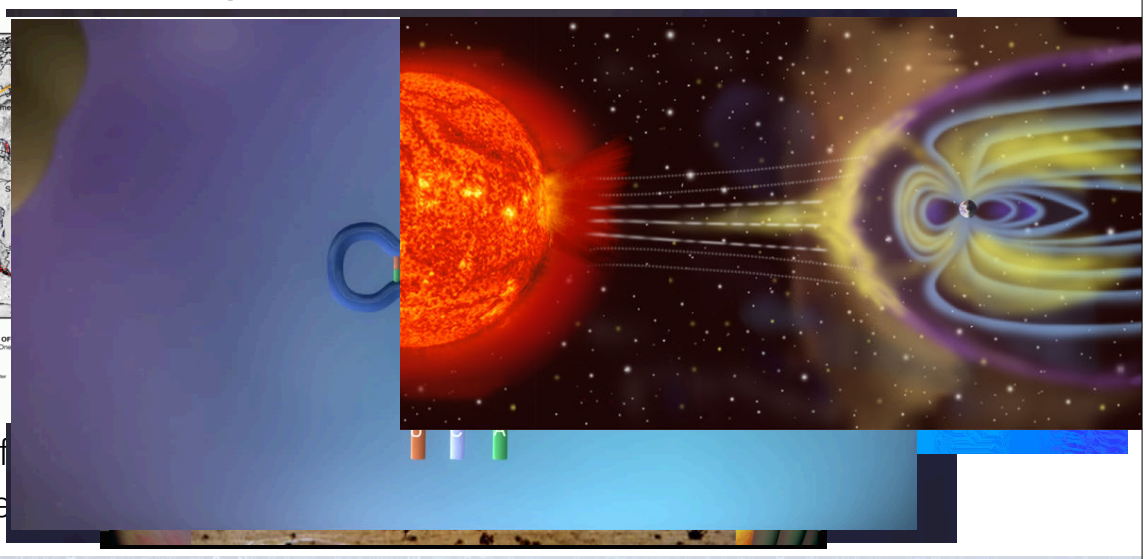
Structuri de Calcul cu Prelucrare Paralela

Paralelismul este Necesar!

- Paralelism la nivel de Bit (Bit-level parallelism)
  - Operatii in virgula mobila
- Paralelism la nivel Instructiune (ILP)
  - Mai multe instructiuni pe ciclu
- Paralelism la nivelul memoriei
  - Suprapunerea intre operatii cu memoria si de calcul
- Paralelism al sistemului de operare
  - Mai multe thread-uri, procese pe CPU-uri multiple, in cadrul aceleasi masini
- Paralelism distribuit
  - Mai multe masini conectate impreuna

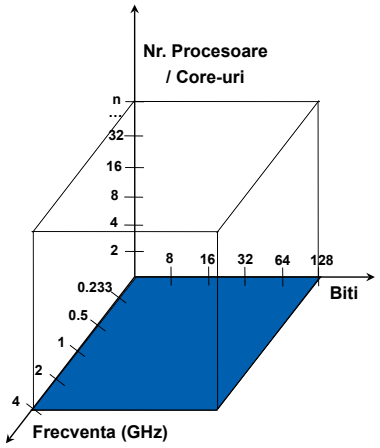
Aplicatii Paralele





- Geof
- Mete

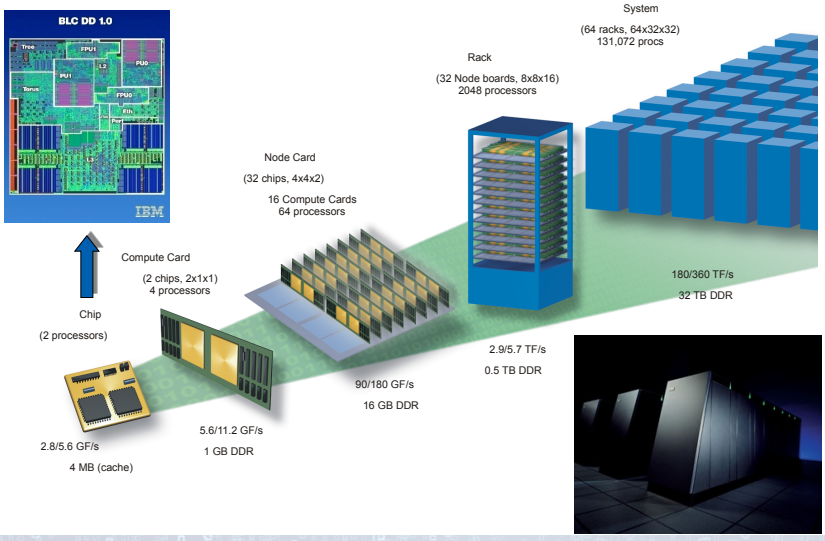
Structuri de Calcul cu Prelucrare Paralela



- Aria → puterea de calcul
- Marimea puterii de calcul: volumul cubului - “n” e limitat “doar” de cost → Arhitecturi paralele de calcul
- Orice problema are un grad de paralelism intrinsec ce depinde de natura ei
- Trebuie tinut seama de:
  - Algoritmi paraleli specifici
  - Limbaje adecvate de programare
  - Limitarile SO
  - Arhitectura intrinseca a SC utilizat

Gradul general de paralelism al codurilor este ~ 10%

IBM BlueGene/L - 131,072 Processors



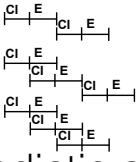
Nivelul Prelucrarii Paralele - CA 23.03

1. Paralelismul la nivel de Bloc/Job:
   - Intre Job-uri diverse
     - Necesare: mecanism de salvare a contextului; un ceas de timp; canale I/O
   - Intre fazele unui Job
     - Citirea sursei programului
     - Compilare
     - Link-are
     - Executarea codului obiect
     - Salvarea rezultatelor
   - Anumite faze ale unui job pot fi suprapuse
2. Paralelismul la nivel de subansamble Hardware:
   - Intre elemente de prelucrare ale vectorilor
   - Intre componentele logice ale dispozitivelor aritmetico/logice (e.g. carry look ahead)

Nivelul Prelucrarii Paralele

3. Paralelismul la nivel de Program:
   - Intre diverse sectiuni (independente) ale unui program
   - Intre buclele (loop-urile) unui program
   - Ambele presupun analiza dependentelor de date!
4. Paralelismul la nivel de Instructiune:
   - Intre diverse faze ale ciclului instructiune:
     - Implementare seriala:
     - Implementare serie-paralela:
     - Implementare paralela:
   - Este necesar un mecanism de predictie al salturilor

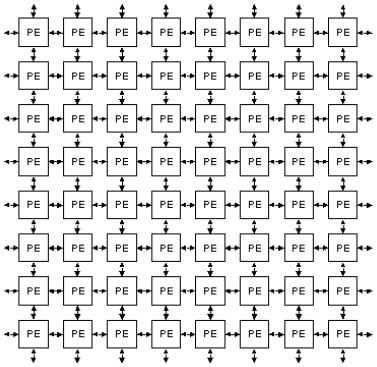
CI = Citire & Interpretare
E = Executie



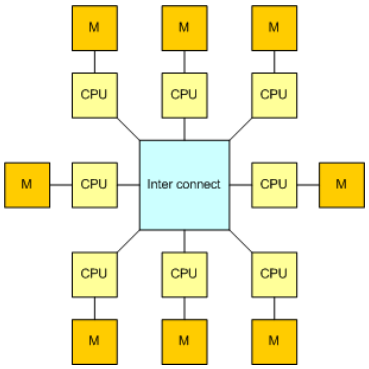
Clasificarea Sistemelor de Calcul

Sisteme Matriceale - Processor Array

- Unitatea de baza a informatiei este vectorul
- Dispun de instructiuni similare SC Von Neumann – operatiile asupra vectorilor sunt efectuate in aceeasi instructiune

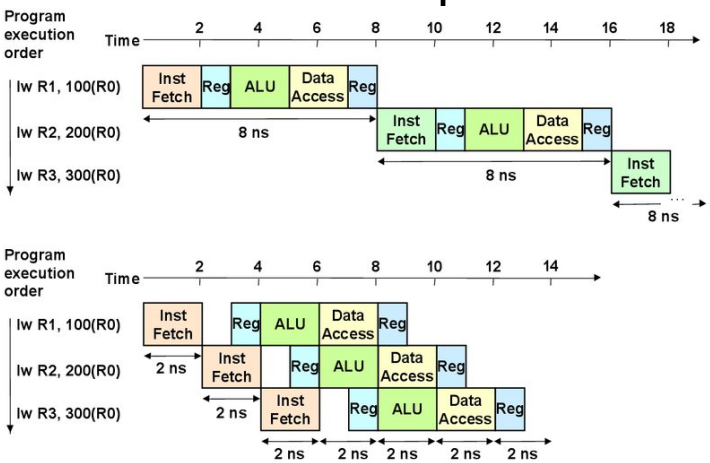


Sisteme Multiprocesor



- Formate din N unitati de prelucrare interconectate printr-o retea de comutare (Strans/Slab cuplata)
- Sistemele lucreaza independent la realizarea aceluiasi Job

Sisteme Pipeline



- Dispun de mai multe unitati de prelucrare asezate in banda de asamblare (RISC):
  - Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

Taxonomia lui Flynn

- Impartirea sistemelor de calcul in functie de:
  - **Fluxul de Instructiuni** – secventa de instructiuni executate de procesor
  - **Fluxul de Date** – secventa de operanzi manipulata de procesor
- Bazate pe acest criteriu se desprind:

I. SISD = Single Instruction Single Data Stream (structura Von Neumann)

II. SIMD = Single Instruction Multiple Data Stream

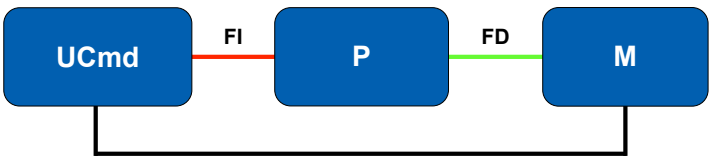
III.MISD = Multiple Instruction Single Data Stream

IV.MIMD = Multiple Instruction Multiple Data Stream

I - SISD

- UCmd = Unitate de comanda
- P = Unitate de prelucrare aritmetica
- M = Memorie
- FI = Flux Instructiuni
- FD = Flux Date

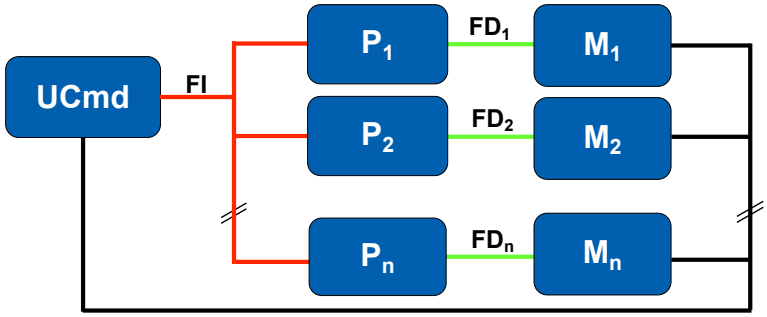
SISD = 1 FI & 1 FD



II - SIMD

- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe Procesoare si Memorii - orice Procesor vede orice Memorie
- Toate procesoarele fac aceeasi operatie impusa de UCmd prin FI

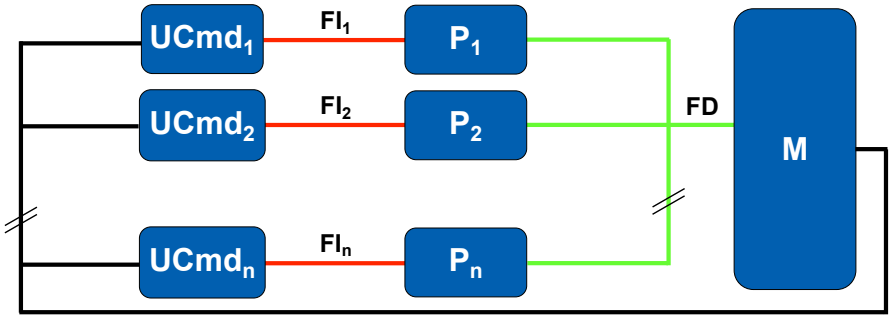
SIMD = 1 FI & n FD



III - MISD

- MISD au:
  - Mai multe UCmd si Procesoare
  - Un singur modul de Memorie
- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)

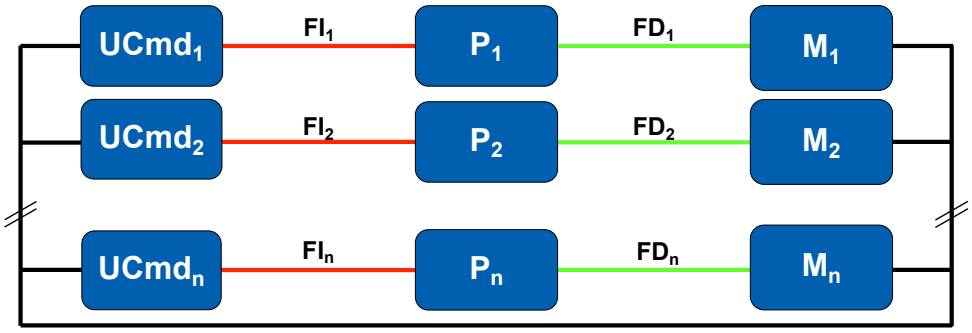
MISD = n FI & 1 FD



IV - MIMD

- MIMD pot comunica: (P-P sau P-M)
- Toate Procesoarele participa la acelasi program
- Mod programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)

MIMD = n FI & n FD



Exemple de Utilizare a Arhitecturilor

SISD, SIMD, MIMD

Exemplu de Utilizare – SISD

- Problema: A[n, n], B[n, n], C = A x B
- Pe structuri de calcul SISD avem 3 for-uri:

```
for i = 0 to n – 1
  for k = 0 to n – 1
    cik = 0
    for j = 0 to n – 1
      cik = cik + aij * bjk
    end j loop
  end k loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^3)$! → nesatisfacatoare! (mai ales cand n e mare…)

Exemplu de Utilizare – SIMD

- Aceeasi Problema: A[n, n], B[n, n], C = A x B
- Avem n procesoare & toate executa aceeasi instructiune odata → in fiecare calcul se calculeaza **cate o linie** si **nu doar un element**
- Considerand ($0 \le k \le n - 1$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii:

```
for i = 0 to n – 1
  cik = 0 (0 ≤ k ≤ n – 1)
  for j = 0 to n – 1
    cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
  end j loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^2)$! → considerabil mai bine ca in cazul SISD

Comentarii si Observatii – SIMD

- Fiecare element al matricei produs C, este o suma ce se efectueaza secvential
- Cele n sume se calculeaza apoi in paralel **!?**
- a_{ij} NU depinde de k:
  - Accesul la aceasta memorie se face **aproximativ secvential**
  - Asadar **nu** e chiar "**sumele se calculeaza in paralel**"!
- Solutia: structurile SIMD **trebuie** sa dispuna de o instructiune de Broadcast si o retea de comutare (RC) pentru acest Broadcast
- P_j citeste a_{ij} prin RC
  - Constanta a_{ij} e difuzata catre toate procesoarele
- Concluzie SIMD: sunt **probleme semnificative** la **comunicatiile** inter-procesoare, la **accesul si organizarea datelor**!

Exemplu de Utilizare – MIMD

- Aceeasi Problema: A[n, n], B[n, n], C = A x B
- O UCmd/P trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale
- Conway a propus o metoda cu 2 primitive: FORK & JOIN
  - FORK: desface un FI in n FI executabile simultan pe procesoare indep
  - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat

```
for k = 0 to n – 2                    // nu si pentru el insusi
   FORK Adr
end k loop
Adr:
   for i = 0 to n – 1
      cik = 0 (0 ≤ k ≤ n – 1) // pe exact k coloane
      for j = 0 to n – 1
         cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
      end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!
→ la fel ca SIMD

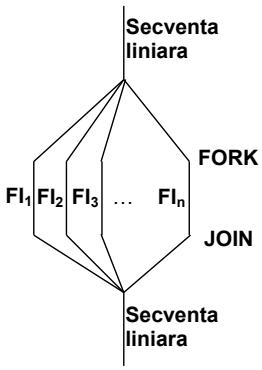
Comentarii si Observatii – MIMD

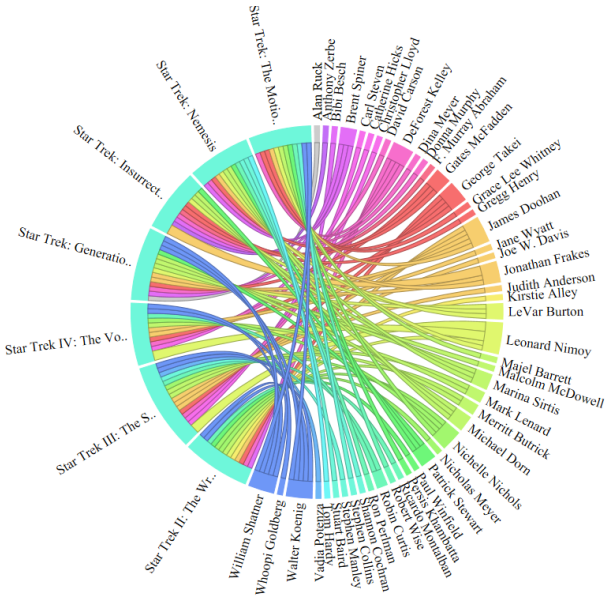
- In mod deliberat programul pt SIMD a fost scris a.i. actiunile P-urilor sa simuleze actiunile din structura MIMD
- Fiecare P_j calculeaza un C_{ik} in paralel
- Diferente SIMD/MIMD:
  - In SIMD procesoarele se sincronizau la nivel de instructiune
  - In MIMD nu exista (neaparat) sincronizari intre FI ale procesoarelor
  - La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
  - La MIMD ∀ procesor poate calcula ∀ element din C (FI-uri diferite)
- Castigul e acelasi: se pot folosi mai multe perechi FORK/JOIN
- Concluzie MIMD:
  - Mai usor de programat & utilizat decat SIMD, dar mai scump!

Totul se plateste…

Implementarea FORK

- Instructiunea JOIN n:
  - Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput
  - Pentru fiecare FI contorul este incrementat si comparat cu n
  - Trebuie ca toate procesele sa se fi terminat pana la n si abia apoi se continua cu urmatoarea secventa liniara pe P1 (master)





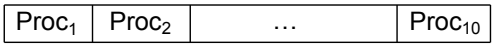
Exemplu cu/fara Dependenta de Date

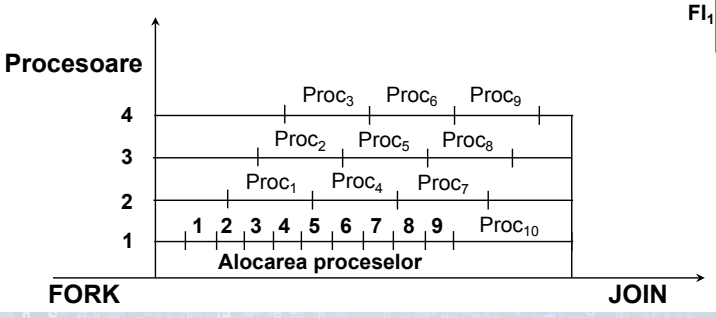
Exemplu fara Dependenta de Date

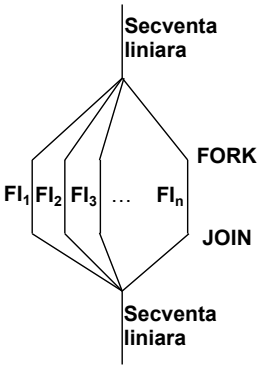
- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Modelul de executie este:

```
for i = 0 to 9
   FORK Adr
end i loop
Adr:
   Proci
   JOIN
```

Coada de asteptare a proceselor:

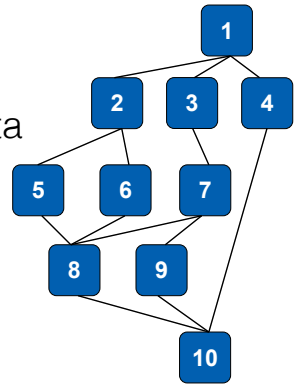


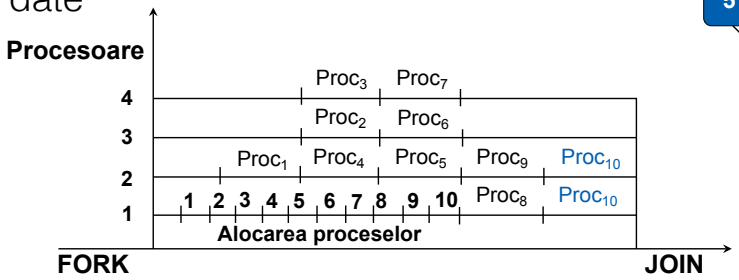




Exemplu cu Dependenta de Date

- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Aceeasi coada de asteptare
- Presupunem urmatoarea dependenta de date:
- Lucrurile stau altfel: exista procesoare ce asteapta datorita dependentei de date

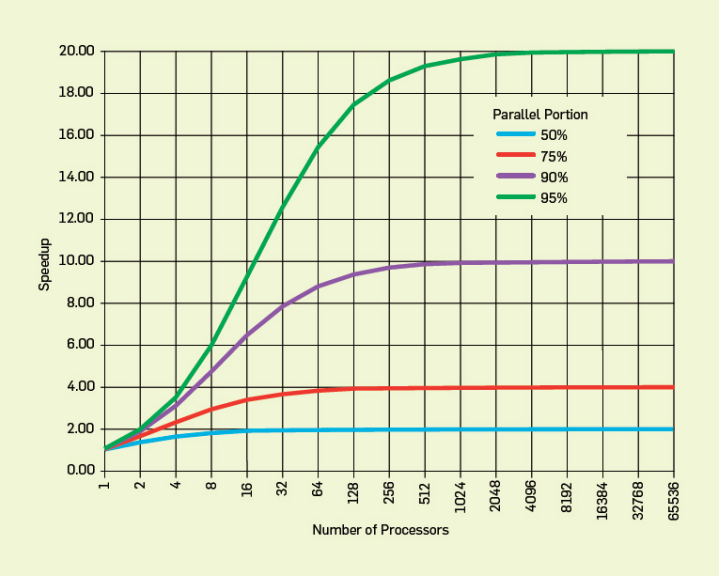




Concluzii Dependenta de Date

- In exemplele prezentate nu apar restrictii de precedenta
- Planificarea FIFO nu e necesar cea mai potrivita
  - Alocare statica
- Alocarea dinamica e mai buna
  - Mai ales cand ∃ procesoare specializate
- Programe de uz general → 10% paralelism
- Programe dedicate → 99+% paralelism
- Dependenta de date → sincronizari in timp
- Eficienta MIMD depinde de:
  - Eficienta algoritmilor de planificare
  - Consumul de timp impus de precedenta de date
- Problema repartizarii pe procesoare → NP completa!

Legea lui Amdahl



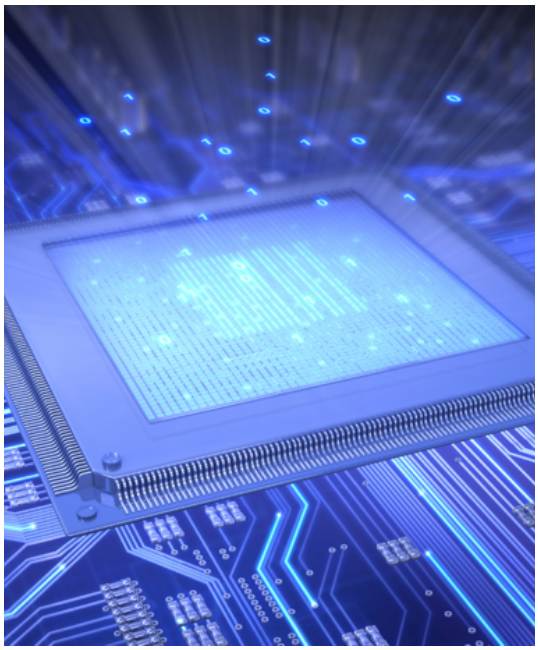
"Keep Them Honest"

Ce am invatat in aces curs?

- De ce avem nevoie de paralelism?
- Care sunt nivelele de prelucrare paralela?
- Clasificarea sistemelor de calcul
- Exemple de utilizare a structurilo: SISS, SIMD, MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD
- Limitele scalabilitatii sistemelor paralele de calcul

Slides & Support Material Acknowledgements

Charles M. Lieber 2011
BCA Research 2013
Coding for Speed 2015
IEEE 2016
Applied Materials 2017
Wired & University of Waterloo 2017
Michale Feldman & Intel Roadmap 2017



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 5
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- De ce avem nevoie de paralelism?
- Care sunt nivelele de prelucrare paralela?
- Clasificarea sistemelor de calcul
- Exemple de utilizare a structurilo: SISD, SIMD, MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD
- Limitele scalabilitatii sistemelor paralele de calcul

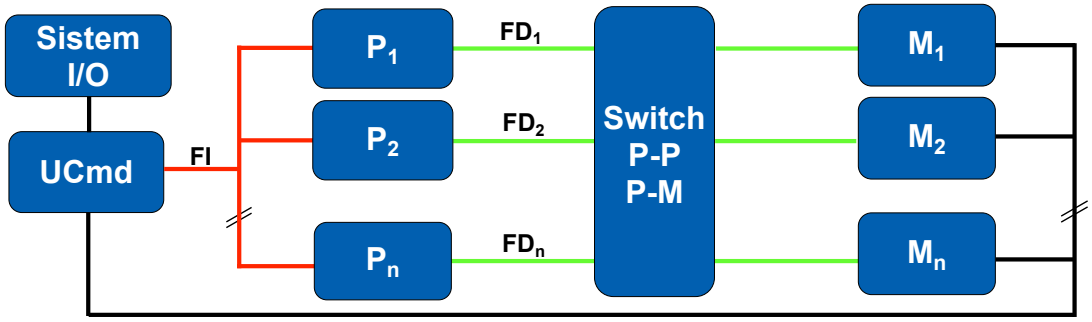
Cuprins

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Structura si Setul de Instructiuni SIMD

Structura SIMD

- Arhitecturile SIMD sunt adecvate prelucrarilor vectoriale:
  - O singura UCmd
  - Mai multe procesoare aritmetice si module de memorie
  - Retea de comutatie P-P & P-M
- Structuri adecvate prelucrarii vectoriale → esentiala organizarea algoritmului de calcul
- Pentru aplicatii neorientate pe aplicatii vectoriale → ineficiente



Componentele Structurii SIMD

- **UCmd**: Citeste, interpreteaza si executa instructiuni de control
  - Necesita registre generale, unitate de prelucrare si memorie locala
  - Executa si instructiunile de ramificatie (nu ca P_i)
- **P_i**: Sunt sincronizate la nivel de instructiune → fiecare procesor lucreaza pe un alt set de date
  - Trebuie sa aiba registre locale
- **M**: Formata din n module
  - Contine instructiuni (de comanda & prelucrare) si date

Componentele Structurii SIMD

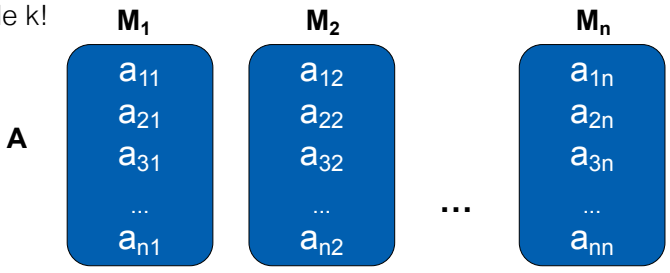
- **Instructiuni**:
  - De control → executate pe UCmd
  - De prelucrare vectoriala → executate de P_i
  - Fluxul de instructiuni e similar cu cel din sisteme secventiale
- **Registre index**:
  - Globale → in UCmd; afecteaza cu aceeasi cantitate adresa operanzilor
    - Accesul pe linii
  - Locale → in fiecare Pi; pot fi independente si se refera la zona de memorie Mi asociata fiecarui procesor
    - Acces pe coloane
    - Altfel se poate ajunge la secventialitatea accesului la date!

Exemplu – Inmultirea de Matrice

- Problema: A[n, n], B[n, n], C = A x B
- Considerand ($0 \le k \le n - 1$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii:

```
for j = 0 to n – 1
    cik = cik + aij * bjk  (0 ≤ k ≤ n – 1)
end j loop
```

- Elementele unei linii sunt in module diferite de memorie!
- a_{ij} nu depinde de k!



- Matricele B & C sunt aranjate similar
- Linia rezultat: $c_{i1}, c_{i2} \ldots c_{in}$ este calculata in paralel pe $P_1, P_2 \ldots P_n$

Setul de Instructiuni Masina SIMD

- P_i are un registru acumulator ACC_i → info locala; UCmd are un registru index INDEX[i]
- Instructiuni masina cu vectori: ($0 \leq k \leq n - 1$)
  - Incarcare:
    - vector: LOAD A ;ACC[k] ← A[0,k] (pentru vectori e suficient A[k])
    - vector indexat: LOAD A[i] ;ACC[k] ← A[INDEX[i],k]
  - Memorare:
    - vector: STO A ;A[0,k] ← ACC[k]
    - vector indexat STO A[i] ;A[INDEX[i],k] ← ACC[k]
  - Adunare vector: ADD A[i] ;ACC[k] ← ACC[k] + A[INDEX[i],k]
  - Inmultire vector: MUL A[i] ;ACC[k] ← ACC[k] * A[INDEX[i],k]
  - Difuzie scalara: BCAST i ;ACC[k] ← ACC[INDEX[i]] (1 perioada de ceas)
- [INDEX[i], k] e organizarea pe linii; [k, INDEX[i]] e organizarea pe coloane
- Instructiunile sunt similare SISD (von Neumman) cu diferenta k-ului ce indica paralelismul

Instructiuni de Ciclare & Comparare

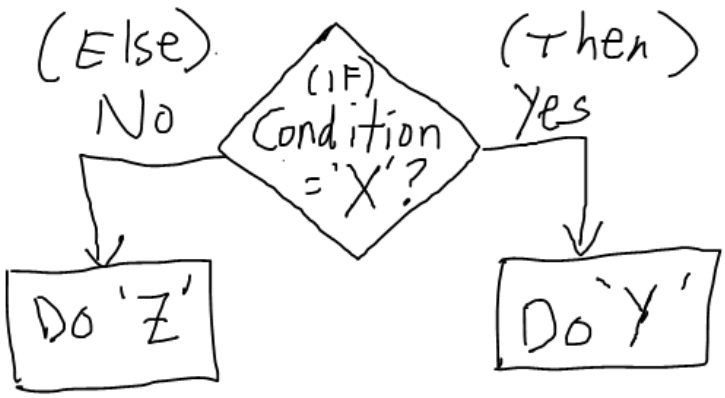
- Instructiuni referitoare le registre INDEX:
  - Incarcare:
    - ENXC i, ct ;INDEX[i] ← ct
    - LDNX i, Adr ;INDEX[i] ← MEM[Adr]
  - Incrementare:
    - INCX i, ct ;INDEX[i] ← INDEX[i] + ct
  - Compara index & salt la adresa:
    - CPNX i, j, Adr ;if INDEX[i] < INDEX [j] then Jump Adr; else continue;
- Adresele de genul A[i], B[j] sunt adrese indexate cu continutul unui registru INDEX de pe UCmd

Exemplu – Inmultirea de Matrice

```
for j = 0 to n – 1
   cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
end j loop
```

- j si n sunt incarcate in doua registre INDEX
- i provine din bucla mare a inmultirii

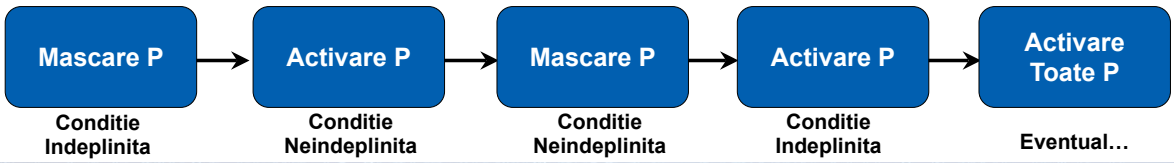
```
     ENXC j, 0        ;INDEX[j] = 0
     LDNX n, adrN     ;la adrN se afla valoarea n
adr: LOAD A[i]        ;ACC[k] ← A[INDEX[i],k]
     BCAST j          ;ACC[k] ← ACC[INDEX[j]]          (f important! aij)
     MUL B[j]         ;ACC[k] ← ACC[k] * B[INDEX[j],k]
     ADD C[i]         ;ACC[k] ← ACC[k] + C[INDEX[i],k]
     STO C[i]         ;C[i] ← ACC[k]                   (depunere rezultat)
     INCX j, 1        ;INDEX[j]++
     CPNX j, n, adr   ;salt la "adr" cand e cazul
```

Implementarea Salturilor Conditionate in Structuri SIMD

Implementarea Salturilor Conditionate

- In sistemele SIMD salturile conditionate se implementeaza prin divizarea fluxului de instructiuni (doar 1FI posibil!)
  - Un flux pentru conditia indeplinita
  - Alt flux pentru conditia neindeplinita
  - Cele doua FI se executa la momente diferite de timp si secvential! → paralelism n/2
  - Pentru implementarea acestor salturi conditionate se vor utiliza operatii de mascare a activitatii procesoarelor P_i



Mascare/Activare Procesoare

- Se considera cate un bit de mascare asociat fiecarui procesor P_i, intr-un registru de masca (n biti): $P_i \leftrightarrow$ MASK[i]:
  - MASK[i] = 1 → procesorul P_i functioneaza
  - MASK[i] = 0 → procesorul P_i in asteptare & nu executa operatia curenta
- Operatiile de testare a mastilor se fac in P-uri & rezultatele → Reg Index din UCmd → necesare instructiuni logice & de acces la registre Index:
  - Pentru stabilirea de conditii
  - Pentru implementarea salturilor
  - Lucru cu registre de masti / mascare efectiva

Instructiuni cu Registre de Masti

- Compara vector:
  - CLSS A, i ;if ACC[k] < A[0,k] then INDEX[i].bitk ← 1
    ;else INDEX[i].bitk ← 0
  - CEQL A, i ;similar doar cu conditia '=' si nu '<'
- Ramificatie fortata:
  - BRALL i, Adr ;Salt la Adr numai daca INDEX[i] = 1
  - BRNON i, Adr ;Salt la Adr numai daca INDEX[i] = 0
- Complementare index:
  - CMP i ;INDEX[i] ← NOT INDEX[i]

Instructiuni cu Registre de Masti

- Incarca/Memoreaza masca de la index:
  - LDMSK i ;MASK ← INDEX[i]
  - STMSK i ;INDEX[i] ← MASK
- Mentionarea/extragerea primului bit (prioritatea):
  - FRST i ;if INDEX[i] ≠ 0 then ramane nemodificat
  - ;else primul bit 1 ramane 1 si restul de biti 1 se trec pe 0
  - FRST se utilizeaaza cand o parte din Procs indeplinesc o conditie si trebuiesc activate
  - NXFIR i: INDEX[i] = indicele primului bit 1
  - NXFIR se utilizeaza impreuna cu BCAST

Exemplu – Normalizarea unei Matrice

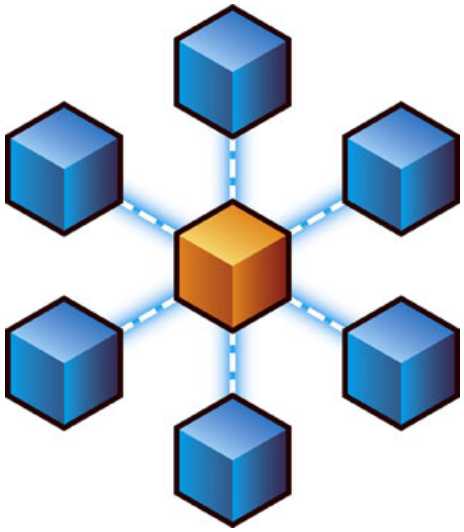
- Normalizarea prin inlocuirea A[i,j] ← A[i,j]/A[0,j] cu A[0,j] ≠ 0:

```
for i = 1 to n - 1
   if (A[0,j]!=0) then Aij /= Aoj (0 ≤ j ≤ n – 1)
end i loop
```

```
for i = 1 to n - 1
   Aij /= Aoj, M[j]
end i loop
```

- Fie M[j] = (A[0,j]!=0) in corespondenta cu mastile Pi:

```
      ENXC i, 1          ; INDEX[i] = 1 (contor)
      SUB ACC            ; Resetezi toate ACC-urile la 0 (XOR AX, AX)
      CEQL A, M          ; if ACC[k] = A[0,k] then bit k din INDEX[M] = 1        (0 ≤ k ≤ n – 1)
                         ;   else bit k din INDEX[M] = 0
      CMP M              ; INDEX[M] ← NOT INDEX[M] (setarea mastii pe conditia dorita)
      LDMSK M            ; Incarcam masca & activam procs
      LDNX n, adrN       ; La adrN e valoarea n
adr:  LOAD A[i]          ; ACC[k] ← A[INDEX[i],k]
      DIV A              ; Doar pe procs unde MASK = 1                          (0 ≤ k ≤ n – 1)
      STO A[i]           ; A[i] ← ACC[k] (depunere rezultat)
      INC i, 1           ; INDEX[i]++
      CPNX i, n, adr     ; Salt la “adr” (sfarsitul ciclului)
```

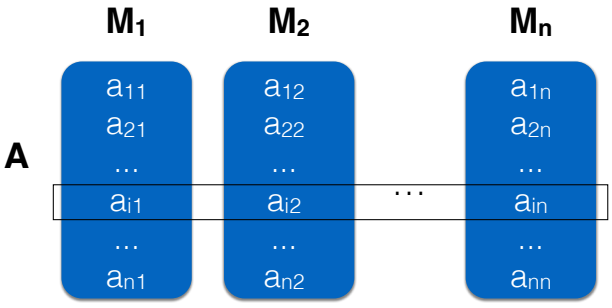
Organizarea Datelor in Structuri SIMD

Organizarea Datelor in Structura SIMD

- Probleme:
  - Repartizarea in memoriile M_i & accesul paralel
  - Citirea vectorilor **intr-o singura instructiune**
  - La matrice: acces la linii si la coloane → necesara transpunerea / deplasarea datelor?
- Memoria:
  - Poate face cel mult un acces la un ciclu de lucru
  - n operanzi in n module diferite pot fi cititi simultan
  - Defavorabila plasarea in acelasi modul al operanzilor → citirea va fi secventiala in n cicli de acces!

Reprezentarea Directa pe Linii

- Matrice A[n,n] & structura SIMD cu n procs si n memorii
- Cu acces simultan la liniile matricei:
  - Acces paralel la linii
  - Acces secvential la coloane

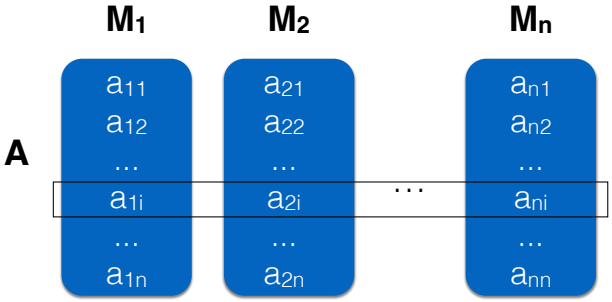


LOAD A[i] – citirea liniei i printr-un singur acces

Reprezentarea Directa pe Coloane

- Matrice A[n,n] & structura SIMD cu n procs si n memorii
- Cu acces simultan la coloanele matricei:
  - Acces secvential la linii
  - Acces paralel la coloane

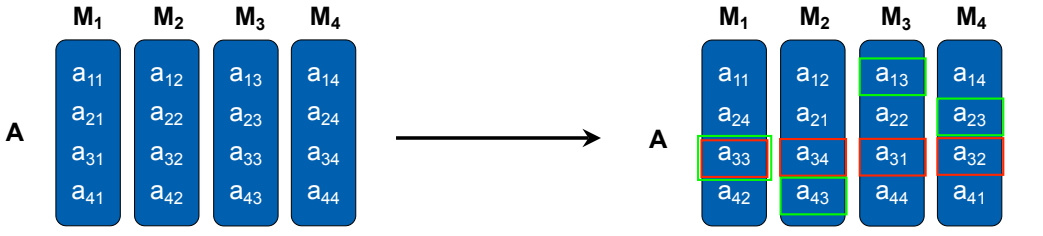
Reprezentarile directe pe linii/coloane sunt incompatibile la acelasi moment de timp



LOAD A[i] – citirea coloanei i printr-un singur acces

Reprezentarea Deplasata Ciclica - EOL CA

- Matrice A[4,4] & structura SIMD cu 4 procs si 4 memorii
- Cu acces paralel liniile si coloanele matricei:
  - Exemplu valid pentru inmultirea matricelor (linia & coloana 3)



Organizarea originala (pe linii)

Organizarea deplasata

Necesar un shuffling cu reteaua de comutatie (a_{i1} nu e pe P_1)

Concluzii Organizarea Datelor

- Trade-off simplu:
  - Programe simple → acces secvential la linii & coloane
  - Programe complicate → acces paralel la linii & coloane
- Cu organizarea deplasata accesul la linii & coloane se face intr-un singur ciclu:
  - La linii – adresa fizica/actuala a liniei
  - La coloane – sunt necesari registre index private in P_i
    - Pt acces la Col i, vectorul index e deplasat ciclic pana 0 e la M_i
    - Se difuzeaza adresa liniei 0 cu indicatia de indexare locala
    - Aducerea in pozitia corecta se face prin reteaua de comutatie P-P
- Costul platit paralelismului este **consistenta datelor**!
- Cu reteaua de comutatie (RC) cross-bar → nu conteaza asezarea datelor
  - Se seteaza cross-barul sau indexul local inainte de operatii

Exemplu – Suma pe Coloane

- Problema: A[n, n] → se doreste suma pe coloane a matricei A intr-un vector SUM[n]
- Cu organizarea pe linii – suma se face secvential, linie cu linie, fiecare P_i calculeaza suma pe cate o coloana:

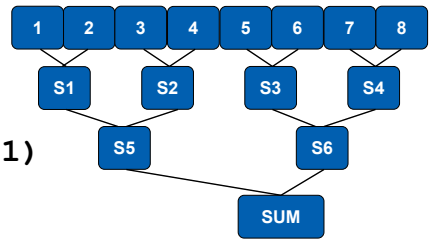
```
SUM[k] = 0 (0 ≤ k ≤ n – 1)
for i = 0 to n – 1
   LOCINDEX[k] = i (0 ≤ k ≤ n – 1)
   SUM[k] += A[LOCINDEX[k],k] (0 ≤ k ≤ n – 1)
end i loop
```

- LOCINDEX e un index local din P
- Fiecare citire din memorie aduce n elemente ce sunt adunate la cele n sume partiale pe cele n P
- Complexitatea este… $O(n)$

Exemplu – Suma pe Linii v1

- Problema: A[n, n] → vrem suma pe linii a matricei A in vectorul SUM[n]
  1. Organizarea pe linii duce la o complexitate $O(n^2)$
  2. Organizarea pe coloane duce la un algoritm de transpunere
  3. Folosind registre temporare asociate P_i-urilor (TEMP[k]) $O(n \log_2 n)$
  4. Se doreste o complexitate $O(n)$ → trebuie cautata alta solutie
- Solutia propusa este:

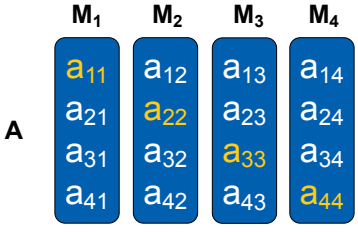
```
for i = 0 to n - 1
   TEMP[k] = A[i,k] (0 ≤ k ≤ n - 1)
   for j = 1 to n - 1
      TEMP[k] += TEMP[k-j] (j ≤ k ≤ n - 1)
   end j loop
   SUM[i] = TEMP[n-1]
end i loop
```



- Complexitatea este… $O(n \log 2n)$

Exemplu – Suma pe Linii v2

- Solutia pentru o complexitate $O(n)$ → va folosi cate un index local pentru fiecare P_i, pentru ca e absolut necesar



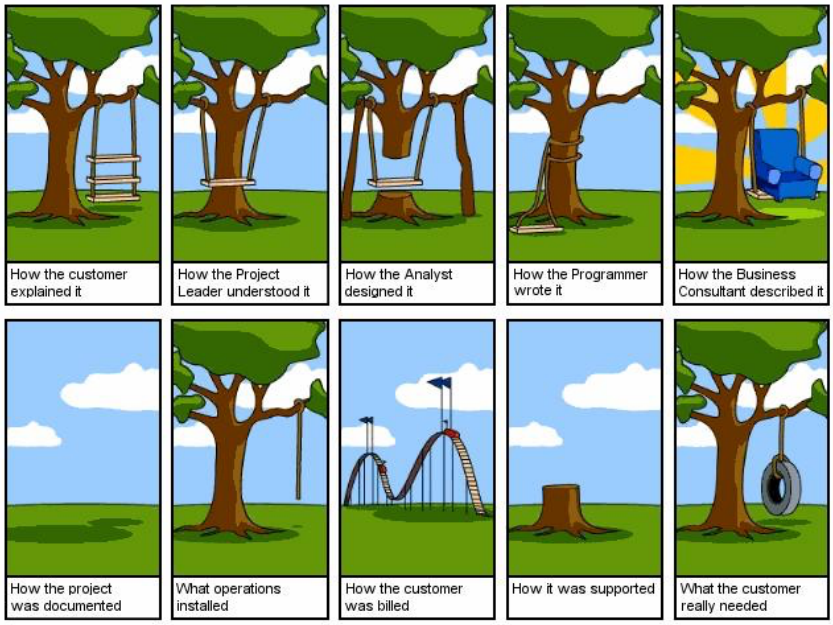
- Algoritmul este:

```
SUM[k] = 0 (0 ≤ k ≤ n - 1)
LOCINDEX[k] = k (0 ≤ k ≤ n - 1)
for i = 0 to n - 1
   SUM[k] += A[LOCINDEX[k],(k - i) mod n] (0 ≤ k ≤ n - 1)
end i loop
```

- Se aduna o diagonala la suma curenta
- Diagonala e deplasata ciclic spre elementul adiacent
- A[LOCINDEX[k],k] nu este altceva decat accesul pe coloane!

Concluzii Algoritmi SIMD

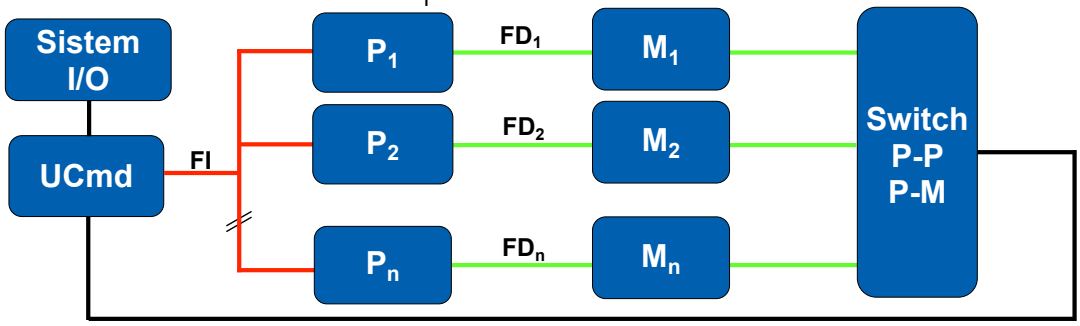
- Elaborarea unor algoritmi eficienti necesita
  - Organizarea corespunzatoare a structurilor de date
  - Registre locale ale sistemului de calcul
  - Existenta unei retele de comutatie P-P
- Etapele elaborarii unui program sunt
  - Descrierea algoritmului
  - Paralelizarea algoritmului
  - Optimizarea paralelizarii curente



Probleme de Comunicatii intr-un Sistem SIMD

Probleme in Structura SIMD

- Arhitecturile SIMD sunt adecvate prelucrarilor vectoriale
- Pentru cresterea vitezei masinii SIMD
  - Datele trebuiesc plasate in module distincte de memorie
- Pentru utilizarea eficienta a procesoarelor
  - Datele trebuiesc permutate pentru a furniza operanzi
- Pentru rezolvarea celor doua probleme sunt necesare RC P-P si P-M



Retele de Comutatie - RC

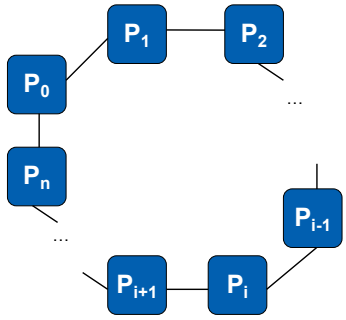
- RC P-P sunt implementate la nivelul procesoarelor
- RC P-M pot fi implementate cu switch-uri Cross-Bar
  - Pentru n procesoare sunt necesare n(n-1)/2 SP
  - Cand n creste costul devine prohibitiv: 65k Procs → 4Mrd+ Switch-uri
- Sunt necesare RC adecvate fiecarei aplicatii
  - Masini dedicate
- Aceste RC ofera
  - Fiabilitate
  - Costuri reduse
  - Viteza mare de comutatie/transfer
  - Facilitati de intretinere

The Trade-off…
Reduced Parallelism!

Deplasarea Ciclica a Datelor intre Procesoare

Interconexiunea Circulara

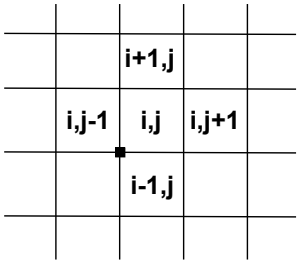
- Deplasarea ciclica a datelor intre procesoare
- Conexiunile sunt permise numai cu vecinii
- Nu exista conexiuni directe intre i si j
- Sunt maxim [(i-j-1)% n] noduri intermediare

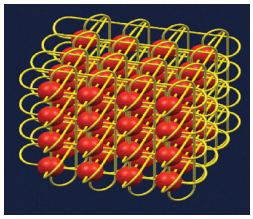


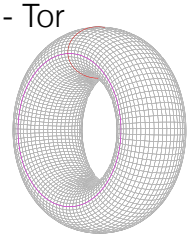
- Exemple de domenii de aplicatie:
  - Adecvate rezolvarii ecuatiilor diferentiale de ordinul 1: x[i] = x[i-1] + x[i+1] – 2x[i]
  - Propagarea caldurii printr-o bara
  - Aplicatii meteo – prognoza vremii in diferite puncte ale globului se face in paralel cu aceleasi date

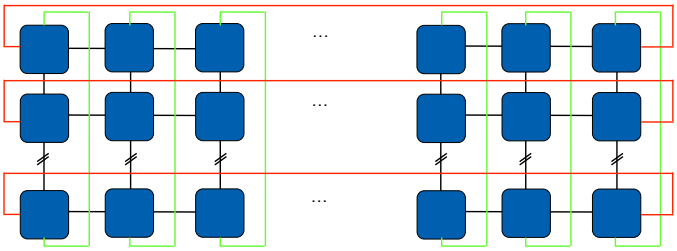
Interconexiunea Matriceala

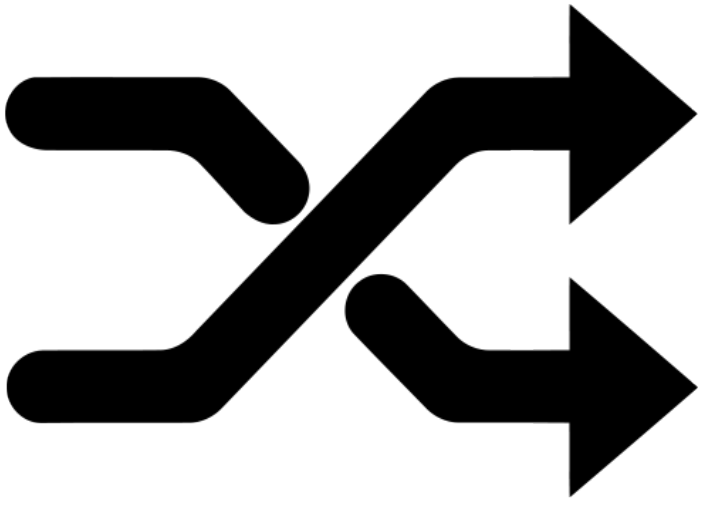
- Asigura comunicarea cu patru procesoare vecine
- Daca dependenta este de felul: x[i,j] = x[i+1,j] + x[i-1,j] + x[i,j+1] + x[i,j-1] – 4x[i,j] → de ex: evoluatia temperaturii in volumul unui material
- ILLIAC 4 – primul SIMD (64 procs) pana la BG
  - 90% la fel si acum - Tor







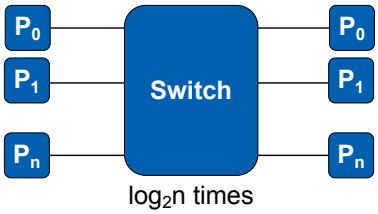




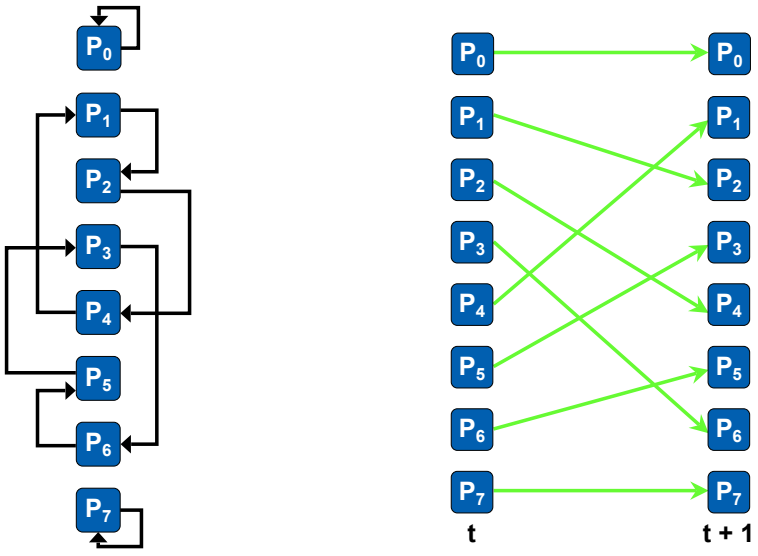
Intercalarea Perfecta – Shuffle

Intercalarea Perfecta – Shuffle

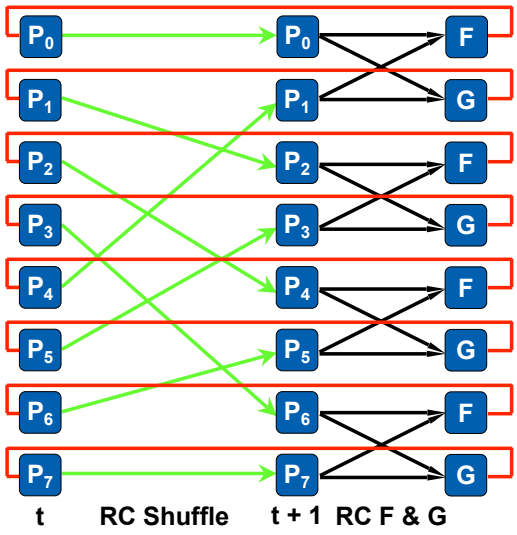
- Este adecvata pentru:
  - Transformata Fourier (FFT – Fast Fourier Transformation)
  - Algoritmi de sortare
  - Transpuneri de matrice
- Asigura conectarea intre procesoarele unei structuri SIMD de n procesoare cu:
  - n-2 comunicatii simultane
  - dupa log2n iteratii datele ajung pe procesorul sursa



Shuffle – 8 Procesoare



Shuffle – 8 Procesoare



- Doua RC:
  - Shuffle
  - Adiacenta (pt F & G):
    - F e pe Y_i = pare
    - G e pe Y_{i+1} = impare
  - $\log_2 n$ iteratii

```
for i = 1 to log2n
   shuffle[y]
   Yi    = F[Yi,Yi+1]
   Yi+1  = G[Yi,Yi+1]
end i loop
```

- Utilizare: sortari, FFT, transpuneri, functii cu operatii recurente

Shuffle – Aplicatii

- Pentru sortari:
  - F calculeaza maximul celor doi operanzi
  - G calculeaza minimul celor doi operanzi
- Pentru FFT:
  - F produce suma ponderata
  - G produce diferenta ponderata
  - Ponderile sunt calculate dinamic la fiecare pas al iteratiei
- Pentru transpunerea matricelor:
  - Organizarea directa a matricei
  - F = Y_i
  - G = Y_{i+1}

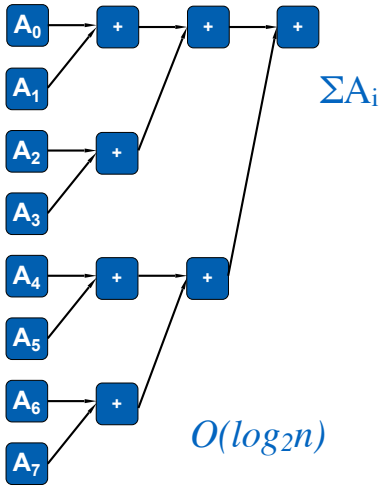
Shuffle – Aplicatii Iterative

- Suma elementelor unui vector
- Daca vectorul e intr-o singura memorie → secvential:

```
sum = 0
for i = 0 to n-1
    sum += ai
end i loop
```

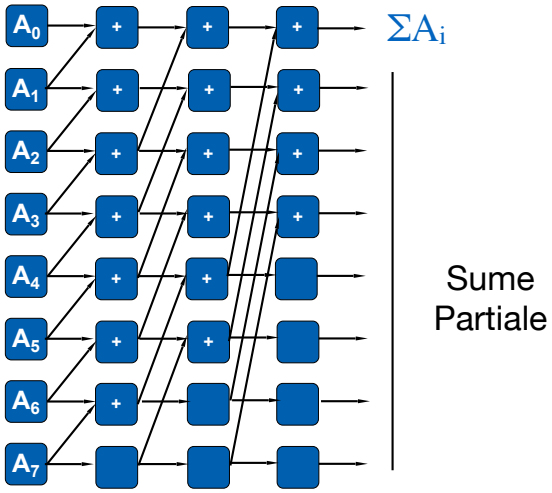
$O(n)$

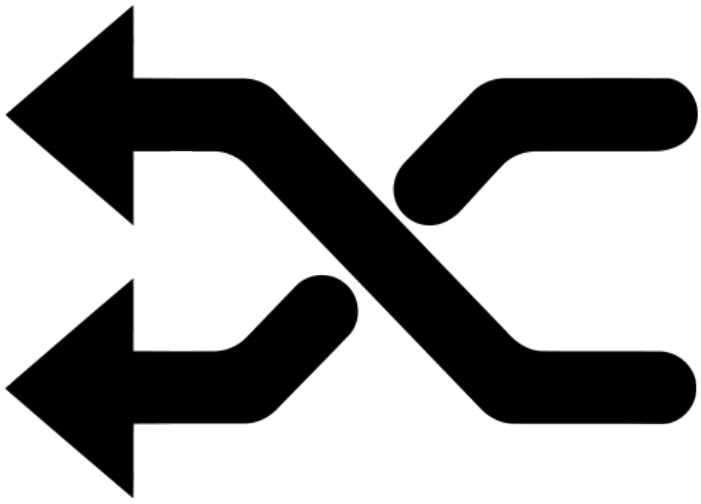
- Se poate reduce $O(n)$ pe SIMD?
- Distribuim vectorul intre memorii
- Probleme:
  - Incarcarea neechilibrata a procs
  - La fiecare pas trebuie schimbata RC in mod dinamic



Shuffle – Aplicatii Iterative

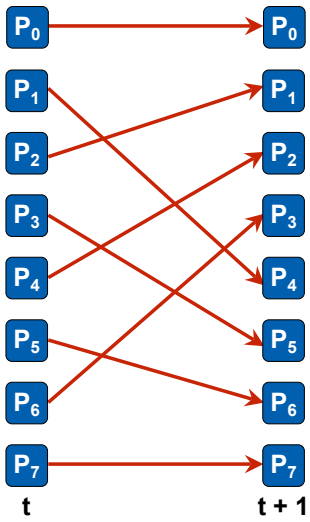
- Sunt necesare
  - Comunicare liniara
  - RC statica
- Reconfigurarea conexiunilor e mai simpla
- Dezavantaj: este in continuare necesara reconfigurarea la fiecare pas!
- Complexitate: $O(log_2 n)$





Conectarea Inversa – Inverse Shuffle

Inverse Shuffle – 8 Procesoare

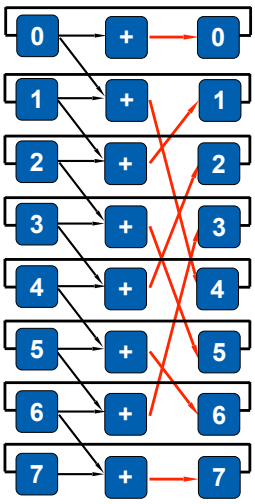


- Adecvat pentru operatii recurente, calculabile in $\log_2 n$ pasi
- Suma elementelor unui vector:

```
Yi = Ai  (0 ≤ i ≤ n-1)
for j = 1 to log2n
   Yj    = Yj + Yj-1
   invshuffle[Y], MASK
   compute MASK
end j loop
```

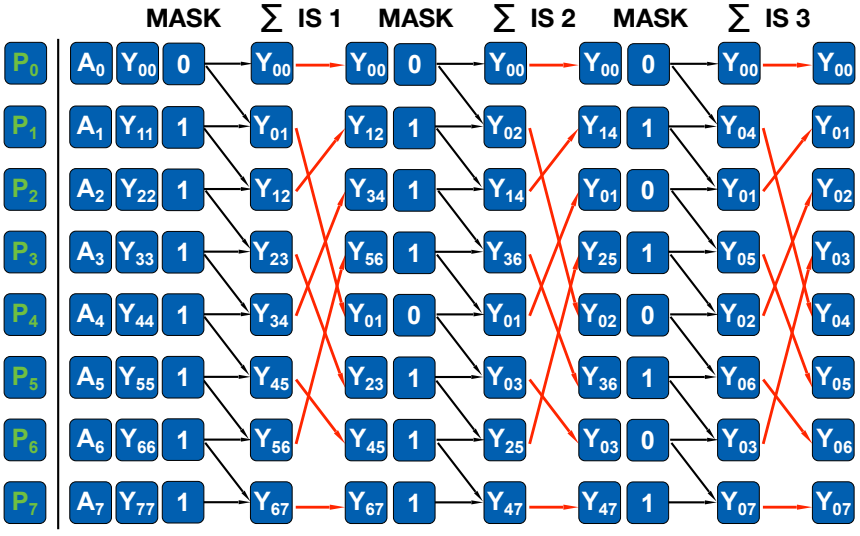
- Legatura identica pentru toate iteratiile:
  - O legatura directa
  - Un Inverse Shuffle

Inverse Shuffle – Aplicatii Iterative



- Legatura (RC) identica pentru toate iteratiile:
  - O legatura directa: $Y_j = Y_j + Y_{j-1}$
  - Un Inverse Shuffle
- Mastile se modifica si ele prin Inverse Shuffle la fiecare iteratie
  - MASK = 0 → procesor inactiv
  - MASK = 1 → procesor activ: (F & G = +)
- La anumite iteratii, unele procesoare pot fi inactivate de masti
- Notam $Y_{ij} = \sum_{k=i}^{j} A(k)$ $O(log_2 n)$

Inverse Shuffle – RC



$Y_{ij} = \sum_{k=i}^{j} A(k)$

Concluzii Structuri SIMD

- Probleme adecvate structurilor SIMD:
  - Calculele trebuiesc descrise de iteratii de tip vector a.i. majoritatea timpului sa fie ocupat cu operatii cu caracter identic
  - Operanzii (simultani) trebuiesc repartizati in module diferite de memorie
  - Trebuiesc identificate conexiunile intre operanzi a.i. ele sa fie cat mai aproape de conexiunile fizice intre procesoare
- Clasa problemelor SIMD nu este universala
  - Doar probleme dedicate se preteaza acestor arhitecturi
- Algoritmii, sistemele de interconectare si structura datelor trebuiesc adaptate in consecinta

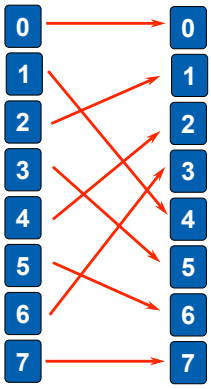
Permutari Elementare

Permutari Elementare

- Specifica conexiunea intre n resurse ca o functie bijectiva pe o multime ordonata
- Se pot realiza in mai multe feluri:
  1. Permutarea de baza – Base Line Permutation
  2. Permutarea cu intercalare perfecta – Shuffle Permutation
  3. Permutarea cu negare de bit – Negate Bit Permutation
  4. Permutarea fluture – Butterfly Permutation
  5. Permutarea cu ordine inversa – Bit Reversal Permutation
  6. Permutarea cu incrementare modulo – Increment Permutation
- Definirea acestor permutari se poate face pe baza reprezentarii binare a adreselor resurselor din multime (Flanders): ADR = (a_n a_{n-1} … a_1 a_0)

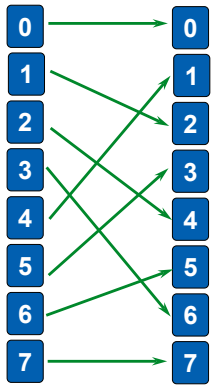
1 – Base Line Permutation

- Bitul cel mai nesemnificativ devine cel mai semnificativ:
  - P_base (a_n a_{n-1} … a_1 a_0) = (a_0 a_n a_{n-1} … a_1)
- Permutarea de baza este intercalarea perfecta inversa!
- Pentru 8 resurse avem P_base (a_2 a_1 a_0) = (a_0 a_2 a_1)
- Variante posibile sunt:
  - Permutarea de baza k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_b_kinf (a_n … a_k a_{k-1} … a_1 a_0) = (a_n … a_k a_0 a_{k-1} … a_1)
    - Asigura conexiunea **in** submultimi de resurse
  - Permutarea de baza k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup (a_n … a_{n-k+1} a_{n-k} … a_0) = (a_{n-k+1} a_n … a_{n-k+2} a_{n-k} … a_0)
    - Asigura conexiunea **intre** submultimi de resurse



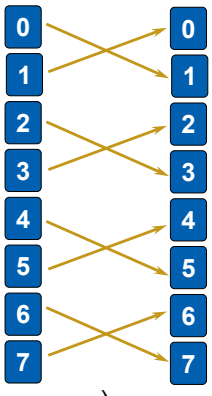
2 – Shuffle Permutation

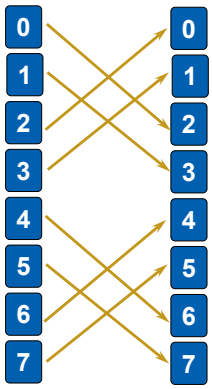
- Bitul cel mai semnificativ devine cel mai nesemnificativ:
  - P_shuffle (a_n a_{n-1} … a_1 a_0) = (a_{n-1} … a_1 a_0 a_n)
- Pentru 8 resurse avem P_shuffle (a_2 a_1 a_0) = (a_1 a_0 a_2)
- Variante posibile sunt:
  - Permutarea shuffle k inferioara:
    - Cei mai nesemnificativi k biti din adresa
    - P_shuffle_kinf (a_n … a_k a_{k-1} … a_1 a_0) = (a_n … a_k … a_1 a_0 a_{k-1})
    - Exemplu: Conexiunea memoriilor - adr baza & offset intr-un modul
  - Permutarea shuffle k superioara:
    - Cei mai semnificativi k biti din adresa
    - P_b_ksup (a_n … a_{n-k+1} a_{n-k} … a_0) = (a_{n-1} … a_{n-k+1} a_n a_{n-k} … a_0)



3 – Negate Bit Permutation

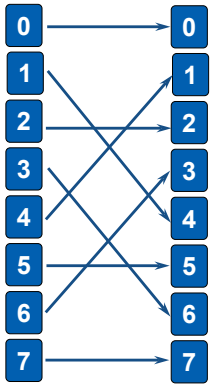
- Bitul cel mai nesemnificativ este negat:
  - P_negate_bit (a_n a_{n-1} … a_1 a_0) = (a_n a_{n-1} … a_1 $\overline{a_0}$)
- Pentru 8 resurse avem:
  - P_negate_bit (a_2 a_1 a_0) = (a_2 a_1 $\overline{a_0}$)
- Varianta posibila este negarea bitului k:
  - P_negate_bit_k (a_n … a_{k-1} … a_1 a_0) = (a_n … $\overline{a_{k-1}}$ … a_1 a_0)
  - Pentru 8 resurse avem P_negate_bit_1 (a2 a1 a0) = (a2 $\overline{a1}$ a0)





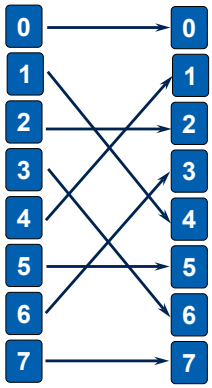
4 – Butterfly Permutation

- Se interschimba bitul cel mai semnificativ cu cel mai nesemnificativ:
  - P_butterfly ($a_n \dots a_1\ a_0$) = ($a_0\ a_{n-1} \dots a_1\ a_n$)
  - P_butterfly ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Butterfly este o permutare simetrica
- Variante posibile (ofera posibilitatea de conectare intre resursele clasei) sunt:
  - Permutarea butterfly k inferioara:
    - P_butterfly_k_inf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0 \dots a_1\ a_{k-1}$)
  - Permutarea butterfly k superioara:
    - P_butterfly_k_sup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1}\ a_{n-1} \dots a_n\ a_{n-k} \dots a_0$)



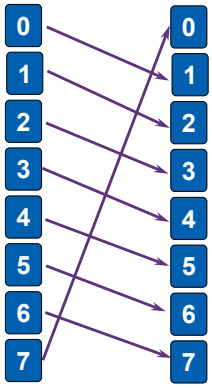
5 – Bit Reversal Permutation

- Se interschimba complet ordinea bitilor:
  - P_reversal ($a_n \dots a_1\ a_0$) = ($a_0\ a_1 \dots a_{n-1}\ a_n$)
  - P_reversal ($a_2\ a_1\ a_0$) = ($a_0\ a_1\ a_2$)
- Cu aceasta permutare se pot obtine echivalente
- Variante posibile sunt:
  - Permutarea reversal k inferioara:
    - P_reversal_k_inf ($a_n \dots a_k\ a_{k-1} \dots a_1\ a_0$) = ($a_n \dots a_k\ a_0\ a_1 \dots a_{k-1}$)
  - Permutarea reversal k superioara:
    - P_reversal_k_sup ($a_n \dots a_{n-k+1}\ a_{n-k} \dots a_1\ a_0$) = ($a_{n-k+1} \dots a_n\ a_{n-k} \dots a_0$)



6 – Increment Permutation

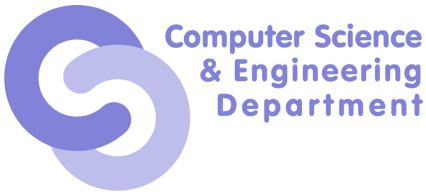
- Se incrementeaza valorile modulo 2^n:
  - P_increment (ADR) = $(V_{ADR} + 1) \bmod (2^{n+1})$
- Aceasta este o conexiune inelara intre module alaturate
- 1 din (V_{ADR} + 1), poate fi orice constanta
- Variante posibile sunt:
  - Permutarea increment k inferioara:
    - Realizeaza conexiuni **in cadrul** submultimii de resurse
  - Permutarea increment k superioara:
    - Realizeaza conexiuni **intre** submultimile de resurse



Ce am invatat in aces curs?

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 6
cs.pub.ro
cs.curs.pub.ro

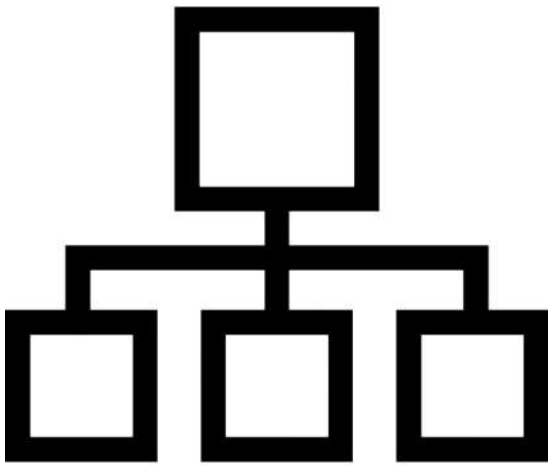
University Politehnica of Bucharest

In cursul anterior

- Structura si Setul de Instructiuni SIMD
- Implementarea Salturilor Conditionate in Structuri SIMD
- Organizarea Datelor in Structuri SIMD
- Probleme de Comunicatii intr-un Sistem SIMD
- Deplasarea Ciclica a Datelor intre Procesoare
- Intercalarea Perfecta – Shuffle
- Conectarea Inversa – Inverse Shuffle
- Permutari Elementare

Cuprins

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare



Comutatoare Ierarhice

Comutatoare

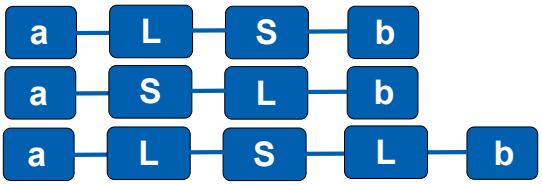
- Se impart in doua mari categorii
  - Ierarhice = conecteaza o componenta de tip a cu una de tip b
    - $a_i \leftrightarrow b_k$ (P ↔ M)

  - Neierarhice = conecteaza doua componenta de acelasi tip a
    - $a_i \leftrightarrow a_k$ (P ↔ P)

Comutatoare Ierarhice

- Comutatorul Simplu - Poarta
  - Asigura comunicarea intre componente de tip a si b
  - Este descris ca : S {poarta, 1a, 1b}

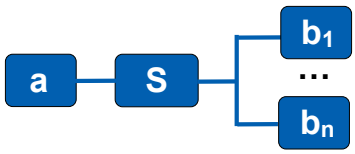
  - Variante posibile:
    - S {poarta, 1a, 1b, comutare la b}
    - S {poarta, 1a, 1b, comutare la a}
    - S {poarta, 1a, 1b, cu legatura la a si b}



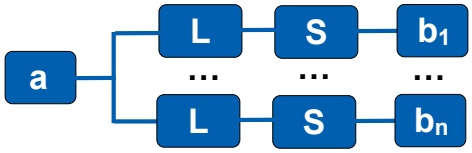
  - Poate conecta o resursa la magistrala sau o magistrala la alta magistrala

Comutatoare Ierarhice Duplex

- Asigura comunicatia intre o componenta de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 1a, nb, concurenta 1, 1Sp}



  - Se pot considera toate variantele de la comutatoarele simple



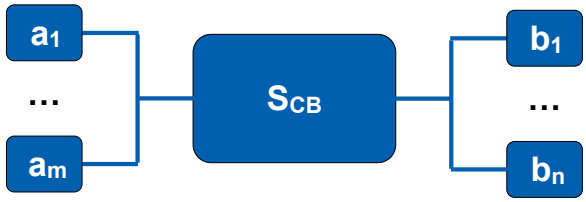
Comutatoare Ierarhice Dual Duplex

- Asigura comunicatia intre doua componente de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 2a, nb, concurenta 2, 2nSp}

  - Sunt posibile aceleasi variante ca la comutatoarele simple

Comutatoare Ierarhice Legatura Multipla

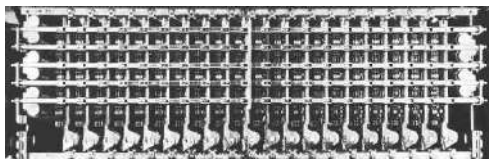
- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}



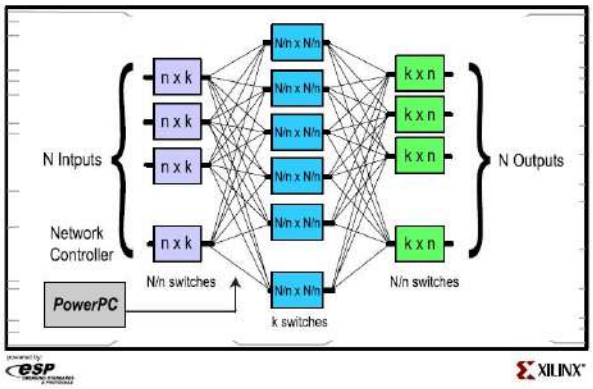
- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB}
  - Trebuie sa rezolve toate posibilele conflicte

Comutatoare Ierarhice CrossBar Switch

- Cross bar de la Bell System din 1960 cu 20 de verticale si 10 niveluri

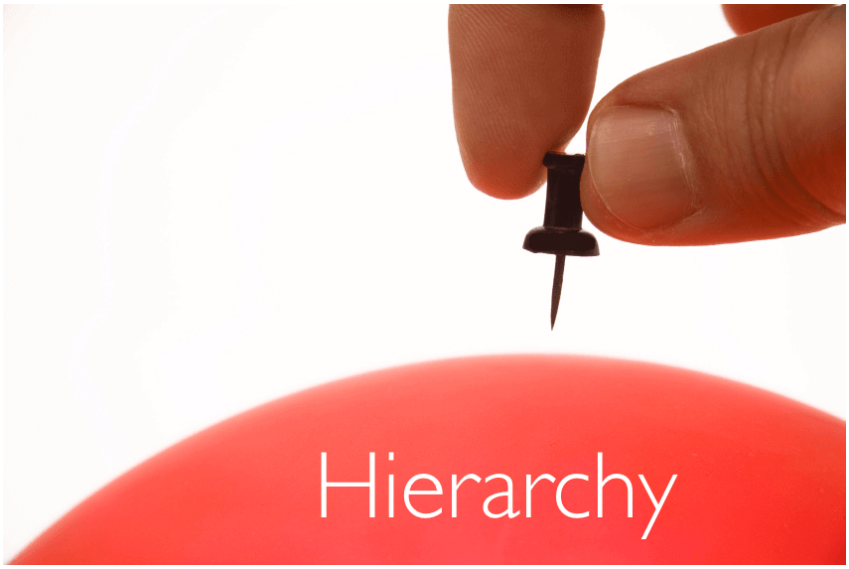


- Un crossbar de la Xilinx in FPGA-urile Virtex-II / Virtex-II Pro/ProX



Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
  - S {TK, ma, nb, c = k, k*(m+n)Sp}
- Orice a poate comunica cu orice b
- Nu totdeauna vei avea min(m, n) comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs si 4 memorii:
    - Pot fi maxim 4 accese la memorii → k=4
    - Nu are sens mai mult → k < n si k < m



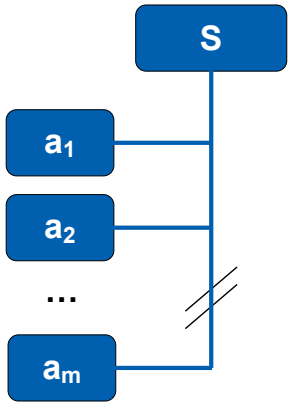
Comutatoare Neierarhice

Comutatoare Neierarhice

- Asigura conexiunea intre elemente de acelasi tip (e.g. P-P; M-M)
- Tipuri:
  - Comutatoare Duplex
  - Legatura multipla – tip central
  - Trunchi K neierarhic
- Structurile cu comutatoare asigura infrastructura de comunicatie

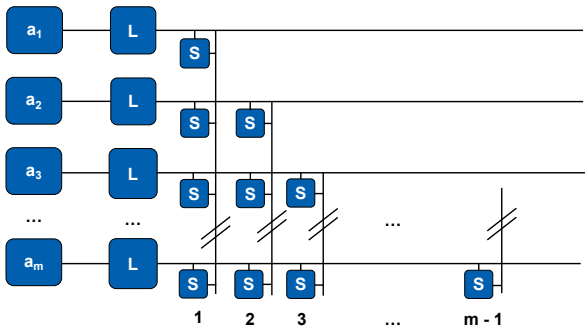
Comutatoare Neierarhice Duplex

- Comutatoare Duplex Neierarhice
  - S{Dn; ma, c:1; mSp}
  - Bus Inlantuit



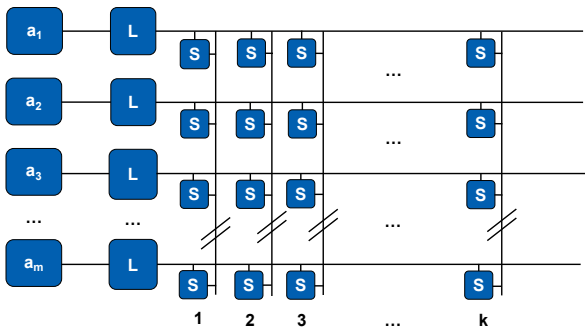
Comutatoare Neierarhice Legatura Multipla

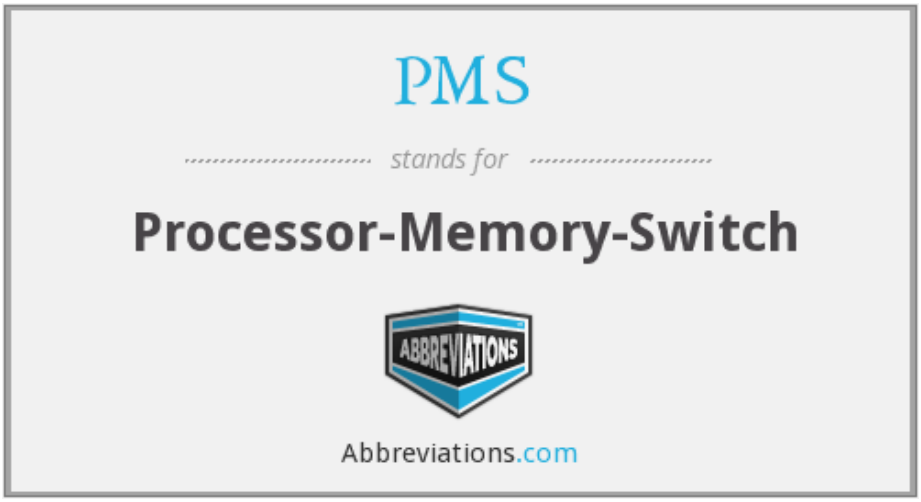
- Legatura multipla – tip central
  - S{lm-neierarhica; ma, c:m/2; m(m-1)/2 Sp}



Comutatoare Neierarhice Trunchi-K

- Trunchi-K neierarhic
  - S{tk; ma, c:min(k,m/2); m*k Sp}

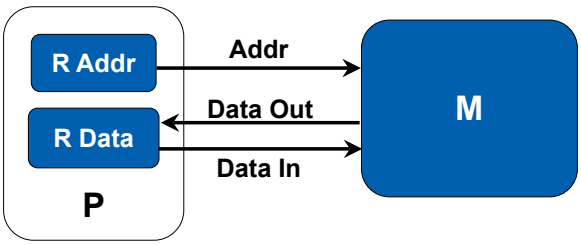




Exemple de Comutatoare Procesor Memorie

Comutatoare Procesor Memorie

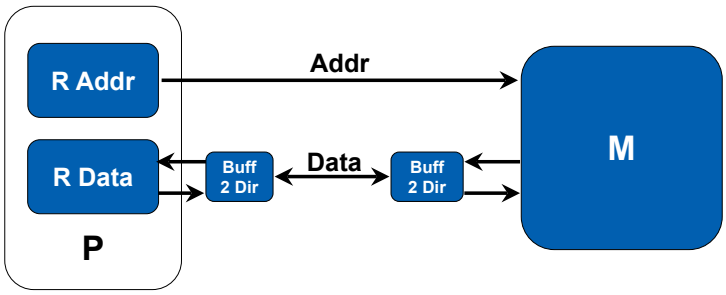
- Switch Processor Memory → S_{PM}
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!
Nu se justifica ambele linii

Comutatoare Procesor Memorie

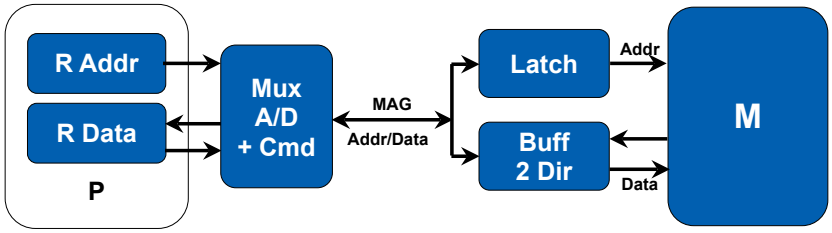
- Switch Processor Memory → S_{PM}
- D_{In} si D_{Out} sunt multiplexate prin magistrala de date



Bufferele sunt fie MUX fie TS

Comutatoare Procesor Memorie

- Switch Processor Memory → S_{PM}
- Addr, D_{In} si D_{Out} sunt multiplexate prin aceeasi magistrala

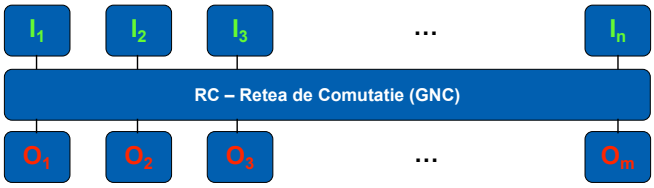


Bufferele sunt fie MUX fie TS

Interconectarea in Sistemele cu Resurse Multiple

Interconectarea in Sistemele cu Resurse Multiple

- In structurile SIMD si MIMD, retelele de comutare (de interconectare a resurselor), au rolul cel mai important in asigurarea performantelor sistemului!
- Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)



- I_i = Intrari & O_j = Iesiri; RC conecteaza n·m resurse
- Net (n, m) = Generalised Connection Network – graf in care arcele conecteaza perechi de resurse

Retele de Comutatie

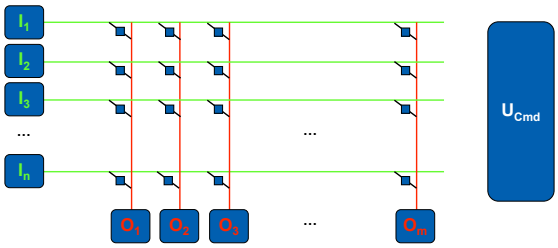
- O astfel de RC trebuie sa respecte niste reguli:
  - O intrare poate fi conectata la orice iesire
  - O intrare poate fi conectata simultan la mai multe iesiri
    - Astfel vom avea Broadcast partial sau total
  - O iesire poate fi conectata numai la o intrare
  - Concurenta maxima e data de min (m, n)
- Din aceste reguli rezulta ca RC ar trebui sa realizeze n·m corespondente == GCN!

GCN – Generalised Connection Network

- Un GCN poate fi implementat
  - Printr-o structura origanizata pe un singur nivel in care se stabilesc legaturi directe intre intrari si iesiri (crossbar)
  - Prin utilizarea unor subretele de dimensiuni (mai) reduse conectate ierarhic pe mai multe niveluri
- Caracteristici pentru GCN sunt doi parametrii
  - Intarzierea de propagare (Δt) – timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
  - Complexitatea (nC) – numarul total de comutatoare elementare utilizate in implementarea GCN-ului

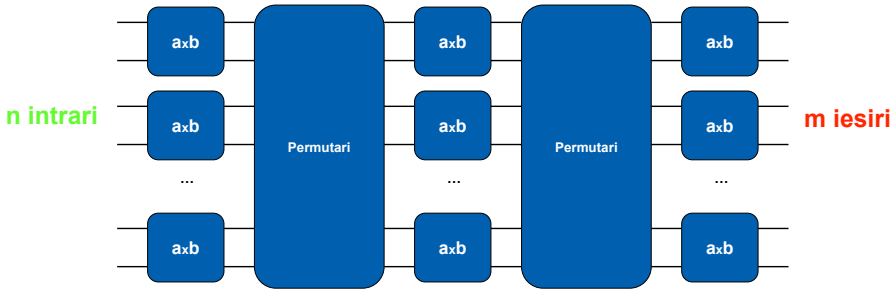
Implementari ale RC (GCN)

- Comutare directa – Cross Bar cu n·m comutatoare (Δt = 1)
- Unitatea de comanda trebuie sa rejecteze 2 I (intrari) / 1 O (iesire)
- O implementare cu comutare directa
  - Este greu de proiectat fizic (nC = n·m)
  - Nu se preteaza la VLSI & este inflexibila la dezvoltare ulterioara
- U_{Cmd} controleaza toate switch-urile elementare



Implementari ale RC (GCN)

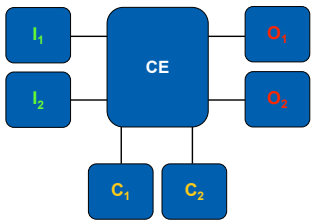
- Prin realizarea unor subretele de comutare organizate ierarhic, pe mai multe niveluri
- Aceasta implementare necesita k ≈ $\log_2 n$ comutatoare elementare → reduce complexitatea (nC) & creste timpul de comutare (Δt)
- Comutatoarele elementare sunt de tip Cross Bar (axb), unde a « n si b « m
- Intre niveluri se aseaza permutarile de interconectare



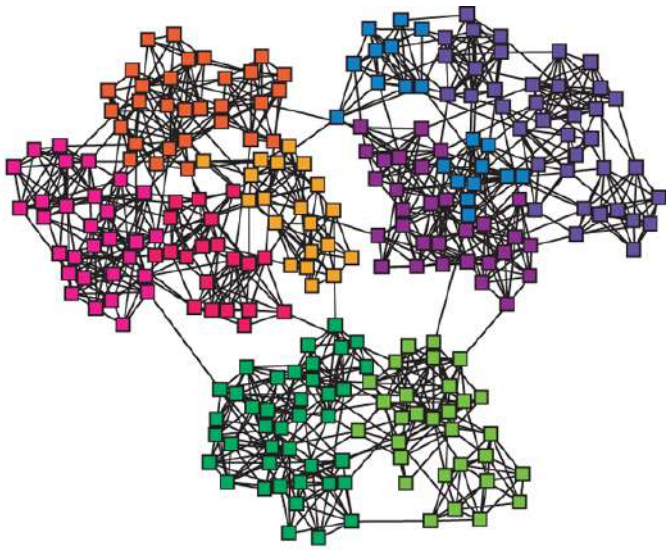
Comutatoare Elementare

- Un comutator elementar are 2 intrari, 2 iesiri si 2 biti de comanda:

C_1	C_2	Connection	
0	0	$O_1 = I_1; O_2 = I_2$	Direct
0	1	$O_1 = I_2; O_2 = I_1$	Inversat
1	0	$O_1 = I_1; O_2 = I_1$	Broadcast Superior
1	1	$O_1 = I_2; O_2 = I_2$	Broadcast Inferior



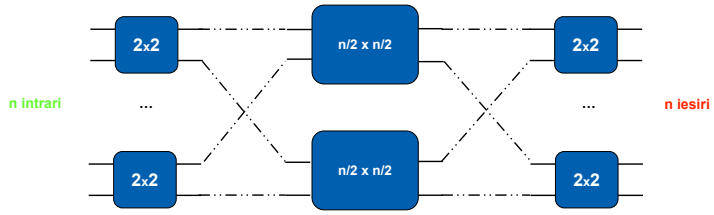
- Aceste module se pot integra pe baza permutarilor elementare
- In acelasi mod se pot construi si comutatoare elementare 4x4, 8x8, etc.



Retele de Comutare Ierarhice

Retele de Comutare Ierarhice

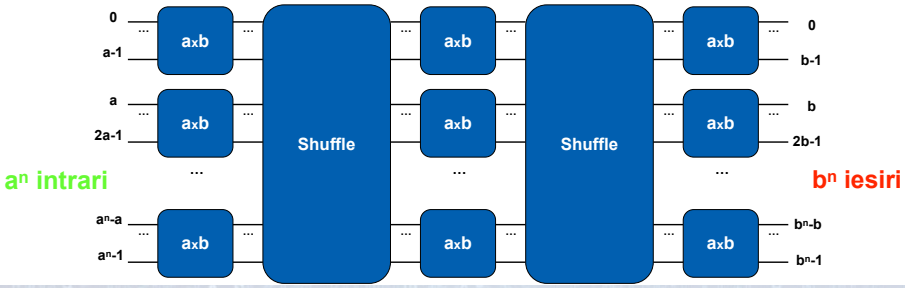
- Realizate din CrossBar-uri de dimensiuni mici asezate pe mai multe niveluri de interconectare:
  - Creste timpul de intarziere datorita utilizarii mai multor module – dar se mentine ∀intrare ↔ ∀iesire
- Exemplu – structura Beizer (Benes):
  - Retea nxn realizata cu doua module n/2 x n/2 (se pot imparti la randul lor in subretele mai mici) si 4n module de tip 2x2
  - Complexitatea este de (4n log n – 2n)



Retele de Comutare de tip Delta

Retele de Comutare de tip Delta

- Sunt comutatoare ierarhice cu a^n intrari si b^n iesiri ce utilizeaza CrossBar-uri (CB) axb si retele de permutare de tip intercalare perfecta (Shuffle)
- Reteaua Delta are n niveluri ierarhice:
  - Nivelul 0 contine a^{n-1} CB-uri de tip axb, pentru a conecta an intrari cu a^{n-1}xb iesiri
  - Nivelul 1 contine asadar a^{n-1}xb intrari conducand la a^{n-2}xb module CB

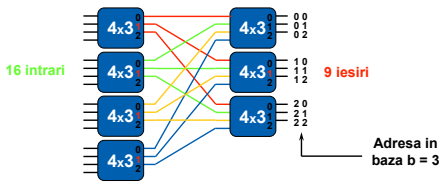


Retele de Comutare de tip Delta

- Astfel, in general:
  - Nivelul i contine $a^{n-i}b^{i-1}$ module CB de tip axb
- Intreaga retea contine:
  - $(a^n-b^n)/(a-b)$ CB-uri de tip axb cand $a \neq b$
  - nb^{n-1} CB-uri de tip axb cand $a = b$
- Destinatia este $D = (d_{n-1}\ d_{n-2}\ \ldots\ d_1\ d_0)$ cu $0 \leq d_i < b$
- Cifrele de reprezentare in baza b, a adresei destinatiei vor controla modulul CB de pe nivelul respectiv (d_i controleaza CB-ul i)
- Functia Shuffle-ului este de a interconecta nivelurile retelei Delta

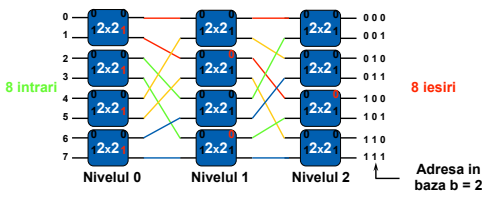
Exemplu – Delta 16x9

- Presupunem ca avem 16 procesoare si 9 memorii
- 16 = 4^2 si 9 = 3^2 → folosim 2 niveluri de CB 4x3
- Daca vrem acces la memoria 4 (11 in baza 3) atunci se merge pe 1 (niv 0) si 1 (niv 1)!



Exemplu – Delta 8x8

- Presupunem ca avem 8 procesoare si 8 memorii
- 8 = 2^3 → folosim 3 niveluri de CB 2x2
- Daca vrem acces la memoria 4 (100 in baza 2) atunci se merge pe 1 (niv 0), 0 (niv 1) si 0 (niv 2)!



Retele Bazate pe Rutare

Retele Bazate pe Rutare

- Au ca nucleu legaturi directe intre o resursa si cele considerate vecine
- Resursele pot fi distincte dar toate au in comun un router ce se ocupa de comunicarea prin mesaje
- Traditional aceste retele sunt modelate prin grafuri cu urmatoarele proprietati de baza:
  - Gradul nodului (al resursei) – defineste numarul de canale de conexiune ale resursei la vecinii sai
  - Diametrul – distanta maxima intre doua resurse ale retelei
  - Regularitatea – nodurile/resursele care au/nu au acest grad
  - Simetria – o retea este simetrica daca “arata la fel” pentru orice nod
- Obs: acest tip de retea nu foloseste comutarea de circuite (CB), ci rutarea – legatura intre noduri/resurse e permisa doar cand este ceruta & permisa

Retele Bazate pe Rutare – Caracteristici

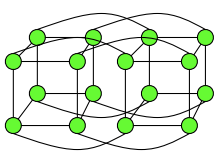
- Topologia – defineste modul in care nodurile sunt interconectate
- Rutarea – stabileste calea de date pe care un mesaj o urmeaza de la sursa la destinatie
- Comutarea – mecanismul ce determina cum si cand un canal de intrare este conectat la unul de iesire
- Obs: trebuie avute in vedere:
  - Alocarea de buffere
  - Controlul fluxului de date

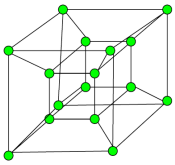
Retele Bazate pe Rutare – Topologii

- Topologii ortogonale:
  - Nodurile sunt aranjate intr-un spatiu n-dimensional ortogonal
  - Fiecare legatura produce o deplasare intr-o singura dimensiune
- Topologii strict ortogonale (mai interesante):
  - Fiecare nod are cel putin o legatura in fiecare dimensiune
  - Trecerea intr-o noua dimensiune se poate face din orice nod
  - Rutarea e mai simpla si implementarea HW e mai eficienta
  - Nodurile pot fi numerotate folosind coordonatele lor in spatiul n-dimensional
  - Procesul de rutare se face pe baza diferentei intre coordonate
  - Obs: Distanta dintre noduri poate fi calculata direct folosind adresele nodurilor respective

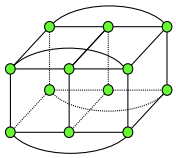
Retele Bazate pe Rutare – Topologii

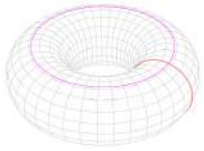
- Hipercub:
  - 2 cuburi unul intr-altul
  - Din orice nod se poate ajunge pe orice nivel
  - Se poate optimiza drumul

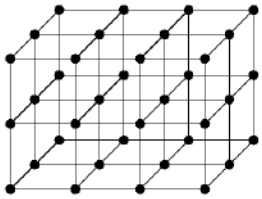




- Tor:



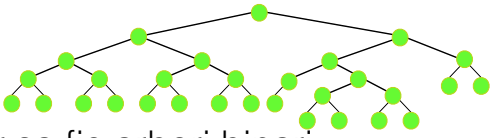




- Plasa n-dimensionala:

Retele de Tip Arbore

- Sunt un alt tip de retele de comutare

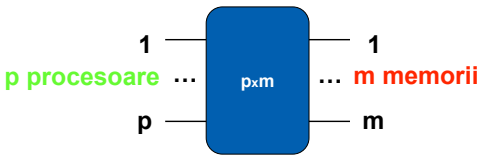


- Nu e neaparat necesar sa fie arbori binari
- Fiecare nod al arborelui poate avea la randul sau un subarbore – subarbori multiplii
- Similar – la structurile cu hipercub, fiecare nod poate fi inlocuit cu un nou cub!
- Exista mai multe cai posibile pentru mesaje – totdeauna se cauta drumul de lungime minima

Performantele Retelelor de Comutare

Performantele Retelelor de Comutare

- Analizam performantele retelelor ce asigura comutarea intre p procesoare si m memorii



- Analiza va fi realizata pentru retele de tip CB si Delta
- Rata de servire – media cererilor de acces la memorii acceptate intr-un ciclu (toate memoriile)
- Ciclu de servire – timpul necesar pentru ca o cerere de acces la memorie sa se propage prin logica de comutare + timpul efectiv de acces la memorie (propagarea inversa)

Presupuneri / Restrictii

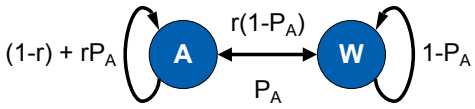
- Nu se fac deosebiri intre ciclurile de citire si de scriere
- Fiecare procesor va genera cereri independente pentru acces la un modul de memorie
- Cererile sunt uniform distribuite in cadrul tuturor modulelor de memorie – fiecare procesor cu un modul de memorie
- La inceputul fiecarui ciclu
  - Fiecare procesor genereaza o noua cerere de acces cu probabilitatea r (media cererilor generate de fiecare procesor intr-un ciclu)
  - Nu exista coada de asteptare → dialog direct
- Cererile neacceptate intr-un ciclu sunt ignorate
  - Cererile din ciclul urmator sunt independente de cele din ciclul anterior
  - Cererile neacceptate trebuiesc relansate intr-un ciclu urmator

Comportarea unui Procesor intr-un CB

- Se va utiliza un graf Markov
- Un procesor are 2 stari posibile:
  - A = starea activa – cererea este acceptata
  - W = starea de asteptare (wait) – cererea este respinsa
- q_A si q_W sunt probabilitatile ca procesorul sa se afle in starile A si W respectiv
- P_A = probabilitatea ca cererea sa fie acceptata – adica de a trece din starea W in starea A
- Astfel, putem defini:

$$q_A = \frac{P_A}{P_A + r(1 - P_A)} \qquad q_W = 1 - q_A$$

Comportarea unui Procesor intr-un CB



- Se considera rata de cereri r ca fiind statica
  - In realitate r e diferita luandu-se in considerare conflictele de acces
- Modulele de memorie primesc r' > r cereri de acces:

$$r' = rq_A + q_W = \frac{r}{r + (1-r)P_A} \qquad P_A = \frac{m}{r'p}\left(1 - \left(1 - \frac{r'}{m}\right)^p\right)$$

Analiza Retelei de Tip CrossBar

- Relatiile dinainte, definesc r' ca o functie de PA si PA ca o functie de r' → definesc un proces iterativ:
  - Initiem rata de transfer r' = r
  - P_A poate fi considerat ca o masura a cererilor de acces nerezolvate
  - Daca P_A este mare atunci sunt putine stari de asteptare
- Fie $\overline{w}$ media numarului de cicli de asteptare pentru o cerere
  - O cerere rejectata de i ori va astepta i ciclii $\overline{w} = \sum_{i=1}^{p} i(1-P_A)^i = \frac{1-P_A}{P_A}$
- In practica, cererile ce produc conflicte de acces la memorii, nu sunt rejectate, ci introduse intr-o coada de asteptare si servite FIFO

Analiza Retelei de Tip Delta

- Consideram o retea Delta ($a^n x b^n$) formata din CB-uri elementare (axb)
  - a^n procesoare si b^n memorii
- Fiecare nivel al retelei Delta e controlat de unul din bitii de reprezentare a adresei destinatie, in baza b!
- Orice modul CB axb e independent de celelalte module → putem considera:
  - Cererile independente si uniform distribuite
- Daca r este rata de cereri la intrarea in axb atunci: $B(a,b) = b - b\left(1 - \frac{r}{b}\right)^a$
- Impartim la b → rata de cereri la fiecare iesire din CB-ul curent: $\frac{B(a,b)}{b} = 1 - \left(1 - \frac{r}{b}\right)^a$

Analiza Retelei de Tip Delta

- Pentru fiecare nivel intermediar al retelei Delta:
  - r_{out} (rata de cereri pentru fiecare iesire) este o functie de r_{in} (rata de cereri de la intrare)

$$r_{out} = 1 - \left(1 - \frac{r_{in}}{b}\right)^a$$

- Rata de cereri de la iesirea unui nivel r_n este rata de cereri de intrare pentru nivelul adiacent:
  - Rata de cereri de la ultimul nivel determina rata de transfer a retelei de tip Delta
- Fie r_i rata de cereri de la iesirea unui nivel in reteaua Delta → rata de cereri a unei RC Delta, generata de fiecare procesor este:

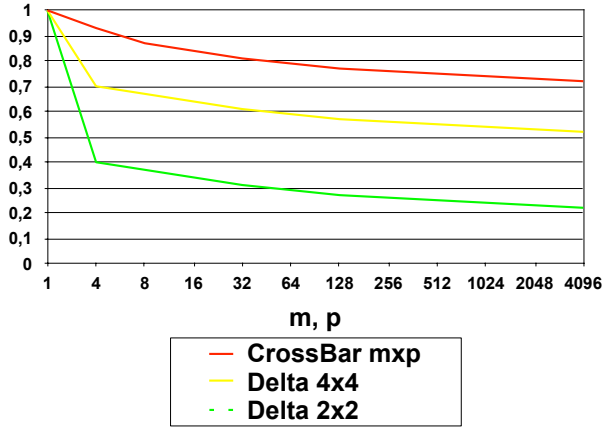
$$B(a^n, b^n) = b^n \cdot r_n \quad \text{unde} \quad r_i = 1 - \left(1 - \frac{r_{i-1}}{b}\right)^a \quad \text{si} \quad r_0 = r$$

Analiza Comparativa Delta / CB

- Probabilitatea ca o cerere sa fie acceptata este:

$$P_A = \frac{b^n \cdot r_n}{a^n \cdot r}$$

- Ce influenta au m si p-ul unei retele?
  - CrossBar **m**x**p** cu r=1 rata de cereri a fiecarui procesor
  - Delta 4^nx4^n cu 4x4
  - Delta 2^nx2^n cu 2x2
- Pentru m mare, P_A-ul scade puternic
  - Multe rejectii si conflicte
  - Limita la 16^nx16^n este 0.6



Ce am invatat in aces curs?

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare

Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 7
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare

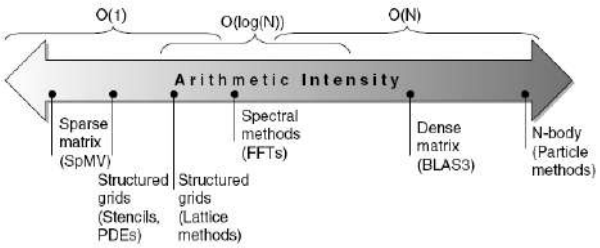
Cuprins

- The Roofline Model
- Architecture
- Programming
- Performance
- Going Forward

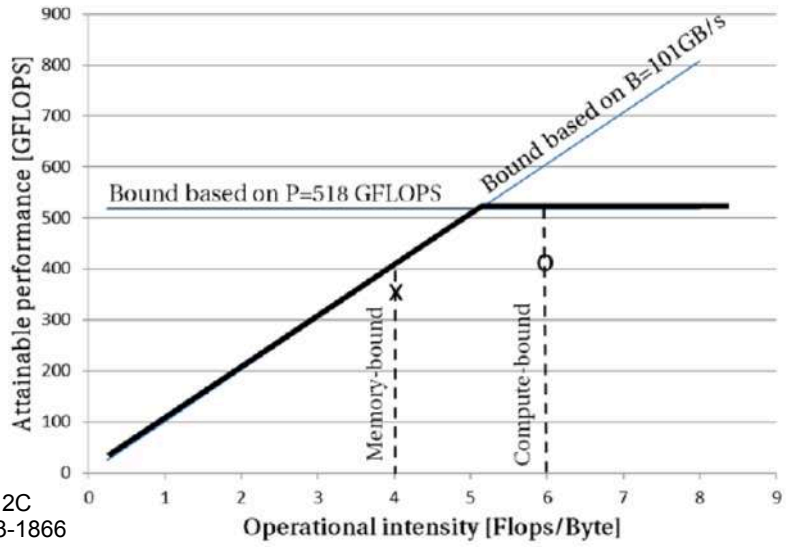
The Roofline Model

The Roofline Performance Model

- Basic Idea
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic Intensity
  - Floating-point operations per byte read



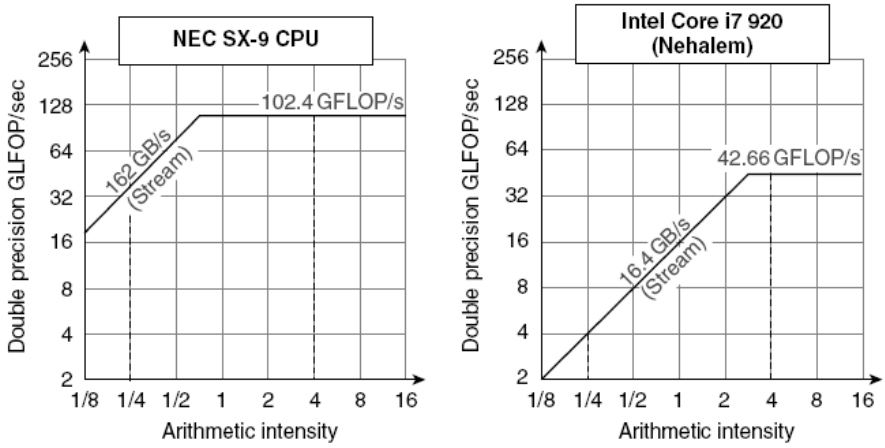
Modelul Roofline



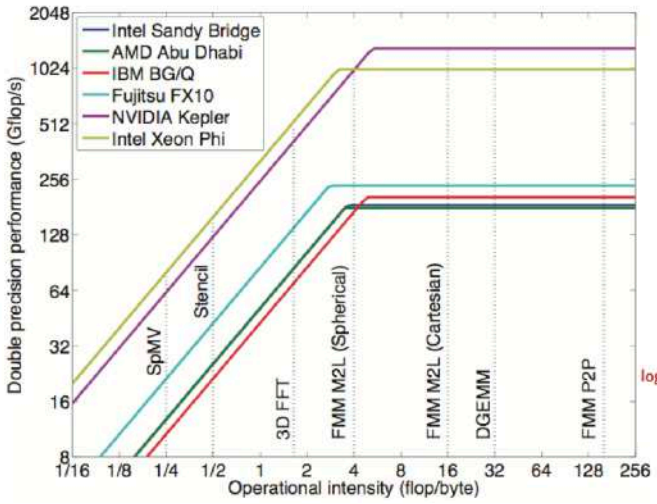
Intel XeonE5-2697v2 12C
2.7GHz cu memorie DDR3-1866

Roofline Performance Examples

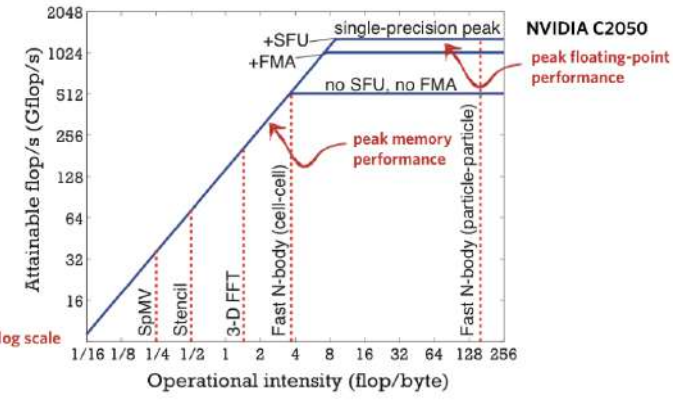
- Attainable GFLOPs/sec Min = (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Performance)



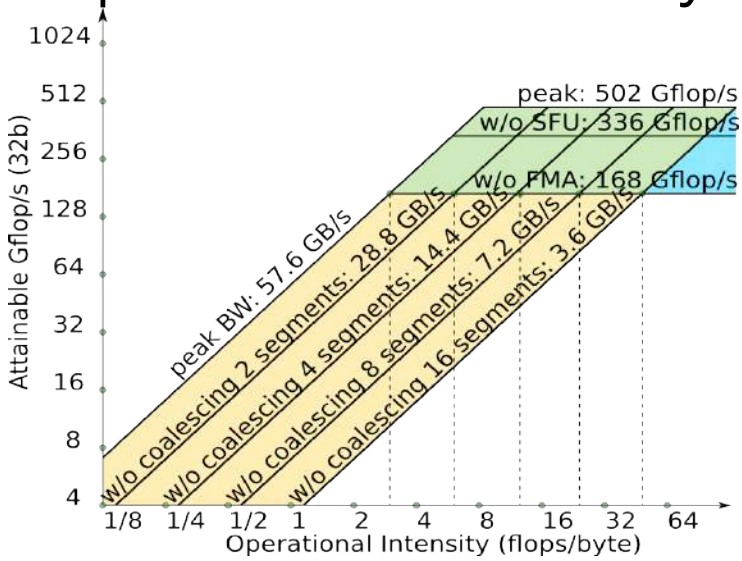
Roofline Performance Examples



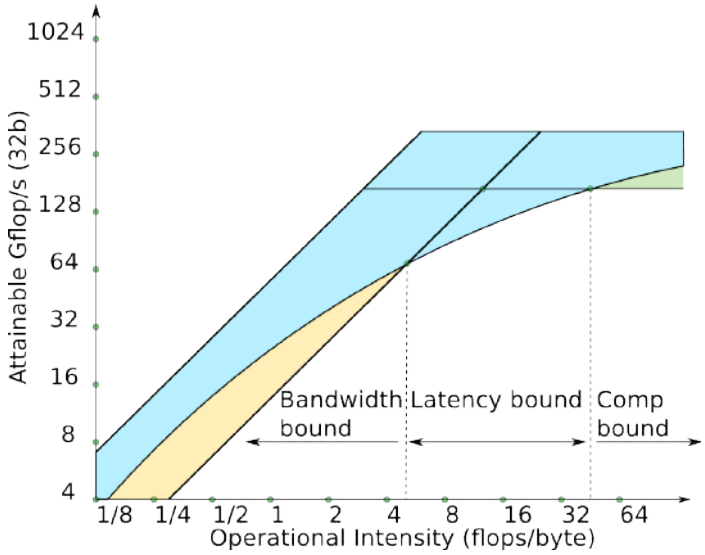
log/log scale



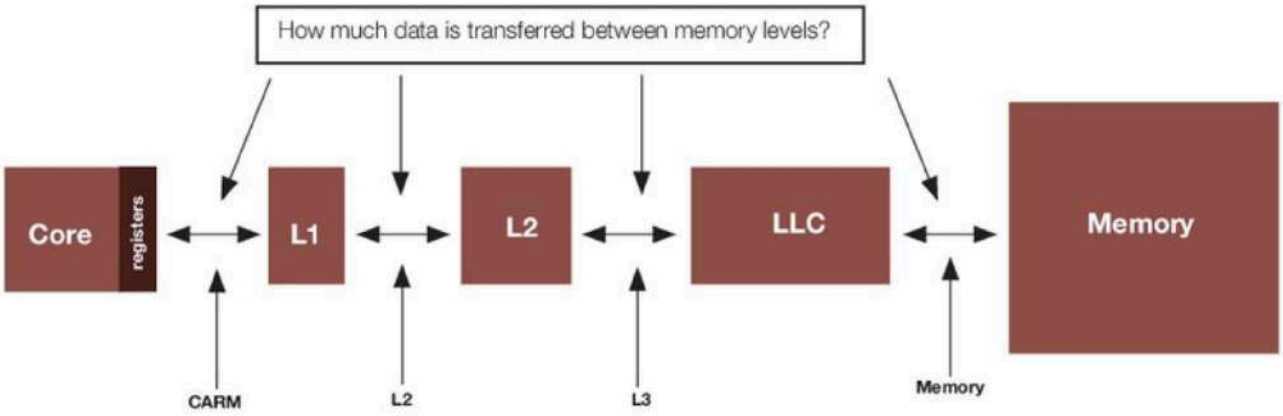
Optimization is Key



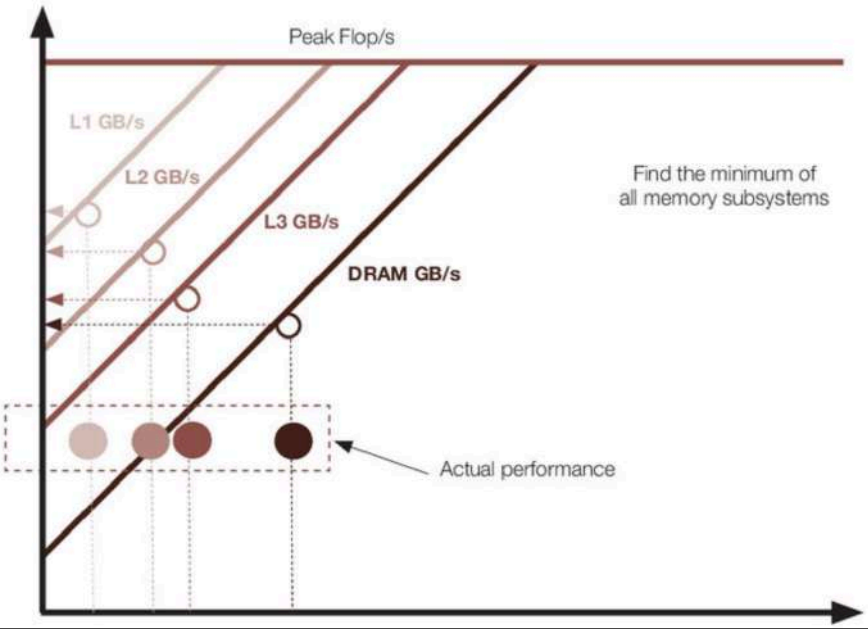
Bottlenecks



Cache-Aware Roofline Model



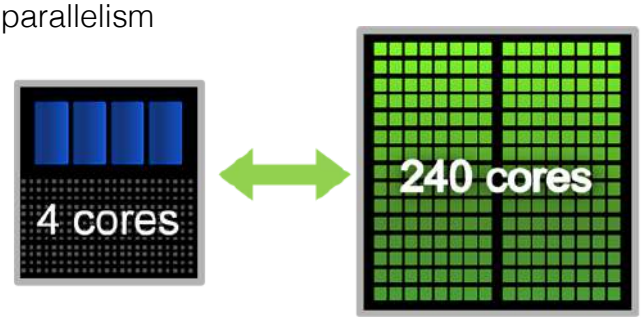
Performance Limitations



Architecture

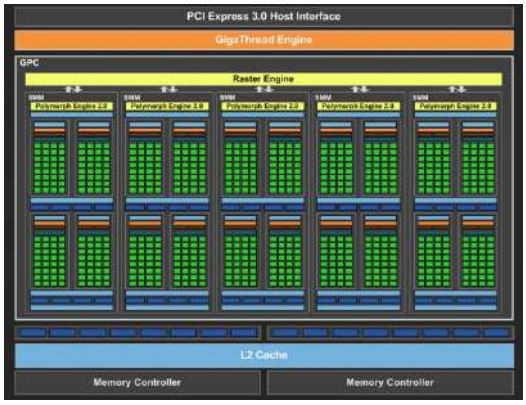
CPUs & GPUs

- Typically GPUs and CPUs coexist in a heterogeneous setting
- “Less” computationally intensive part runs on CPU
  - Coarse-grained parallelism
- More intensive parts run on GPU
  - Fine-grained parallelism

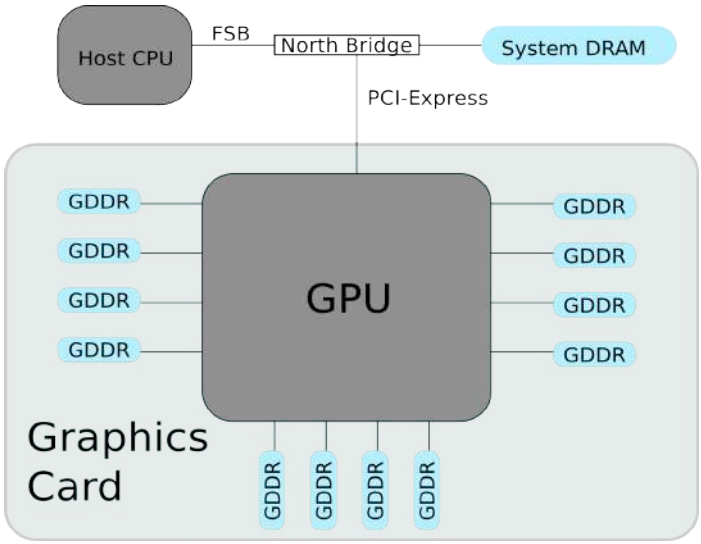


NVIDIA Kepler K40

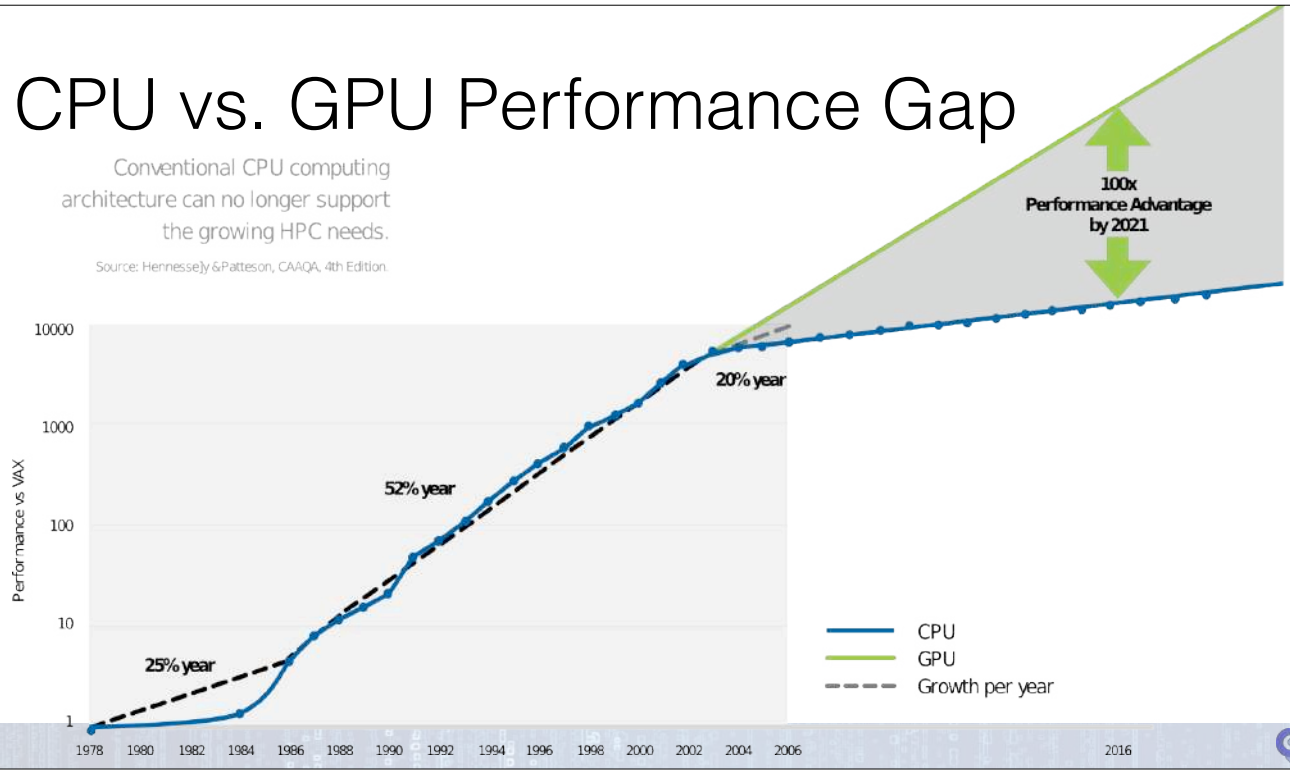
- 2880 Streaming Processors/cores (SPs) organized as 15 Streaming Multiprocessors (SMs)
- Each SM contains 192 cores
- Memory size of the GPU system: 12 GB
- Clock speed of a core: 745 MHz



System Architecture



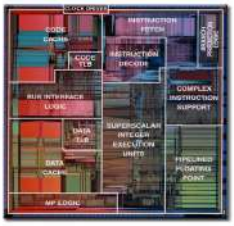
CPU vs. GPU Performance Gap



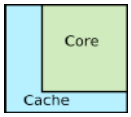
CPU/APU Architectures

Pentium Processors Evolution

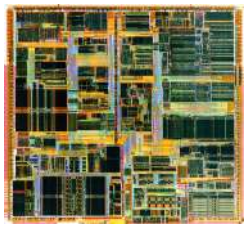
Pentium I

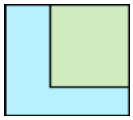


Chip area breakdown

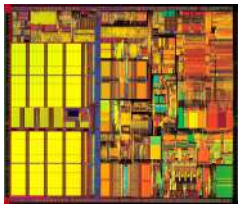


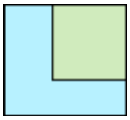
Pentium II



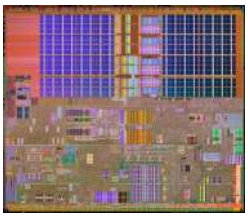


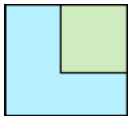
Pentium III





Pentium IV

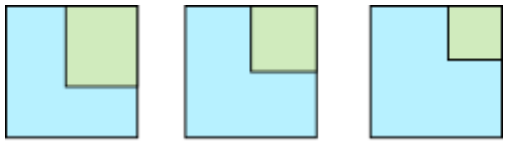




Q: What can you observe? Why?

Extrapolation of Single Core CPUs

If we extrapolate the trend, in a few generations, Pentium will look like this:

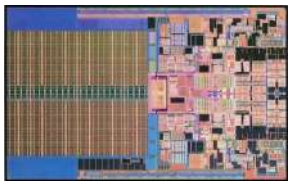


Of course, we know it did not happen.

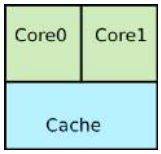
Q: What happened instead? Why?

Multi-Core CPUs Evolution

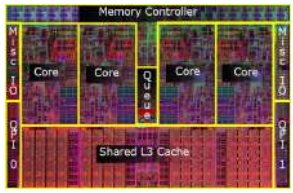
Penny

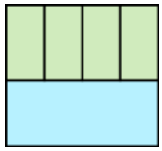


Chip area breakdown

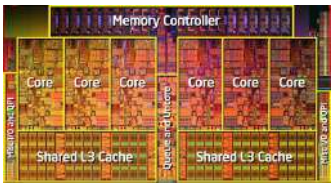


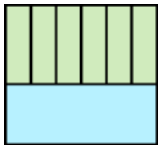
Bloomfield



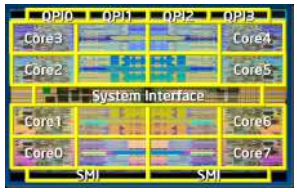


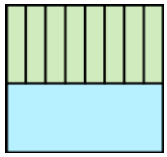
Gulftown





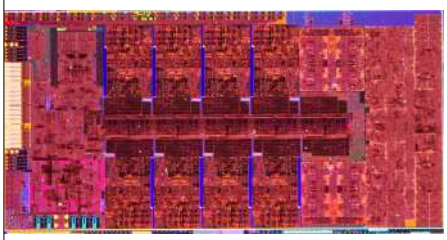
Beckton



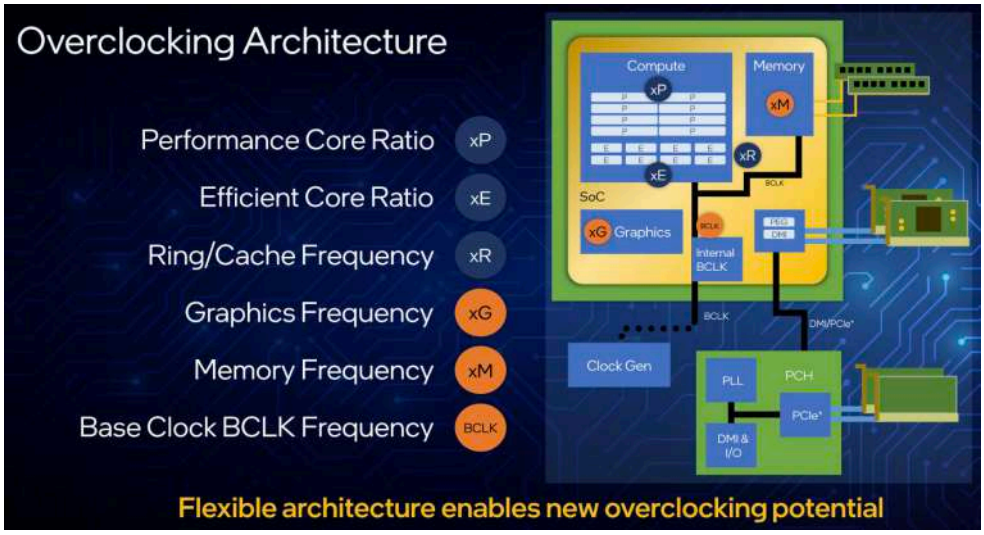


Q: What can you observe? Why?

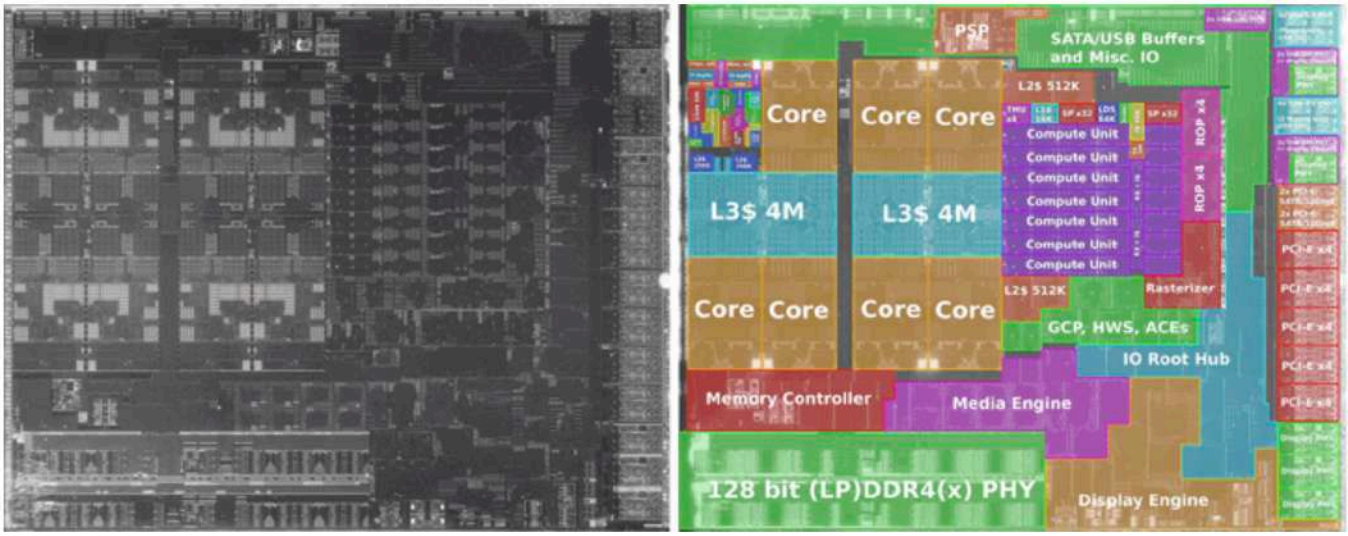
Intel Core i9-12900K



Intel Overclocking Architecture



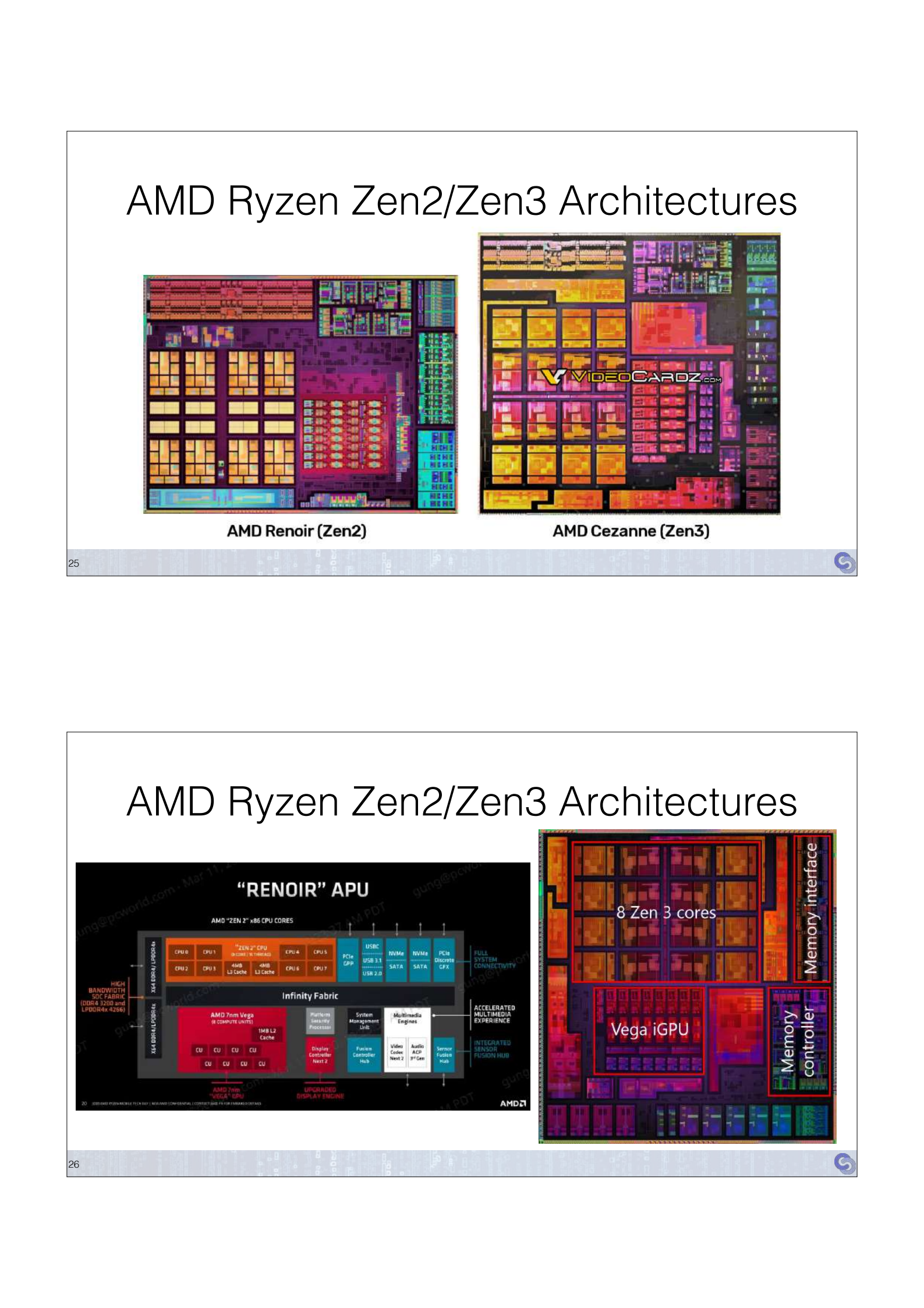
AMD Ryzen Architecture



AMD Ryzen Zen2/Zen3 Architectures

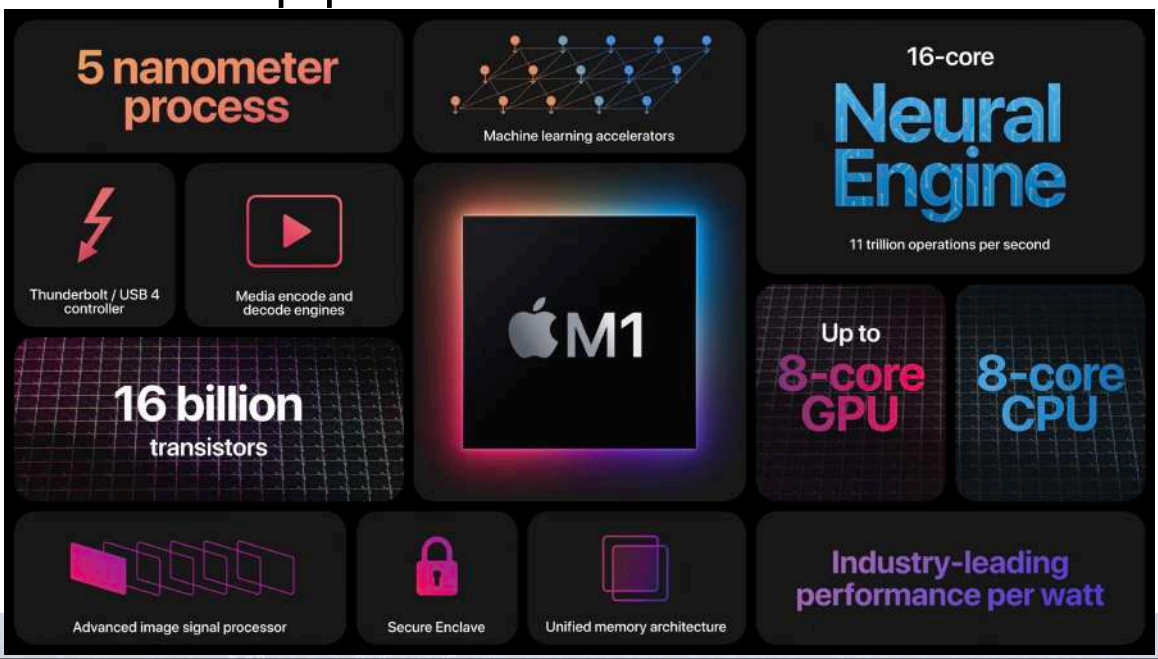
AMD Renoir (Zen2)

AMD Cezanne (Zen3)

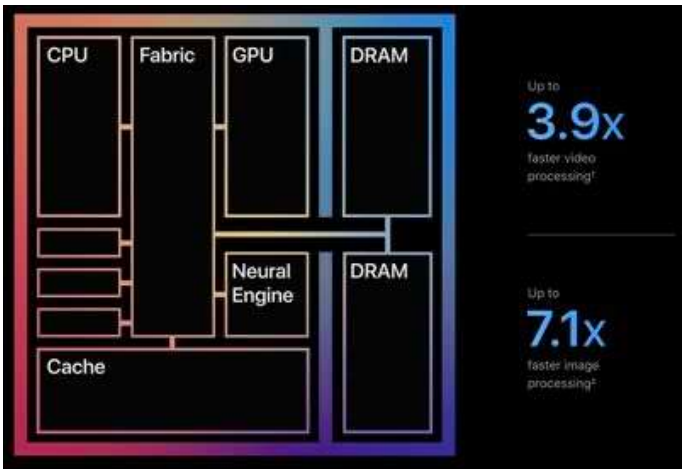


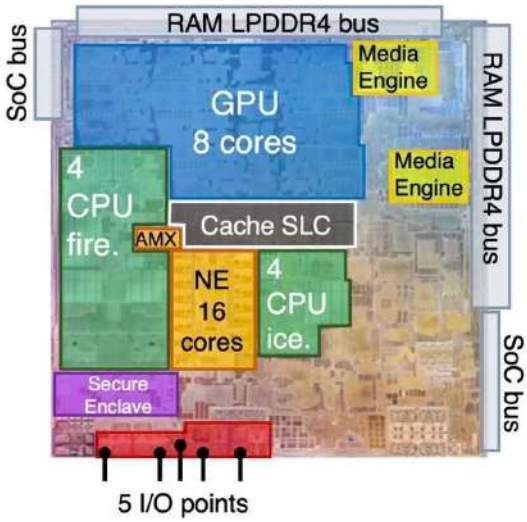
AMD Ryzen Zen2/Zen3 Architectures

Apple M1 Processor



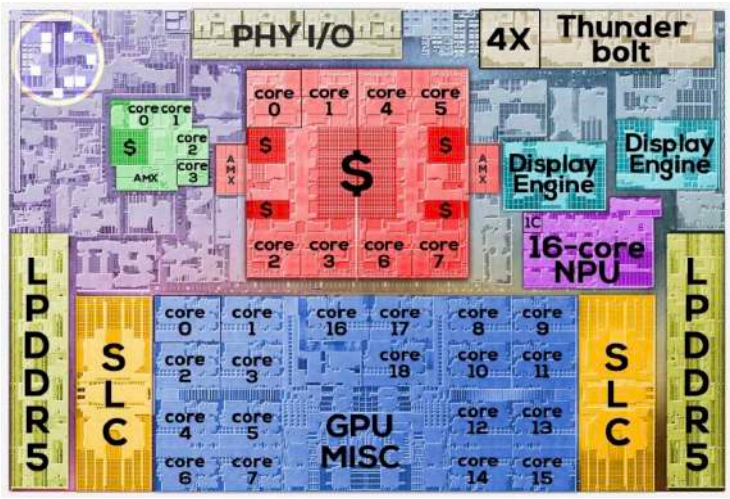
Apple M1 Architecture

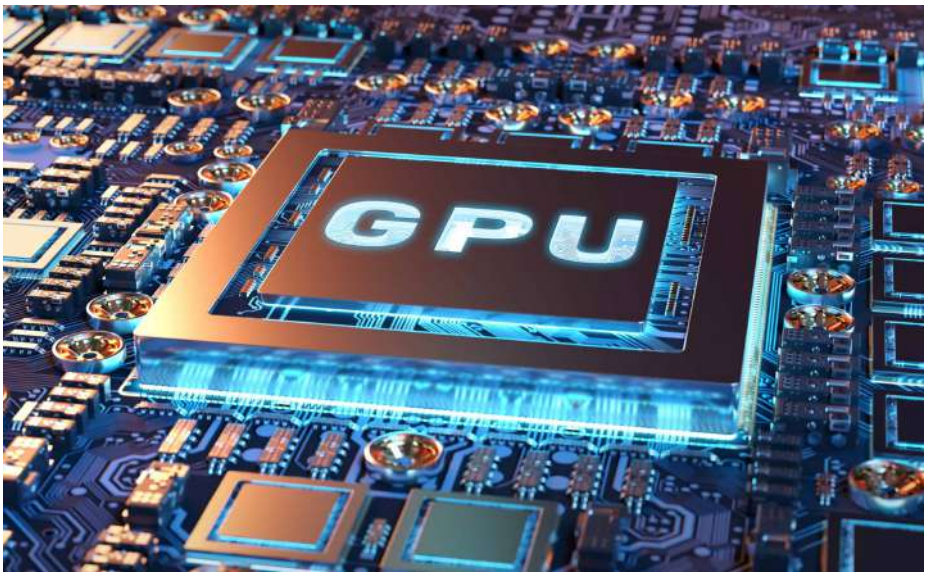




Apple M2 Processor

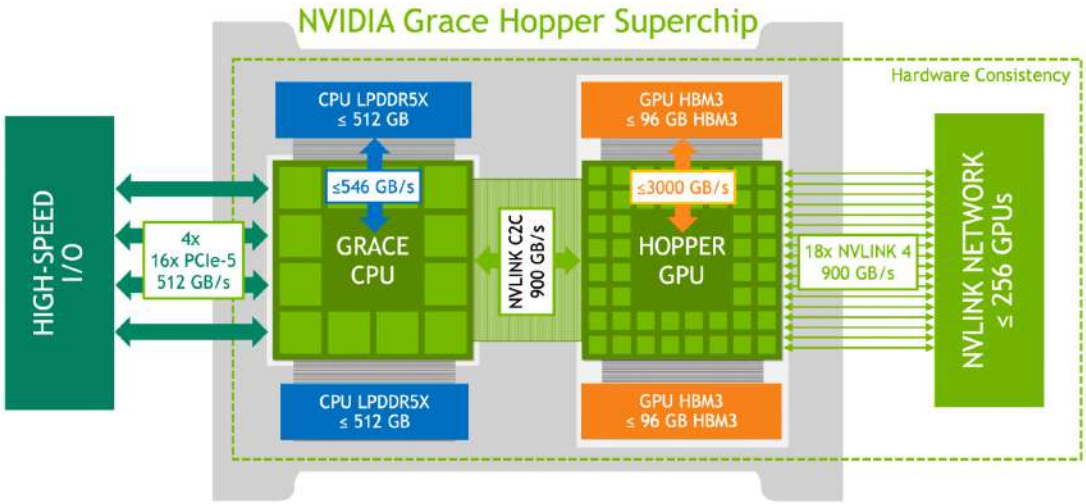
Apple M2 Architecture





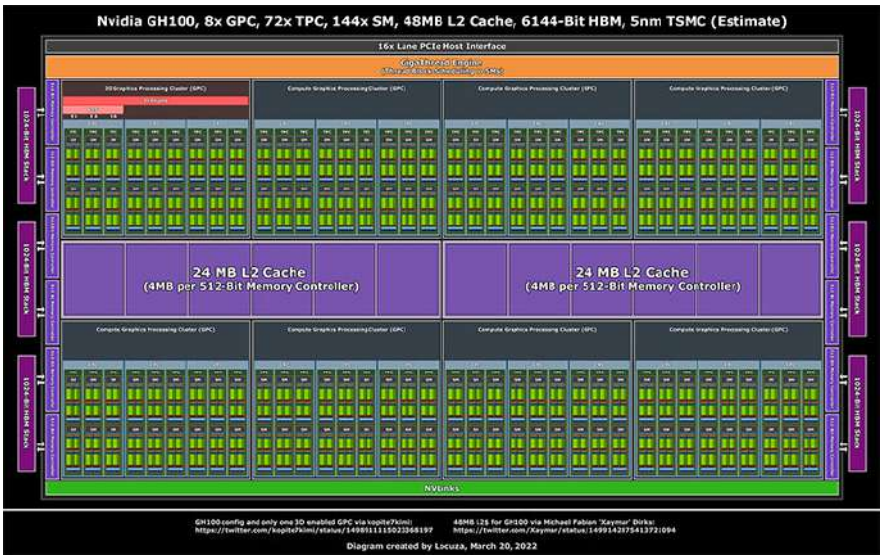
GPU Architectures

NVidia Grace/Hoper Superchip

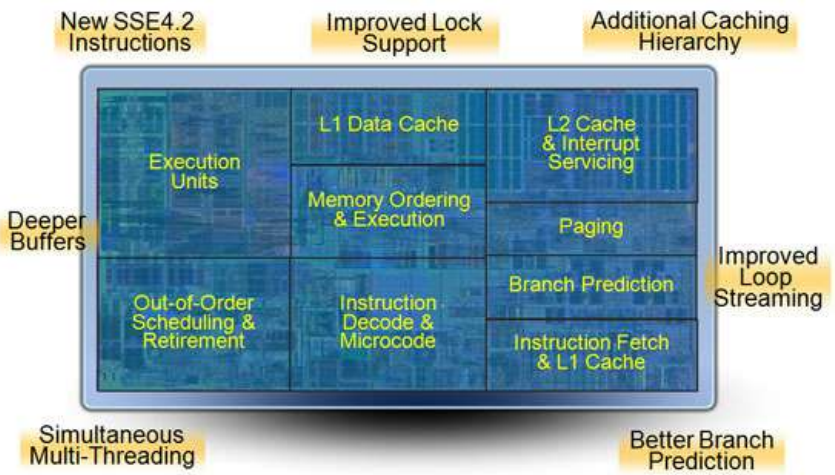


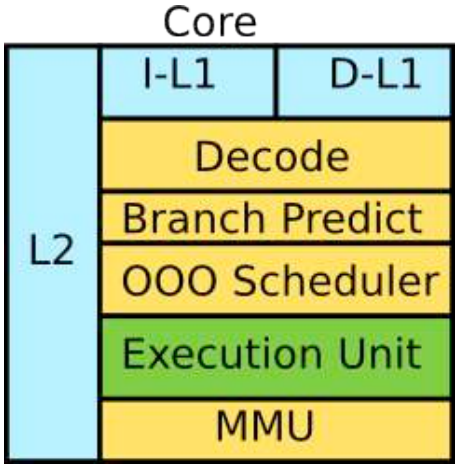
NVidia 144-core Grace CPU (Arm v9)

NVidia Hopper GPU



Let's Zoom(in)to an IA Core

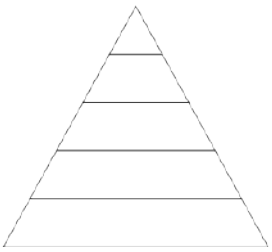




Less than **10%** of total chip area is used for the real execution.

Q: Why?

Revisiting the Memory Hierarchy



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

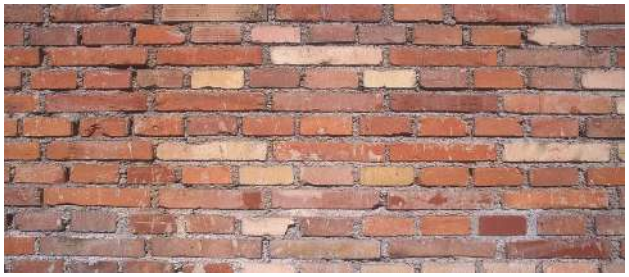
Notes on Energy at 45nm:
64-bit Int ADD takes about 1 pJ.
64-bit FP FMA takes about 200 pJ.

It seems we can not further increase the computational density!

The Brick Wall

Power Wall: power expensive, transistors “free”
Memory Wall: Memory slow, multiplies fast
ILP Wall: diminishing returns on more ILP HW

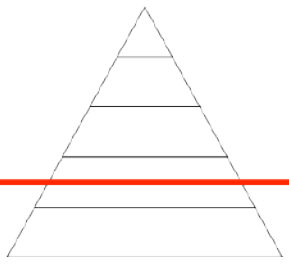
Power Wall + Memory Wall + ILP Wall = Brick Wall



Q: How to Break the Wall?
Exploit the parallelism inside the application…

Step 1: Trade Latency with Throughput

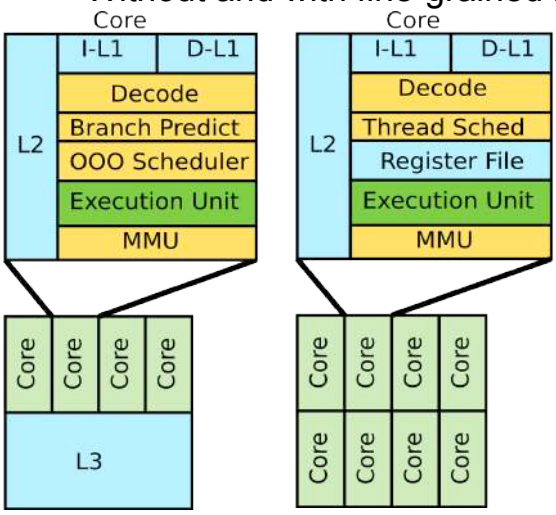
	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	~~1K~~ 16K	~~10~~ 20	~~1~~ 2~3	1000
L1	32K	20	5	100
L2	256K	100	10	100
~~L3~~	~~8M~~	~~200~~	~~50~~	~~100~~
Off-chip	4G	2000	100	10



Hide the memory latency through fine-grained interleaved threading

Fine-Grained Interleaved Threading

Without and with fine-grained interleaved threading



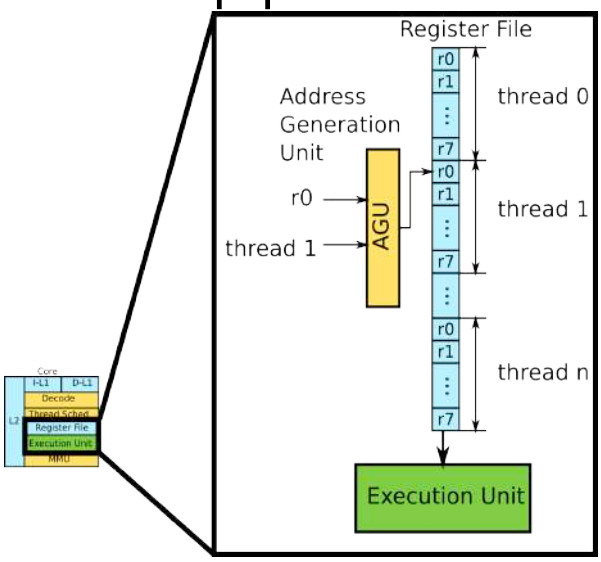
Pros:
Reduce cache size
No branch predictor
No OOO scheduler

Cons:
Register pressure
Thread scheduler
Requires huge parallelism

OOO = Out-of-order Execution

Hardware Support

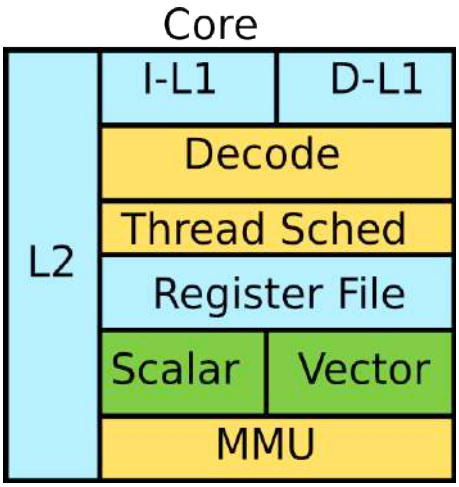
Register file supports zero overhead context switch between interleaved threads



Can we make Further Improvements?

- Reducing large cache gives 2x computational density.
- Q: Can we make further improvements?

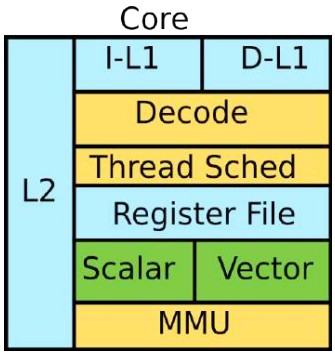
Hint:
We have only utilized thread level parallelism (TLP) so far.



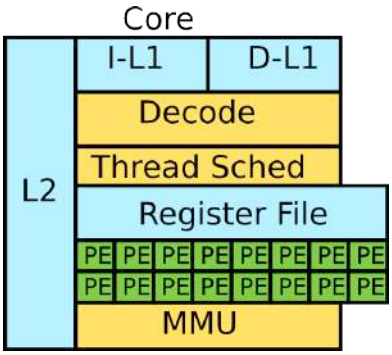
Step 2: Single Instruction Multiple Data

GPU uses wide SIMD: 8/16/24/... processing elements (PEs)
CPU uses short SIMD: usually has vector width of just 4

SSE has 4 data lanes

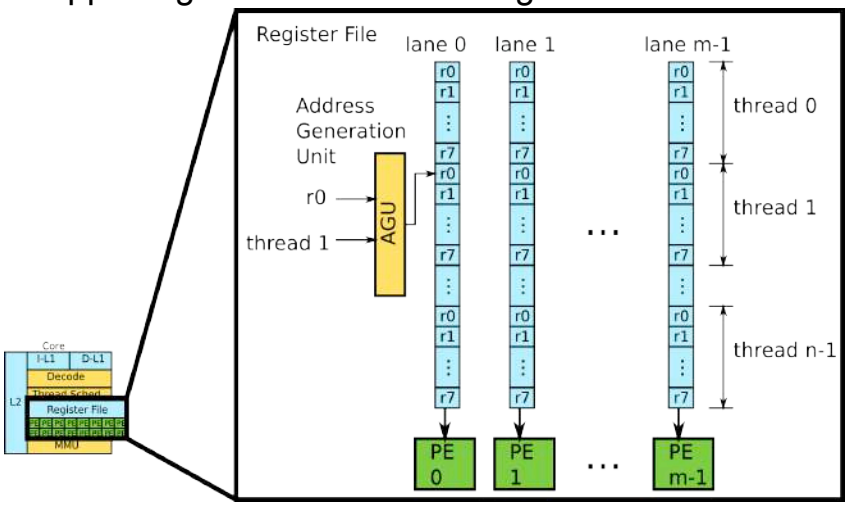


GPUs have 8/16/24/... data lanes



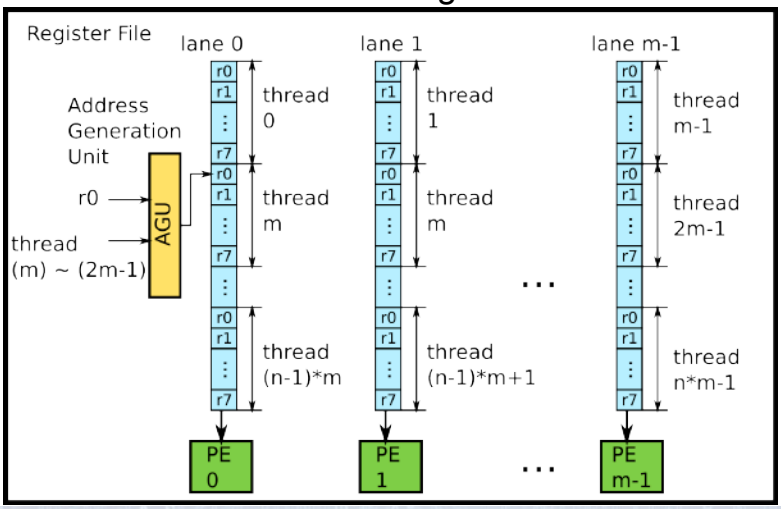
Hardware Support

Supporting interleaved threading + SIMD execution



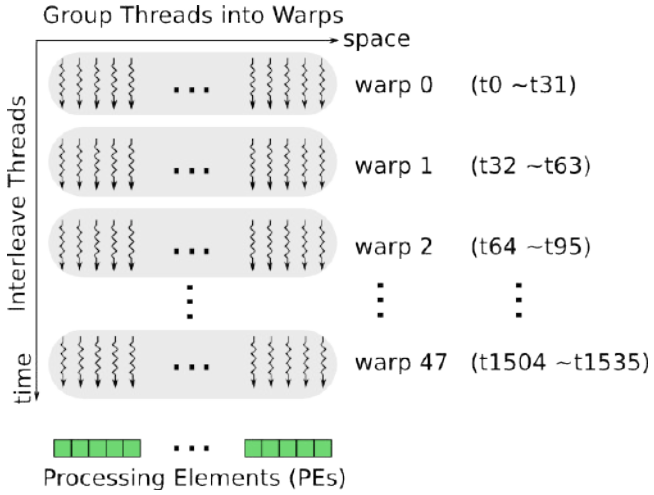
Single Instruction Multiple Thread (SIMT)

Hide vector width using scalar threads



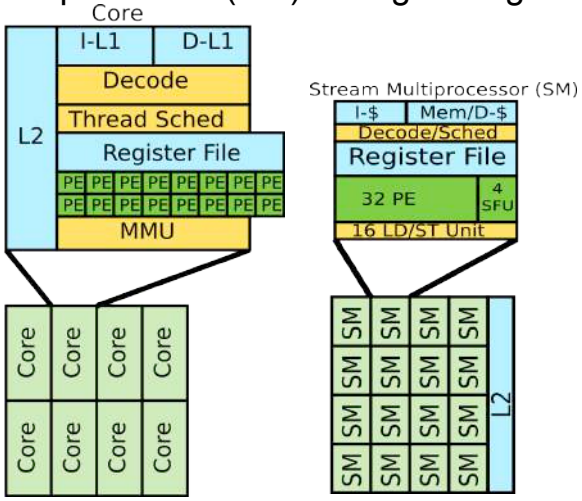
Example of SIMT Execution

Assume 32 threads are grouped into one warp.



Step 3: Simple Core

Stream Multiprocessor (SM) is a light-weight core compared to a “classical” IA core



Light weight PE:

Fused Multiply Add (FMA)

SFU:

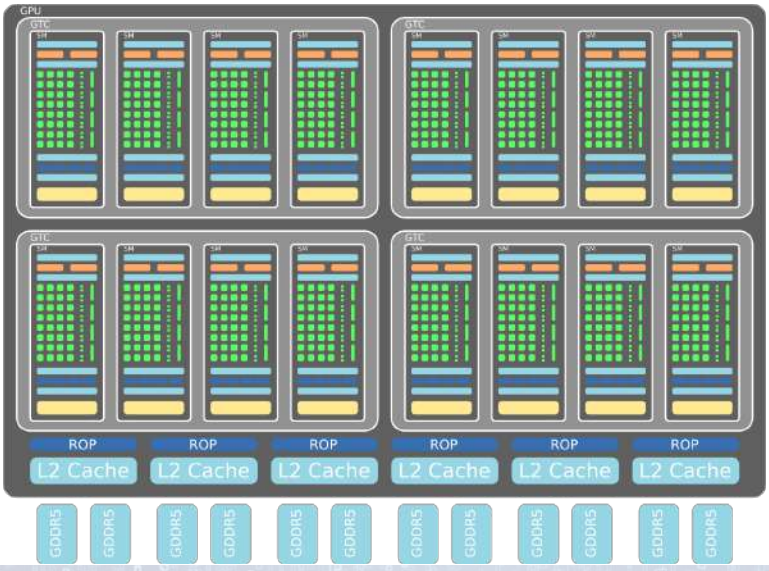
Special Function Unit

NVIDIA's Motivation of a Simple Core

“The [multiple IA-core] approach is analogous to trying to build an airplane by putting wings on a train."

Bill Dally, NVIDIA

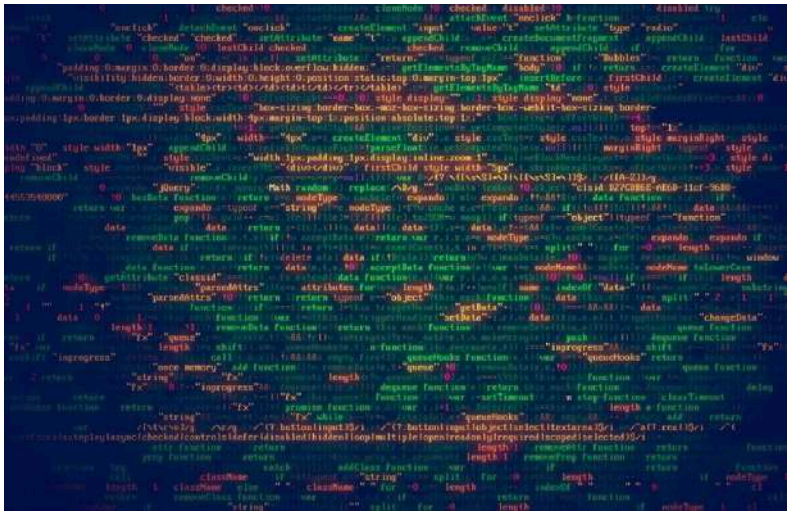
How Do We Reach Here?



NVIDIA Fermi, 512 Processing Elements (PEs)

Throughput Oriented Architectures

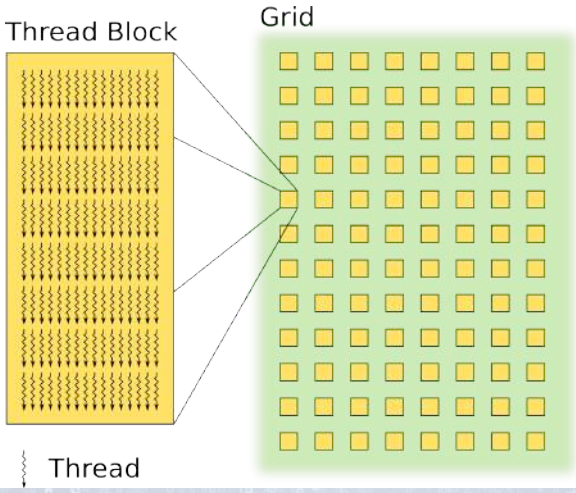
- Fine-grained interleaved threading (~2x comp density)
- SIMD/SIMT (>10x comp density)
- Simple core (~2x comp density)



Programming

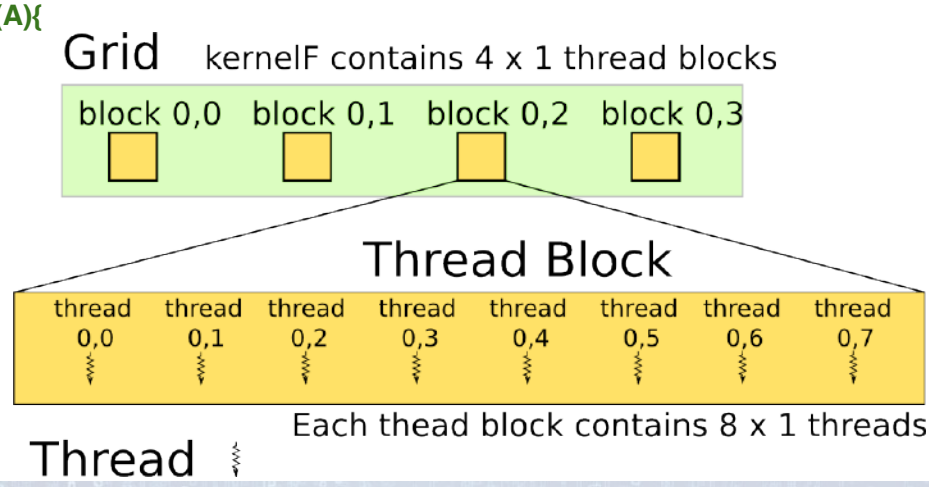
Multi-Threading at another Level

- Massive number (>10000) of **light-weight** threads



Two Levels of Thread Hierarchy

```
kernelF<<<(4,1),(8,1)>>>(A);

__device__   kernelF(A){
   i = blockIdx.x;
   j = threadIdx.x;
   A[i][j]++;
}
```

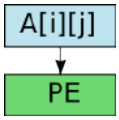


Vector Programming

- Vector width is exposed to programmers

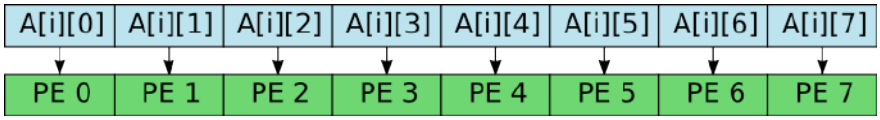
Scalar program

```
float A[4][8];
do-all(i=0;i<4;i++){
   do-all(j=0;j<8;j++){
      A[i][j]++;
   }
}
```



Vector program (vector width of 8)

```
float A[4][8];

do-all(i=0;i<4;i++){
   movups xmm0, [ &A[i][0] ]
   incps xmm0
   movups [ &A[i][0] ], xmm0
}
```



GPU Program

- Program expresses data level parallelism (DLP) in terms of thread level parallelism (TLP)
- Hardware converts TLP into DLP at run time

Scalar program

```
float A[4][8];
do-all(i=0;i<4;i++){
   do-all(j=0;j<8;j++){
      A[i][j]++;
   }
}
```

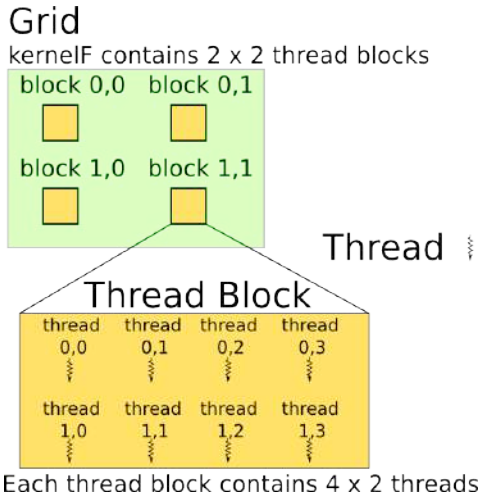
GPU program

```
float A[4][8];

kernelF<<<(4,1),(8,1)>>>(A);

__device__    kernelF(A){
   i = blockIdx.x;
   j = threadIdx.x;
   A[i][j]++;
}
```

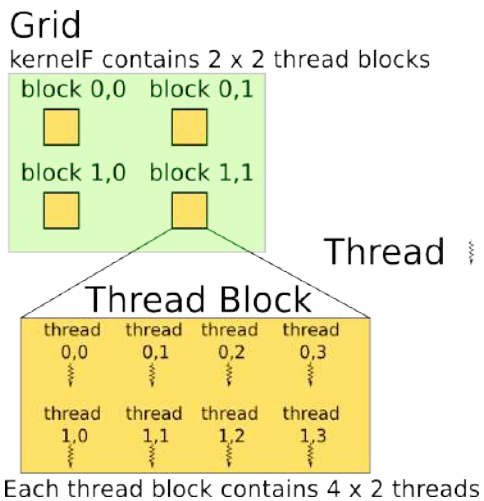
Multi-dimension Thread and Block ID

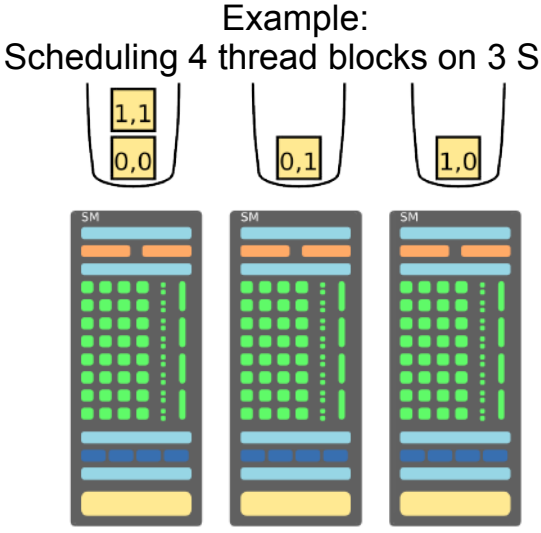
- Both grid and thread block can have two dimensional index

```
kernelF<<<(2,2),(4,2)>>>(A);

__device__   kernelF(A){
   i = blockDim.x * blockIdx.y
      + blockIdx.x;
   j = threadDim.x * threadIdx.y
      + threadIdx.x;
   A[i][j]++;
}
```



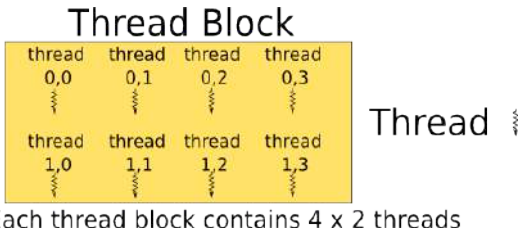
Scheduling Thread Blocks on SM



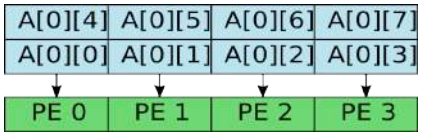


Executing Thread Block on SM

```
kernelF<<<(2,2),(4,2)>>>(A);

__device__   kernelF(A){
   i = blockDim.x * blockIdx.y
      + blockIdx.x;
   j = threadDim.x * threadIdx.y
      + threadIdx.x;
   A[i][j]++;
}
```

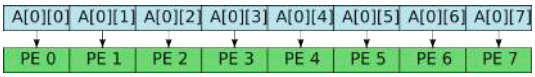


Executed on machine with width of 4:

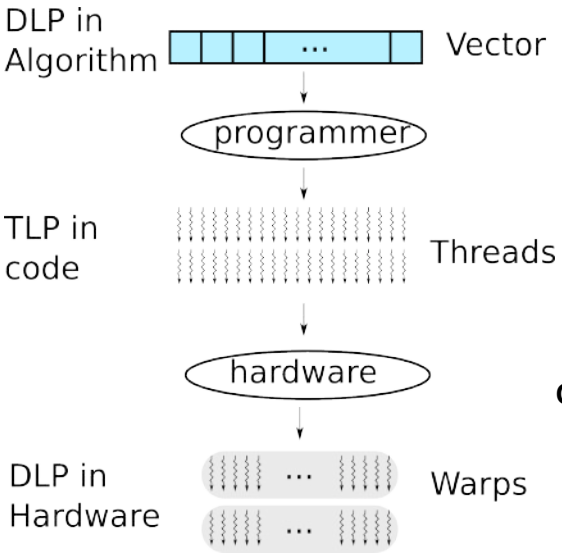


Note: The number of Processing Elements (PEs) is transparent to programmer

Executed on machine with width of 8:



Programmers Think in Threads



Q: Why make this hassle?

Why Use Thread instead of Vector?

- Thread **Pros**:
  - Portability: Machine width is transparent in ISA (Instruction Set Architecture)
  - Productivity: Programmers don't take care of vector width on processor arch
- Thread **Cons**:
  - Manual sync: Give up lock-step within vector
  - Scheduling of threads could be inefficient
  - Debug: "Threads considered harmful". Thread programming is notoriously hard to debug
- Programmers **explicitly express DLP in terms of TLP**
- Programmers **explicitly manage memory hierarchy**

Hierarchical Parallelism

- Parallel computations arranged as grids
- One grid executes after another
- Grid consists of blocks
- Blocks assigned to SM
  - A single block assigned to a single SM
  - Multiple blocks can be assigned to a SM
- Block consists of elements
- Elements computed by threads

Thread Blocks

- Thread block
  - Array of concurrent threads that execute the same program and can cooperate to compute the result
  - Consists of up to 1024 threads (K40) - hp-sl.q queues / 2048+ others
  - Has shape and dimensions (1d, 2d or 3d) for threads
- A thread ID has corresponding 1d, 2d or 3d indices
- Each SM executes up to 16 thread blocks concurrently (K40)
- Threads of a thread block share memory
- Maximum threads per SM: 2048 (K40)

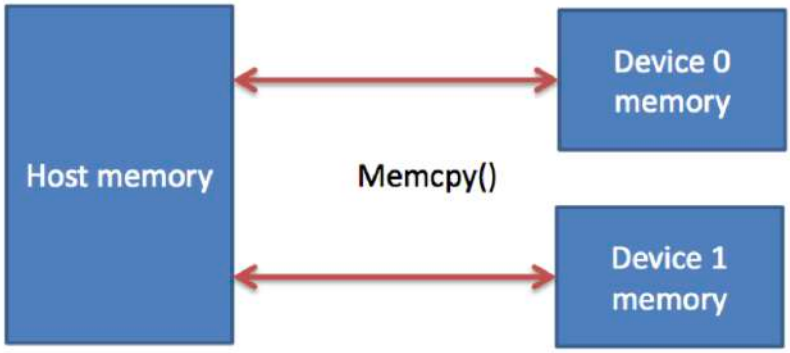
General GPU Programming Steps

1. Initialize the data on the CPU / space on the GPU
2. Copy data from CPU to GPU
3. Compute on GPU
4. Copy data back from GPU to CPU

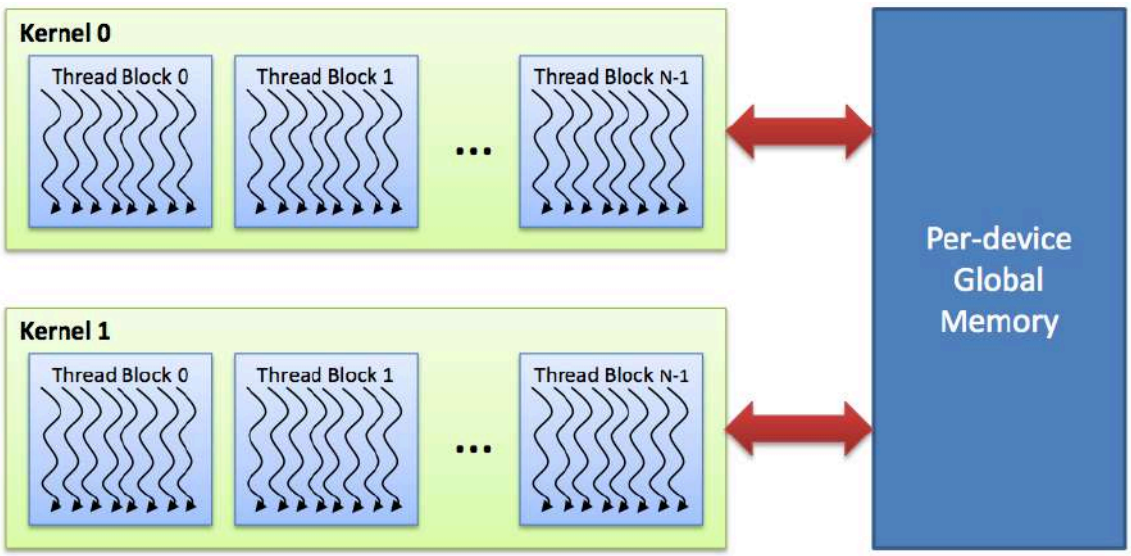
- By default, execution on host doesn't wait for kernel to finish
- General rules:
  - **Minimize** data transfer between CPU & GPU
  - **Maximize** number of threads on GPU

GPU Memory Hierarchy

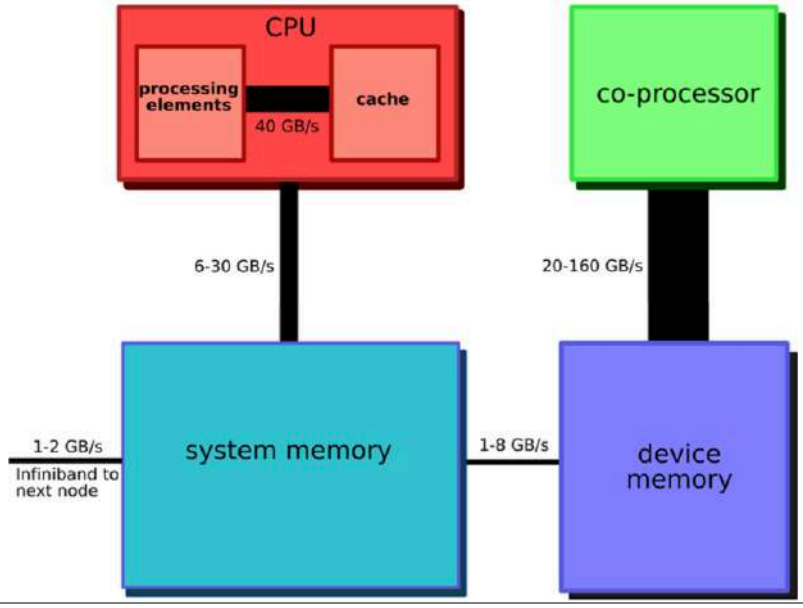
- Global (device) memory
  - Accessible by all threads as well as host (CPU) – Data lifetime is from allocation to deallocation



GPU Memory Hierarchy

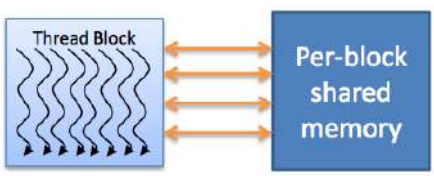


Bandwidth in a CPU-GPU System



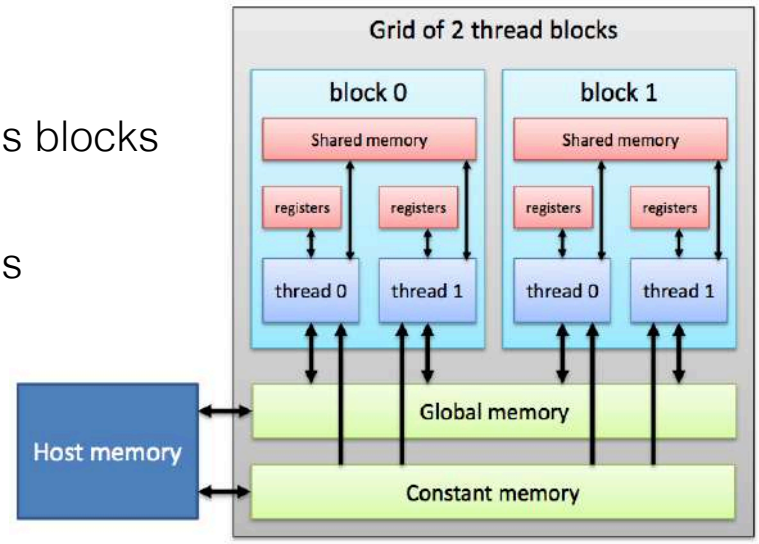
GPU Memory Hierarchy

- Local storage
  - Each thread has own its local storage
  - Mostly registers - managed by the compiler
  - Data lifetime = thread lifetime
- Shared memory
  - Each thread block has its own shared memory
    - Accessible only by threads within that block
  - Data lifetime = block lifetime

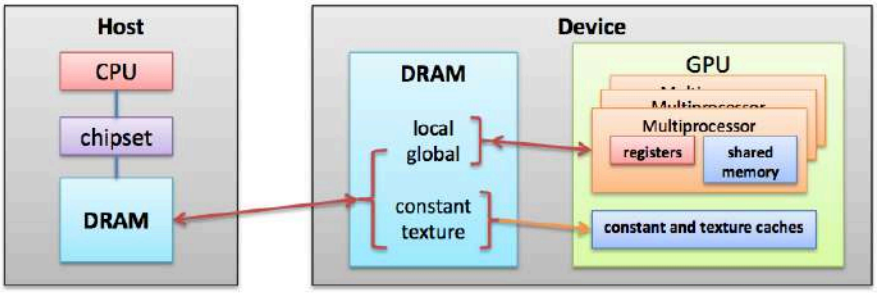


GPU Memory Hierarchy

- 1D grid
  - 2 threads blocks
- 1D block
  - 2 threads

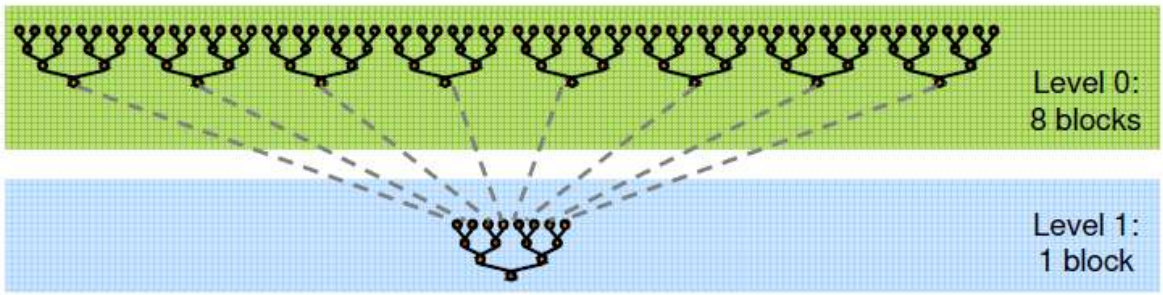


GPU Memory Hierarchy



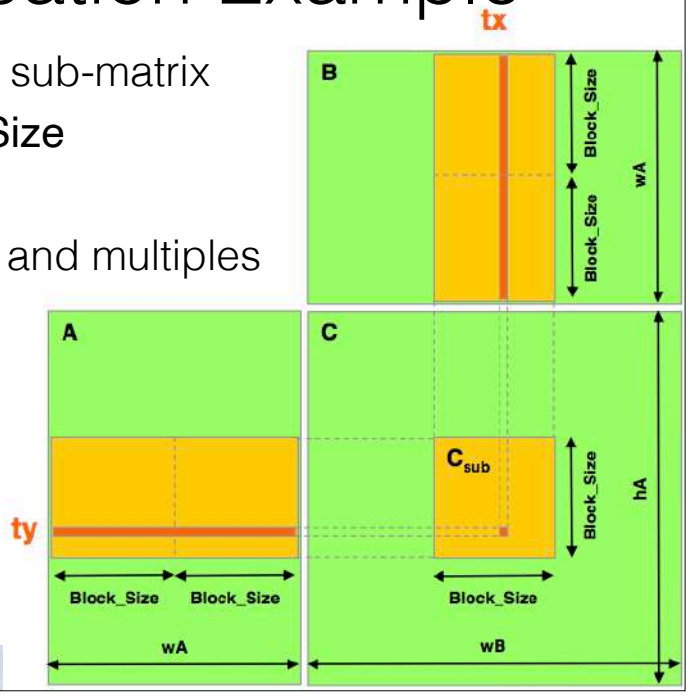
Memory	Location	Cached	Access	Scope	Lifetime
Register	On-chip	N/A	R/W	One thread	Thread
Local	Off-chip	No	R/W	One thread	Thread
Shared	On-chip	N/A	R/W	All threads in a block	Block
Global	Off-chip	No	R/W	All threads + host	Application
Constant	Off-chip	Yes	R	All threads + host	Application
Texture	Off-chip	Yes	R	All threads + host	Application

Reduction Example

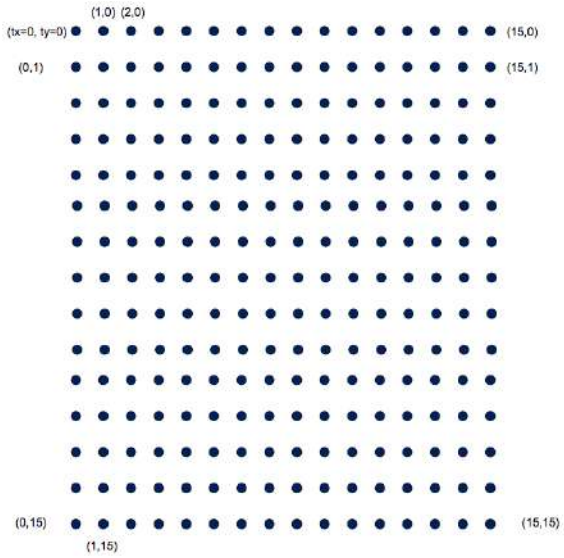


Matrix Multiplication Example

- One block computes one square sub-matrix C_{sub} of size **Block_Size x Block_Size**
- One thread one entry of C_{sub}
- Assume that of A & B are square and multiples of **Block_Size**
  - We will have **Block_Size=16x16**

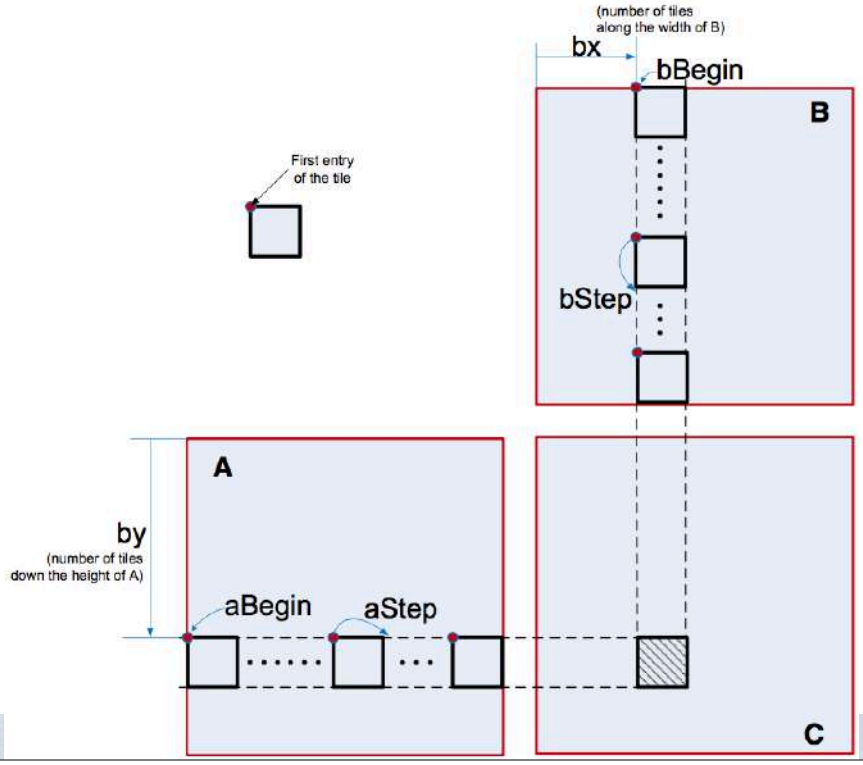


A Block of 16 X 16 Threads



Matrix Multiplication - Host Code

```
// Thread block size
#define BLOCK_SIZE 16

// Forward declaration of the device multiplication func.
__global__ void Muld(float*, float*, int, int, float*);

// Host multiplication function
// Compute C = A * B
// hA is the height of A
// wA is the width of A
// wB is the width of B
void Mul(const float* A, const float* B, int hA, int wA, int wB, float* C) {
 int size;

  // Load A and B to the device
 float* Ad;
 size = hA * wA * sizeof(float);
 cudaMalloc((void**)&Ad, size);
 cudaMemcpy(Ad, A, size, cudaMemcpyHostToDevice);

 float* Bd;
 size = wA * wB * sizeof(float);
 cudaMalloc((void**)&Bd, size);
 cudaMemcpy(Bd, B, size, cudaMemcpyHostToDevice);

  // Allocate C on the device
 float* Cd;
 size = hA * wB * sizeof(float);
 cudaMalloc((void**)&Cd, size);

 // Compute the execution configuration assuming
 // the matrix dimensions are multiples of BLOCK_SIZE
 dim3 dimBlock (BLOCK_SIZE, BLOCK_SIZE);
 dim3 dimGrid (wB/dimBlock.x, hA/dimBlock.y);

  // Launch the device computation
 Muld<<<dimGrid, dimBlock>>>(Ad, Bd, wA, wB, Cd);

  // Read C from the device
 cudaMemcpy(C, Cd, size, cudaMemcpyDeviceToHost);

  // Free device memory
 cudaFree(Ad);
 cudaFree(Bd);
 cudaFree(Cd);
}
```



Matrix Multiplication - Device Code

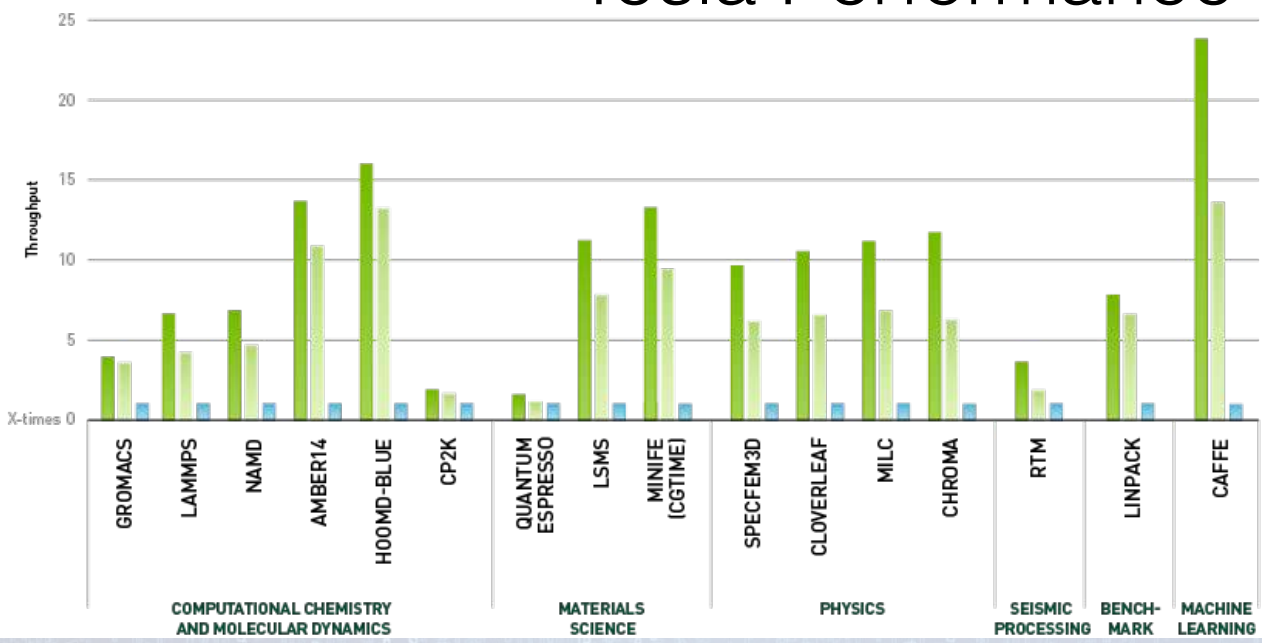
```
// Device multiplication function called by Mul()
// Compute C = A * B
// wA is the width of A
// wB is the width of B
__global__ void Muld(float* A, float* B, int wA, int wB, float* C){

  // Block index
 int bx = blockIdx.x; // the B & C matrix sub-block column index
 int by = blockIdx.y; // the A & C matrix sub-block row index

  // Thread index
 int tx = threadIdx.x; // the column index in the sub-block
 int ty = threadIdy.y; // the row index in the sub-block

  // Index of the first sub-matrix of A processed by the block
 int aBegin = wA * BLOCK_SIZE * by;
  // Index of the last sub-matrix of A processed by the block
 int aEnd = aBegin + wA - 1;
  // Step size used to iterate through the sub-matrices of A
 int aStep = BLOCK_SIZE;
  // Index of the first sub-matrix of B processed by the block
 int bBegin = BLOCK_SIZE * bx;
  // Step size used to iterate through the sub-matrices of B
 int bStep = BLOCK_SIZE * wB;
  // The element of the block sub-matrix that computed by the thread
 float Csub = 0;
  // Shared memory for the sub-matrix of A
 __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
  // Shared memory for the sub-matrix of B
 __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];
 // Loop over all the sub-matrices of A and B required to compute
 // the block sub-matrix
 for (int a = aBegin, b = bBegin;
  a <= aEnd;
  a += aStep, b += bStep){
     // Load the matrices from global memory to shared memory;
     // each thread loads one element of each matrix
  As[ty][tx] = A[a + wA * ty + tx];
  Bs[ty][tx] = B[b + wB * ty + tx];
     // Synchronize to make sure the matrices are loaded
  __syncthreads();
     // Multiply the two matrices together; each thread computes
     // one element of the block sub-matrix
  for (int k = 0; k < BLOCK_SIZE; ++k)
    Csub += As[ty][k] * Bs[k][tx];
     // Synch to make sure the preceding computation is done before
     // loading two new sub-matrices of A and B in the next iteration
  __syncthreads();
  }
     // Write the block sub-matrix to global memory;
     // each thread writes one element
  int c = wB * BLOCK_SIZE * by + BLOCK_SIZE * bx;
  C[c + wB * ty + tx] = Csub;
}
```

Further Matrix-Multiply Performance

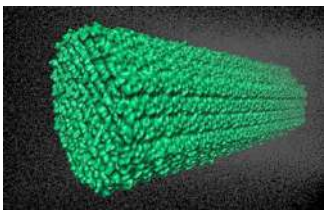
- Different implementations
  - Reference **kij**-ordered version on a 3GHz Xeon SC is 1.7 GFLOPS
  - V1 SIMD 32 i loop: 27 GFLOPS
  - V2 SIMD 32 k loop + Local Memory: 44 GFLOPS
  - V3 unroll 2 values from i loop: 43 GFLOPS
  - V4 unroll 2 values from j loop: 169 GFLOPS
  - V3+V4: 173 GFLOPS
- The bottom line
  - **Relatively easy to get something to run on GPUs**
  - **Difficult to get it to run fast…** (e.g. algorithms, memory, I/O, etc.)

Performance

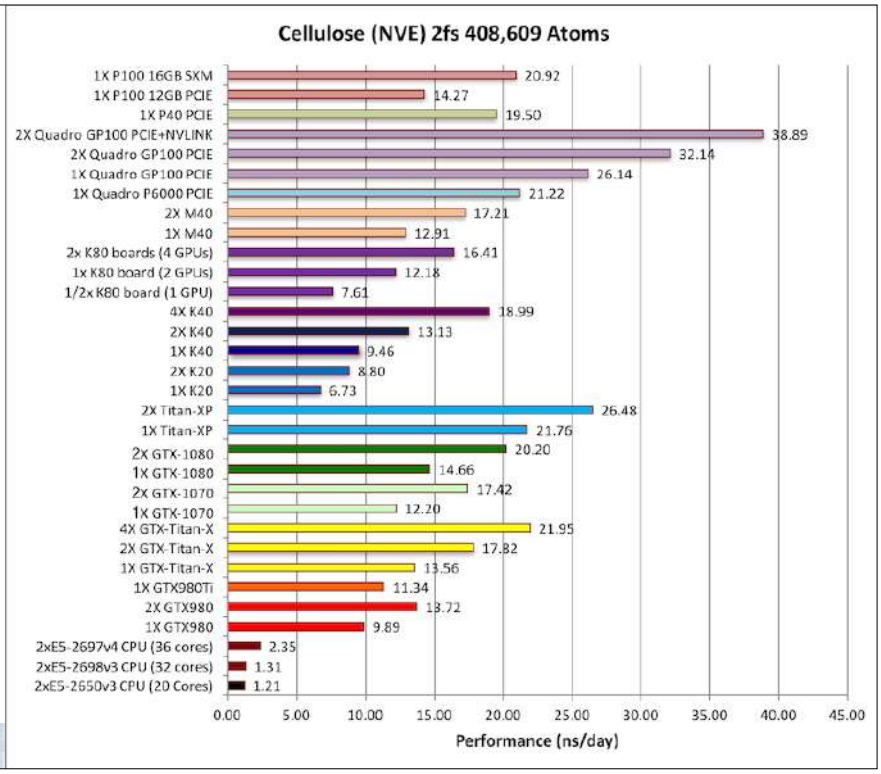
Tesla Performance



Amber Molecular Simulations



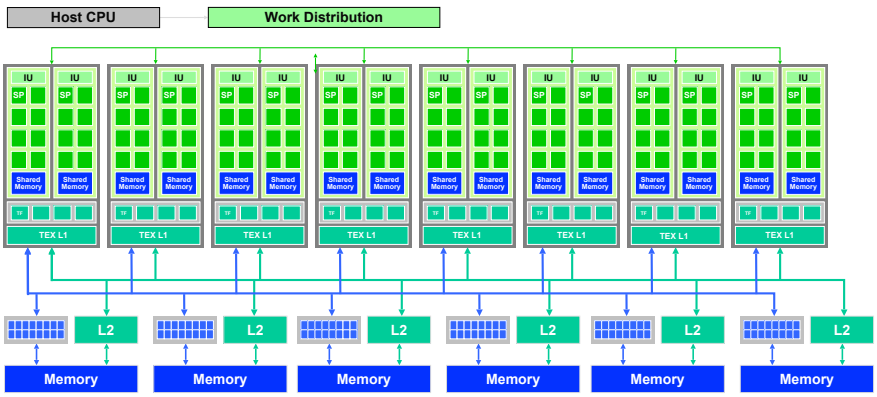
Cellulose



Going Forward

The Future of Computing is Parallel

- CPU clock rate growth is ~slowing, future computing growth will be from parallelism
- GPU Accelerators are a massively parallel computing platforms
  - 12,288 concurrent threads, hardware managed
  - 128 SP Thread Processor cores at 1.35 GHz == 518 GFLOPS peak



Implications and Opportunities

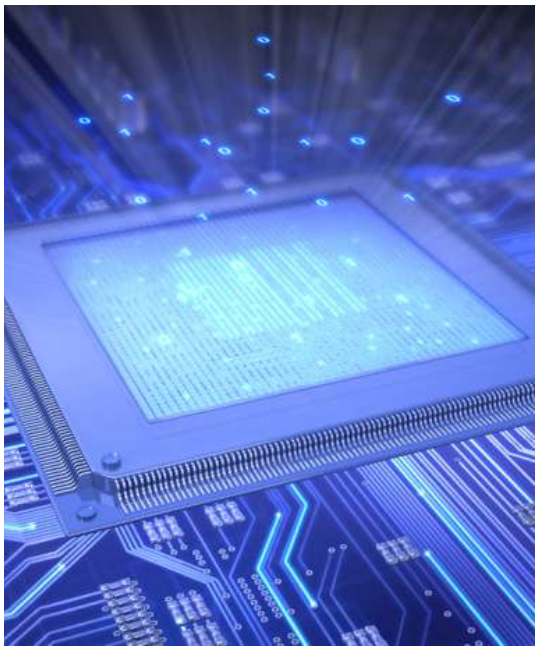
- Massively parallel computing allows
  - Drastic reduction in “time to discovery”
  - The new 3rd paradigm for research: computational experimentation
  - The “democratization of supercomputing”
    - $1,000/Teraflop in personal computers today
    - $1,000,000/Petaflops in clusters today
    - HW cost will no longer be the main barrier for big science
    - Global competition will be won with abilities to create and use parallel systems for discovery
- Future winner academic institutions will be leaders in
  - Research in Parallel Programming and Parallel Architecture
  - Teaching massively parallel programming to CS/EE students & scientists

Ce am invatat in aces curs?

- The Roofline Model
- GPU Architecture
- GPU Programming
- GPU Performance
- Going Forward

Slides & Support Material Acknowledgements

Elsevier Inc. 2012
Ye Mesman Corporaal - 2010
Sathish Vadhiyar - 2009
David Patterson - Computer Architecture is Back 2007
Michael Garland. David B. Kirk, "Understanding throughput-oriented architectures", CACM 2010
David Kirk - NVidia 2014-2021
Dan Negrut & Dan Melanz & Andrew Seidl - 2013
Michael Wolfe - Portland Compiler Group - 2009
Robert Strzodka - Max Plank Institute - 2011
Cache-Aware Roofline Model - 2018



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 8
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

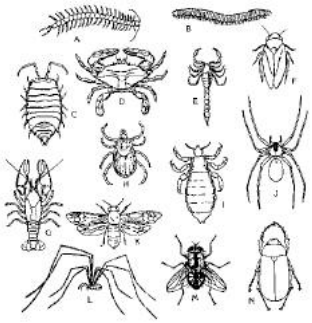
Cuprins

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Debugging and Performance Evaluation

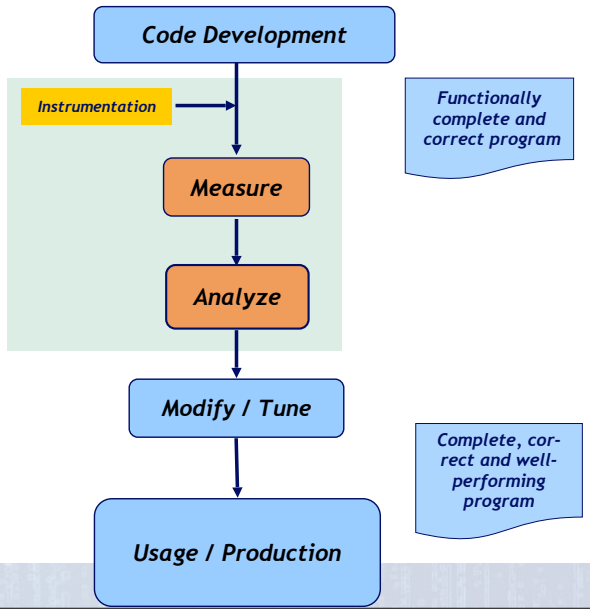
Debugging and Performance Evaluation

- Common errors in parallel programs
- Debugging tools
- Overview of benchmarking and performance measurements

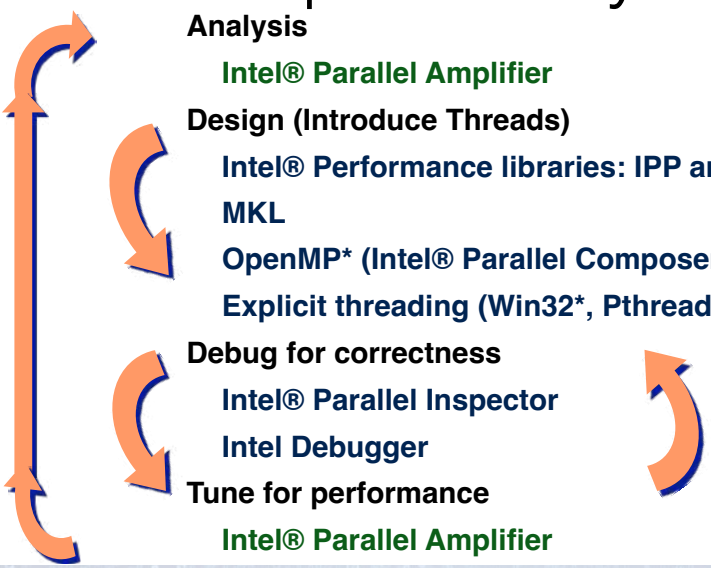


Concepts and Definitions

- The typical performance optimization cycle



Development Cycle



Parallel Code Development

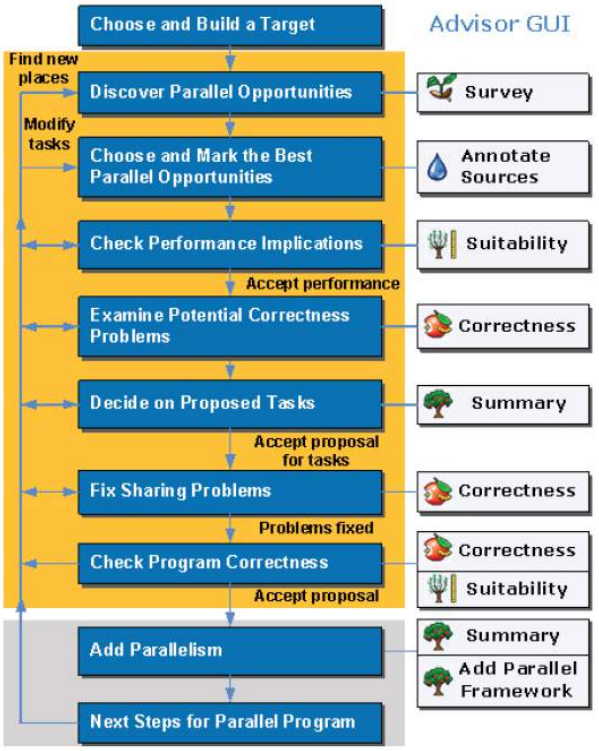
- Decide where to add the parallelism
  - Analyze the serial program
  - Prepare it for parallelism
- Add the parallelism
  - Threads, CUDA, OpenMP, Cilk, TBB, etc.
- Find logic problems
  - Only fails sometimes
  - Place of failure changes
- Find performance problems

Workflow

- Transforming many serial algorithms into parallel form takes five easy high-level steps
- Often existing algorithms are over-constrained by serial language semantics, and the underlying mathematics has a natural parallel expression if you can just find it

Advisor Overview

- If you look at these steps in more detail, you find decisions you will need to make
- You do not have to choose the perfect answer the first time, so you can go back and modify your choices

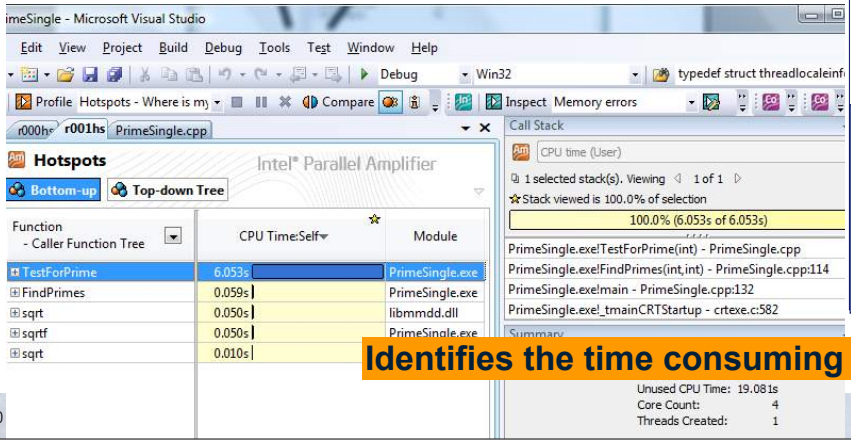


Hotspot Analysis

- Use Parallel Amplifier to find hotspots in application

```
bool TestForPrime(int val)
{   // let's start checking from 3
    int limit, factor = 3;
    limit = (long)(sqrtf((float)val)+0.5f);
    while( (factor <= limit) && (val % factor))
            factor ++;

    return (factor > limit);
}
```

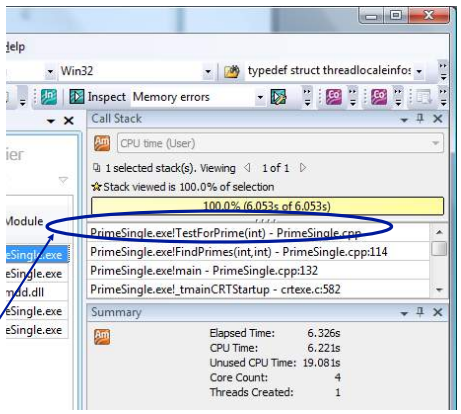
```
void FindPrimes(int start, int end)
{
    // start is always odd
    int range = end - start + 1;
    for( int i = start; i <= end; i+= 2 ){
        if( TestForPrime(i) )
            globalPrimes[gPrimesFound++] = i;
        ShowProgress(i, range);
    }
}
```



Identifies the time consuming regions

Analysis - Call Stack

- Inspect the code for the leaf node
- Look for a loop to parallelize
  - If none are found, progress up the call stack until you find a suitable loop or function call to parallelize (FindPrimes)

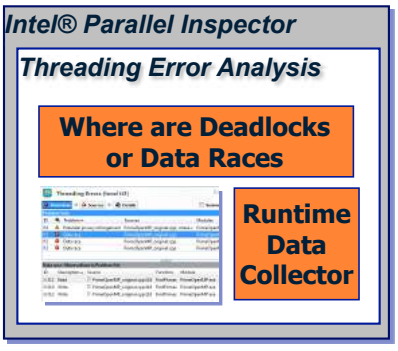


This is the level in the call tree where we need to thread

Used to find proper level in the call-tree to thread

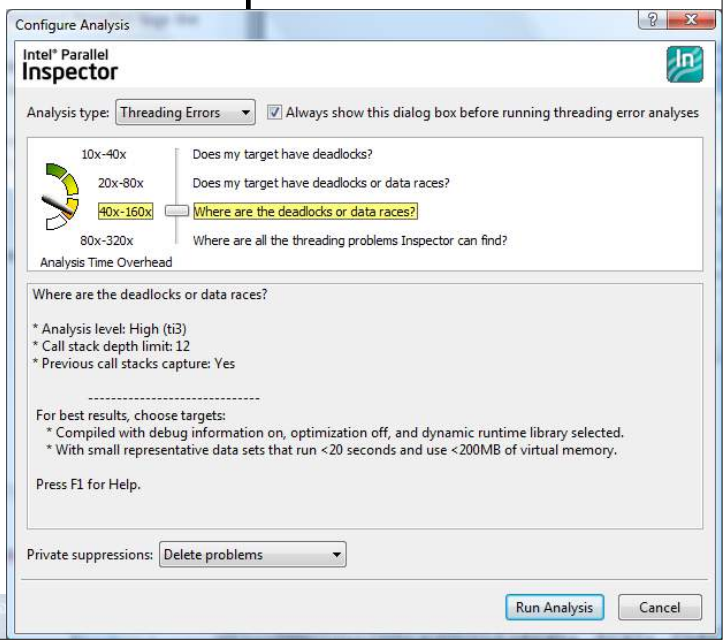
Debugging for Correctness

- Intel® Parallel Inspector pinpoints notorious threading bugs like data races and deadlocks



Intel® Parallel Inspector

- Select info regarding both data races & deadlocks
- View the Overview for Threading Errors
- Select a threading error and inspect the code



Motivation

- Developing threaded applications can be a complex task
- New class of problems are caused by the interaction between concurrent threads
  - Data races or storage conflicts
    - More than one thread accesses memory without synchronization
  - Deadlocks
    - Thread waits for an event that will never happen

Debugging Parallel Codes

- Debugging tool for threaded software
  - Plug-in to Microsoft* Visual Studio*
- Finds threading bugs in OpenMP*, Intel® Threading Building Blocks, and Win32* threaded software
- Locates bugs quickly that can take days to find using traditional methods and tools
  - Isolates **problems**, not the **symptoms**
  - **Bug does not have to occur to find it!**

Parallel Inspector: Analysis

- Dynamic as software runs
  - Data (workload)-driven execution
- Includes monitoring of:
  - Thread and Sync APIs used
  - Thread execution order
    - Scheduler impacts results
  - Memory accesses between threads

Code path must be executed to be analyzed

Parallel Inspector: Before You Start

- Instrumentation: background
  - Adds calls to library to record information
    - Thread and Sync APIs
    - Memory accesses
  - Increases execution **time** and **size**
- Use **small** data sets (workloads)
  - Execution time and space is **expanded**
  - Multiple runs over different paths yield best results

Workload selection is important!

Workload Guidelines

- Execute problem code once per thread to be identified
- Use smallest possible working data set
  - Minimize data set size
    - Smaller image sizes
  - Minimize loop iterations or time steps
    - Simulate minutes rather than days
  - Minimize update rates
    - Lower frames per second

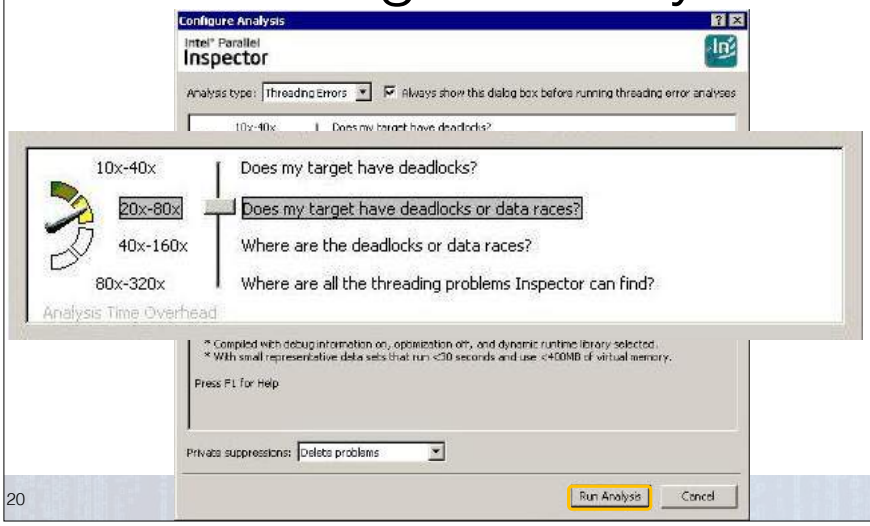
Finds threading errors faster!

Binary Instrumentation

- Build with supported compiler
- Running the application
  - Must be run from within Parallel Inspector
  - Application is instrumented when executed
  - External DLLs are instrumented as used

Parallel Inspector Startup

- The Configure Analysis window pops up

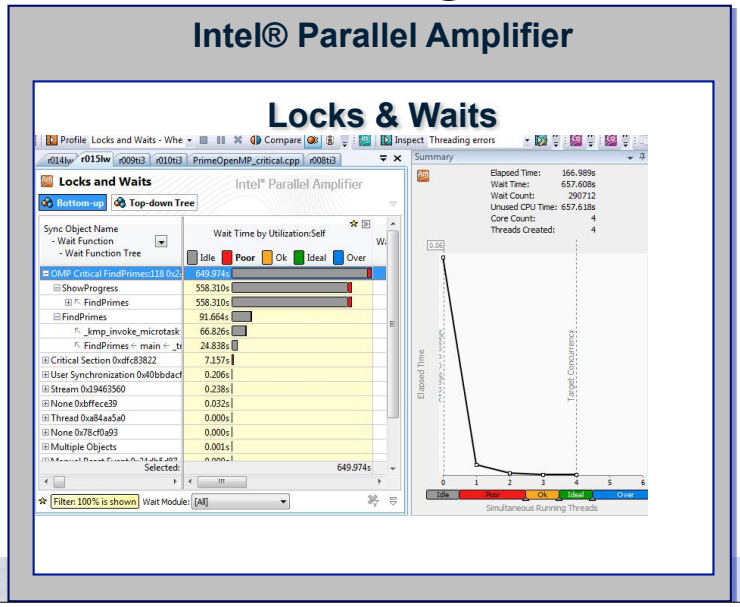


Select the level of analysis to be carried out by Parallel Inspector

The deeper the analysis, the more thorough the results and the longer the execution time

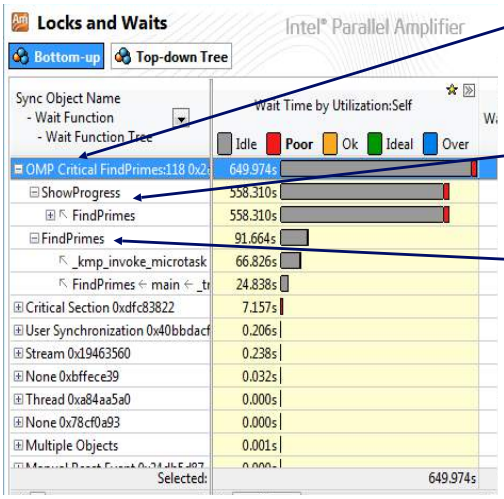
Click Run Analysis

Tuning for Performance



- Parallel Amplifier (Locks & Waits) pinpoints performance bottlenecks in threaded applications

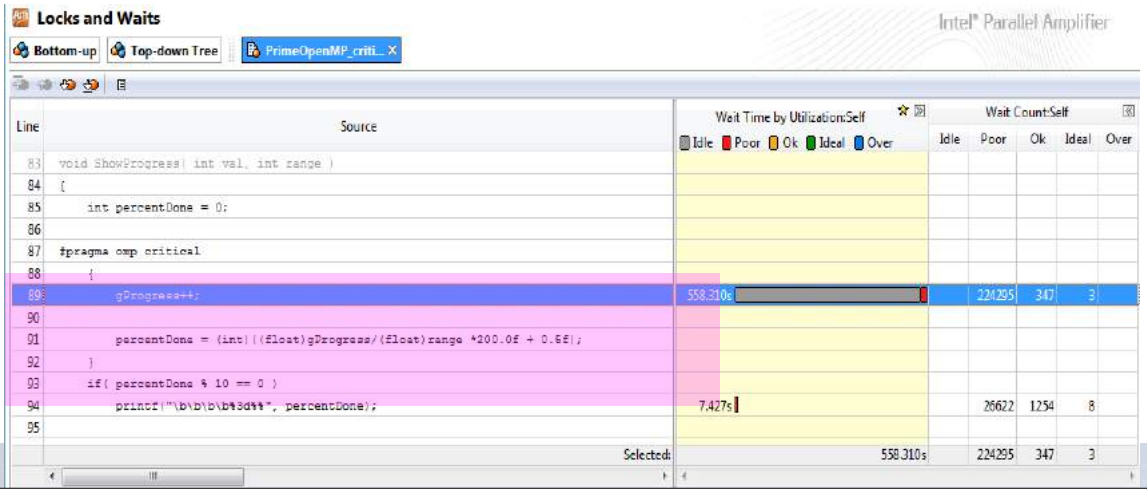
Parallel Amplifier - Locks & Waits



- Graph shows significant portion of time in idle condition as result of a critical section
- FindPrimes() & ShowProgress() are both excessively impacted by the idle time occurring in the critical section

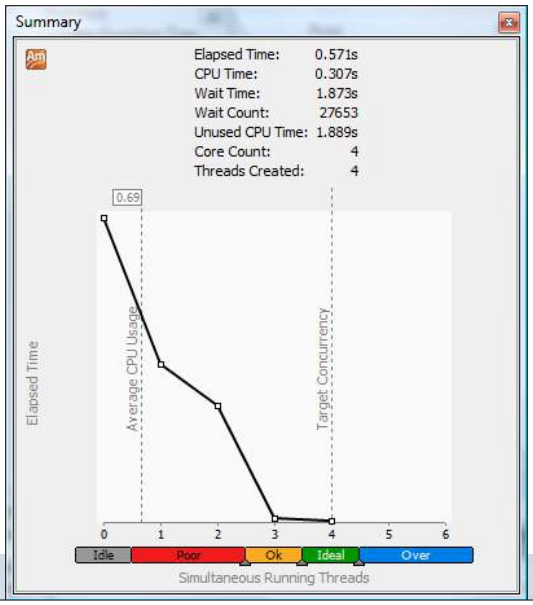
Parallel Amplifier - Locks & Waits

- ShowProgress() consumes 558/657 (85%) of the time idling in a critical section
- Double Click ShowProgress() in largest critical section to see the source code



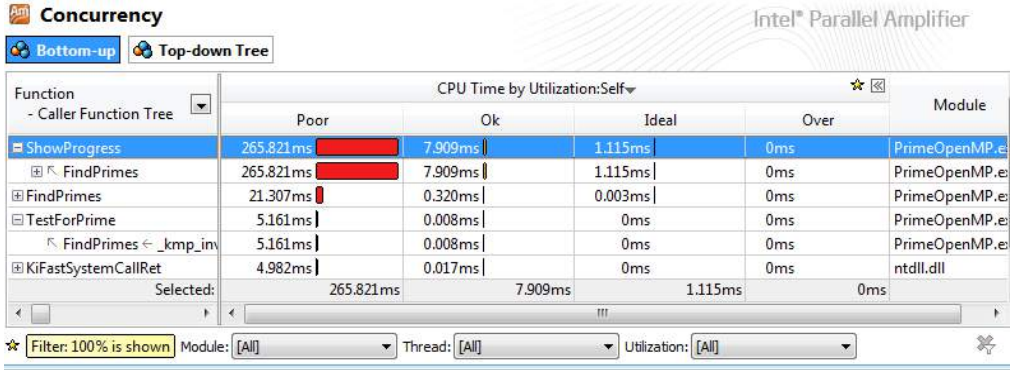
Parallel Amplifier Summary

- Elapsed Time shows 0.571 sec
- Wait Time/ Core Count:
  - 1.87/4 =0.47 sec
- Waiting 82% of elapsed time in critical section
- Most of the time 1 core and occasionally 2 are occupied



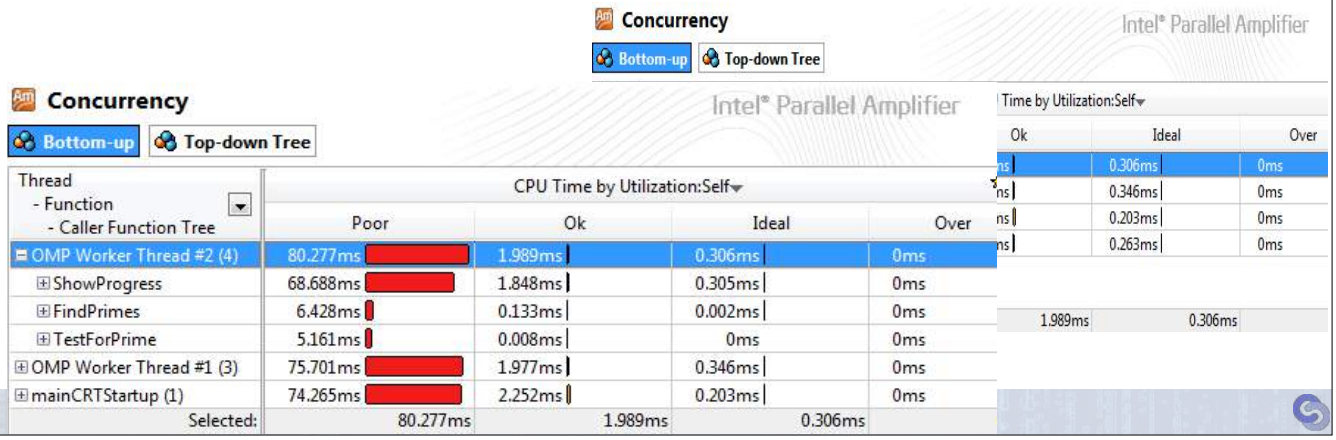
Parallel Amplifier - Concurrency

- Function - Caller Function Tree
- ShowProgress is called from FindPrimes and represent the biggest reason concurrency is poor



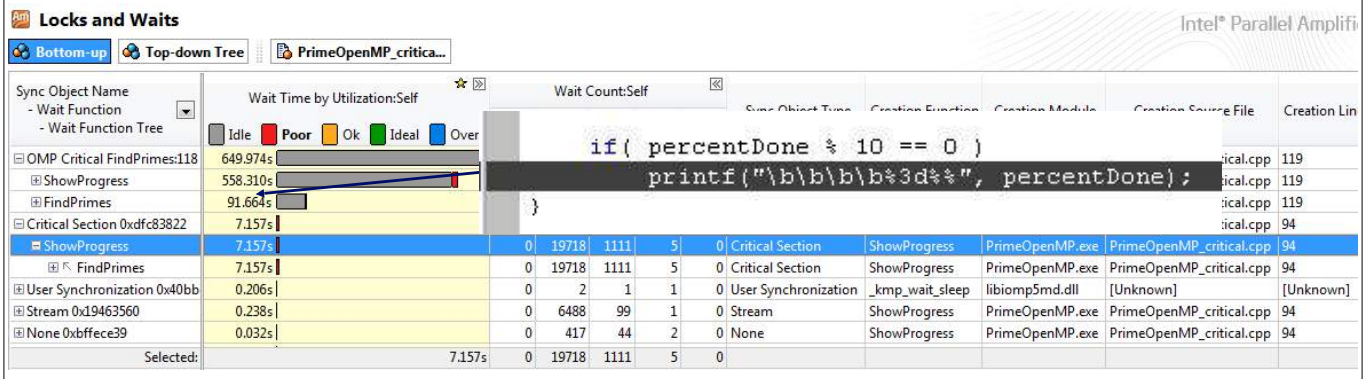
Parallel Amplifier - Concurrency

- Thread –Function –Caller Function Tree
  - This view shows how each thread contributes to the concurrency issue
- Expanding any thread will reveal the functions that contribute most



Performance

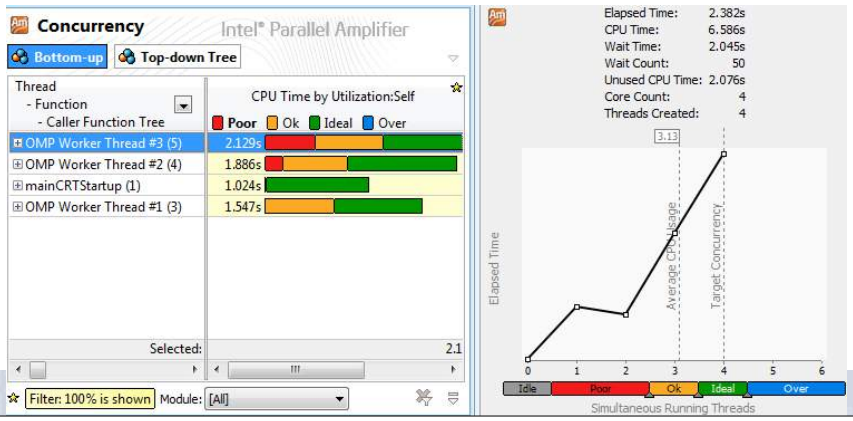
- Double Click ShowProgress in second largest critical section
- This implementation has implicit synchronization calls - printf
- This limits scaling performance due to the resulting context switches



Back to the design stage

Load Balance Analysis

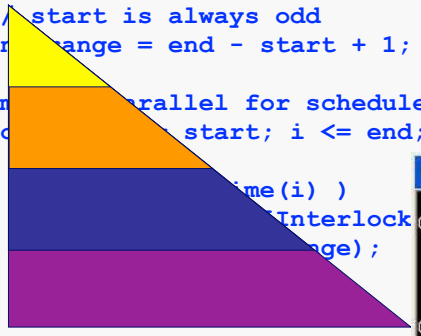
- Use Parallel Amplifier – Concurrency Analysis
- Select the “Thread – Function – Caller Function Tree”
- Observe that the 4 threads do unequal amounts of work



Fixing the Load Imbalance

- Distribute the work more evenly

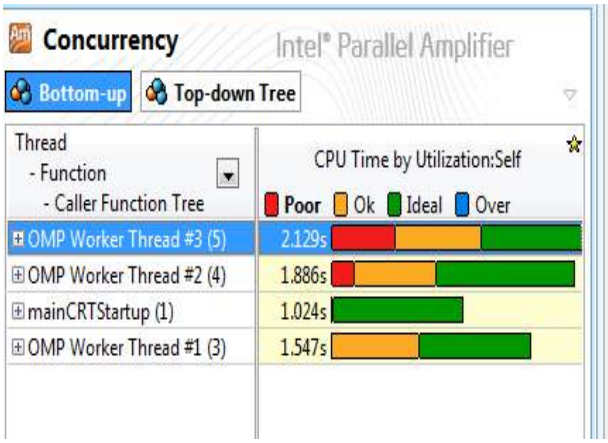
```
void FindPrimes(int start, int end)
{
    // start is always odd
    int range = end - start + 1;

#pragma parallel for schedule(sta
    for start; i <= end; i +=
    {
        ime(i) )
        Interlock
        nge);
    }
}
```

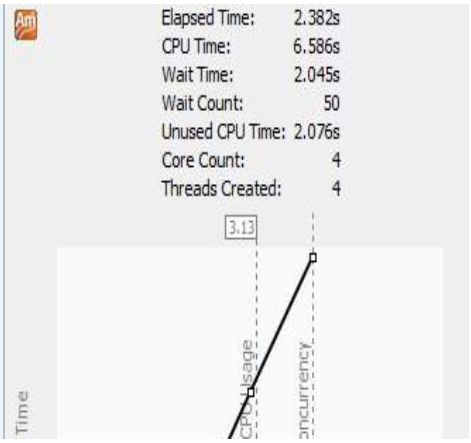


```
C:\WINDOWS\system32\cmd.exe
C:\classfiles\PrimeOpenMP\Debug>PrimeOpenMP.exe 1 5000000
 90%
  348513 primes found between      1 and 5000000 in     4.22 secs
C:\classfiles\PrimeOpenMP\Debug>_
```

Speedup achieved is 1.68X

Comparative Analysis





Threading applications require multiple iterations of going through the software development cycle

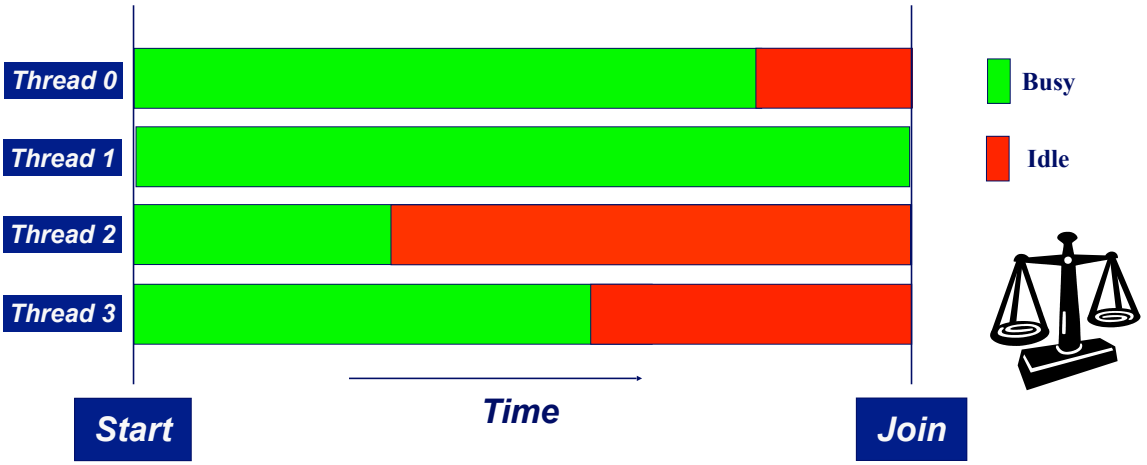
Performance Issues

Common Performance Issues

- **Load balance**
  - Improper distribution of parallel work
- **Synchronization**
  - Excessive use of global data / contention for same synchronization object
- Parallel Overhead
  - Due to thread creation, scheduling
- Granularity
  - Not sufficient parallel work

Load Imbalance

- Unequal work loads lead to idle threads and wasted time



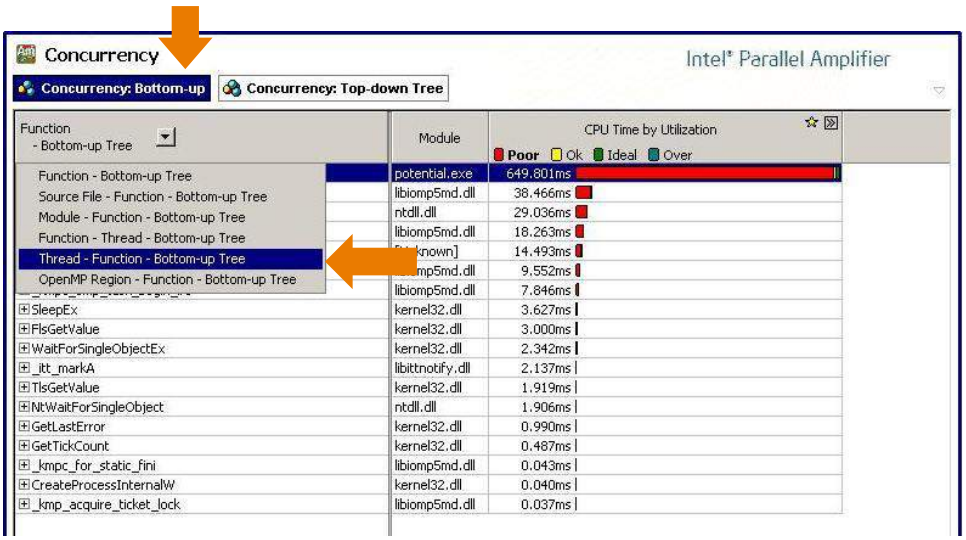
Redistribute Work to Threads

- Static assignment
  - Are the same number of tasks assigned to each thread?
  - Do tasks take different processing time?
    - Do tasks change in a predictable pattern?
    - Rearrange (static) order of assignment to threads
    - Use dynamic assignment of tasks

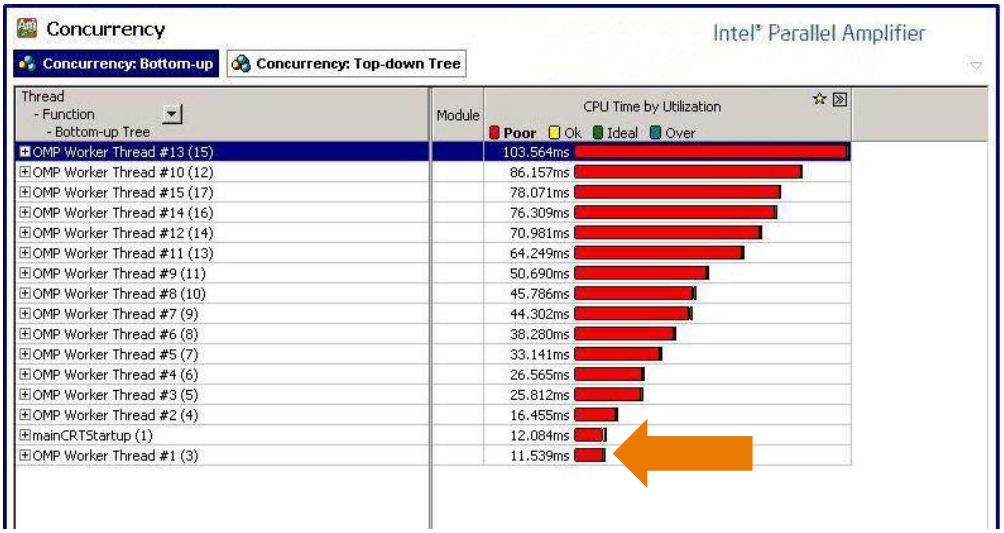
Redistribute Work to Threads

- Dynamic assignment
  - Is there one big task being assigned?
    - Break up large task to smaller parts
  - Are small computations agglomerated into larger task?
    - Adjust number of computations in a task?
    - More small computations into single task?
    - Fewer small computations into single task?

Unbalanced Workloads

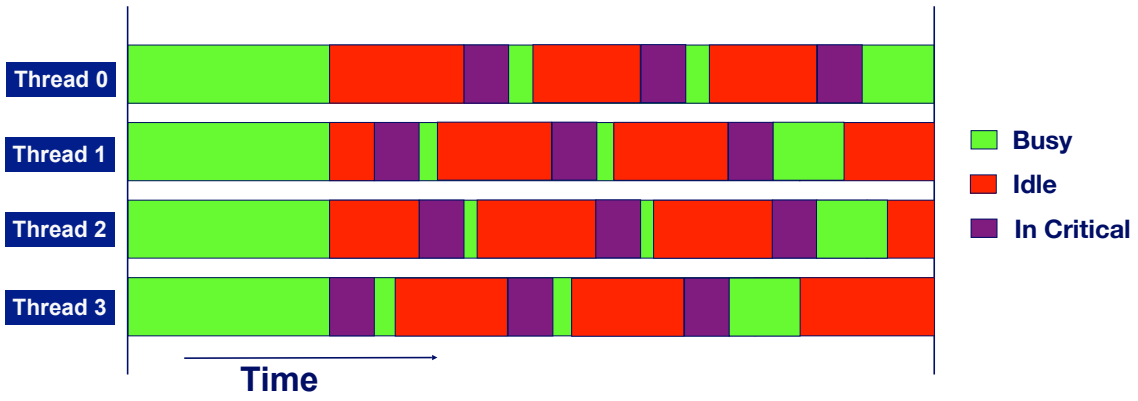


Unbalanced Workloads



Synchronization

- By definition, synchronization serializes execution
- Lock contention means more idle time for threads



Synchronization Fixes

- Eliminate synchronization
  - Expensive but necessary “evil”
  - Use storage local to threads
    - Use local variable for partial results, update global after local computations
    - Allocate space on thread stack (**alloca**)
    - Use thread-local storage API (**TlsAlloc**)
  - Use atomic updates whenever possible
    - Some global data updates can use atomic operations

General Optimizations

- Serial Optimizations
  - Serial optimizations along the critical path should affect execution time
- Parallel Optimizations
  - Reduce synchronization object contention
  - Balance workload
  - Functional parallelism
- Analyze benefit of increasing number of processors
- Analyze the effect of increasing the number of threads on scaling performance

Profiling and Tracing

Measurement: Profiling vs. Tracing

- Profiling
  - Summary statistics of performance metrics
    - Number of times a routine was invoked
    - Exclusive & inclusive time
    - Hardware performance counters
    - Number of child routines invoked, etc.
    - Structure of invocations (call-trees/call-graphs)
    - Memory, message communication sizes
- Tracing
  - When and where events took place along a global timeline
    - Time-stamped log of events
    - Message communication events (sends/receives) are tracked
    - Shows when and from/to where messages were sent
    - Large volume of performance data generated leads to more perturbation in the program

Measurement: Profiling

- Profiling
  - Helps to expose performance bottlenecks and hotspots
  - 80/20 – rule or Pareto principle: often 80% of the execution time in 20% of your application
  - Optimize what matters, don't waste time optimizing things that have negligible overall influence on performance
- Implementation
  - Sampling: periodic OS interrupts or hardware counter traps
    - Build a histogram of sampled program counter (PC) values
    - Hotspots will show up as regions with many hits
  - Measurement: direct insertion of measurement code
    - Measure at start and end of regions of interests, compute difference

Measurement: Tracing

- Tracing
  - Recording of information about significant points (events) during program execution
    - Entering/exiting code region (function, loop, block, …)
    - Thread/process interactions (e.g., send/receive message)
  - Save information in event record
    - Timestamp
    - CPU identifier, thread identifier
    - Event type and event-specific information
  - Event trace is a time-sequenced stream of event records
  - Can be used to reconstruct dynamic program behavior
  - Typically requires code instrumentation

Performance Counters

Hardware Performance Counters

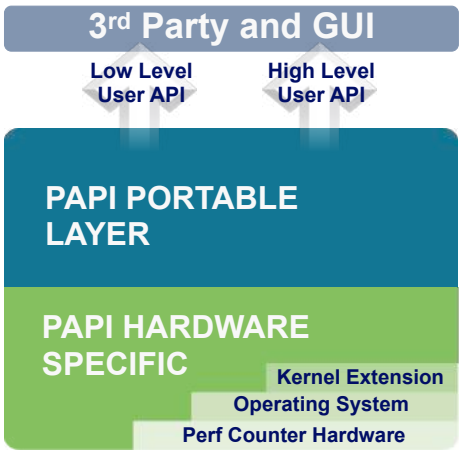
- Specialized hardware registers to measure the performance of various aspects of a microprocessor
- Originally used for hardware verification purposes
- Can provide insight into:
  - Cache behavior
  - Branching behavior
  - Memory and resource contention and access patterns
  - Pipeline stalls
  - Floating point efficiency
  - Instructions per cycle
- Counters vs. events
  - Usually a large number of countable events - hundreds
  - On a small number of counters (4-18)

What is PAPI

- Middleware that provides a consistent and efficient programming interface for the performance counter hardware found in most major microprocessors.
- Countable events are defined in two ways:
  - Platform-neutral Preset Events (e.g., PAPI_TOT_INS)
  - Platform-dependent Native Events (e.g., L3_CACHE_MISS)
- Preset Events can be derived from multiple Native Events
  - PAPI_L1_TCM (Total Cache Misses) **might** L1 Data Misses + L1 Instruction Misses
- Preset events are defined in a best-effort way
  - No guarantees of semantics portably
  - Figuring out what a counter actually counts and if it does so correctly can be hairy

PAPI Counter Interfaces

- PAPI provides 3 interfaces to the underlying counter hardware:
  - A low level API manages **hardware events** in user defined groups called EventSets.
  - A high level API provides the ability to **start**, **stop** and **read** the **counters** for a specified list of events.
  - Graphical and end-user tools provide facile data collection and visualization.



PAPI Example Low Level API Usage

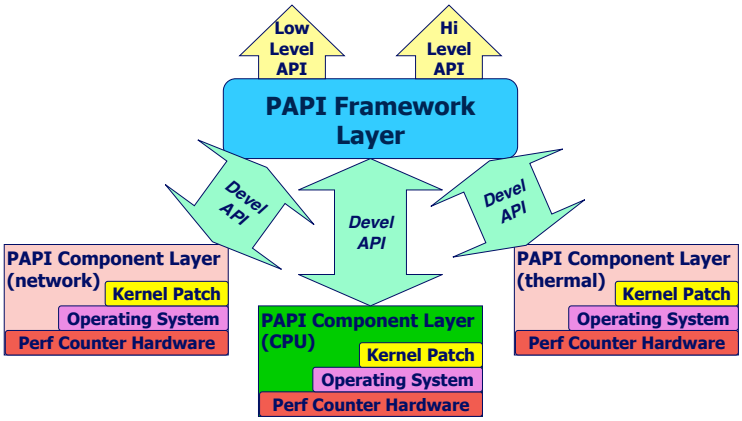
```
#include "papi.h"
#define NUM_EVENTS 2
int Events[NUM_EVENTS]={PAPI_FP_OPS,PAPI_TOT_CYC},
int EventSet;
long long values[NUM_EVENTS];
/* Initialize the Library */
retval = PAPI_library_init (PAPI_VER_CURRENT);
/* Allocate space for the new eventset and do setup */
retval = PAPI_create_eventset (&EventSet);
/* Add Flops and total cycles to the eventset */
retval = PAPI_add_events (&EventSet,Events,NUM_EVENTS);
/* Start the counters */
retval = PAPI_start (EventSet);

do_work();  /* What we want to monitor*/

/*Stop counters and store results in values */
retval = PAPI_stop (EventSet,values);
```

Using PAPI Through Tools

- You can use PAPI directly in your application, but most people use it through tools
- The tool might have a predefined set of counters or lets you select counters through a configuration file/environment variable, etc.
- Tools using PAPI
  - Scalasca, KOJAK (FZ Juelich, UTK)
  - TAU (UO)
  - PerfSuite (NCSA)
  - HPCToolkit (Rice)
  - Open|Speedshop (SGI)
  - ompP (UCB)
  - IPM (LBNL)

Component PAPI Design

Re-Implementation of PAPI w/ support for multiple monitoring domains (CPU, network, thermal)



Ce am invatat in aces curs?

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 9
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Debugging and Performance Evaluation
- Performance Issues
- Profiling and Tracing
- Performance Counters

Cuprins

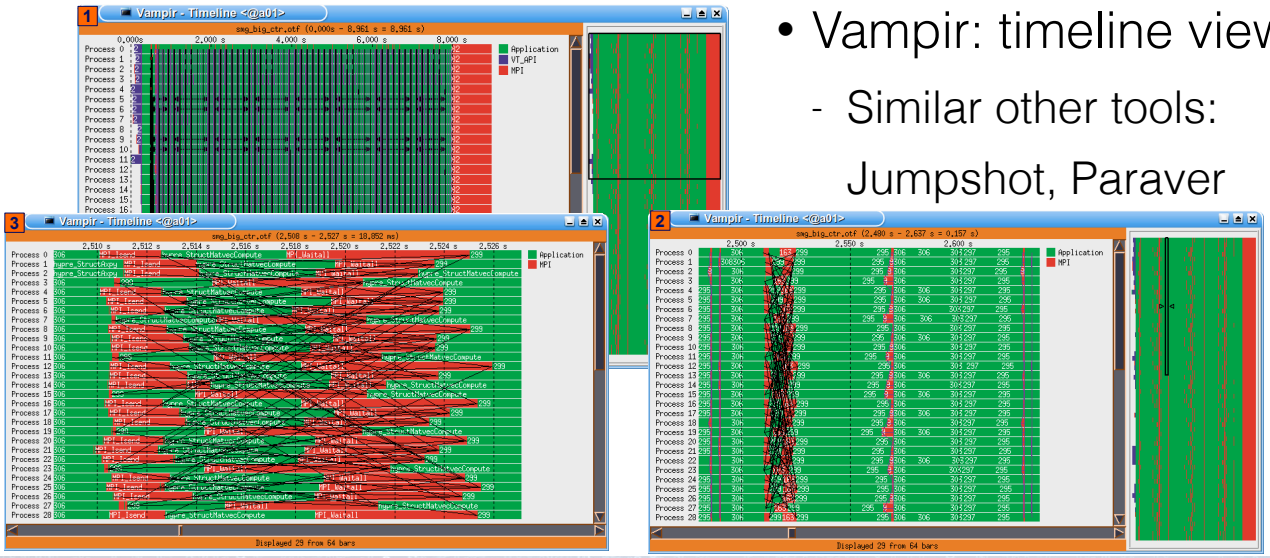
- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing

Performance Analysis - Tools & Techniques

Performance Data Analysis

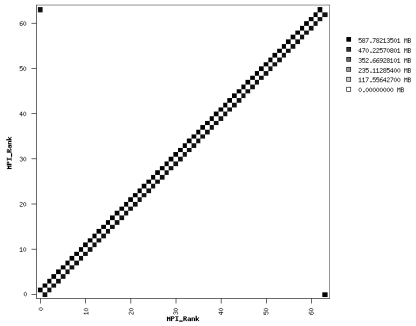
- Draw conclusions from measured performance data
- Manual analysis
  - Visualization
  - Interactive exploration
  - Statistical analysis
  - Modeling
- Automated analysis
  - Try to cope with huge amounts of performance by automation
  - Examples: Paradyn, KOJAK, Scalasca, Periscope

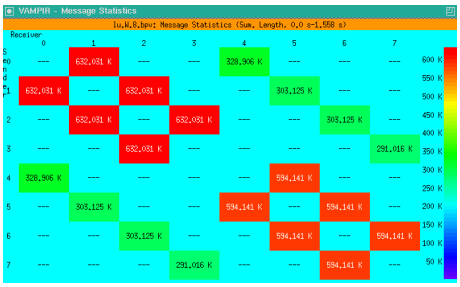
Trace File Visualization

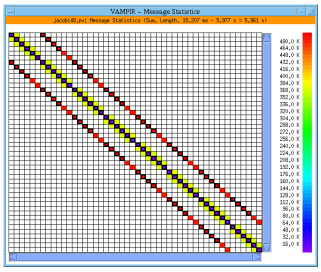


- Vampir: timeline view
  - Similar other tools: Jumpshot, Paraver

Trace File Visualization



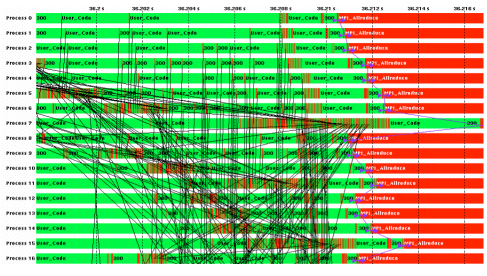




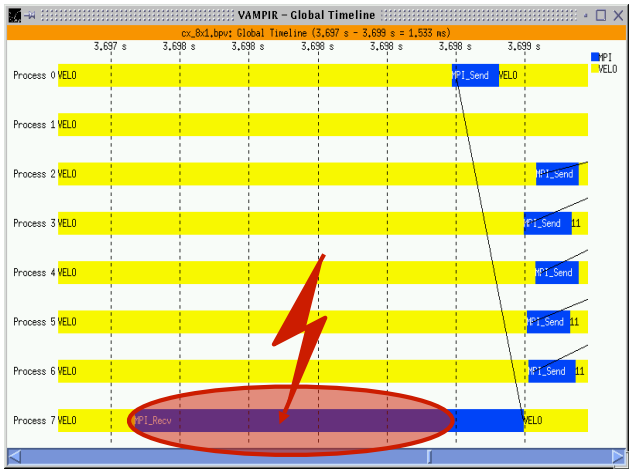
- Vampir/IPM: message communication statistics

Automated Performance Analysis

- Reason for Automation
  - Size of systems: several tens of thousand of processors
  - LLNL Sequoia: 1.6 million cores
  - Trend to multi-core
- Large amounts of performance data
  - Several gigabytes or even terabytes
- Not all programmers are performance experts
  - Scientists want to focus on their domain
  - Need to keep up with new machines
- Automation can solve some of these issues

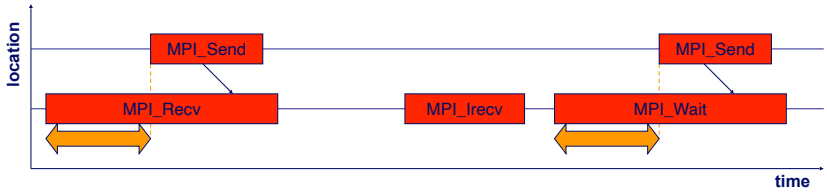


Automation Example

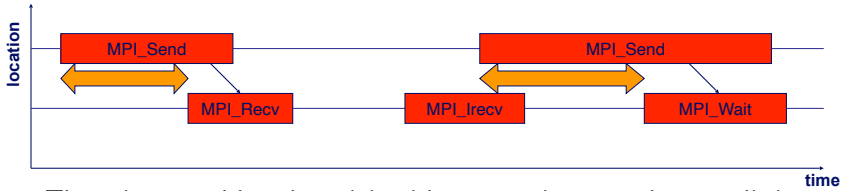


- „Late sender“ pattern
- This pattern can be detected automatically by analyzing the trace

MPI Pattern: Late Sender / Receiver

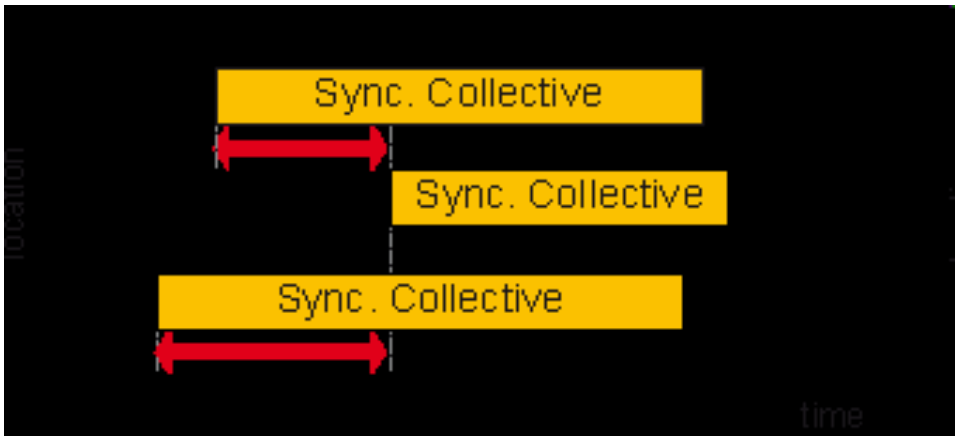


- Late Sender: Time lost waiting caused by a blocking receive operation posted earlier than the corresponding send operation



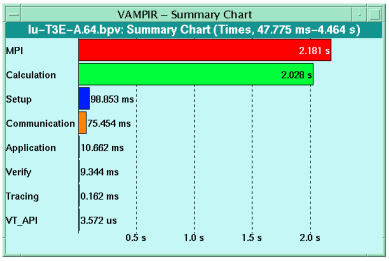
- Late Receiver: Time lost waiting in a blocking send operation until the corresponding receive operation is called

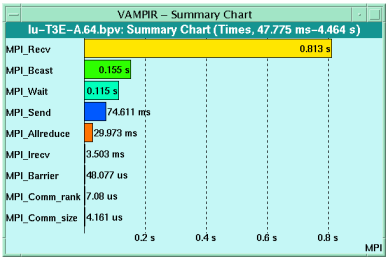
MPI Pattern: Wait at Barrier



- Time spent in front of MPI synchronizing operation such as barriers

Vampir Overview Statistics



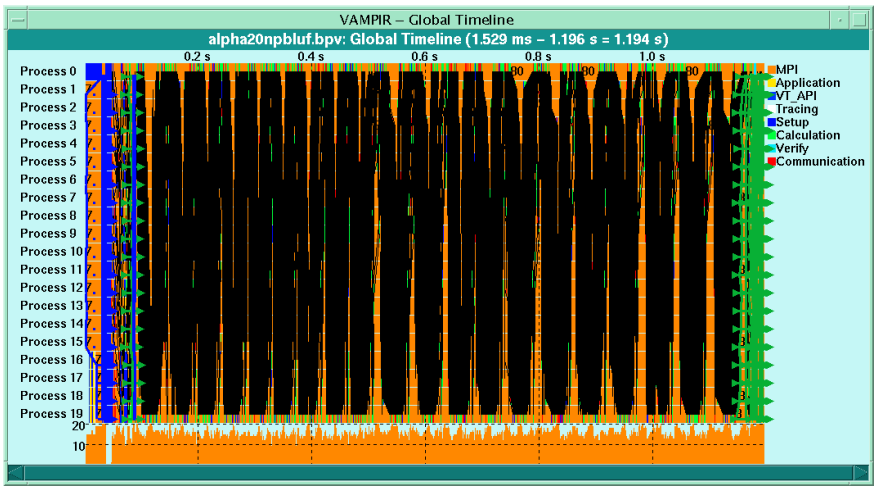


- Aggregated profiling information
  - Execution time
  - Number of calls
- This profiling information is computed from the trace
  - Change the selection in main timeline window
- Inclusive or exclusive of called routines

Timeline Display – Contents

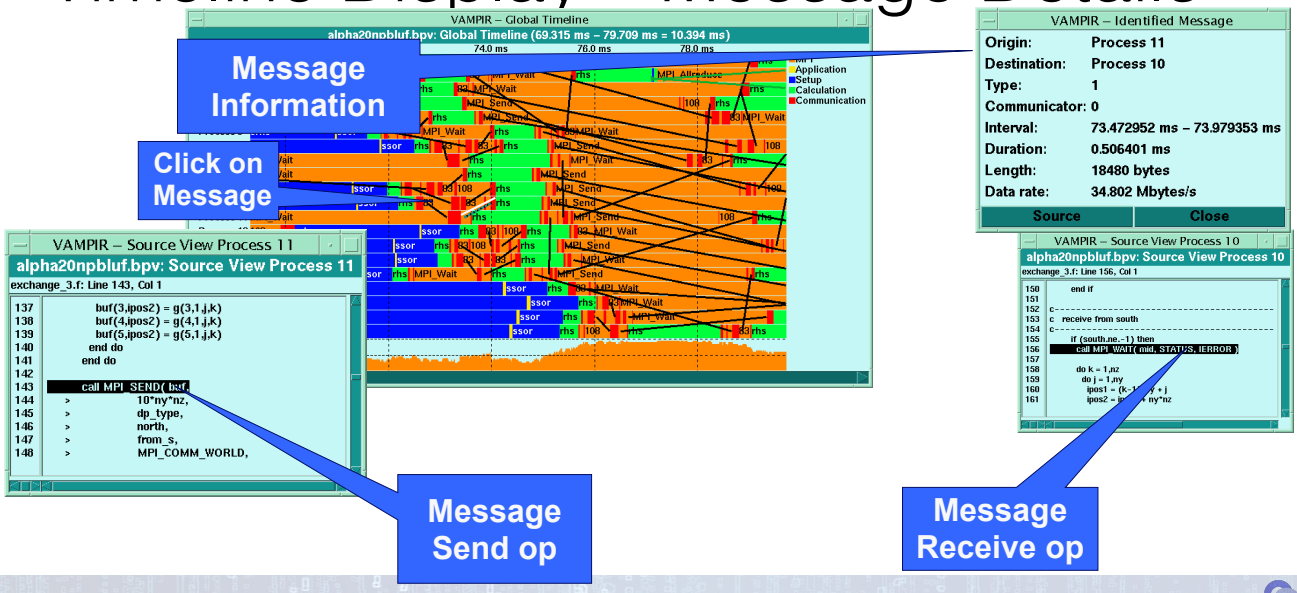
- Shows all selected processes
- Shows state changes (activity color)
- Shows messages, collective and MPI–IO operations
- Can show parallelism display at the bottom

Timeline Display

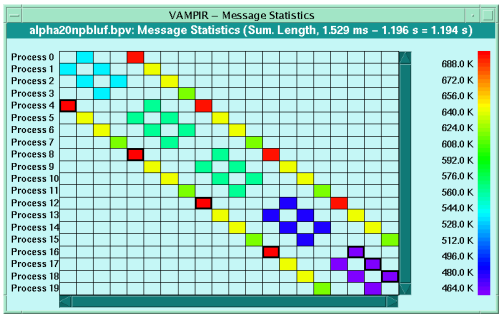


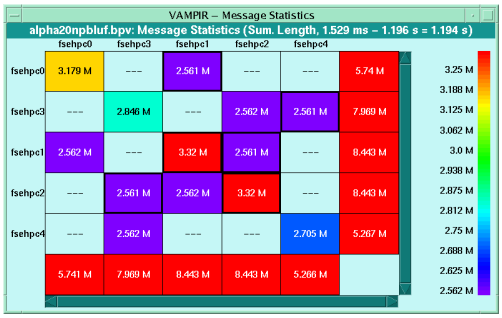
- To zoom, mark region with the mouse

Timeline Display – Message Details



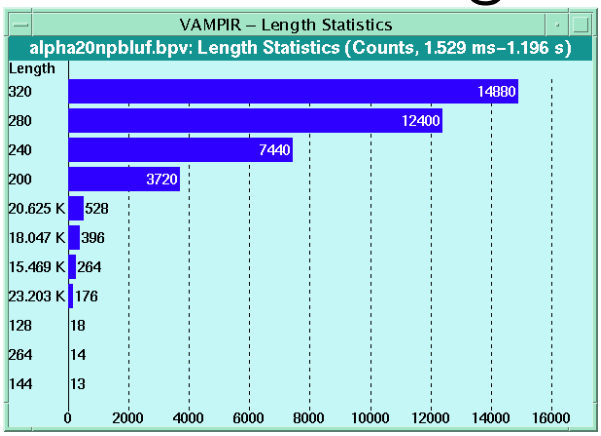
Communication Statistics

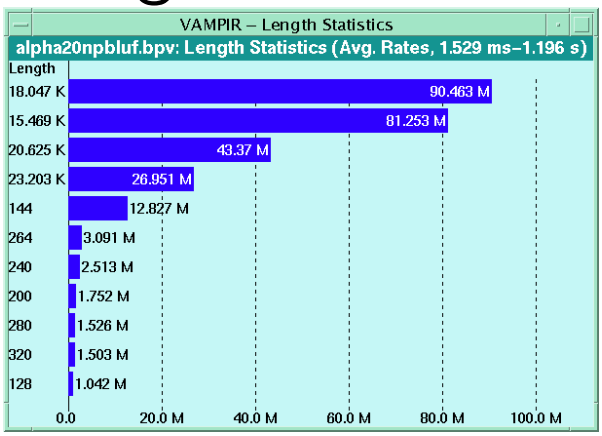




- Message statistics for each process/node pair:
  - Byte and message count
  - Min/max/avg message length, bandwidth

Message Histograms

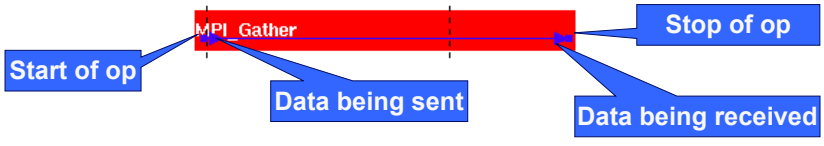


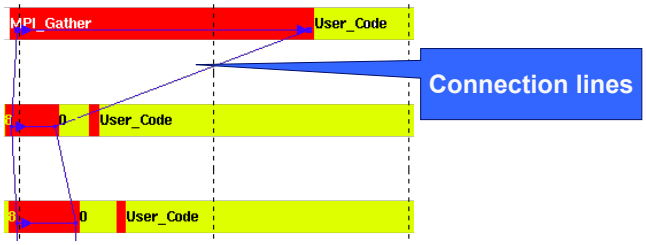


- Message statistics by length, tag or communicator
  - Byte and message count
  - Min/max/avg bandwidth

Collective Operations

- For each process: mark operation locally

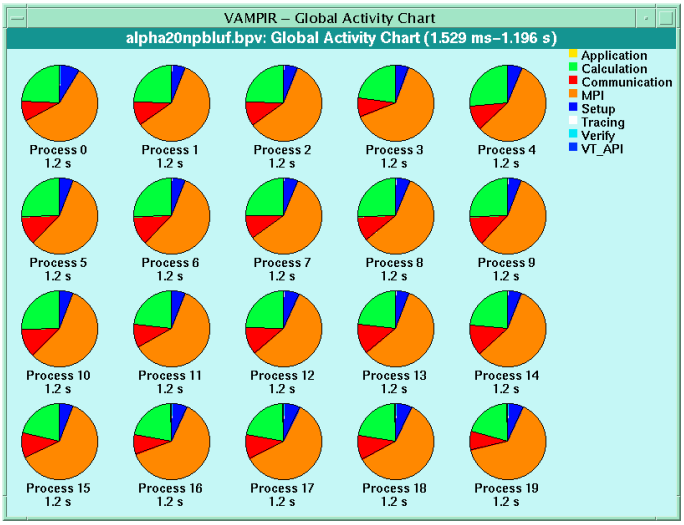




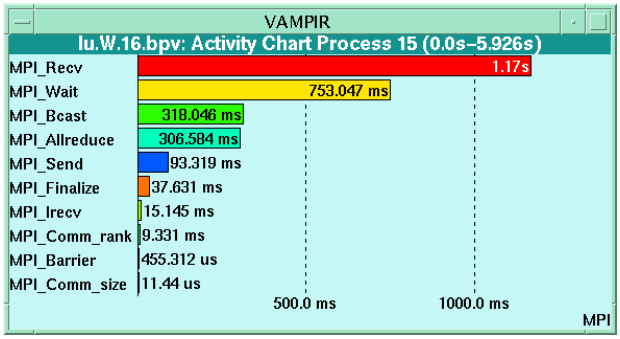
- Connect start/stop points by lines

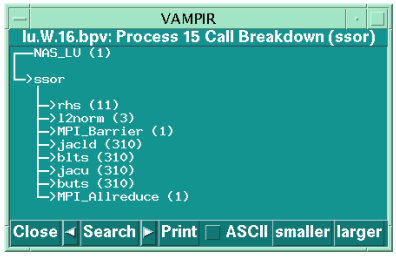
Activity Chart

- Profiling information for all processes

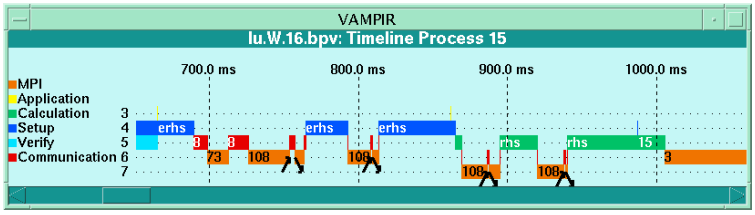


Process – Local Displays

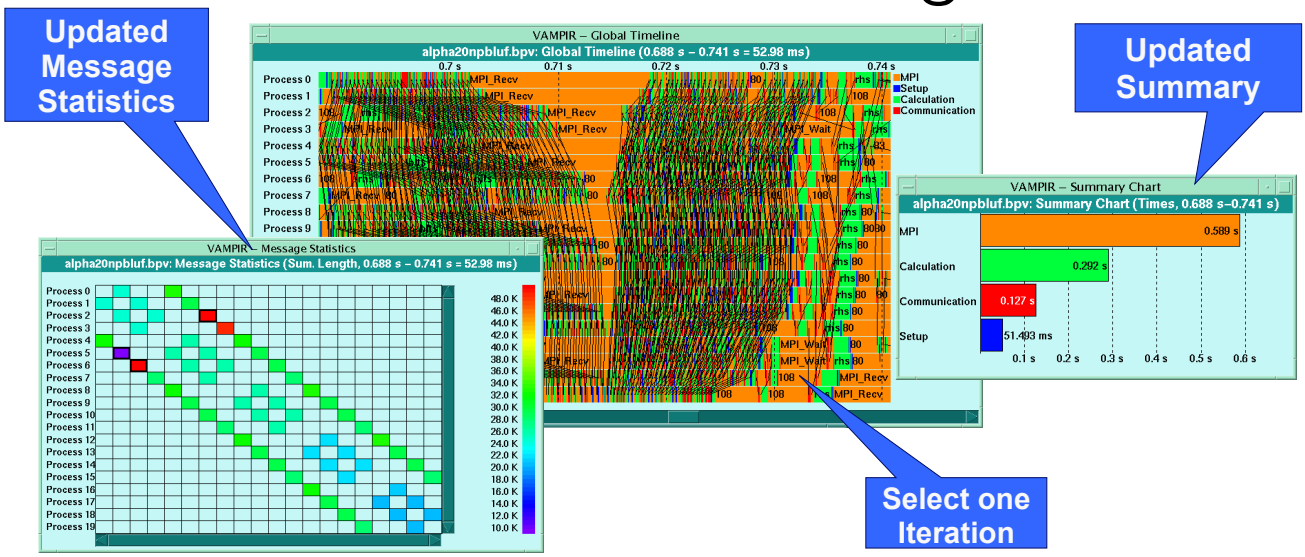




- Timeline: calling levels
- Activity chart
- Calling tree: number of calls

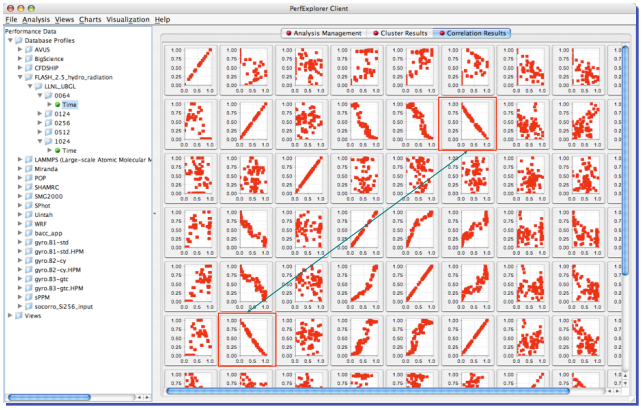


Effects of Zooming



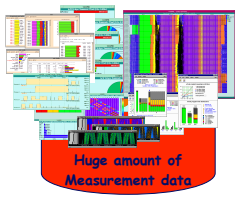
PerfExplorer – Correlation Analysis (Flash)

- Describes strength and direction of a linear relationship between two variables (events) in the data

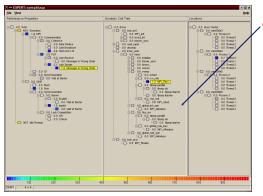


Scalasca / KOJAK – Basic Idea

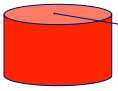
- "Traditional" Tool



- Automatic Tool



Simple:
1 screen +
2 commands +
3 panes

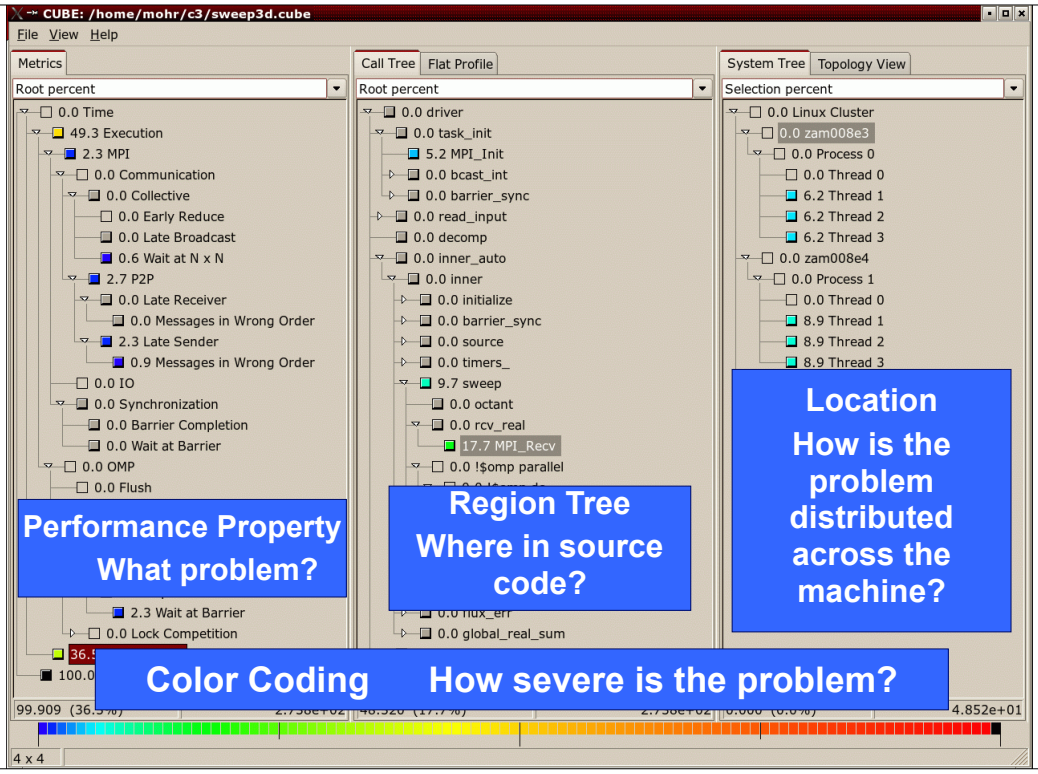


Relevant
problems
and data

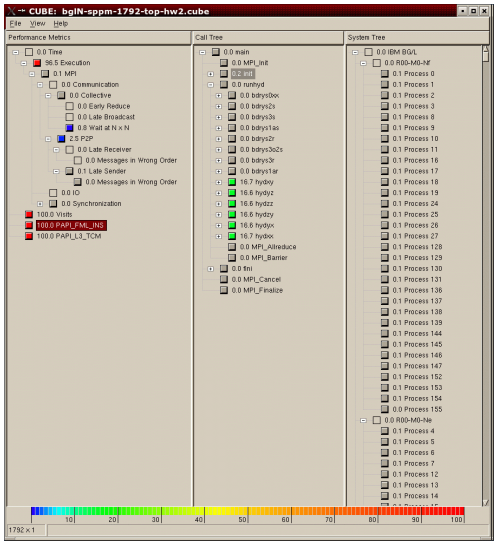
- For non-standard / tricky cases (10%)
- For expert users

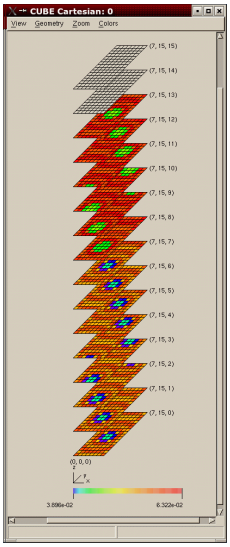
- For standard cases (90% ?!)
- For "normal" users
- Starting point for experts

⇨ **More productivity for performance analysis process!**

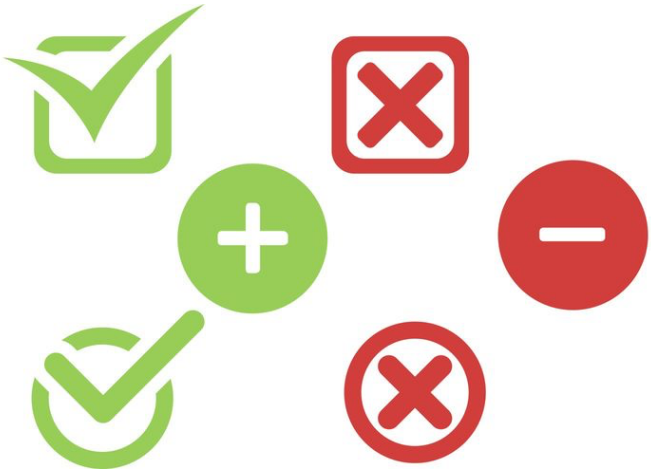


KOJAK: sPPM run on (8x16x14) 1792 PEs





- Topology display
- Shows distribution of pattern over HW topology
- Easily scales to even larger systems



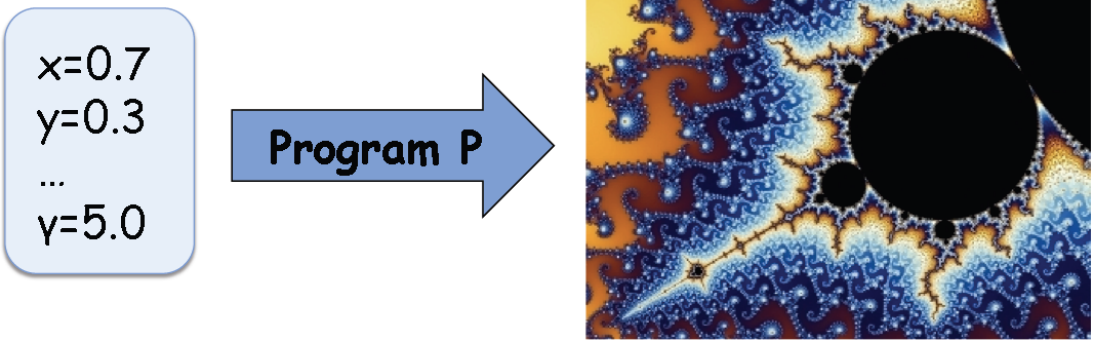
Parallel Correctness Challenges

Parallel Correctness Challenges

- Parallel programming presents a number of new challenges to writing correct software
  - New kinds of bugs: data races, deadlocks, etc.
  - More difficult to test programs and find bugs
  - More difficult to reproduce errors
- Key Difficulty: Potential non-determinism
  - Order in which threads execute can change from run to run
  - Some runs are correct while others hit bugs

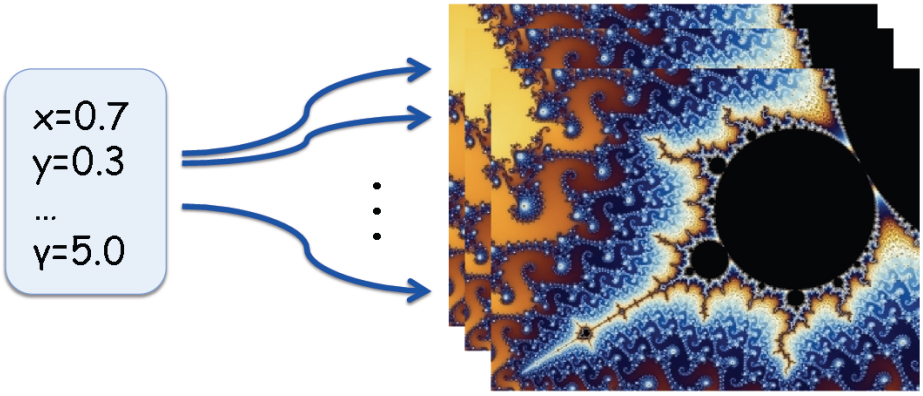
Parallel Correctness Challenges

- For sequential programs, we typically expect that same input → same output



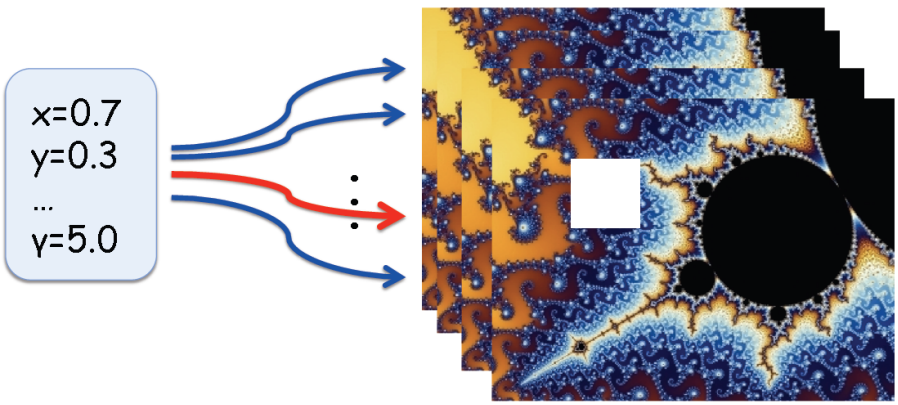
Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run



Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run



Parallel Correctness Challenges

- But for parallel programs, threads can be scheduled differently each run
- A bug may occur under only rare schedules
  - In 1 run in 1000 or 10,000 or …
- May occur only under some configurations:
  - Particular OS scheduler
  - When machine is under heavy load (> 80%)
  - Only when debugging/logging is turned off!

Testing Parallel Programs

- For **sequential** programs:
  - Create several test inputs with known answers.
  - Run the code on each test input.
  - If all tests give correct output, have some confidence in the program
  - Have intuition about which “edge cases” to test.
- But for **parallel** programs:
  - Each run tests only a single schedule.
  - How can we test many different schedules?
  - How confident can we be when our tests pass?

Testing Parallel Programs

- Possible Idea: Can we just run each test thousands of times?
- Problem: Often not much randomness in OS scheduling
  - May waste much effort, but test few different schedules
  - Recall: Some schedules tend to occur only under certain configurations – hardware, OS, etc
  - One easy parameter to change: load on machine

Stress Testing

- Idea: Test parallel program while oversubscribing the machine
  - On a 4-core system, run with 8 or 16 threads
  - Run several instances of the program at a time
  - Increase size to overflow cache/memory
  - Effect: Timing of threads will change, leading to
    - **Different thread schedules**
- Pro: Very simple idea, easy to implement
  - And often works!

Noise Making / Random Scheduling

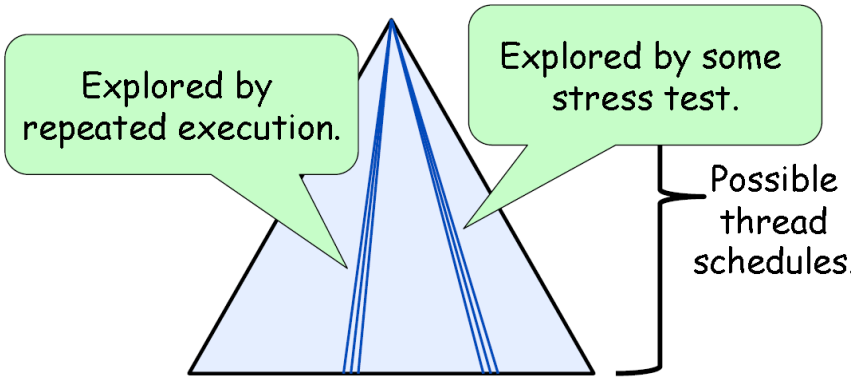
- Idea: Run with random thread schedules
  - E.g., insert code like:
    - if (rand() < 0.01) usleep(100)
    - if (rand() < 0.01) yield()
  - Can add to “suspicious” or “tricky” code
  - Or use tool to seize control of thread scheduling
- Pros: Still fairly simple and often effective
  - Explores different schedules than stress testing
  - Many tools can perform this automatically

Noise Making / Random Scheduling

- IBM’s ConTest: Noise-making - Java
- Berkeley’s Thrille - C + pThreads
- CalFuzzer does simple random scheduling - Java
- Microsoft Research’s Cuzz - .NET
- Many of these tools provide replay
  - Same random number seed → same schedule

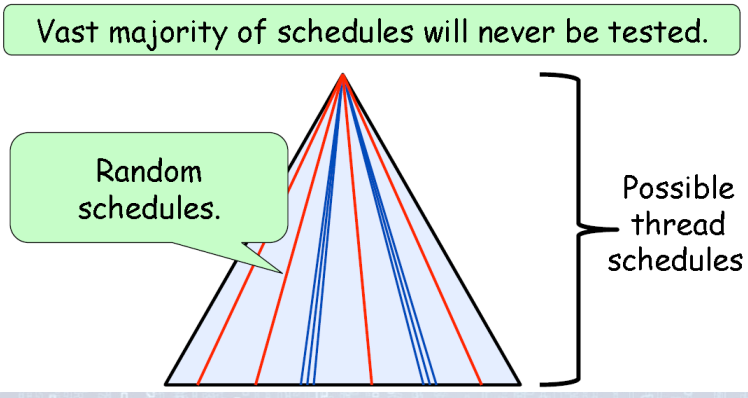
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



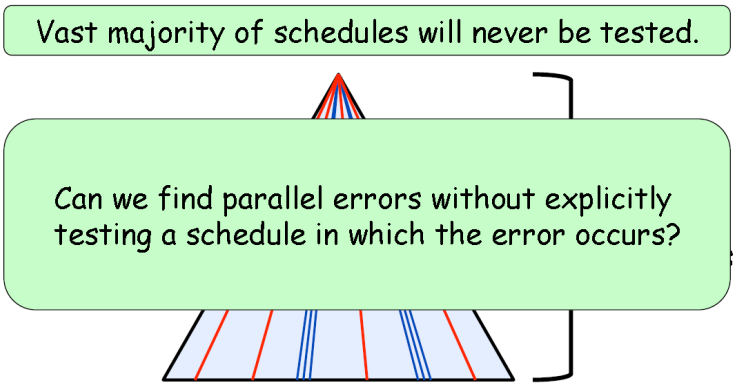
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



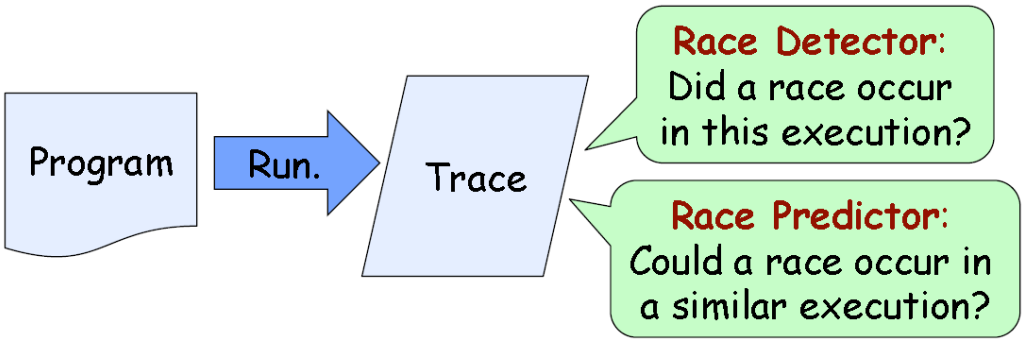
Limitations of Random Scheduling

- Parallel programs have huge number of schedules – exponential in length of a run



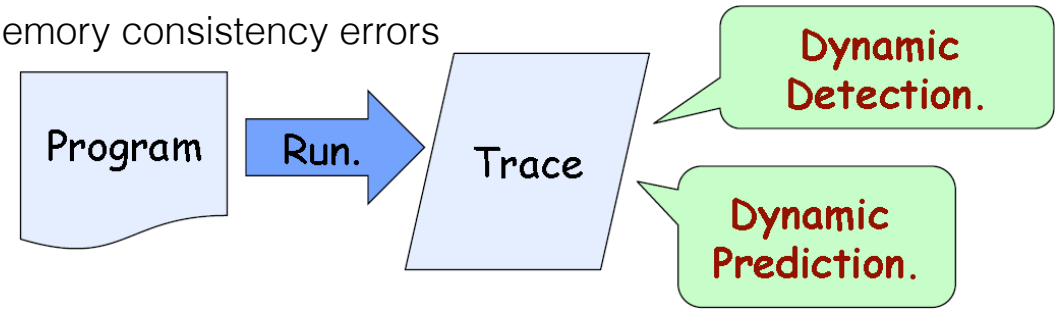
Detecting/Predicting Parallel Bugs

- Say we observe a test run of a parallel program that doesn't obviously fail
- Key Question: Can we find possible parallel bugs by examining the execution?



Detecting/Predicting Parallel Bugs

- Techniques/tools exist for:
  - Data races
  - Atomicity violations
  - Deadlocks
  - Memory consistency errors



Static Analysis

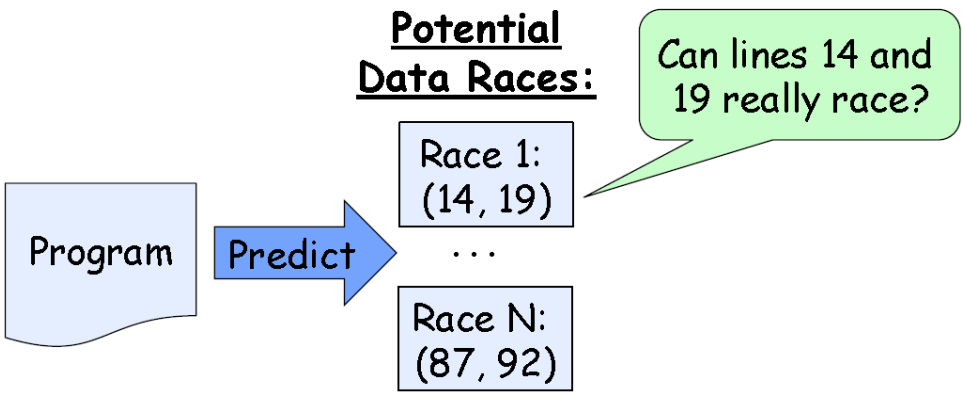
- So far we've only discussed dynamic analyses
  - Examine a real run/trace of a program
- Static analyses predict data races, deadlocks, etc., **without** running a program
  - Only examine the source code
  - Area of active research for ~20 years
  - Potentially much better coverage than dynamic analysis – examines all possible runs
  - But typically also more false warnings

Active Random Testing Overview

- **Problem**: Random testing can be very effective for parallel programs, but can miss many potential bugs
- **Problem**: Predictive analyses find many bugs, but can have false warnings
  - Time consuming and difficult to examine reported bugs and determine whether or not they are real
- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug

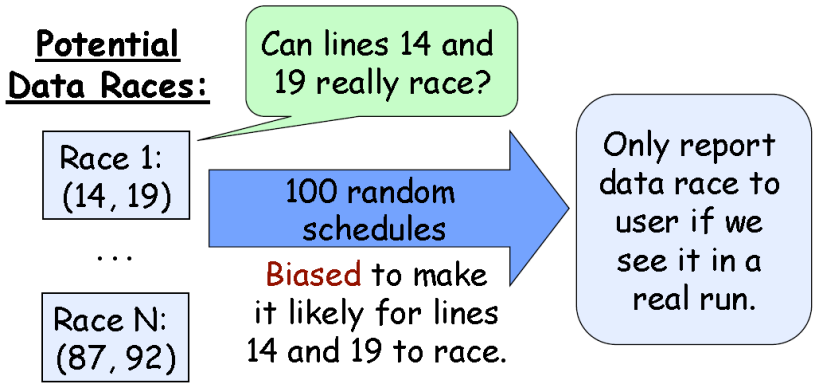
Active Random Testing

- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug



Active Random Testing

- **Key Idea**: Combine them – use predictive analysis to find potential bugs, then **biased** random testing to actually create each bug



Active Random Testing Tools

- CalFuzzer - extensible, open-source tool for active testing of Java programs
  - Data races, atomicity bugs, and deadlocks
  - RaceFuzzer is the active testing algorithm for data races
- Thrille for C / pThreads & Data Races
- And UPC-Thrille for Unified Parallel C
  - Part of the Berkeley UPC system

Testing & Debugging Conclusions

- Many tools available right now to help find bugs in parallel software
  - Data races, atomicity violations, deadlocks
- **No silver bullet solution!**
  - Design how threads will coordinate and share/protect data
  - Tools help catching mistakes when design is not followed
  - Ad hoc parallelization likely to **never** be correct, even with help from tools

Tools References

- IBM's ConTest: Noise-making - Java
  - https://www.research.ibm.com/haifa/conferences/hvc2010/present/Testing_and_Debugging_Concurrent_Software.pdf
- Cuzz: Random scheduling for C++/.NET
  - http://research.microsoft.com/en-us/projects/cuzz/
- Intel Inspector - Locate and debug threading, memory errors in C/C++
  - https://software.intel.com/content/www/us/en/develop/tools/oneapi/components/inspector.html?wapkw=Thread%20Checker%20and%20Parallel%20Inspector#gs.0pp194
- Helgrind, DRD, ThreadSanitizer Dynamic Data Race Detection/Prediction for C/C++
  - http://valgrind.org/docs/manual/hg-manual.html
  - http://code.google.com/p/data-race-test/
- CHORD - Static Race/Deadlock Detection for Java
  - https://www.cse.msu.edu/~cse914/Overheads/mmcgill-java-static-race-detector-6per.pdf

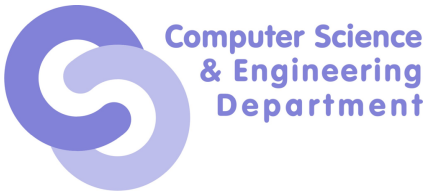
Tools References

- CalFuzzer - Java
  - https://github.com/ksen007/calfuzzer
  - https://www.seas.upenn.edu/~mhnaik/papers/cav09.pdf
- Thrille - C
  - http://github.com/nicholasjalbert/Thrille
- CHESS - C++/.NET Model Checking, Race Detection
  - https://www.microsoft.com/en-us/research/publication/chess-a-systematic-testing-tool-for-concurrent-software/
- Java Path Finder - Model Checking for Java
  - https://blog.devgenius.io/introduction-to-java-pathfinder-80828b53309a
- Tau Performance System - Fortran, C, C++, Java, Python
  - http://www.cs.uoregon.edu/research/tau/home.php
- Scalasca Performance Analysis - C/C++ and Fortran
  - https://www.scalasca.org/
- Performance Application Programming Interface - PAPI & Exa-PAPI
  - http://icl.cs.utk.edu/exa-papi/

Ce am invatat in aces curs?

- Performance Analysis
  - Programming
  - Tools
  - Techniques
- Parallel Correctness Challenges
  - Debugging
  - Testing

Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 10
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cuprins

- Arhitecturi MIMD – Caracteristici
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in Cadrul unui Cluster
- Comunicarea Intercluster

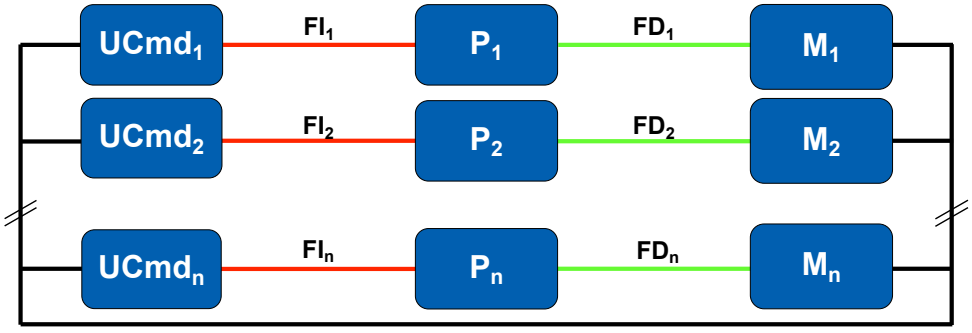
MIMD

Arhitecturi MIMD

MIMD

- MIMD pot comunica: (P-P sau P-M)
- Toate Procesoarele participa la acelasi program
- Mult mail des folosite ca SIMD
- Mod programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)

MIMD = n FI & n FD



Arhitecturi MIMD – Caracteristici

- Structura & arhitectura procesoarelor e eterogena
- Procesoarele pot functiona autonom ca un sistem universal
  - Pot prelua task-urile altor procesoare din sistem
  - Astfel se previne degradarea performantelor sistemului
- Organizarea memoriei
  - Fiecare procesor ar trebui sa aiba acces la toata memoria sistemului, dar…
    - Capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei → mecanisme speciale
  - Trebuie asigurata protectia memoriei pentru asigurarea integritatii datelor din sistem

Arhitecturi MIMD – Caracteristici

- Modularitatea presupune
  - Expandabilitate
    - Numarul de procesoare poate fi modificat fara a fi necesara reconfigurarea sistemului
  - Reconfigurabilitate
    - Structurala
      - Procesul de refacere a starii sistemului la aparitia unor defecte → se realizeaza prin distribuirea structurala a functiilor
      - Permite preluarea sarcinilor unui procesor de catre alte procesoare din sistem
    - Arhitecturala: schimbarea caracteristicilor de baza
      - Dimensiunea registrilor de memorie
      - Lungimea cuvantului
      - Structura UAL

Arhitecturi MIMD – Probleme

- Decompozitia algoritmilor
  - Manual – la conceptia algoritmilor
  - Automat – prin limbaje de programare concurenta
- Planificarea pentru executie a task-urilor si maparea pe structura de procesoare utilizata
- Sincronizarea proceselor/task-urilor astfel incat sa nu se ajunga la blocarea sistemului
- Siguranta in functionare e asigurata de
  - Disponibilitate: capacitatea sistemului de a raspunde cererilor utilizatorului
  - Integritate: capacitatea sistemului de a mentine consistenta starii in conditiile aparitiilor unor defecte

System Multiprocesor / Multicalculator

Clasificarea Structurilor MIMD

- Din punct de vedere arhitectural, structurile MIMD se pot imparti in doua mari categorii:
  - Sisteme multiprocesor:
    - Sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub controlul unui “executiv” unic
  - Sisteme multicalculator:
    - Mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe

Sisteme Multiprocesor

- Au un singur spatiu de adresare - comunicare prin shared memory
- Se impart in sisteme:
  - Cu memorie distribuita / scalabilitate:
    - Gestiune dinamica a accesului la memoria distribuita
    - Gestiune statica a accesului la memoria distribuita
  - Cu memorie centrala / nu exista scalabilitate:
    - Comunicarea se face prin retele de comutatie (e.g. crossbar switch)
    - Conexiuni de tip magistrala (bus) multiplu

Sisteme Multicalculator

- Au spatii multiple de adresare - comunicare prin transfer de mesaje
- Se impart in sisteme:
  - Multicalculator distribuite:
    - Mesh – plasa de interconectare P-M, NEC…
    - Conexiuni Butterfly (Perfect Shuffle) – CM5
    - Hypercube – la sistemele NCUBE, BlueGene…
    - Retele LAN rapide: Myrinet, ATM, GigaEthernet, InfiniBand, MPLS …
  - Multicalculator centralizate

Clasificarea Structurilor MIMD

- Din punct de vedere al accesului la memorie, structurile MIMD sunt:
  - Sisteme cu memorie partajata - Shared Memory

  - Sisteme cu memorie distribuita - Distributed Memory

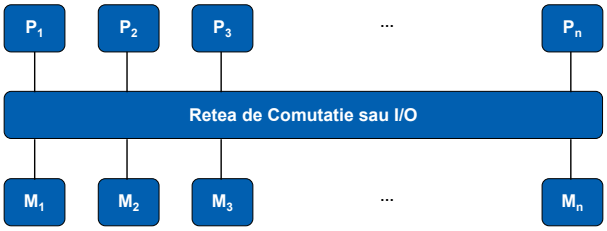
Sisteme cu Memorie Partajata

Sisteme cu Memorie Partajata

- Sistemele cu memorie partajata (Shared Memory) pot fi implementate in una din urmatoarele configuratii:
  - UMA = Uniform Memory Access
  - NUMA = Non-Uniform Memory Access
    - CC-NUMA = Cache-Coherent NUMA
  - COMA = Cache-Only Memory Architecture

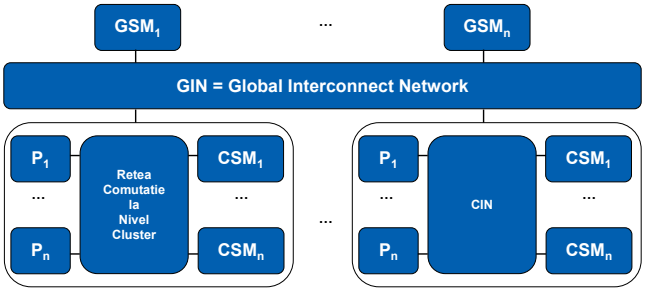
Structura UMA

- UMA = Uniform Memory Access
- Accesul este unic: orice procesor se poate conecta la un modul de memorie in mod uniform
- Reteaua de comunicatie (I/O) poate fi de orice tip



Structura NUMA

- CSM = Cluster Shared Memory
- CIN = Cluster Interconnect Network
- GSM = Global Shared Memory
- GIN = retea de comutare globala la care se conecteaza clustere formate din procesoare
- Accesul la memorii depinde de modulul unde trebuie efectuat accesul (CSM_i sau GSM_j)

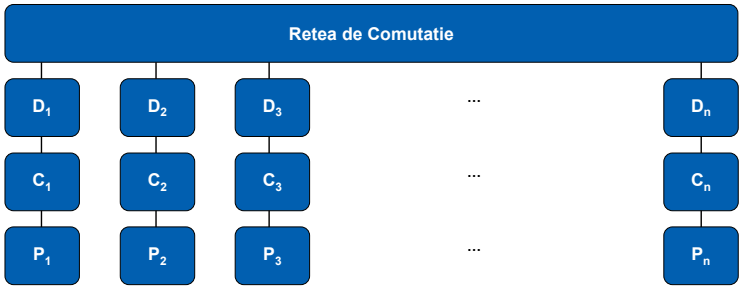


Structura CC-NUMA

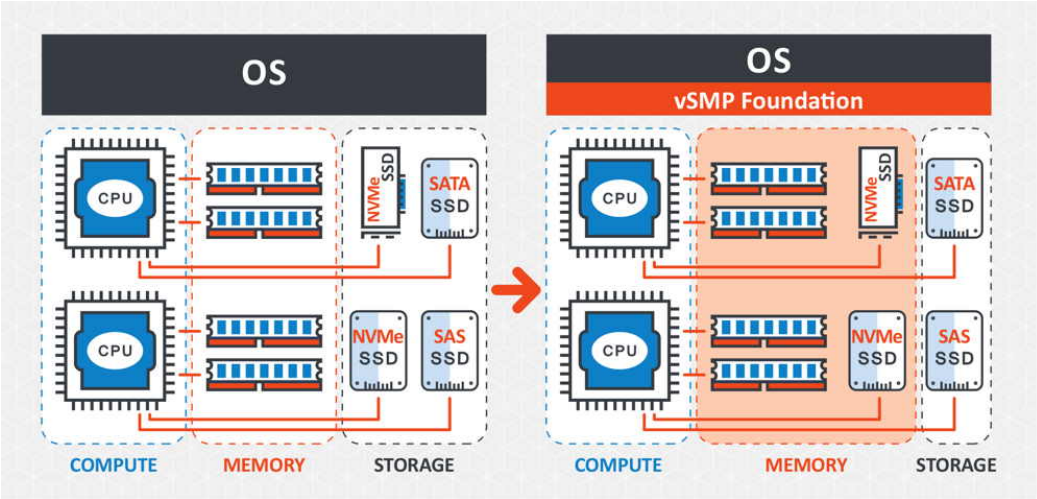
- CC-NUMA = Cache Coherent NUMA
- Coerenta accesului la memorie e mai bine stabilita ca la NUMA:
  - Silicon Graphics - Origin 2000:
    - 2 procesoare pe nod
    - Acces divizat la componentele nodului printr-un Bus intern
    - Hub de conectare asigura accesul partajat la Cache-ul nodului → coerenta cache-ului
  - Sequent NUMA-Q - structuri de noduri interconectate cu structura inelara:
    - Standardul IEEE – SCI = Scalable Coherent Interface – structura de comunicare "rapida" (1GB/s in the 00's)
  - Standford DASH (Directory Architecture for SHared memory) – 4 MIPS R3000 Procs

Structura COMA

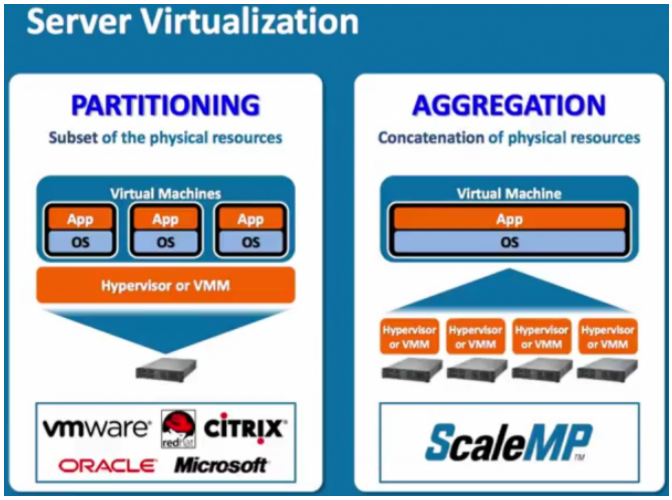
- Procesoarele nu au acces direct
  - La propria zona de date
  - La cea a celorlate procesoare
- Procesoarele au acces direct doar la Cache! → Cache Only
  - D_i = Memoria de Date
  - C_i = Memoria Cache
  - P_i = Procesoare

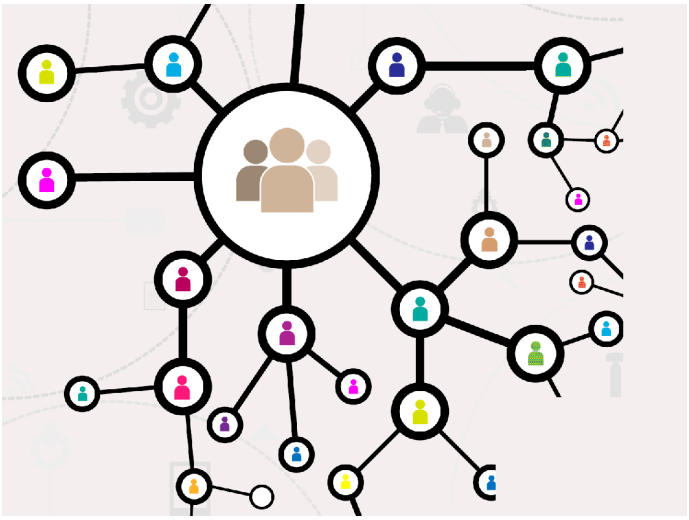


Exemplu de SW COMA



Server Virtualization

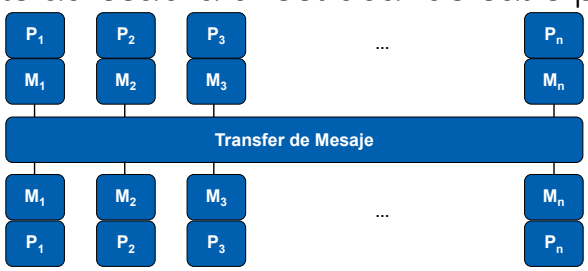




Sisteme cu Memorie Distribuita

Sisteme cu Memorie Distribuita

- Accesul se face prin transfer de mesaje
- Sistemele au la baza mai multe noduri legate printr-o retea de comutatie de mesaje (nu neaparat fizica)
- Conexiunea este statica, punct la punct intre noduri
- Memoria locala este adresabila direct doar de catre procesorul local



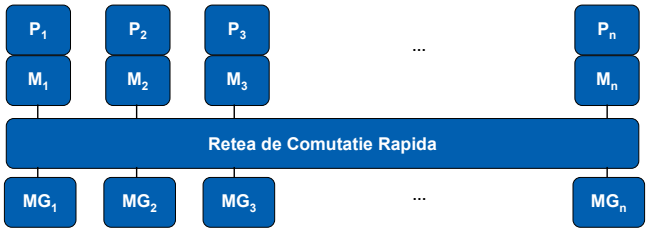
Exemplu @cs.pub.ro

- SUN Enterprise 10000 – Starfire (1996-2003 @cs.pub.ro)
  - Reteaua de comutatie este de tip Gigaplane XB (102.4GB/s; p2p: 1.6GB/s)
  - Comutare de pachete cu cai separate de adrese si date XB
  - Adresele distribuite printr-un router ce face broadcast
  - Acest fapt permite
    - Partitionarea dinamica a setului de procesoare si crearea unor partitii
    - Aceste partitii sunt complet izolate intre ele si pot fi utilizate ca atare pentru sarcini distincte
  - Suporta paralelism de tip shared-memory in Fortran & C/C++ OpenMP

Sisteme Strans / Slab Cuplate

Sisteme Strans / Puternic Cuplate

- Dispun de procesoare P_j cu memorie locala asociata M_i de tip multiport
- O retea de comutatie rapida RC (dar fixa) ce permite accesul uniform la memoria globala partajata MG_i
- Comunicarea si sincronizarea se face prin MG_i cu o rata de transfer similara cu a memoriei locale
- Asigura o scalabilitate redusa datorita faptului ca RC este fixa si de aceea au o structura dedicata
- Performantele sistemului sunt date de modul de acces la memoria partajata

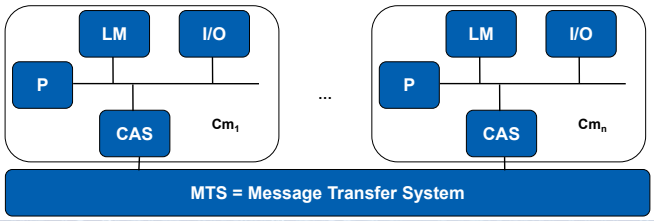


Sisteme Slab Cuplate

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)
  - Multicalculator centralizate

Sisteme Slab Cuplate

- Resurse locale ale CM_i = Computer Module:
  - LM = local memory (acces din interior/exterior)
  - I/O = magistrala de intrare/iesire (acces din interior/exterior)
  - MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
  - MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
  - CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
    - Un singur transfer odata daca MTS este magistrala
    - Mai multe transferuri simultane daca MTS este de tip retea

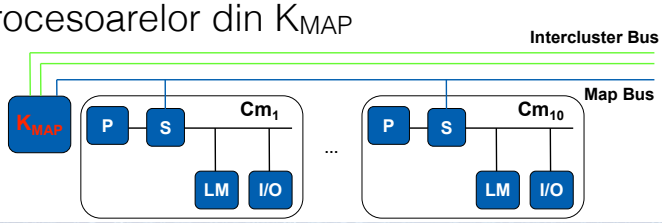


Exemplu – Cm*

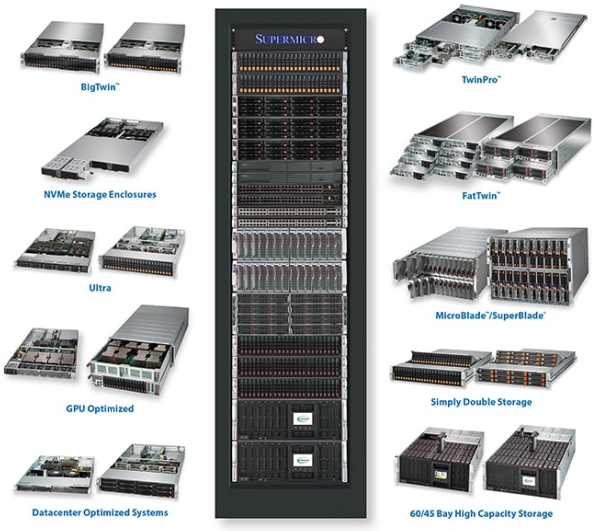
- Structura Cm* a fost dezvoltata arhitectural si implementata la Carnegie Mellon in anii ‘70
- Cm* e proiectat cu intentia de a obtine in acelasi timp:
  - Performante bune
  - Fiabilitate sporita
- In 1975 structura e compusa din 10 procesoare
- Ajunge in 1977 la 50 de procesoare
- Cm* beneficiaza de doua sisteme de operare distincte

Arhitectura Cm*

- Structura Cm* conecteaza mai multe module individuale → structura multipla de calculatoare == cluster
- K_{MAP} = gestioneaza magistrala Map Bus si cererile pentru Cm-uri
- Un cluster e format din mai multe Cm-uri, K_{MAP} & Map Bus
- Cm-urile sunt fiabile → asigura semiparalelism
- Performantele sunt determinate de interconectarea clusterelor & viteza de comunicatie a procesoarelor din K_{MAP}



Analogie - Clustere / Servere

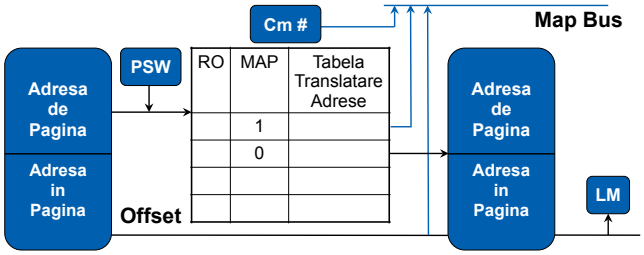


Accesul la Resursele Locale – Switch-ul S

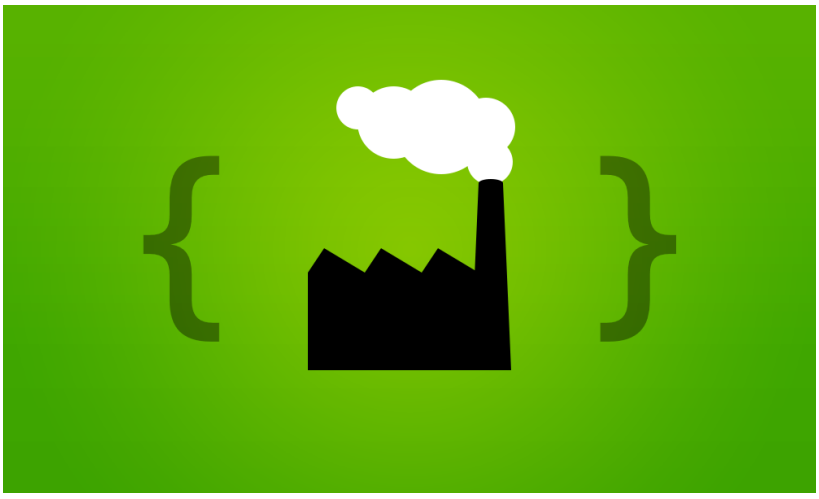
Accesul la Resursele Locale

- Functiile switch-ului S dintr-o structura Cm:
  - Primeste si interpreteaza cererile de acces de la P-uri locale si externe la memoria locala si la sistemul I/O
  - S permite unui P local sa acceseze resurse externe Cm
- Pentru a realiza interpretarea cererilor locale si externe S-ul asigura:
  - O translatie a adreselor la nivel local
  - Formarea adresei externe prin concatenarea adresei in cadrul paginii si numarul de identificare al procesorului
- Memoria este paginata cu dimensiunea paginii de 4K
  - 16 pagini in total → 64K
- Spatiul total de adresare este de 28 de biti - 256M

Mecanismul de Adresare pentru Referinte la Memoria Locala



- PSW = Processor Status Word – ofera adresa in tabela de translatare (adresa fizica/ virtuala)
- RO = Read Only bit (0 = RO; 1 = RW)
- MAP = Maparea adreselor (0 = adresa locala; 1 = adresa globala)
- Cm # = in ce Cm se afla adresa externa; K_{MAP} interpreteaza si decide daca adresa este in cluster-ul curent sau inafara lui



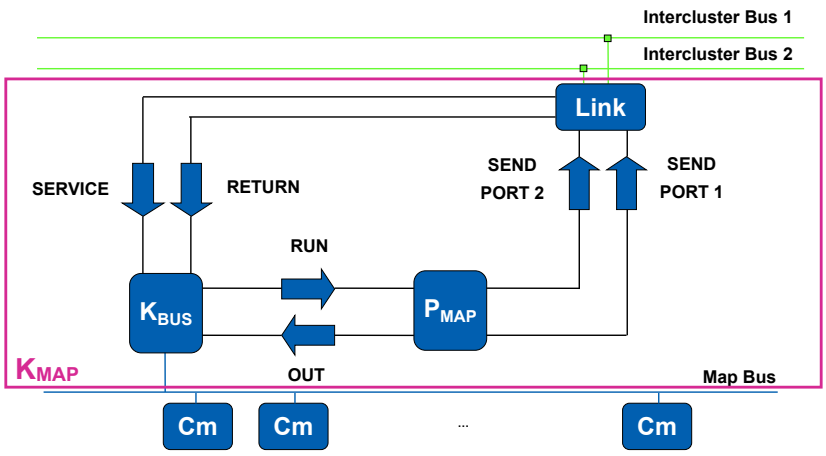
Structura si Functiile K_{MAP}

Functiile K_{MAP}

- K_{MAP} asigura:
  - Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus si arbitrare intre Cm-urile din acelasi cluster
  - Comunicarea intre clustere prin Intercluster Bus
- Functiile de baza ale K_{MAP}:
  - Mapare de adrese
  - Comunicare
  - Sincronizare
- O serie din primitivele OS sunt implementate in K_{MAP}
- Nucleul OS din Cm se simplifica → transparenta

Structura K_{MAP}

- K_{MAP} este compus din 3 procesoare: P_{MAP}, Link, K_{BUS}:



Elementele K_{MAP}

- $K_{BUS:}$ unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala
- P_{MAP}: procesor de mapare care face toate translatarile de adresa
  - Maxim 8 procese simultan
- Link: gestioneaza si supravegheaza transmiterea mesajelor intercluster Send/Recv
- Exista doua tipuri de comutatie:
  - De circuite – in cadrul cluster-ului
  - De mesaje – intre clustere
- K_{MAP} prelucreaza tranzactii (contexte) concurente
- Fiecare context are asociate registre speciale si registre de legatura la subrutine pt translatarea adreselor

Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu
  - Fiecare va fi rezident in P_{MAP}
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi: RUN/OUT

Comunicatia K_{BUS} – P_{MAP}

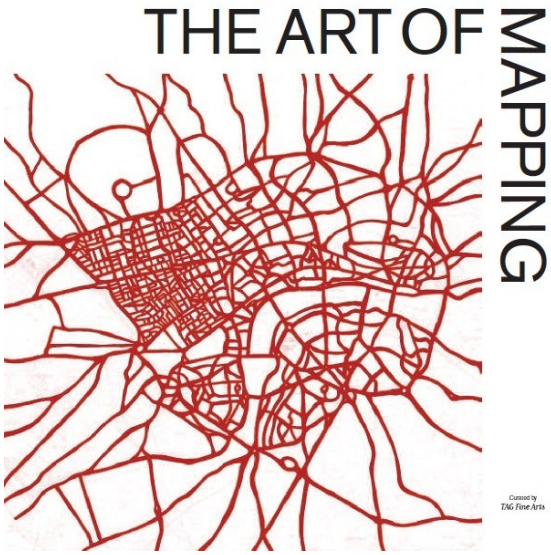
- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu

Comunicatia Link – P_{MAP}

- P_{MAP} trimite in SEND o cerere de transmitere mesaj inter-cluster
- Se suspenda contextul initiator al mesajului intercluster & il trece intr-o coada de asteptare
- La primirea raspunsului pentru acest context se pune in RETURN
- Link reactiveaza contextul in RUN
- La primirea raspunsului inter-cluster in Link acesta il plaseaza in SERVICE
- Se emite spre K_{BUS} o cerere de alocare a unui nou context in RUN

Comunicatia K_{BUS} – Map Bus

- K_{BUS} trebuie sa execute o mare varietate de task-uri printre care si tratarea inteligenta a erorilor
- K_{BUS} este un procesor microprogramat cu o memorie de control ROM de 256 de cuvinte a cate 40 de biti fiecare
- Map Bus este o magistrala de comunicatii de 38 de biti dintre care 20 bidirectionali de adrese / date intre K_{BUS} si S-urile locale
- K_{BUS} controleaza toate accesele pe Map Bus precum si cozile si registrele pentru comunicarea cu P_{MAP}
- Maparea Map Bus este sincrona cu K_{BUS}

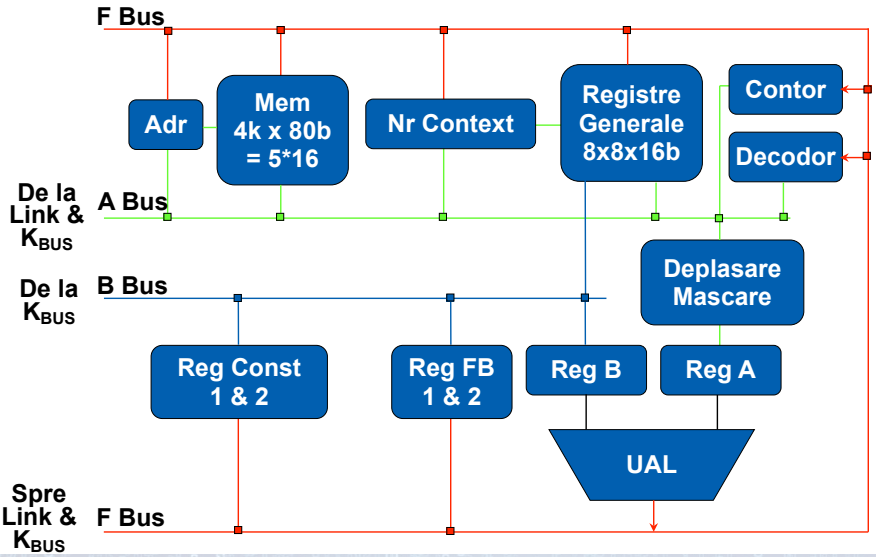


Arhitectura Procesorului de Mapare P_{MAP}

Complexitatea Arhitecturii P_{MAP}

- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare

Arhitectura Procesorului P_{MAP}

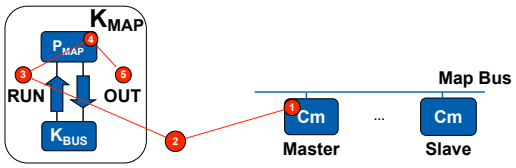


Componentele Procesorului P_{MAP}

- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele PMAP trebuie sa aibe acces rapid la informatia pt translatarea adreselor virtuale in adrese fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster

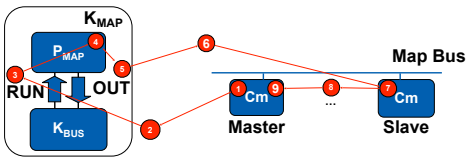
Comunicarea in Cadrul unui Cluster

Etapele de Acces
in Cadrul unui Cluster



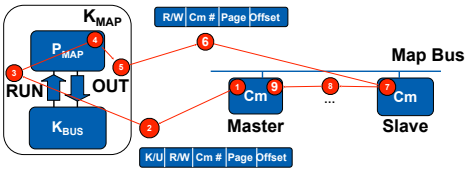
1. In Cm Master se initiaza acces la o memorie nelocala
2. Cm Master emite adresa virtuala catre K_{BUS}
3. K_{BUS} activeaza un context (crearea structurii de date specifica tranzitiei) ce se pune in coada RUN catre P_{MAP}
4. P_{MAP} trateaza contextul si face translatarea adresei
5. P_{MAP} pune in coada OUT o cerere pentru ciclu de memorie pentru Cm Slave din cluster-ul curent

Etapele de Acces
in Cadrul unui Cluster

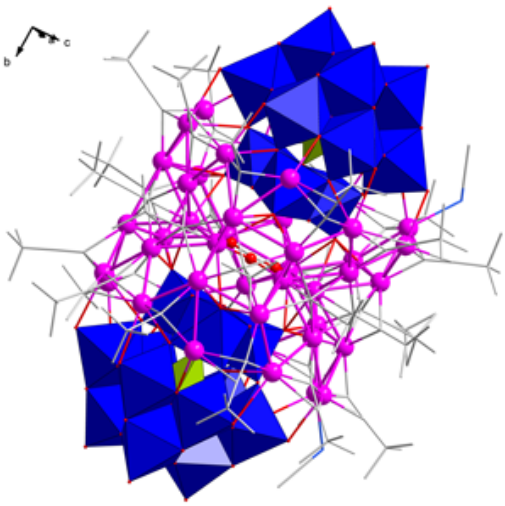


6. K_{BUS} trimite adresa fizica catre Cm Slave prin Map Bus
7. S-ul local din Cm Slave face acces la memoria locala prin furt de ciclu
8. K_{BUS} “permite” ca rezultatul operatiei de acces la memorie sa fie furnizat catre Cm Master
9. Cm Master preia datele, termina operatia in curs si continua executia

Formatul Pachetelor Transmise

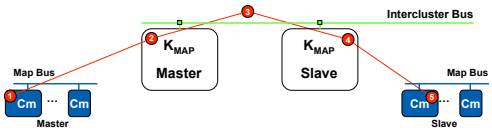


- Pachetele trimise au urmatorul format:
  - U/K – User/Kernel
  - R/W – Read/Write
  - Cm # – numarul Cm-ului ce efectueaza transferul
  - Page – utilizata pentru translatarea locala
  - Offset – In cadrul paginii, adresa relativa
- Memoriile Cm-urilor trebuie sa aiba posibilitatea de dublu acces



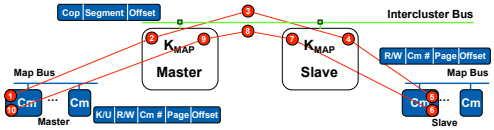
Comunicarea Intercluster

Comunicarea Intercluster



1.Cm Master trimite o cerere de transfer la K_{MAP} Master
2.K_{MAP} Master pregateste mesaj/pachet intercluster si codifica cererea
3.Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
4.K_{MAP} Slave decodifica cererea primita si o trimite catre Clusterul sau local
5.Cererea de ciclu de memorie este trimisa la Cm Slave

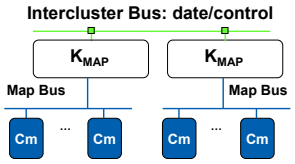
Comunicarea Intercluster



6. Cm Slave transmite rezultatul catre K_{MAP} Slave
7. K_{MAP} Slave pregateste mesajul intercluster - reactivarea contextului respectiv
8. K_{MAP} Slave transmite rezultatul catre K_{MAP} Master
9. K_{MAP} Master receptioneaza si interpreteaza mesajul primit
10.Rezultatul este trimis catre Cm Master

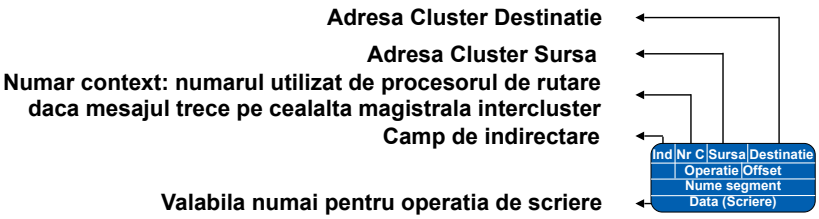
Protocolul Magistralei Intercluster

- Magistrala intercluster
  - Contine linii de date si control
  - Nu contine linii de adresa pt ca se realizeaza comutare de pachete si nu de circuite
- Comunicarea este:
  - Asicrona
  - De tip intrebare-raspuns (Q & A)
  - Cu interblocare completa – pentru fiecare actiune se primeste raspuns si abia apoi se anuleaza comanda
- Mesajele intercluster sunt formate din 1-8 cuvinte
- Cele mai utilizate: mesajele directe si de raspuns

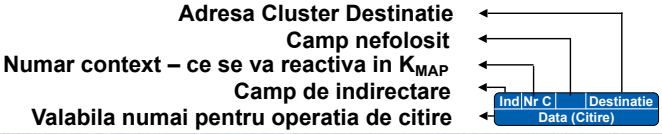


Formatul Mesajelor Intercluster

- Mesajul Direct – format din 4 cuvinte



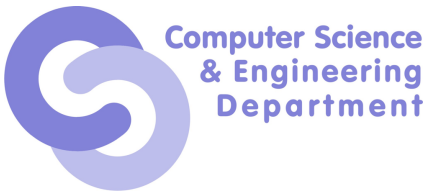
- Mesajul de Raspuns – format din 2 cuvinte



Ce am invatat in aces curs?

- Caracteristicile arhitecturilor MIMD
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in cadrul unui Cluster
- Comunicarea Intercluster

Thank you for your attention.



emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

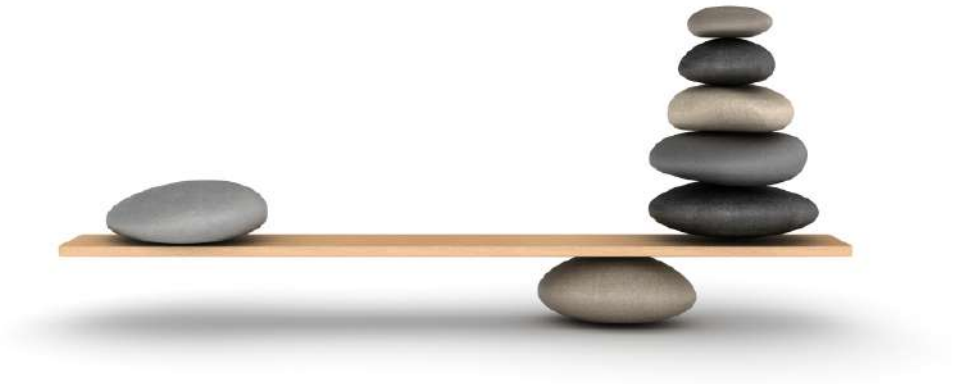
Arhitectura Sistemelor de Calcul

Curs 11
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

Cuprins

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”



Nevoia de a Compara Sisteme de Calcul

Motivatie

- Lectura specificatiilor unor sisteme de calcul nu este suficienta pentru a ne oferi o imagine completa asupra performantelor acestora
- Asadar, a fost necesara crearea unor teste (benchmark-uri) care sa permita compararea performantelor intre sisteme/arhitecturi diverse
- Testele constau din seturi de programe care sunt menite sa scoata in evidenta performantele diverselor componente ale unui SC:
  - Floating Point CPU Performance
  - Caracteristici de interconectare (Bandwidth/Latenta)
  - Memorii externe (Hard-Drives)

Caracteristici & Probleme

- Sunt create pentru a “simula” comportamentul aplicatiilor reale (sintetice)
- Cele mai bune benchmark-uri sunt aplicatiile utilizator in sine…
- Trebuie sa fim intotdeauna circumspecti cand utilizam benchmark-uri!
- Cateva probleme:
  - Producatorii HW fac “tuning”
  - Performantele masinilor se degradeaza cand load-ul depaseste o valoare de prag
  - Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea NU sunt obiectul unui benchmark

Tipuri de Benchmark-uri

- Aplicatii reale: procesoare de text, aplicatii utilizator
- Kernel: contine doar parti esentiale dintr-o varietate de programe (i.e. Linpack), cu rezultate in MFlops
- Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componente ale sistemului
- Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe care sa respecte proportia aflata astfel (Whetstone/ Dhrystone)
- Pentru sisteme I/O
- Pentru sisteme paralele

Cum Masuram Performanta?

Latency vs. Throughput

- **Latency** (timpul de executie): timpul necesar terminarii unui task anume
- **Throughput** / bandwidth (latimea de banda): numarul de taskuri/transferuri intr-un timp fix
  - Exploateaza paralelismul pentru throughput si nu pentru latenta
  - Deseori latenta si throughputul pun presiuni contradictorii pe sisteme
  - Definitii diferite in functie de cod
    - Simulare Stiintifica - **Latency**
    - Web-Server - **Throughput**

Exemplu: Latency vs. Throughput

- Cum mutam oameni 10 km?
  - Masina: capacitate = 5; viteza = 60 km/h
  - Autobuz: capacitate = 60; viteza = 20 km/h
- Latency: **Masina = 10 min**; Autobuz = 30 min
- Throughput: Masina = 15 ppl/h; **Autobuz = 60 ppl/h**

Comparand Performanta - Speedup

- A este de X ori mai rapid ca B daca
  - X = Latency(B)/Latency(A)
    - Imparte la cel mai rapid
  - X = Throughput(A)/Throughput(B)
    - Imparte la cel mai incet
- A este cu X% mai rapid ca B daca
  - Latency(A) = Latency(B) / (1 + X/100)
  - Throughput(A) = Throughput(B) * (1 + X/100)

Exemplu: Latency vs. Throughput

- Latency
  - Masina este de **3** ori (si **200%**) mai "rapida" ca Autobuzul

- Throughput
  - Autobuzul este de **4** ori (si **300%**) mai "rapid" ca Masina

Speedup si % Crestere/Descrestere

- Programul A ruleaza 200 cicli - facand un task X
- Programul B ruleaza 350 cicli - facand acelasi task X
- % crestere si descrestere **nu sunt la fel**:
  - % crestere: ((350-200)/200)*100 = **75%**
  - % descrestere: ((350-200)/350)*100 = **42,3%**
- Speedup:
  - 350/200 = 1.75: A este de 1.75 ori mai rapid ca B
  - % crestere: (1.75 - 1)*100 = 75%
- Daca C e 1.5x mai rapid ca A, cati cicli ruleaza C?
  - 133 cicli, adica 50% mai rapid ca A

Calculul Performantei - Medii

- Media Aritmetica: (1/N) * $\sum_{P=1..N}$ Latenta(P)
  - Pentru marimi ce sunt proportionale cu timpul
- Media Armonica: N / $\sum_{P=1..N}$ 1/Throughput(P)
  - Pentru marimi ce sunt invers proportionale cu timpul
- Media Geometrica: $\sqrt[N]{\prod_{P=1..N}}$ Speedup(P)
  - Pentru marimi adimensionale

Exemplu de Medie Armonica

- Conduceti 2 km
  - 30 km/h pentru primul km
  - 90 km/h pentru al doilea km
- Care este viteza medie?
  - Atentie: raspunsul **nu este 60 km/h**
  - 0.033 h / km pentru km unu
  - 0.011 h / km pentru km doi
  - 0.022 h / km in medie = **45 km/h**

Performanta Procesoarelor vs. Workloads

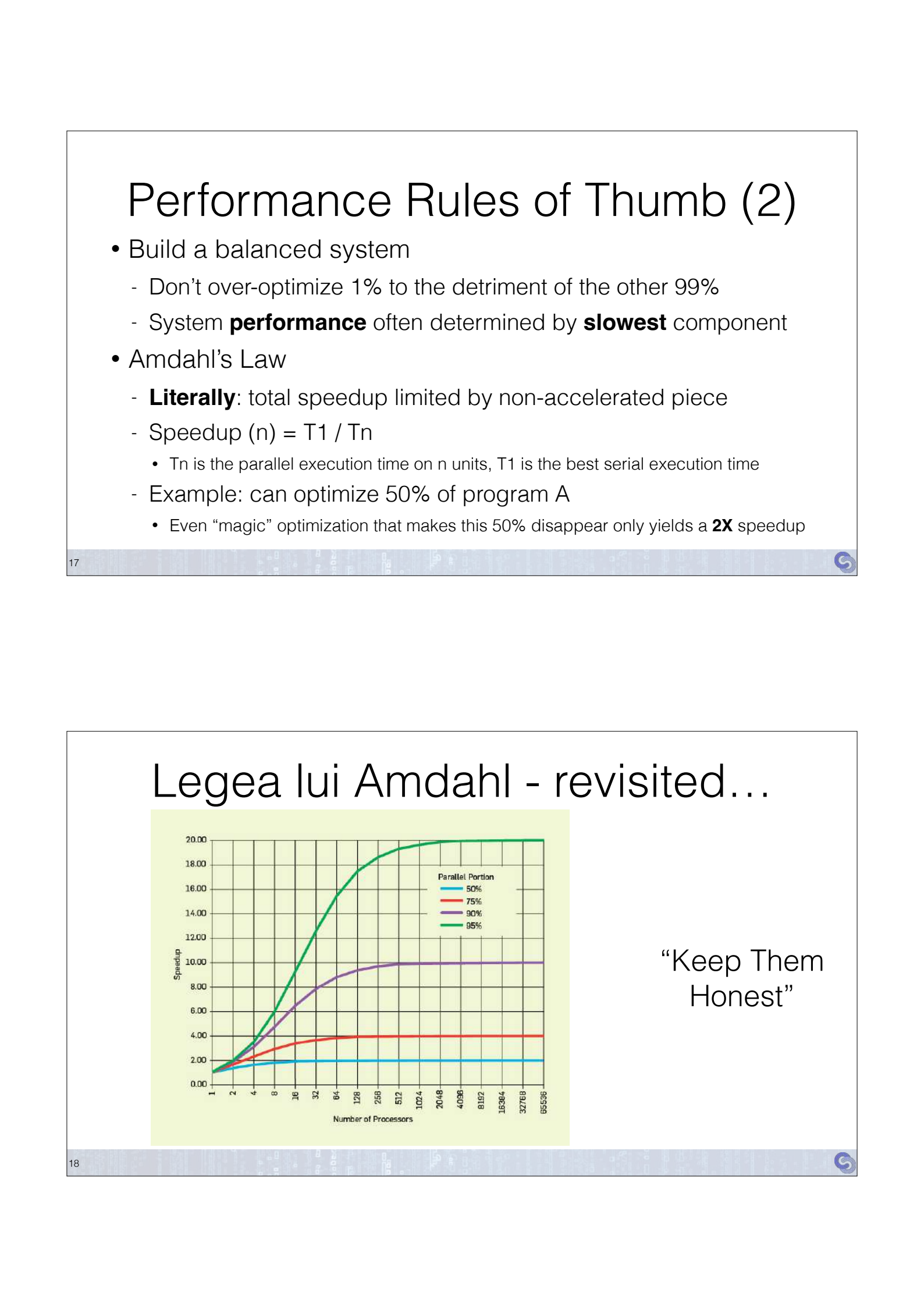
- Q: Ce inseamna performanta unui procesor?
- A: Nimic fara un workload asociat.
- Benchmarks
  - Reprezinta un set standard de workload-uri
  - Utilizate pentru a compara performanta mai multor sisteme
  - Reprezentative programelor reale sau chiar programe reale
- Micro-Benchmarks
  - Set non-standard de non-workload-uri
  - Programe mici utilizate pentru izolarea unor chestiuni de performanta
  - Nereprezentative pentru manifestari complexe: turnuri Hanoi, 8-queens, etc.

Performance Rules of Thumb

- Design for actual performance, not peak performance
  - Peak performance: “Performance you are guaranteed not to exceed”
  - Greater than “actual” or “average” or “sustained” performance
  - For actual performance X, machine capability must be > X
- Easier to “buy” bandwidth than latency
  - Which approach is easier, to transport more cargo via train?
    - Build another track?
    - Make a train that goes twice as fast?
  - Use bandwidth to reduce latency

Performance Rules of Thumb (2)

- Build a balanced system
  - Don't over-optimize 1% to the detriment of the other 99%
  - System **performance** often determined by **slowest** component
- Amdahl's Law
  - **Literally**: total speedup limited by non-accelerated piece
  - Speedup (n) = T1 / Tn
    - Tn is the parallel execution time on n units, T1 is the best serial execution time
  - Example: can optimize 50% of program A
    - Even "magic" optimization that makes this 50% disappear only yields a **2X** speedup



Legea lui Amdahl - revisited…

"Keep Them Honest"

Benchmark-uri pentru Masini “Seriale”

SPEC

- SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web
- Incearca sa simuleze situatii reale:
  - SPECweb testeaza performantele unui server de web pentru cereri HTTP paralele
  - SPEC CPU testeaza CPU-ul ruland 29 de aplicații INT si FP
- SPEC sunt scrise in C sau Fortran
  - Compilatoarele pot avea optimizari pentru acest tip de aplicatie
- Se distribuie cu o licenta
- Exista un conflict cu GPL (foloseste cod GPL)… http://www.spec.org/

Performanta SPEC

- Un sistem cu SPEC 150 este de 1.5 ori mai rapid decat un sistem cu SPEC 100
- Numarul SPEC generic se calculeaza din performanta celor 29 de aplicatii:
  - Medie geometrica
  - Medie aritmetica
  - Medie aritmetica ponderata in functie de prioritatile utilizatorului

GeekBench

- Benchmark-uri multicore multi-platforma
  - Laptop, workstation, iPhone, Android, etc.
- Performanta: Int, FP, Memorie, Bandwidth, etc.
- Salvarea rezultatelor online
  - Usor de comparat cu alte sisteme/procesoare
- Problema: workload-uri simple ce nu “streseaza” complet sistemele

Exemple GeekBench

- Laptop:
  - Intel “Ivy bridge” at 3.4 GHz (4 cores) 11,456
- Workstation
  - Intel Core i7-3520M at 2.9 GHz (2 cores) 7,807
- Platforme mobile:
  - iPhone 4S - Apple A5 at 0.8 GHz (2 cores) 642
  - iPhone 5 - Apple A6 at 1 GHz (2 cores) 1,589
  - Samsung G S3 - Snapdragon S3 – 1.5 GHz (2 cores) 1,429

TPC & BAPCo & Sandra

- TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date:
  - http://www.tpc.org/
- Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc:
  - WEBmark, SYSmark, si MobileMark
  - http://www.bapco.com/
- SiSoftware Sandra (System ANalyser, Diagnostic and Reporting Assistant) este un program de testare si diagnoza:
  - http://www.sisoftware.net/

Benchmark-uri pentru Masini “Paralele”

The LINPACK Benchmark

- Linpack folosit pentru sisteme paralele pentru ca
  - Este extrem de raspandit si utilizat
  - Date si teste pentru acest program sunt disponibile pentru **toate** sitemele relevante din lume
- Benchmark-ul masoara cat de repede rezolva computer-ul un sistem dens de ecuatii liniare $A \cdot x = b$ de dimensiune $n \cdot n$
- Solutia se bazeaza pe metoda eliminarii Gaussiene, utilizand pivotarea partiala
- Conduce la $2/3 \cdot n^3 + n^2$ operatii in virgula mobila $O(n^3)$
- Se exclude utilizarea metodelor optimizate de inmultire a matricelor de genul metodei lui Strassen $O(n^{2.807})$ sau Coppersmith–Winograd $O(n^{2.376})$
- In acest mod se vor genera milioane de operatii in virgula mobila pe secunda (Mflop/s)
- Rezolvarea sistemelor de ecuatii liniare este o problema intalnita **frecvent** in aproape orice aplicatie stiintifica sau inginereasca

The LINPACK Benchmark (+)

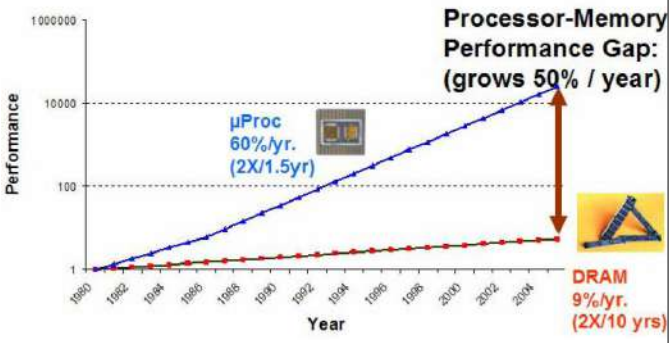
- Avantaje:
  - Ofera ca rezultat un **singur** numar
    - Rmax = performanta maxima (**obtinuta**)
    - Rpeak = performanta maxima teoretica de varf (**estimata**)
  - Usor de definit si usor de clasificat pe baza lui
  - Permite modificarea dimensiunii problemei (n x n)
  - Ocupa sistemul testat cu un job de lunga durata
  - Disponibil la: http://www.netlib.org/benchmark/hpl/

The LINPACK Benchmark (-)

- Dezavantaje
  - Scoate in evidenta **doar** viteza “peak” si numarul de procesoare
  - **Nu** evidentiaza (explicit) rata de transfer locala (bandwidth)
  - **Nu** evidentiaza (explicit) reteaua de comutare
  - **Nu** testeaza (explicit) mecanisme de tipul gather/scatter
  - Ignora legea lui Amdahl – permite doar scalare “slaba”
- Un singur numar nu poate caracteriza performanta totala a unui sistem de calcul!

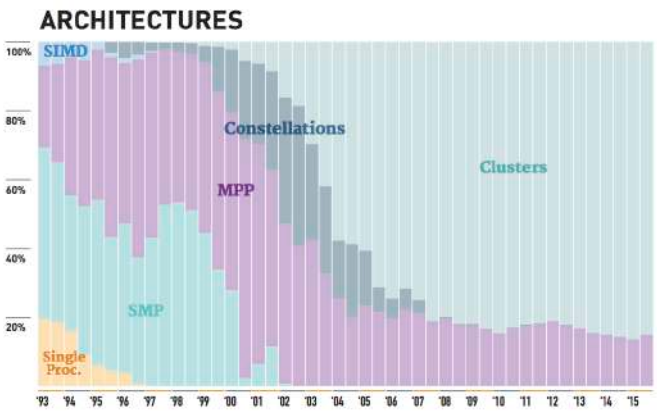
Relevanta LINPACK

- Daca luam in considerare anii 80'-90', poate LINPACK era cel mai potrivit benchmark
- Zidul memoriei era mai mult o treapta
- Memoria era mai "apropiata" de CPU:
  - n ciclii / executie instructiune
  - n ciclii / aducerea unui cuvant din memorie
- LINPACK arata de asemenea gradul de optimizare al compilatoarelor



Schimbari ale Sistemelor de Calcul

- In ultimii 40 de ani s-au schimbat foarte multe in hardware-ul sistemelor de calcul:
  - Arhitecturi vectoriale, superscalare, distribuite, cu memorie partajata, sisteme multicore, etc
- Cu toate ca LINPACK a fost modificat, nu toate modificarile reflecta progresele hardware
- Ierarhia memoriilor este mult mai complexa acum



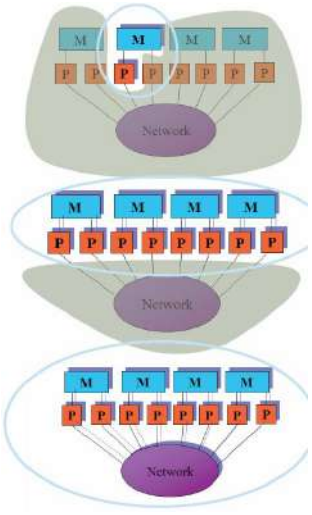
HPC Challenge Benchmark

- Un Benchmark ce evidentiaza
  - Procesoarele
  - Memoria
  - Reteaua de interconectare
- Avantajul major al HPCC
  - Arhitecturile pot fi descrise cu mult mai multe metrici decat FLOPS-urile LINPACK
- Scopuri ale HPCC
  - Complementar cu TOP 500
  - Exprimarea performatei aplicatiilor ca functie a caracteristilor accesului la memorie (localitate spatiala / temporala)
  - Permite optimizari
  - Inregistreaza efortul necesar pentru tuning-ul aplicatiilor
  - Rezultate arhivate si verificabile

HPCS = High Productivity Computing Systems
DARPA = Defense Advanced Research Projects Agency

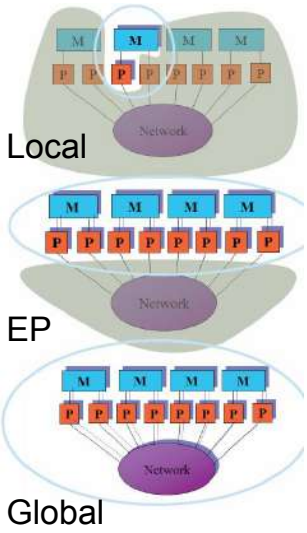
Testarea unui singur Procesor si a intregului Sistem de Calcul

- **Local**: un singur procesor participa la calculele dorite
- **Embarrasingly Parallel**: fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
- **Global**: toate procesoarele din sistem participa la calcule si comunica explicit intre ele

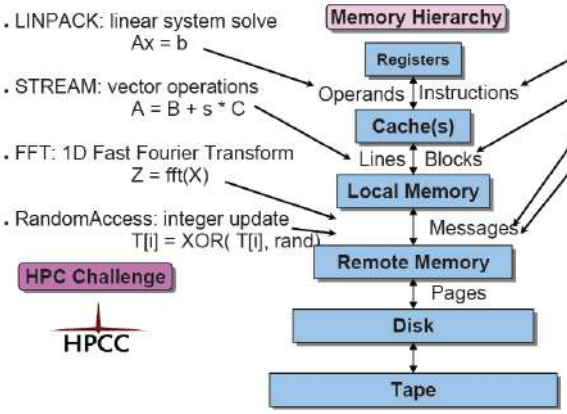


Componente HPCC

- HPCC e format din 7 benchmark-uri relevante:
  1. **HPL** (LINPACK) – MPI Global (Ax=b)
  2. **STREAM** – Local / ***STREAM** – EP (A ← B + s*C)
  3. **PTRANS** – MPI Global (A ← A + B^T)
  4. **RandomAccess** – Local, MPI Global / ***RandomAccess** – EP (T[i] ← XOR(T[i], rand))
  5. **Bandwidth** and **Latency** – MPI
  6. **FFT** – Local, Global & EP (Z ← fft(X))
  7. **Matrix Multiply** – Local & EP
- HPCC == framework pentru adaugarea de benchmark-uri



Scopul HPCC



Max	Relative
2 Pflop/s	8x
6.5 Pbyte/s	40x
0.5 Pflop/s	200x
64000 GUPS	2000x

DARPA HPCS

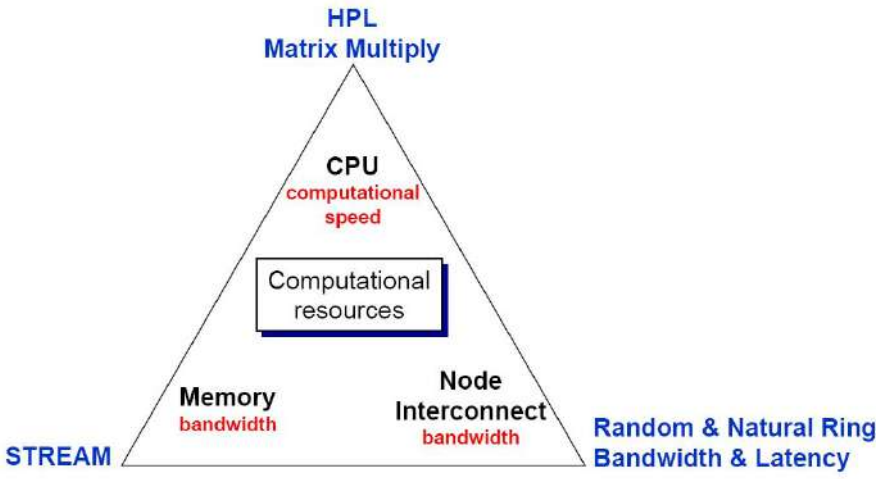
Performance Targets

- HPCS incearca prin HPCC sa:
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

Functionarea HPCC

- Benchmark-ul functioneaza ca un program de sine statator
- Similar ca fisier de intrare/specificare cu HPL
- HPCC este format dintr-un “Base Run”, urmat de eventuale “Optimization Runs”:
  - Base Run este rularea Benchmark-ului “out of the box”
  - Utilizatorul trebuie sa instaleze pe masinile de test MPI si BLAS
  - In Optimized Runs se pot inlocui anumite subrutine, pentru a creste performantele unei anume componente
- Rezultatele se arhiveaza la: http://icl.cs.utk.edu/hpcc/
- Se pot genera apoi tabele html sau Excel cu rezultatele obtinute:
  - In mod intentionat nu se va oferi un singur criteriu de evaluare
  - Astfel NU se vor obtine clasificari generale!
- Scopul declarat al HPCC: sa NU necesite mai mult de dublul unei rulari HPL

Resurse vs HPCC



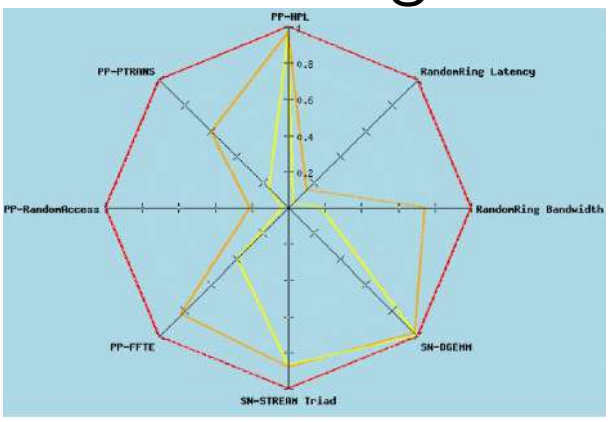
- Din ce sunt formate componentele HPCC?

Condensed Results - Base Runs Only - 114 Systems - Generated on Tue Nov 14 17:13:55 2006

System Information System - Processor - Speed - Count - Threads - Processes MA/PT/PS/PC/TH/PR/CM/CS/IC/LA/SD				G-HPL TFlop/s	G-PTRANS GB/s	G-Random Access Gup/s	G-FFTE GFlop/s	EP-STREAM Sys GB/s	EP-STREAM Triad GB/s	EP-DGEMM GFlop/s	RandomRing Bandwidth GB/s	RandomRing Latency usec
Cray Inc. Red Storm/XT3 AMD Opteron	2.4GHz	12960	125920	91.0250000	2355.9700	1.7401300	1354.0700	34840.499	2.1138	4.39989	0.03911	16.29
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	80.6830000	339.2840	0.0657312	2178.1100	53553.868	0.8172	1.80619	0.01084	8.84
IBM p5-575 Power5	1.9GHz	10240	110240	57.8670000	553.0090	0.1693440	842.5000	55184.179	5.3891	7.08552	0.11015	110.59
IBM p5-575 Power5	1.9GHz	8192	16192	45.7019000	2626.1700	0.3239760	908.6920	44455.936	5.4268	7.06423	0.08871	11.05
Cray Inc. XT3 Dual-Core AMD Opteron	2.6GHz	10404	110404	43.4033000	778.3650	0.8235630	1107.2100	25774.557	2.4774	4.78935	0.06537	14.32
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	37.0540000	4665.9100	0.1648000	1762.8200	62899.707	0.9596	2.47017	0.01039	9.63
IBM p5-575 Power5	1.9GHz	8192	26192	33.3173000	575.8230	0.2066390	965.6680	43802.460	5.3470	6.08616	0.07698	51.99
Cray Inc. XT3 AMD Opteron	2GHz	10330	110330	32.9869000	1013.0600	1.0176560	1119.2900	43581.780	4.2109	3.66719	0.16189	10.32
IBM Blue Gene/L PowerPC 440	0.7GHz	32768	132768	31.2381000	87.7818	0.2786090	1112.8100	29913.678	0.9129	2.17447	0.01197	9.91
Cray Inc. XT3 AMD Opteron	2.4GHz	5200	15200	20.5270000	874.8990	0.2653830	644.7300	26020.800	5.0040	4.39535	0.14682	25.80
Cray Inc. XT3 AMD Opteron	2.4GHz	5208	15208	20.4086000	944.2270	0.8724120	761.7290	24268.442	4.6598	4.41173	0.20636	9.20
Cray Inc. XT3 AMD Opteron	2.6GHz	4096	14096	16.9752000	302.9790	0.3330720	905.5690	20656.456	5.0431	4.75166	0.16886	9.44
Cray Inc. XT3 AMD Opteron	2.6GHz	4128	14128	16.6421000	674.7660	0.6767580	821.6770	19295.676	4.6743	4.75946	0.22245	8.22
Cray Inc. XT3 AMD Opteron	2.4GHz	3744	12744	14.3040000	608.5660	0.2202980	417.1720	18146.382	4.8468	4.41330	0.16164	25.32
Cray Inc. X1 Cray E	1.13GHz	1008	11008	12.0263000	168.0190	0.0861199	82.3684	15522.691	15.3983	14.50000	0.13667	16.30
HP XC Intel Itanium 2	1.6GHz	2048	12048	10.5616000	306.4190	0.2633320	339.8360	4606.031	1.9591	6.26484	0.15920	6.04

System Information System - Processor - Speed - Count - Threads - Processes MA/PT/PS/PC/TH/PR/CM/CS/IC/LA/SD				G-HPL TFlop/s	G-PTRANS GB/s	G-Random Access Gup/s	G-FFTE GFlop/s	EP-STREAM Sys GB/s	EP-STREAM Triad GB/s	EP-DGEMM GFlop/s	RandomRing Bandwidth GB/s	RandomRing Latency usec
SGI Columbia 2048 Intel Itanium 2	1.6GHz	2024	12024	9.0196500	18.1901	0.0481621	45.7760	3996.493	1.9758	6.23637	0.12271	6.98
NEC SX-8	2GHz	576	1 576	9.0085900	312.7670	0.0192617	160.9490	22555.750	40.9954	15.22220	0.92024	22.27
IBM p655 Power4+	1.5GHz	2048	12048	6.2054900	103.0640	0.1417230	106.3440	3651.636	1.7703	5.35073	0.08560	15.36
SGI Altix 3700 Bx2 Intel Itanium 2	1.6GHz	1008	11008	5.1283200	105.6660	0.0325982	15.6619	1907.509	1.8924	5.88404	0.20288	6.82
HP XC Intel Itanium 2	1.6GHz	1024	11024	4.8532700	154.1190	0.1615660	184.1270	2602.423	1.9555	6.24250	0.13878	5.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7823400	217.9230	0.1370020	266.6600	5274.696	4.7952	4.81050	0.23638	25.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7276600	253.3460	0.3035680	328.2860	5161.124	4.6919	4.77440	0.39964	7.29
IBM PS P575+ Power5+	1.9GHz	512	1 512	3.4100800	219.5560	0.0848539	52.2538	3997.745	5.8550	7.62342	0.05243	36.14
Cray Inc. X1E Cray X1 MSP	1.13GHz	252	1 252	3.1940900	65.2640	0.0140684	15.5052	2438.908	9.6823	14.10470	0.36024	14.90
IBM p655 Power4+	1.5GHz	1024	11024	3.1102600	51.4248	0.0983533	82.1055	1733.284	1.6927	3.72701	0.09024	10.13

Diagrame Kiviat



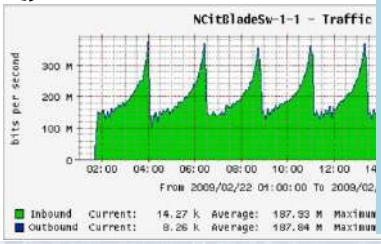
Cray XD1 AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - RapidArray Interconnect System - 11-22-2004
Dalco Opteron/GbNet Linux Cluster AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - GbNet11 - 11-04-2004
Sun Fire V20z Cluster AMD Opteron - 64 procs - 2.2 GHz
1 thread/MPI process (64) - Gigabit Ethernet, Cisco 6509 switch - 03-06-2005

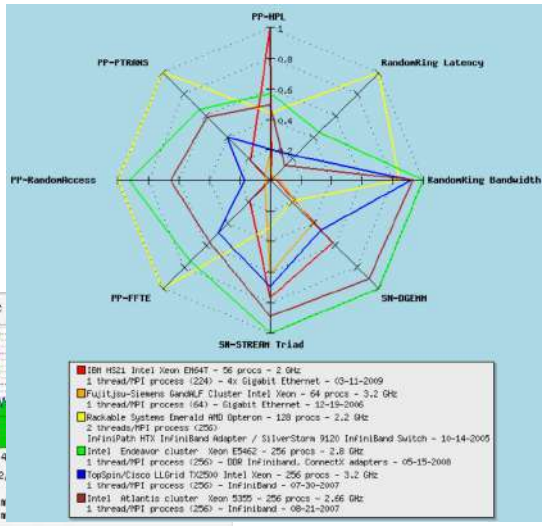
- HPL, STREAM, DGEMM: tehnologia de interconectarea nu conteaza; HPL scaleaza bine iar STREAM si DGEMM nu comunica (sunt locale)
- RandomRing Bandwidth: Cray-ul are cel mai mare bandwidth
- RandomRing Latency: Cray-ul are si cea mai mica latenta
- Random Access: Tehnologia de interconectare conteaza mult – senzitivitate la latenta
- PTRANS & FFTE: Tehnologia de interconectare conteaza mult – senzitivitate la bandwidth

HPL & HPCC @cs.pub.ro

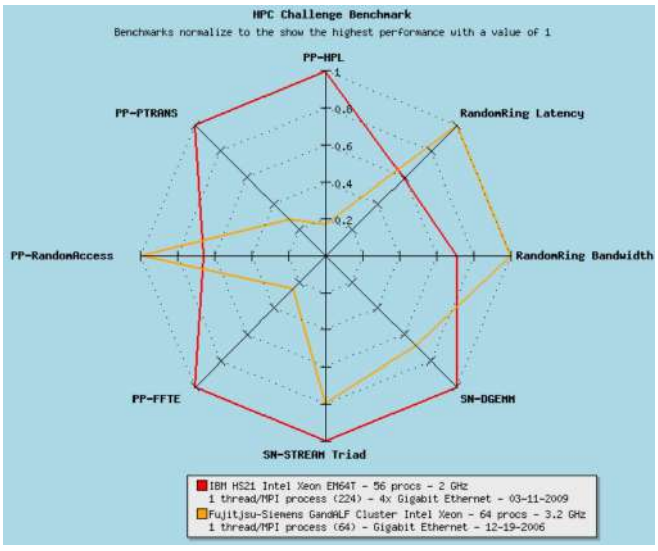
- In 2009 we broke the 1TFlops barrier with 28 of our 32 IBM HS21 blades (2xQuad-Core)
- HPL performance:
  - **1003 GFlops Max**
  - 1792 GFlops Peak
- System dimension:
  - 230.000 (Ax=b)

55.88%





Interconectare in Clustere cs.pub.ro



Dezvoltarea Viitoare a HPCC

- Proiectul doreste crearea unui cadru pentru dezvoltearea de benchmark-uri
- Se doreste dezvoltarea unor “semnaturi” ale fiecarei masini
- In viitor, colectia de benchmark-uri trebuie marita
  - De exemplu pentru operatii cu matrici rare
- Portarea HPCC pe cat mai multe sisteme
- Implementarea diversa a HPCC
  - In noi limbaje: UPC, Co-Array, Chapel, Charm++,…
  - In noi medii si paradigme: Spark, 0MQ, Flink,…

Alte Benchmark-uri Paralele

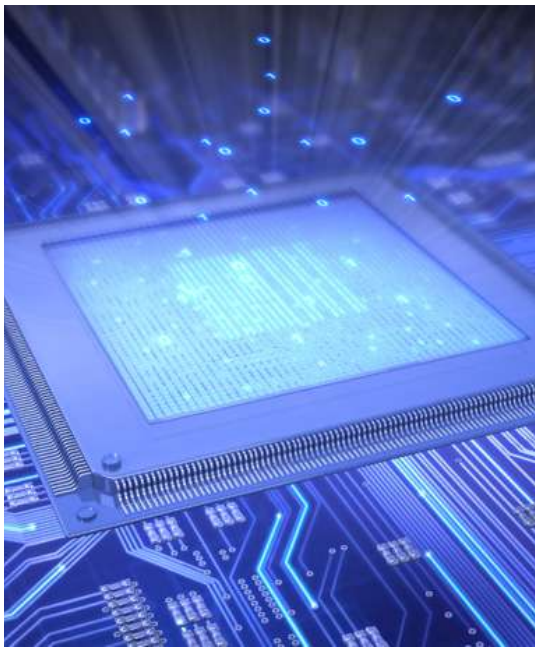
- SPLASH2 - Stanford Parallel Applications for Shared Memory
  - http://www.capsl.udel.edu/splash/
- NAS Parallel Benchmark
  - https://www.nas.nasa.gov/publications/npb.html
- SPECopenMP
  - Parallelized (OpenMP) versions of SPECfp 2000
- SPECjbb
  - Java multithreaded database-like workload

Ce am invatat in aces curs?

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”

Slides & Support Material Acknowledgements

CIS 371 - University of Pennsylvania 2013
CS/ECE 3810 - University of Colorado 2017
University of Tennessee 2017
http://www.netlib.org 2017



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 12
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”

Cuprins

- Top 500 List
- Arhitecturi si Aplicatii Relevante
- Trend de dezvoltare

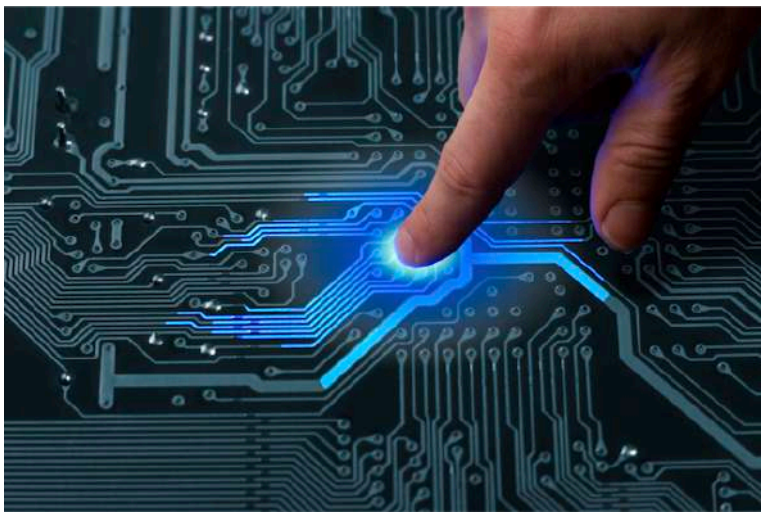
Top 500 List

top500.org

- Proiect demarat in Iunie 1993
- Lista actualizata a celor mai puternice 500 de sisteme de calcul de uz general, disponibile comercial in lume si folosite la aplicatii complexe
- Statisticile referitoare la supercalculatoare sunt interesante pentru
  - Dezvoltatorii de sisteme (producatorii de HW & SW)
  - Utilizatori
  - Potentialii viitori utilizatori
- Se doreste cunoasterea
  - Numarului si locatiei sistemelor
  - Domeniilor de aplicatie ale sistemelor → facilitarea unor potentiale colaborari

top500.org

- Lista este publicata de 2 ori pe an
- Benchmark – versiune paralela a Linpack – HPL (Jack Dongarra @ ICL, University of Tennessee)
  - http://www.netlib.org/benchmark/hpl/
  - Rezolvarea unui sistem dens de ecuatii liniare Ax = b
  - Nu reflecta intr-un mod exhaustiv performanta intregului sistem!
  - Dimensiunea problemei poate fi modificata pentru a se potrivi cel mai bine pe arhitectura sistemului masurat
- Listele sunt prezentate la @Supercomputing Conference – US si International Supercomputing Conference – Germania

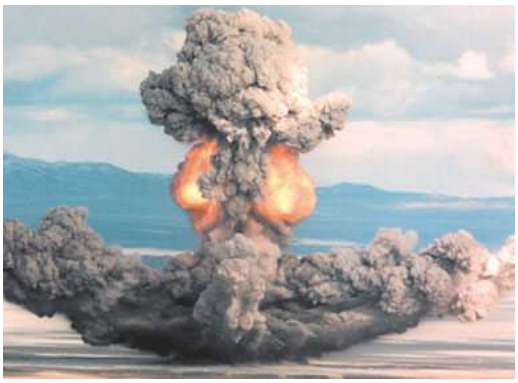


Arhitecturi si Aplicatii Relevante

Motivatie - HW Then…

- Cylindrical so wires are short
- Freon Cooling
- 8M Pounds
- Aldermaston Atomic Weapons Establishment

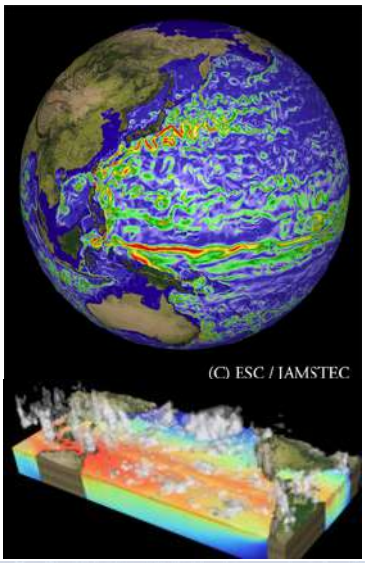
Motivatie - Aplicatii



No (more) Kaboom…

Motivatie - The Earth-Simulator

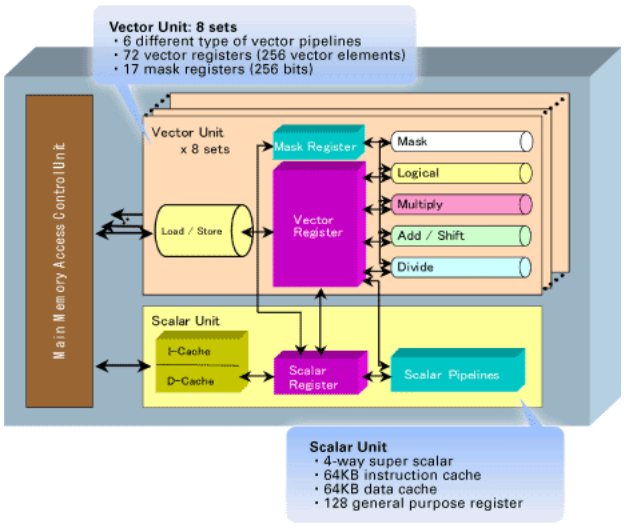
- Simulari atmosferice pentru Agentia Aerospatiala a Japoniei
- Simulari oceanice pentru Centrul Japonez de Stiinte si Tehnologii Marine
- Simulari industriale
- Simulari atomice pentru Insititutul Japonez de **Cercetari Atomice**



Sistemul NEC Earth Simulator

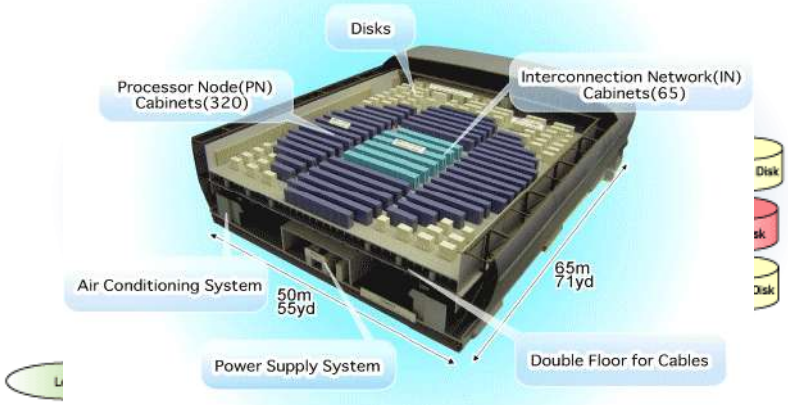
- Site: The Earth Simulator Center
- Familia de sisteme: NEC Vector
- Model: SX6
- Procesor: NEC 1000MHz
- OS: Super-UX
- Arhitectura: MPP
- Aplicatii: Studiul mediului, Cercetari Atomice
- Anul instalarii: 2002
- Numar de procesoare: 5120
- Rmax(TFlops): **35**.86
- Rpeak(TFlops): **40**.96
- Interconectare: Multi-stage crossbar
- No 1 intre 2002-2004

Procesoare Vectoriale SX6



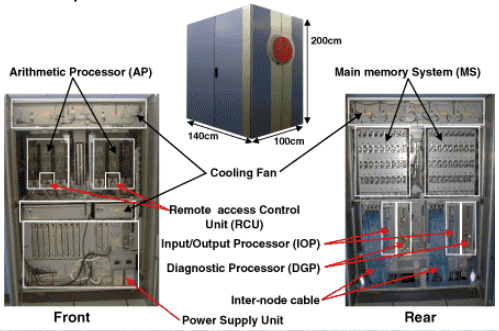
- Unitate Scalara (SU)
- Unitate Vectoriala (VU)
- Frecventa – 500 MHz
- SU – procesor super-scalar: 64Kb instr cache; 64Kb data cache; 128 registrii de uz general
- VU – 72 registrii vectoriali cu 256 elemente & 8 seturi de 6 benzi de asamblare pentru operatii logice si aritmetice
- Tehnologie de fabricaţie: LSI, 0.15µm CMOS, 60 mil. tranzistori
- Consum 140 W

Arhitectura HW Earth-Simulator



Consum & Structura Earth Simulator

- Fiecare 2 noduri sunt instalate într-un container (1x1.4x2m)
- Fiecare container consumă 20 KW → 8MW
- Memoria sistemului este de 10TB
- Spatiu de stocare pe disc 700 TB
- Stocare in masa pe banda 1.6 PB



Columbia / SGI Altix @NASA

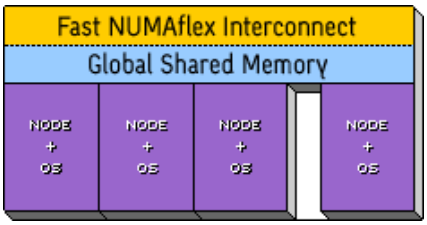
- Site: NASA/Ames Research Center/NAS
- Familia de sisteme: SGI® Altix™
- Model: SGI® Altix™ 3700, Voltaire Infiniband
- Procesoare: Intel IA-64 Itanium 2 1.5 GHz
- OS: SuSE Linux Enterprise Server 9
- Arhitectura: MPP
- Aplicatii: Cercetari aerospatiale
- Anul instalarii: 2004
- Numar de procesoare: 10240
- Rmax(TFlops): **51**.870
- Rpeak(TFlops): **60**.960
- Interconectare: Numalink/Infiniband

Arhitectura Columbia

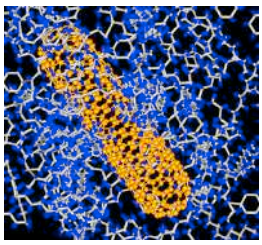
- Numele sistemului
  - In memoria echipajului de pe Columbia - 1 februarie 2001
- 20 superclustere Altix™ din seria 3000
  - 8 SGI Altix 3700 si 12 Altix 3700 Bx2
  - 4 din Bx2-uri formeaza un 2048-PE (Processor Shared Memory Environment)
- Procesoare Intel Itanium 2
  - 1.5GHz, 6MB Cache
- Pana la 24 TB de Global Shared Memory pe fiecare cluster

Interconectare Columbia

- Nodurile sunt conectate
  - Voltaire InfiniBand
  - Ethernet de 1 si 10 Gb/s
- Columbia este conectat la un on-line RAID printr-o conexiune Fibre Channel → 440TB
- Conectarea procesoarelor prin SGI®NUMAlink™ → design modular
  - 2048 de procesoara folosesc NumaLink si pentru conectarea dintre noduri
  - Singura tehnologie care ofera global shared memory si intre noduri, nu doar in cadrul unui nod



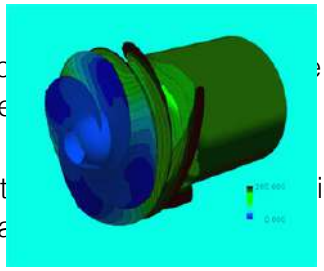
Software Columbia

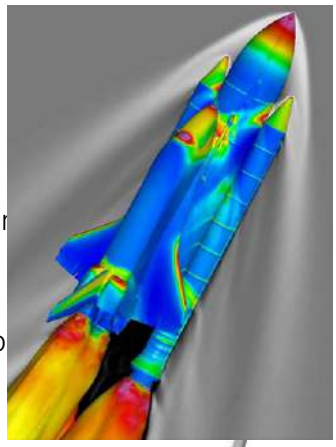


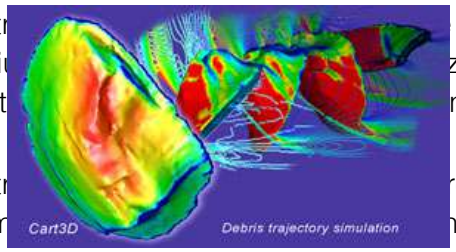
- OS: SGI ProPack 4 – bazat pe SUSE Linux
- Compilatoare Intel® Fortran/C/C++ & Gnu
- Fiecare nod de 512 procesoare ofera
  - Latenţă mică la accesul mem (<1 ms) → reduce overheadul in comunicatie
  - Global shared mem de 1TB → procesele mari rămân rezidente
- Optim pentru aplicatii cu comunicatie masiva intre procesoare
  - Simulari fizice in care domeniul este discretizat → CFD
  - Prognoza meteo & Nanotehnologii
  - N-Body simulations → Astrofizica

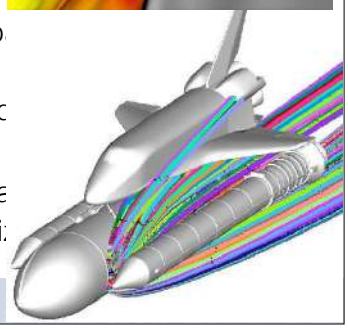
Aplicatii Columbia

- Cart3D
  - Prezice traiectoria unei buc e in timpul ascensiu
  - Culorile reprezinta presiune
- Debris
  - Calculeaza traiectoriile rest icolul orbital in timp
  - Evalueaza periculozitatea a
- Overflow
  - CFD utilizat pentr vehiculelor aerosp
  - Campul de presi za de 2.46Mach, la
  - Culorile reprezint n jurul vehiculului, c
- Phantom
  - CFD utilizat pentr re in motoare cu rea
  - Presiunea in pom navetei spatiale utili







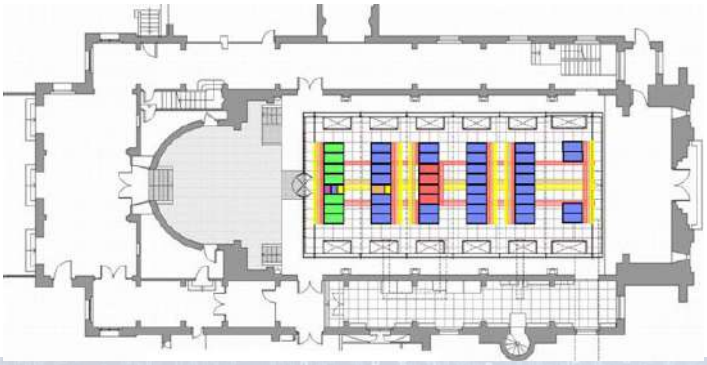


MareNostrum JS20 Cluster

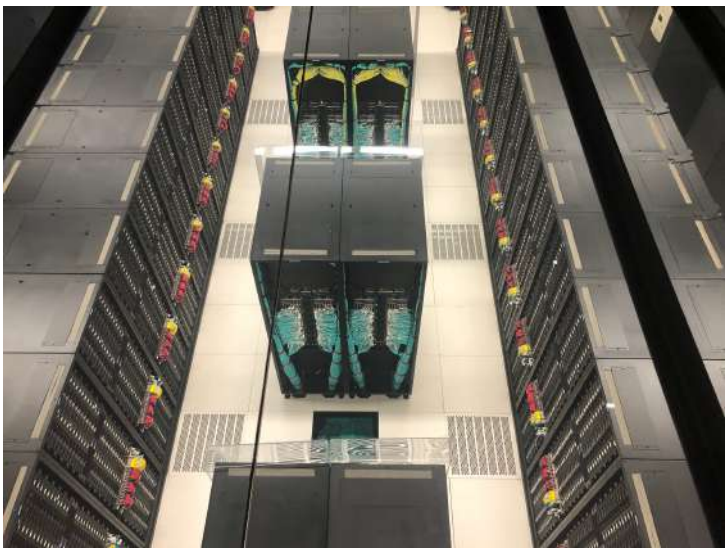
- Site: Barcelona Supercomputing Center
- Familia de sisteme: IBM Cluster
- Model: BladeCenter JS20 Cluster
- Procesoare: PowerPC 970 2.3 GHz
- OS: SuSE Linux Enterprise Server 9
- Arhitectura: Cluster
- Aplicatii: Medicina
- Anul instalarii: 2006
- Numar de procesoare: 10240
- Rmax(TFlops): **62**.63
- Rpeak(TFlops): **94**.21
- Interconectare: Red Myrinet

Arhitectura MareNostrum

- Memorie totala 20TB
- Capacitate de stocare: 370TB
- Este format din 44 de rack-uri ce ocupa 120m^2

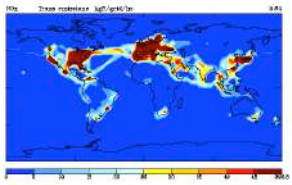


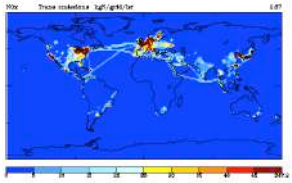
Arhitectura MareNostrum

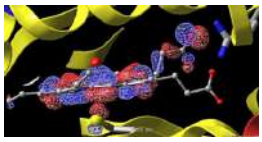


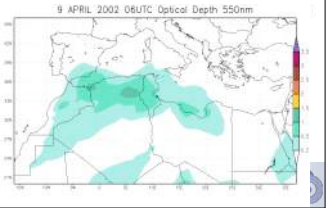
Aplicatii MareNostrum

- Computational Sciences
  - Computer Architecture
  - Performance Tools
  - Grid Computing & Clusters
- Earth Sciences
  - Air Quality
  - Meteorological Modelling
  - Mineral Dust
  - Climate Change
- Life Sciences
  - Molecular Modelling & Bioinformatics
  - Computational Genomics
  - Electronic & Atomic Protein Modelling



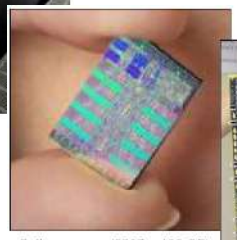






IBM RoadRunner

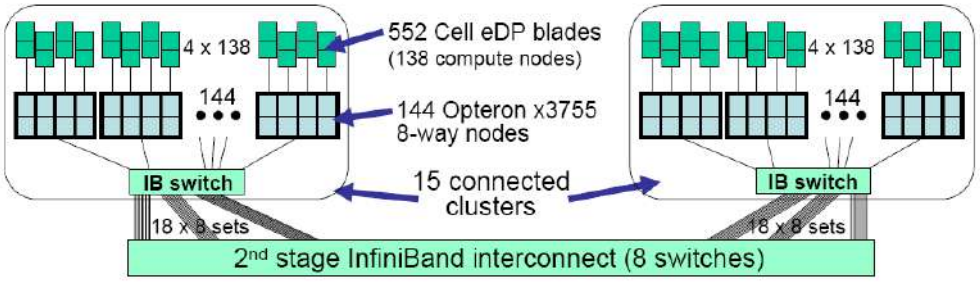
- Site: DOE/NNSA/Los Alamos NL
- Familia de sisteme:
- Model: BladeCenter QS22 Cluster
- Procesoare: PowerXCell 8i 3.2 GHz
- OS: Linux
- Arhitectura: Cluster
- Memorie: 104TB
- Aplicatii: Cercetare
- Anul instalarii: 2008
- Numar de procesoare: 122.400
- Rmax(PFlops): **1**,042 (NMax 2,25M)
- Rpeak(PFlops): **1**,376
- Consum: 2345 KW
- Interconectarea: Voltaire Infiniband



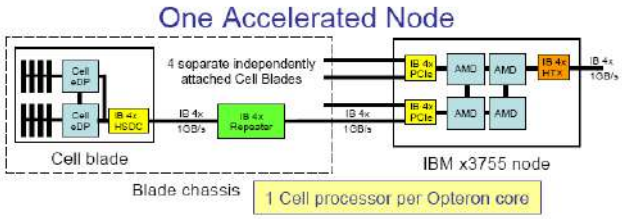
Cell processor (2007, ~100 GF)

CM-5 board with vector units (1994, 1 GF)

Arhitectura RoadRunner



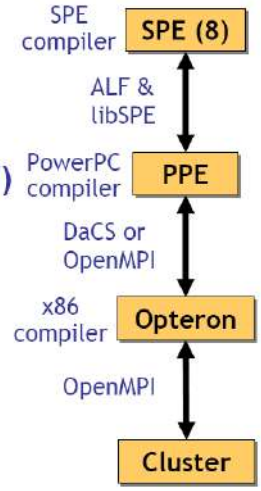
8,640 dual-core Opterons
⇨ **76 TeraFlops**
16,560 Cell eDP chips
⇨ **1.7 PetaFlops**



Los Alamos NATIONAL LABORATORY

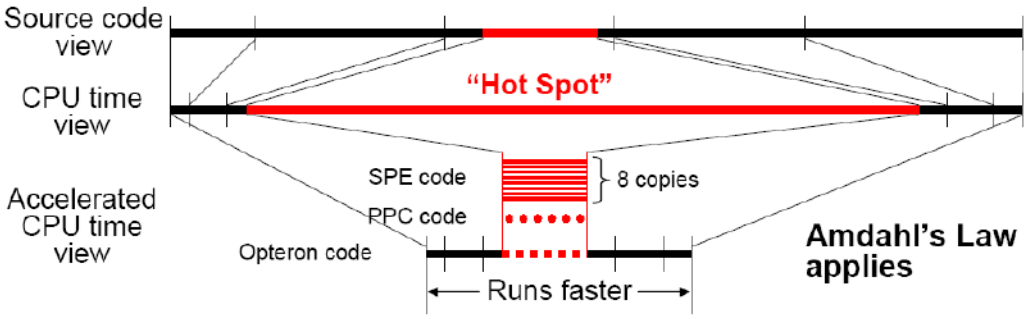
Programare RoadRunner

- **Computational Library (ALF w/ IBM)**
  - Task & work block queuing & management
  - Streaming & user-defined data partitioning
  - Process management
  - Error handling
- **Communication Library (DaCS w/ IBM)**
  - Data movement & synchronization
  - Process management & synchronization
  - Topology description
  - Error handling
  - First implementation may leverage OpenMPI
- **Longer term**
  - ALF & DaCS support in tools
  - ALF from Opteron ⇒ Cell directly
  - Compilers supporting some of this



Los Alamos NATIONAL LABORATORY

Programare Hibrida RoadRunner



- Hybrid programming will be a challenge!
  - No compiler switches to "just use the Cells"
  - Not even a single compiler – 3 of them
  - Code developer/architect must decompose application and create cooperative program pieces

JuQueen BlueGene/Q

- Site: Forschungszentrum Juelich (FZJ)
- Model: IBM-BG/Q – 28 Racks
- Procesor: IBM PowerA2, 1.6 GHz
- OS: CNK/Linux
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2012
- Numar de core-uri: 458.752
- Rmax(PFlops): **5**,009
- Rpeak(PFlops): **5**,872
- Memorie: 448 TB
- Interconectare: Custom 5D Torus

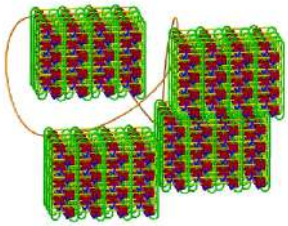
ASC@UPB: Monica Bugeanu & Alex Calotoiu

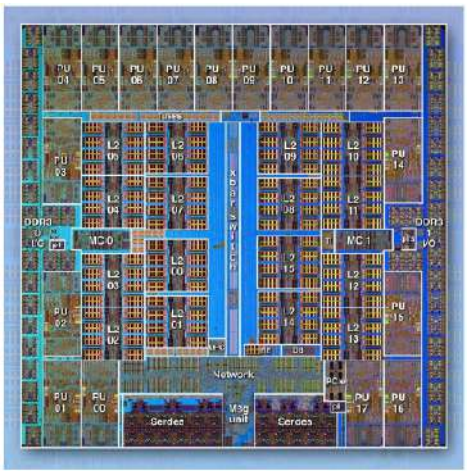
Arhitectura IBM BlueGene

- Program initiat de IBM in 1999 pentru a construi “a petaflop scale machine”
- BlueGene/L – primul pas, bazat pe procesoare PowerPC
  - Spatiu de adresare mare
  - Compilatoare standard
  - Bazat pe middleware de “message passing” deja existent
  - A necesitat adaugiri semnificative fata de sistemul PowerPC standard
- Un nod computational = computer-on-a-chip – ASIC
  - Procesoare IBM PowerPC
  - Memorie DRAM embedded
  - L1-3 cache embedded
  - Multiple module de interconectare folosind retele de comutare de mare viteza

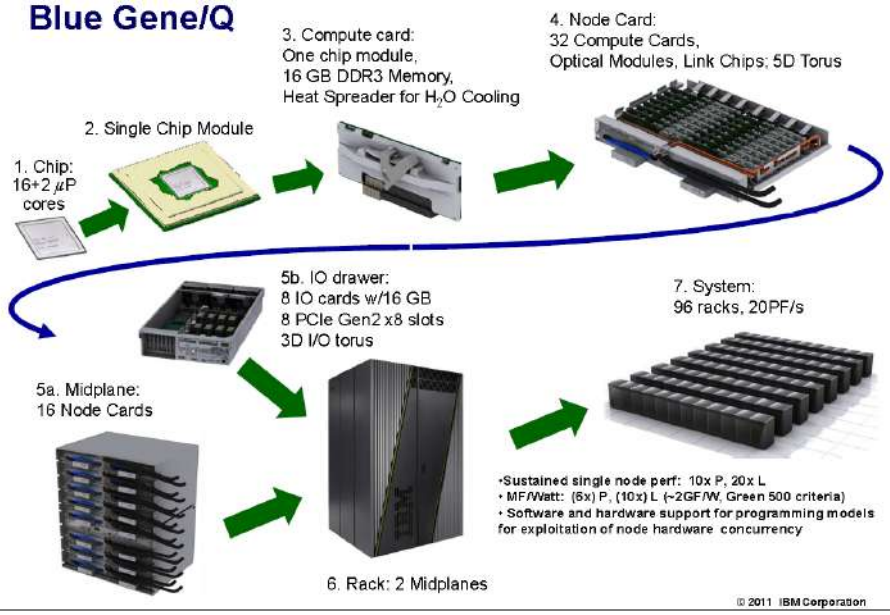
Arhitectura IBM BlueGene

- Memorie
  - 1GB DDR RAM / core – 16GB pe nod
  - Doua controlere de memorie externa de tip DDR integrat on-chip la 1.33GHz
- Procesor
  - 18C @ 1.6GHz: 16 computationale, unul pentru OS, unul de backup
  - 32MB DRAM L2 Cache: transactional memory & executie speculativa
  - Quad SIMD floating point unit
  - 45nm cu performanta maxima de 204.8 GFLOPS
  - Consuma 55W la 19.19mm si 1.47 B tranzistoare
- Interconectare
  - Tor 5D cu un router pe nod cu 11 link-uri
  - Procesor I/O extern dedicat – 2GB/s
  - Core-urile legate printr-un crossbar
- I/O Extern
  - Noduri dedicate pentru I/O extern
  - Reteaua este de tip arborescent
- OS - CNK/Linux
- Software: MPI, OpenMP, Charm++, GlobalArrays, Posix IO

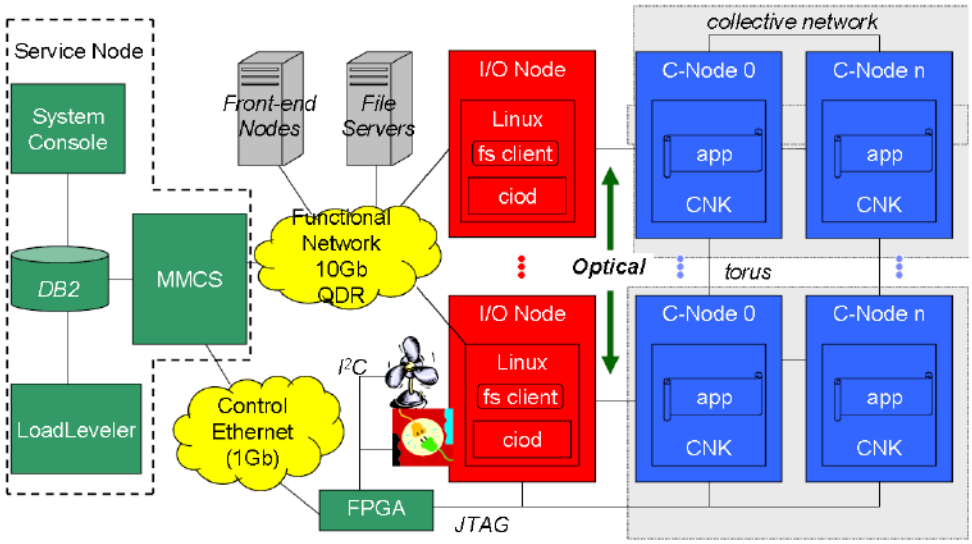




Arhitectura IBM BlueGene

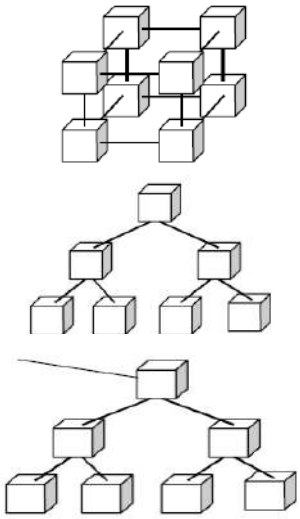


Arhitectura IBM BlueGene

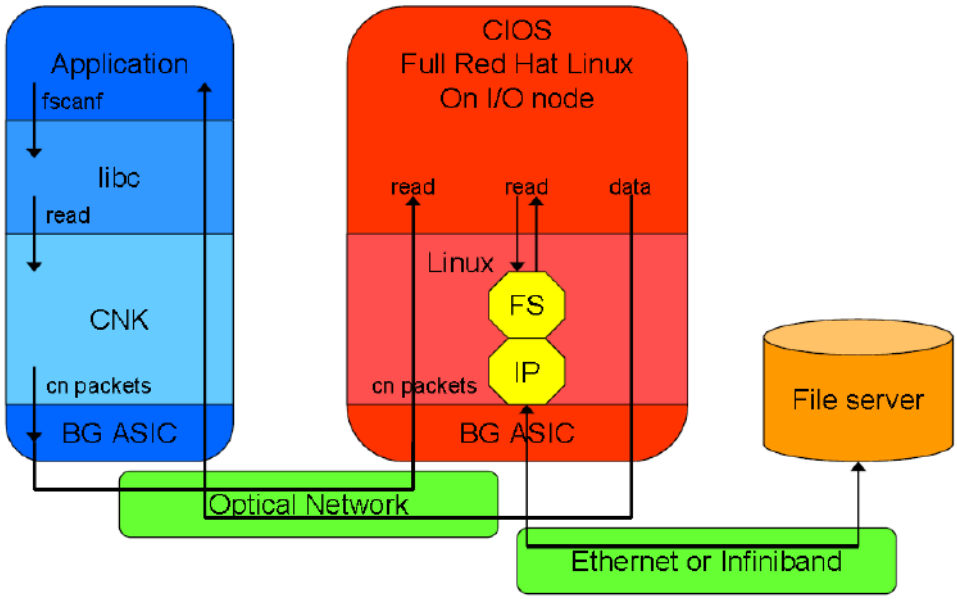


Retele de Comunicatie IBM BlueGene

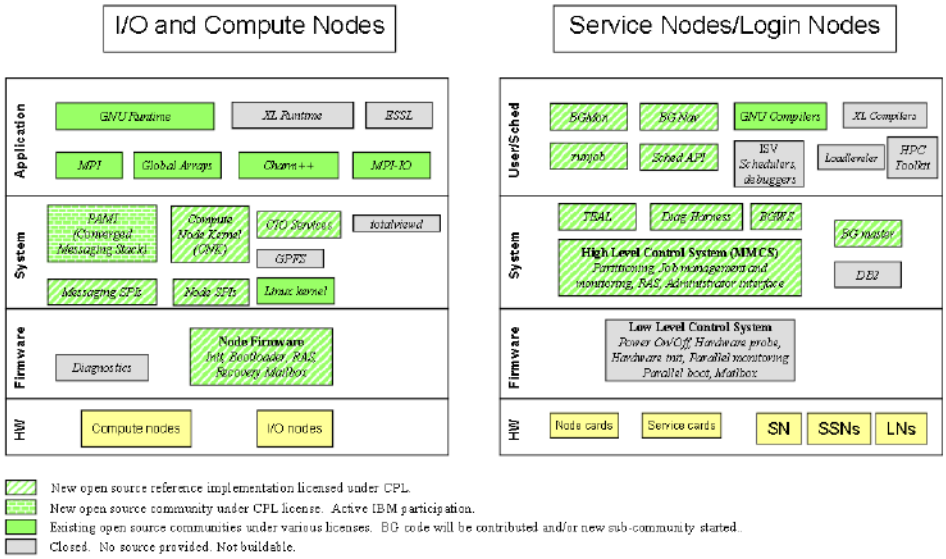
- Nodurile sunt atasate la 5 retele de comunicatie
  - Retea toroidala 5D pentru comunicatii intre noduri
  - Retea colectiva de comunicatii
  - Retea globala de intreruperi si bariere
  - Pentru I/O – Gigabit Ethernet
  - Joint Test Access GroupGigabit Ethernet – pentru control si monitorizare
- Favorizeaza aplicatii care comunica cu vecinii imediati
- BG e dezvoltat pentru aplicatii cu volum mare de date



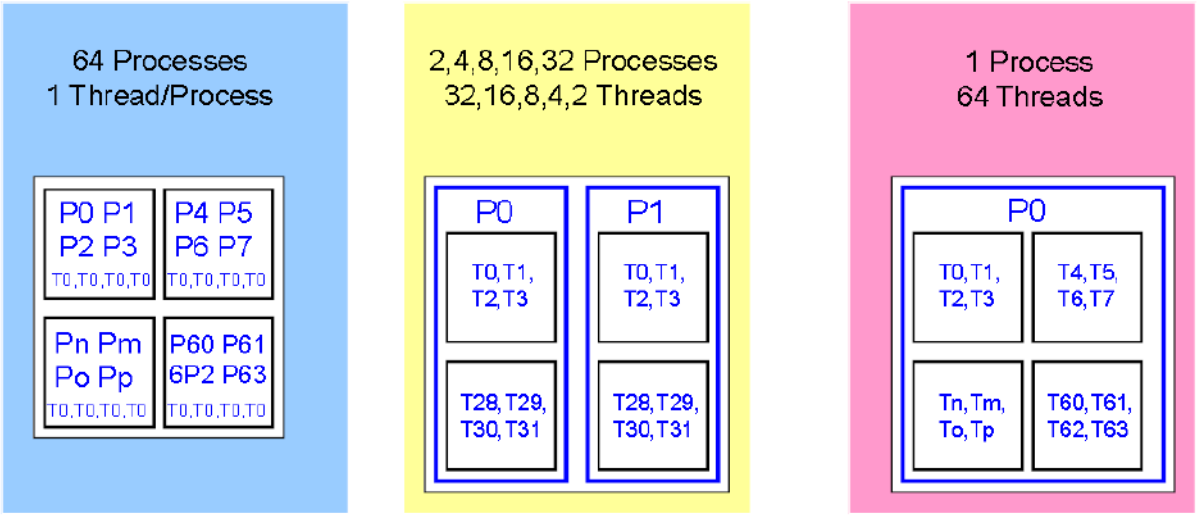
I/O IBM BlueGene



IBM BlueGene Software Stack

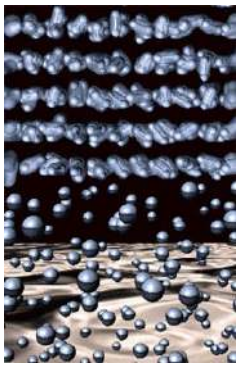


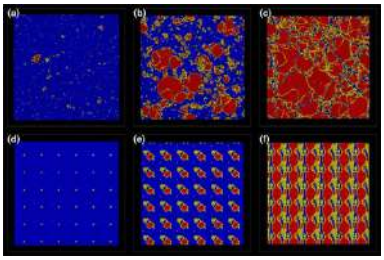
IBM BlueGene – Mod de executie



Aplicatii IBM BlueGene

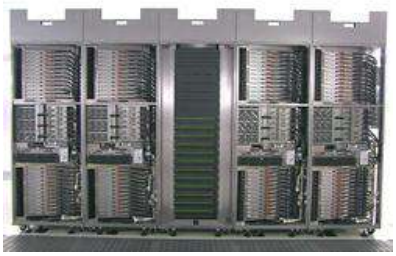
- Analiza proteinelor
- Interactiunea intre medicamente si proteine
- Data Mining
- Fizica atomica: similar cu ASC Purple
- Dinamica moleculara (inclusiv ab-initio)
- Hidrodinamica steady state si turbulenta
- Astrofizica

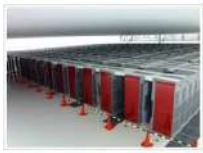




K Computer SPARC64

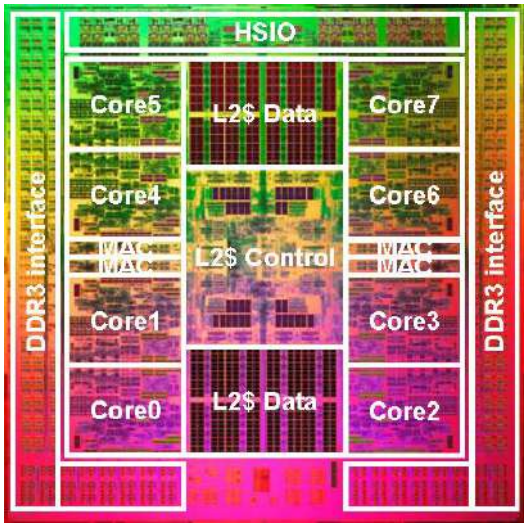
- Site: RIKEN Advanced Institute for Computational Science
- Familia de sisteme: Fujistu – SPARC64 VIIIfx 2.0GHz
- Procesor: SPARC64 VIIIfx 2.0GHz (8-core)
- OS: Linux
- Arhitectura: Cluster
- Aplicatii: Cercetare
- Anul instalarii: 2011
- Numar de core-uri: 705.024
- Rmax(PFlops): **10**,510
- Rpeak(PFlops): **11**,280
- HPCG(TFlops): **602**,5
- Memorie: 1,410 TB
- Consum: 12.660 kW
- Interconectare: Custom – Tofu



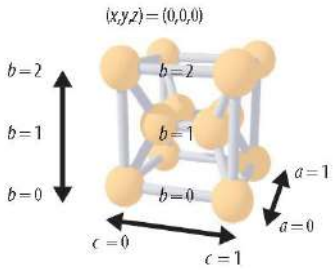


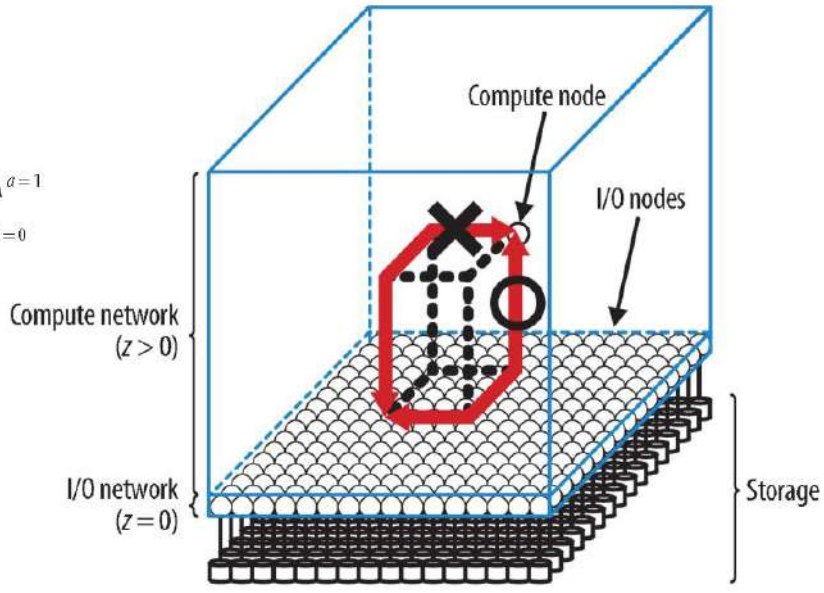
K Computer Processor

- 8C @ 2GHz – SoC design
- Embedded Memory Controller
- Shared 5M L2 Cache
- 45nm – 760M tranzistori
- 128GFlops (peak)
- 64GB/s transfer la memorie
- 58W si racire cu apa
- Set de registri mare 192 int / 256 fp
- Unitati SIMD
- Software Managed Cache



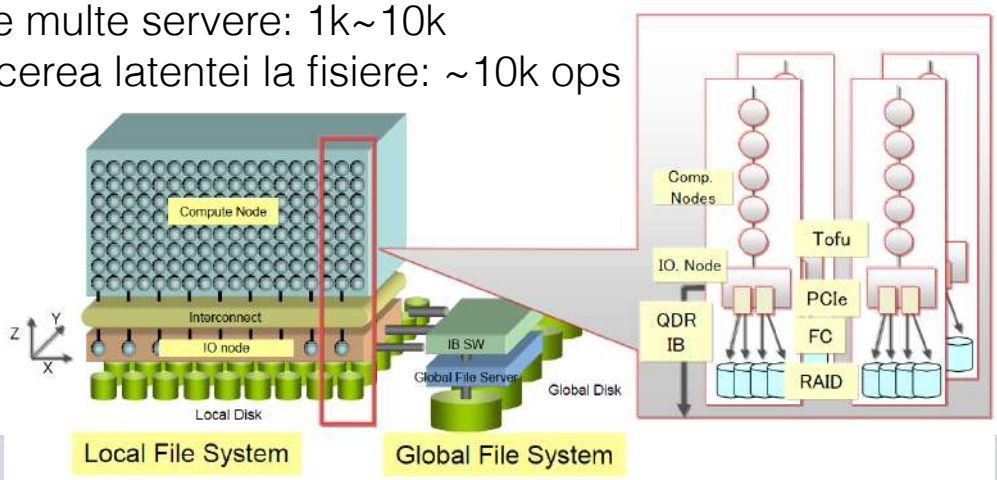
K Computer 6D Mesh/Torus Tofu Interconnect



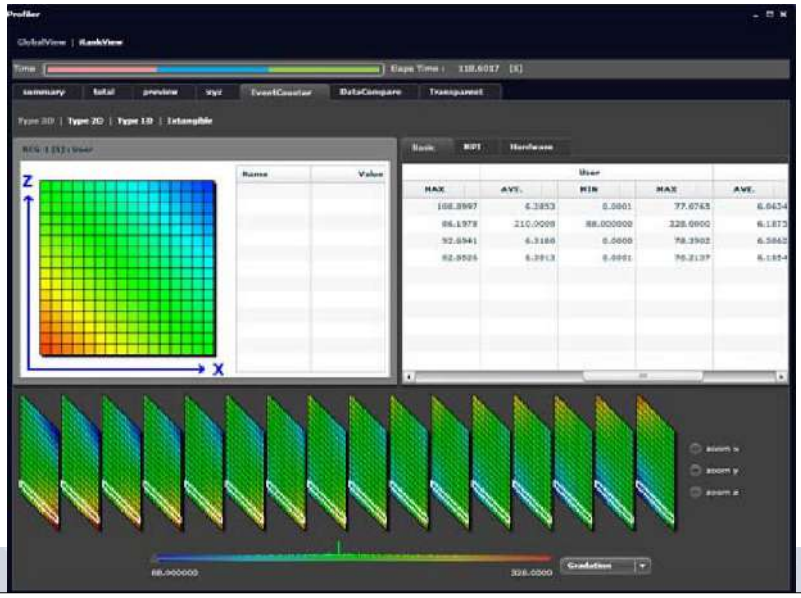


K Computer Fujitsu Exabyte File System

- Date foarte multe: 100PB~1EB
- Foarte multi clienti: 100k~1M
- Foarte multe servere: 1k~10k
- Reducerea latentei la fisiere: ~10k ops



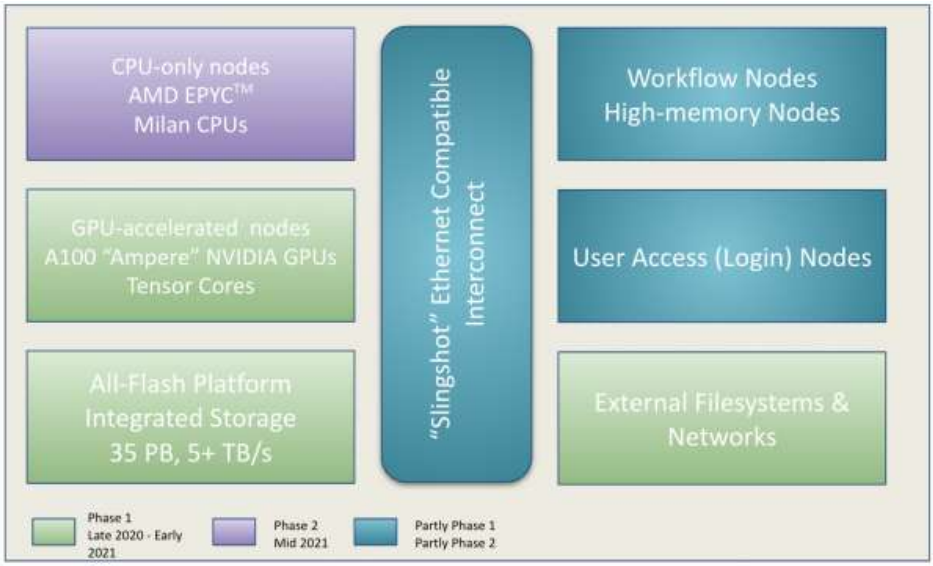
K Computer Performance Tuning



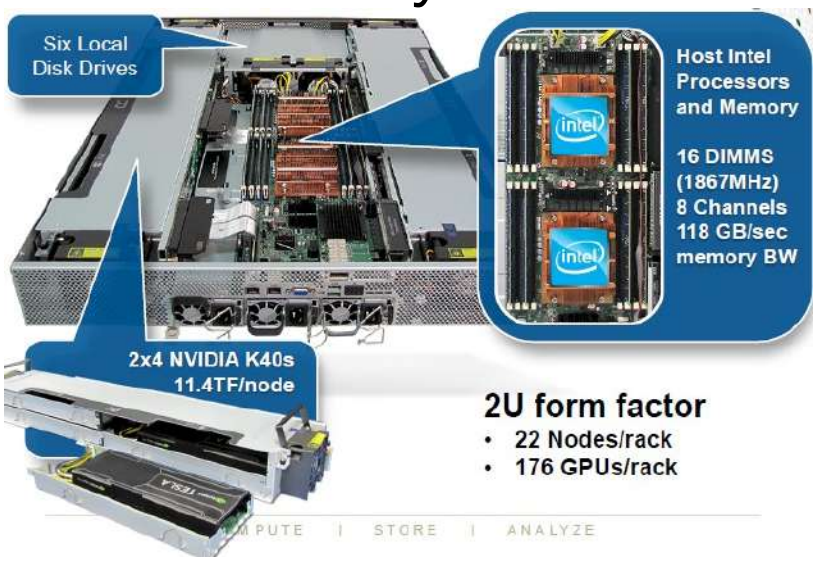
8/2022 - Perlmutter

- Site: LBNL
- Familia de sisteme: HPE Cray EX235n
- Procesor: AMD EPYC 7763 64C 2.45GHz, NVIDIA A100 SXM4 40 GB
- OS: HPE Cray OS
- Arhitectura: MPP
- Aplicatii: Research
- Anul instalarii: 2021
- Numar de core-uri: 761,865
- Rmax(PFlops): **70**,90
- Rpeak(PFlops): **93**,75
- HPCG(TFlops): **1,905**
- Memorie: **421** TB
- Consum: **2.58** kW / **27.37** GFlop/W
- Interconectare: GBEth / Slingshot-10 (274 GB)

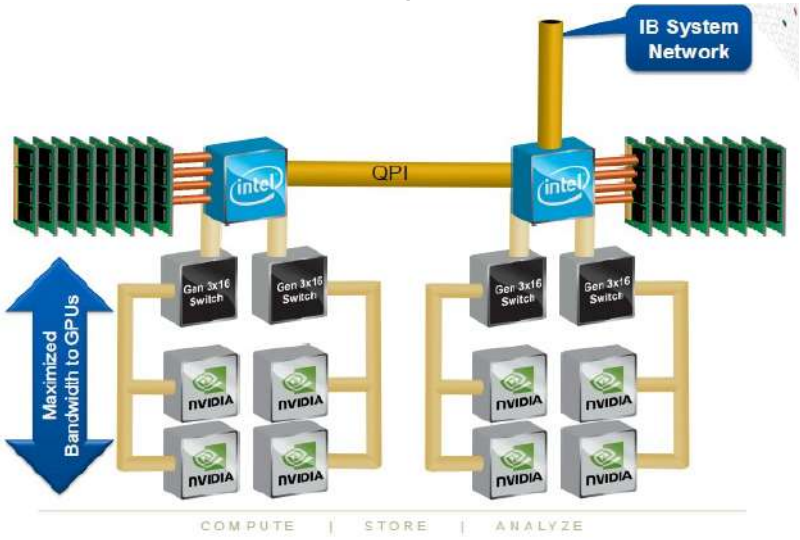
Perlmutter Architecture



Arhitectura Cray XC Series – 1



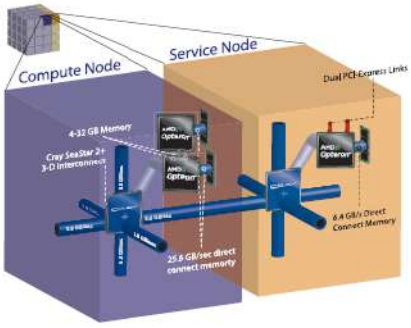
Arhitectura Cray XC Series – 2

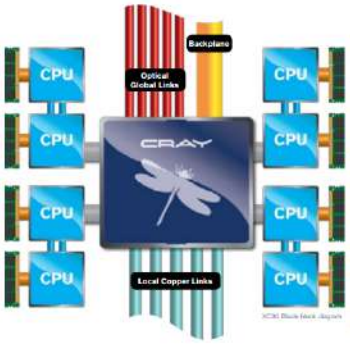


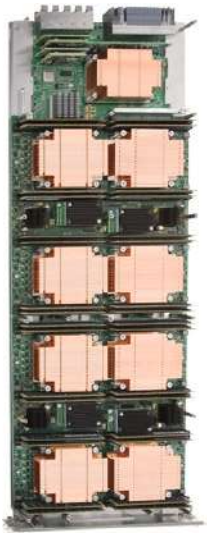
Cray XC – Software Stack

Cray HPC Cluster Software Stack						
HPC Programming Tools	Development & Performance Tools	Cray PE on CS	Intel® Cluster Studio	PGI Cluster Development Kit®	GNU Toolchain	NVIDIA® CUDA®
	Application Libraries	Cray® LibSci™, LibSci_ACC	Intel® MPI	IBM Platform MPI	MVAPICH2	OpenMPI
	Debuggers	Rogue Wave TotalView®	Allinea DDT, MAP	Intel® IDB	PGI PGDBG®	GNU GDB
Schedulers File Systems and Management	Resource Management / Job Scheduling	SLURM	Adaptive Computing Moab®, Maui, Torque	Altair PBS Professional	IBM Platform™ LSF®	Grid Engine
	File Systems	Lustre®	NFS	GPFS	Panasas PanFS®	Local (ext3, ext4, XFS)
	Cluster Management	Cray® Advanced Cluster Engine (ACE™) Management Software				
Operating Systems and Drivers	Drivers & Network Mgmt.	Accelerator Software Stack & Drivers		OFED		
	Operating Systems	Linux® (Red Hat, CentOS)				

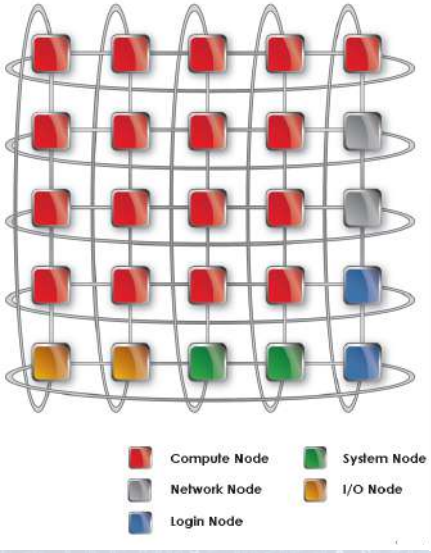
Arhitectura Sisteme Cray

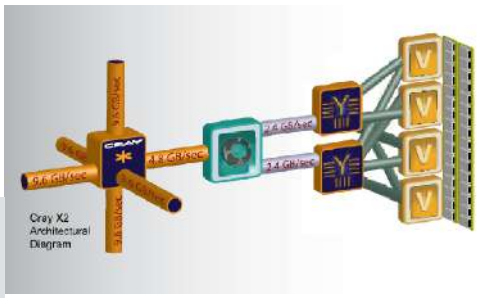




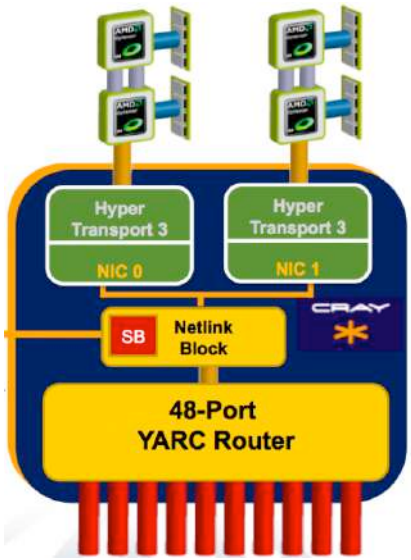


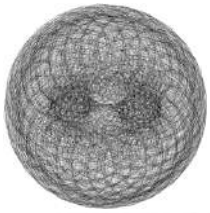
Arhitectura Sisteme Cray





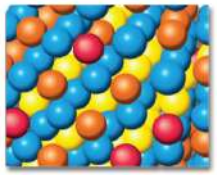
Retele de conectare Cray

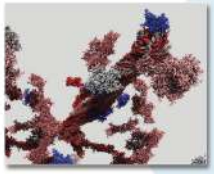




Cray - Aplicatii

WL-LSMS
Role of material disorder, statistics, and fluctuations in nanoscale materials and systems.

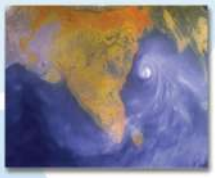




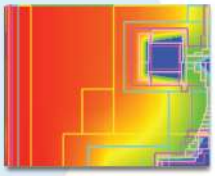
LAMMPS
A multiple capability molecular dynamics code.

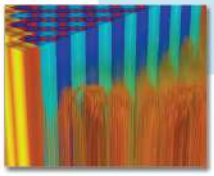
S3D
Combustion simulations to enable the next generation of diesel/bio fuels to burn more efficiently.

CAM-SE
Answers questions about specific climate change adaptation and mitigation scenarios.



NRDF
Radiation transport – important in astrophysics, laser fusion, combustion, atmospheric dynamics, and medical imaging – computed on AMR grids.



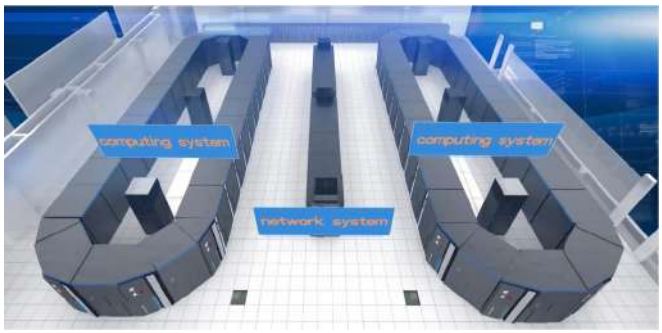


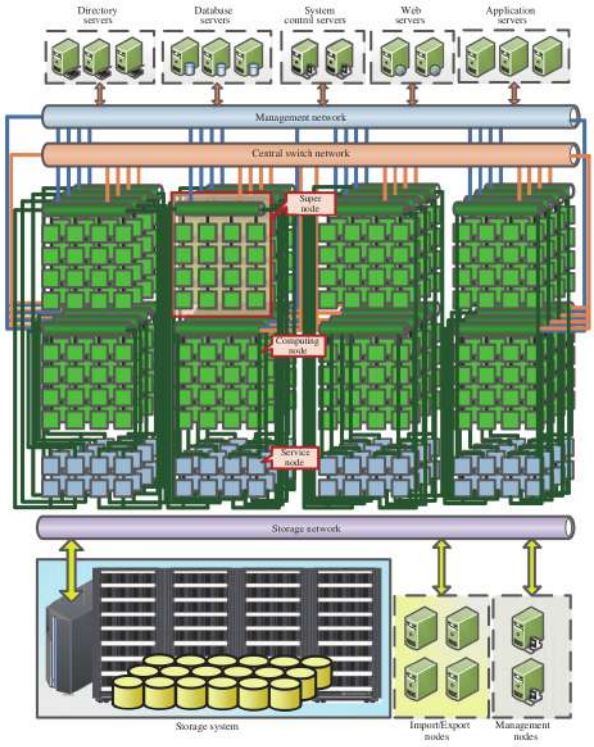
Denovo
High-fidelity radiation transport calculations that can be used in a variety of nuclear energy and technology applications.

7/2022 - Sunway TaihuLight

- Site: National Super Computer Center in Wuxi
- Model: NRCPC MPP
- Procesor: Sunway SW26010 260C 1.45GHz
- OS: Sunway RaiseOS 2.0.5
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2016-2018
- Numar de core-uri: 10.649.600
- Rmax(PFlops): **93**,02
- Rpeak(PFlops): **125**,44
- HPCG(TFlops): **481**
- Memorie: **1,311** TB
- Consum: **15,371** kW
- Interconectare: Custom Sunway

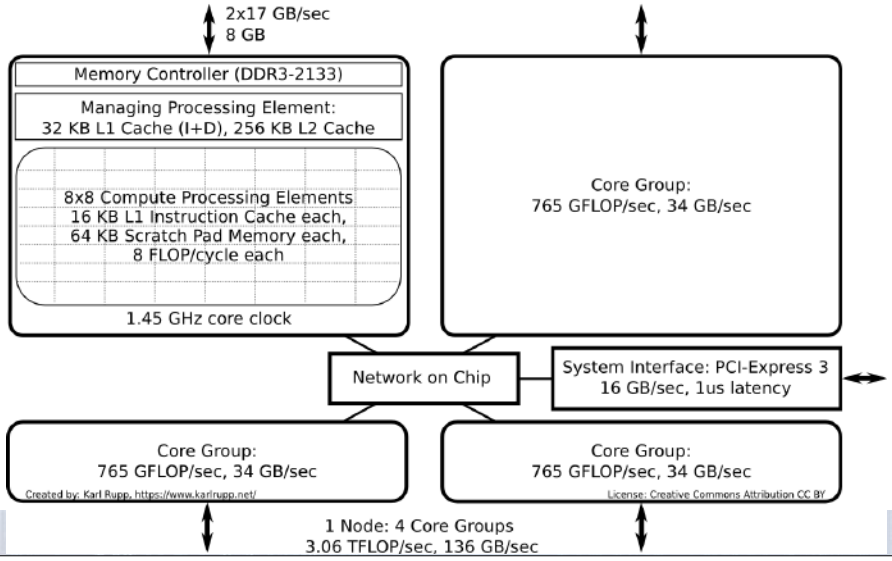
Arhitectura Sunway TaihuLight





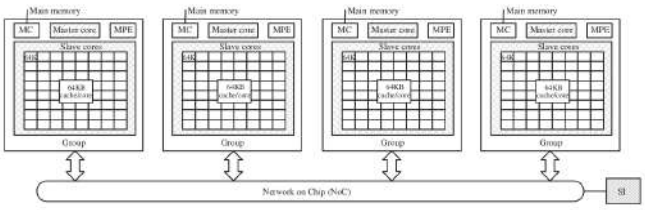
Node Architecture Sunway TaihuLight

1 Node (of 40 960) of Sunway TaihuLight

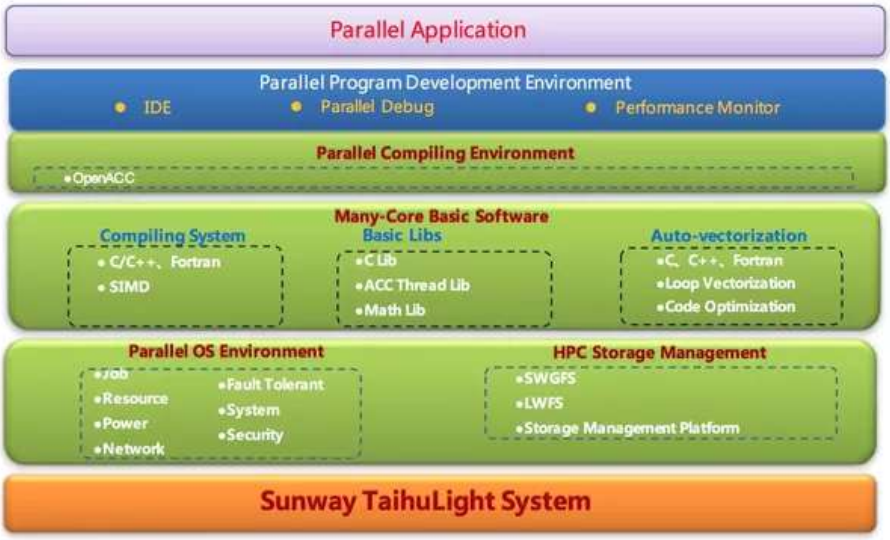


Arhitectura Sunway TaihuLight

- Performance / Watt: 6.05 GFLOPS/W - 4th Top500 Green
- 40960 Processors, each with 260 Cores having:
  - Scratchpad Memory: 64kB Data / 16kB Instruction
- Memory Bandwith: 5591 TB/s
- Network link Bandwith: 14GB/s
- System Storage: 20PB
- I/O Bandwidth: 288GB/s
- NoC Interconnect



Software Stack Sunway TaihuLight



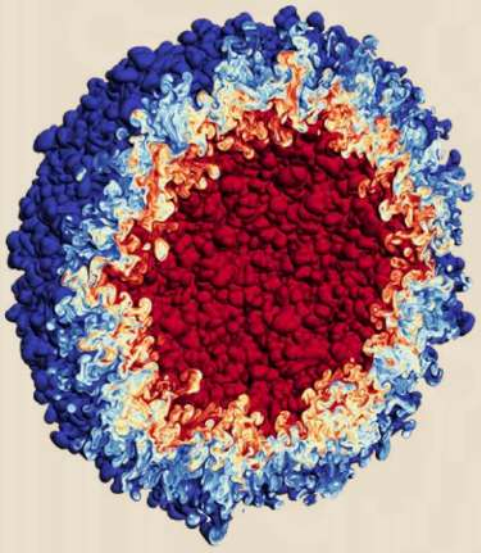
Software Sunway TaihuLight

- Simulation/Modeling Packages:
  - ANSYS: Fluent, CFX, Multiphysics
  - Comet
  - Star CD / CCM+
  - MSC Nastran
  - LS-Dyna
- Pre/post processing utilities:
  - HyperMesh
  - Tecplot
  - Gambit
  - MSC Patran

6/2022 - Sierra Power System S922

- Site: DOE/NNSA/LLNL
- Familia de sisteme: IBM Power Systems
- Procesor: IBM POWER9 22C 3.1GHz, NVIDIA Volta V100
- OS: Red Hat Enterprise Linux
- Arhitectura: Cluster
- Aplicatii: Cercetare
- Anul instalarii: 2018
- Numar de core-uri: 1,572,480
- Rmax(PFlops): **94**,64
- Rpeak(PFlops): **125**,71
- HPCG(PFlops): **1,796**
- Memorie: **1.382** TB
- Consum: **7.438** kW
- Interconectare: Dual-rail Mellanox EDR Infiniband

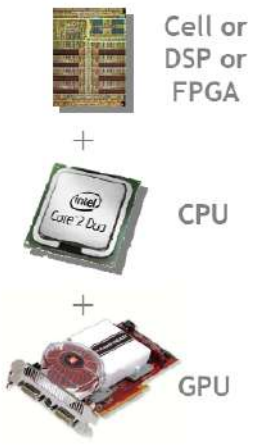
Application Highlight



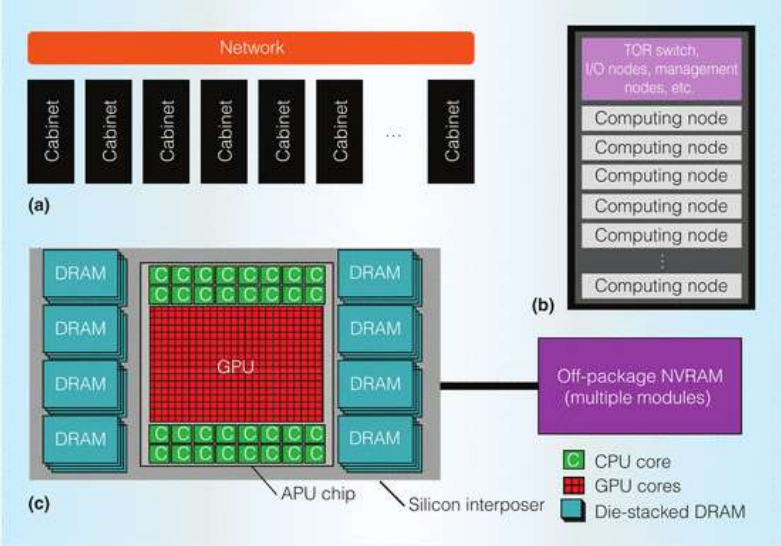
- Nuclear Security Administration's stockpile stewardship mission
- 3D Simulation instead of underground testing
- Performance of nuclear weapon systems
- Understand key issues of nuclear physics

Future Trends Processors

- New processors & accelerators:
  - Multi-core to many-core: 8, 16, 32, ... 80, 128, 1000?
  - IBM Cell , AMD Fusion, Intel Polaris, NVidia G8800
  - Distributed memory & caches at core level
  - FPGAs, GPGPU, Clearspeed CSX600, IBM Cell, XtremeData XD1000, Nvidia G80, AMD Stream Processor
- Connection standards
  - AMD Torrenza, Intel/IBM Geneseo, AMD HyperTransport Initiative
- Programming
  - IBM Roadrunner Cell libraries, RapidMind, Peakstream, Impulse C, Standford's Sequoia, NVidia CUDA, Clearspeed C, Mercury MFC, stream programming
- Heterogeneous architectures
  - Clusters of mixed node
  - Hybrid accelerated node (e.g. Roadrunner, Clearspeed, FPGA)
  - Hybrid on the same bus (e.g. CPU+GPUs, Intel's Geneseo, AMD's Torrenza)
  - Within processor itself (e.g. Cell, AMD Fusion)



Future Trends System Architecture



ASCR Roadmap Example

ASCR Computing Upgrades At a Glance

System attributes	NERSC Now	OLCF Now	ALCF Now	NERSC Upgrade	OLCF Upgrade	ALCF Upgrades	
Name Planned Installation	Edison	TITAN	MIRA	Cori 2016	Summit 2017-2018	Theta 2016	Aurora 2018-2019
System peak (PF)	2.6	27	10	> 30	200	>8.5	180
Peak Power (MW)	2	9	4.6	< 3.7	13.3	1.7	13
Total system memory	357 TB	710TB	768TB	~1 PB DDR4 + High Bandwidth Memory (HBM)+1.5PB persistent memory	> 2.4 PB DDR4 + HBM + 3.7 PB persistent memory	>480 TB DDR4 + High Bandwidth Memory (HBM)	> 7 PB High Bandwidth On-Package Memory Local Memory and Persistent Memory
Node performance (TF)	0.460	1.452	0.204	> 3	> 40	> 3	> 17 times Mira
Node processors	Intel Ivy Bridge	AMD Opteron Nvidia Kepler	64-bit PowerPC A2	Intel Knights Landing many core CPUs Intel Haswell CPU in data partition	Multiple IBM Power9 CPUs & multiple Nvidia Voltas GPUS	Intel Knights Landing Xeon Phi many core CPUs	Knights Hill Xeon Phi many core CPUs
System size (nodes)	5,600 nodes	18,688 nodes	49,152	9,300 nodes 1,900 nodes in data partition	~4,600 nodes	>2,500 nodes	>50,000 nodes
System Interconnect	Aries	Gemini	5D Torus	Aries	Dual Rail EDR-IB	Aries	2nd Generation Intel Omni-Path Architecture
File System	7.6 PB 168 GB/s, Lustre®	32 PB 1 TB/s, Lustre®	26 PB 300 GB/s GPFS™	28 PB 744 GB/s Lustre®	120 PB 1 TB/s GPFS™	10PB, 210 GB/s Lustre initial	150 PB 1 TB/s Lustre®

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	1,679.82	22,703
2	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, **Fujitsu** RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
3	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE** EuroHPC/CSC Finland	2,220,288	309.10	428.70	6,016
4	**Leonardo** - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, **Atos** EuroHPC/CINECA Italy	1,824,768	238.70	304.47	7,404
5	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, **IBM** DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	200.79	10,096

The Current Top 5 @Top 500

Ce am invatat in aces curs?

- Top 500 List
- Arhitecturi si Aplicatii Relevante
- Trend de dezvoltare

Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 13
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Top 500 List
- Arhitecturi si Aplicatii Relevante

Cuprins

- Top 5
- Concluzii Top 500

Top 5

The HPCG Benchmark

- The High Performance Conjugate Gradients (HPCG) Benchmark
  - New metric for ranking HPC systems
  - **Complementary** to the High Performance LINPACK (HPL) benchmark
  - Designed to exercise computational and data access patterns that more closely match a different and broad set of important applications
  - Implementation written in C++ with MPI and OpenMP support
  - Offers extensive GPU support

The HPCG Benchmark

- HPCG is a stand-alone code measuring performance of basic operations:
  - Sparse matrix-vector multiplication
  - Vector updates
  - Global dot products
  - Local symmetric Gauss-Seidel smoother
  - Sparse triangular solve
  - Driven by multi-grid preconditioned conjugate gradient algorithm exercising key kernels on a nested set of coarse grids

Top 5 @HPCG

Rank	TOP500 Rank	System	Cores	Rmax (PFlop/s)	HPCG (TFlop/s)
1	2	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	16004.50
2	1	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	14054.00
3	3	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,220,288	309.10	3408.47
4	4	**Leonardo** - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, Atos EuroHPC/CINECA Italy	1,824,768	238.70	3113.94
5	5	**Summit** - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	2925.75

JUNE 2023

			SITE	COUNTRY	CORES	RMAX PFLOP/S	POWER MW
1	**Frontier**	HPE Cray EX235a, AMD Opt 3rd Gen EPYC (64C 2GHz), AMD Instinct MI250X, Slingshot-11	DOE/SC/ORNL	USA	8,699,904	1,194.0	22.7
2	**Fugaku**	Fujitsu A64FX (48C, 2.2GHz), Tofu Interconnect D	RIKEN R-CCS	Japan	7,630,848	442.0	29.9
3	**LUMI**	HPE Cray EX235a, AMD Opt 3rd Gen EPYC (64C 2GHz), AMD Instinct MI250X, Slingshot-11	EuroHPC/CSC	Finland	2,220,288	309.0	6.01
4	**Leonardo**	Atos Bullsequana intelXeon (32C, 2.6 GHz), NVIDIA A100 quad-rail NVIDIA HDR100 Infiniband	EuroHPC/CINEC	Italy	1,824,768	238.7	7.40
5	**Summit**	IBM POWER9 (22C, 3.07GHz), NVIDIA Volta GV100 (80C), Dual-Rail Mellanox EDR Infiniband	DOE/SC/ORNL	USA	2,414,592	148.6	10.1

Top 5 @HPL

5 - Summit Power System AC922

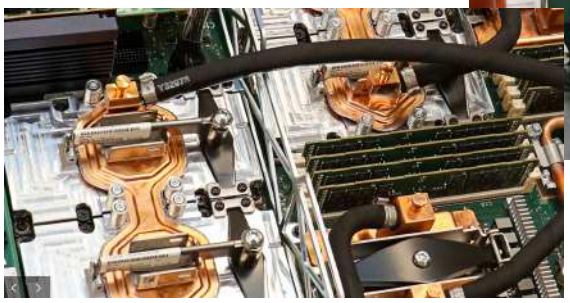
- Site: DOE/SC/Oak Ridge National Laboratory
- Familia de sisteme: IBM Power Systems
- Procesor: IBM Power9 22C 3.1GHz, NVIDIA Volta V100
- OS: Red Hat Enterprise Linux
- Arhitectura: Cluster
- Aplicatii: Cercetare
- Anul instalarii: 2018
- Numar de core-uri: 2,414,592
- Rmax(PFlops): **148**,6
- Rpeak(PFlops): **200**,8
- HPCG(PFlops): **2,**93
- Memorie: **2,802** TB
- Consum: **10.096** kW
- Interconectare: Dual-rail Mellanox EDR Infiniband

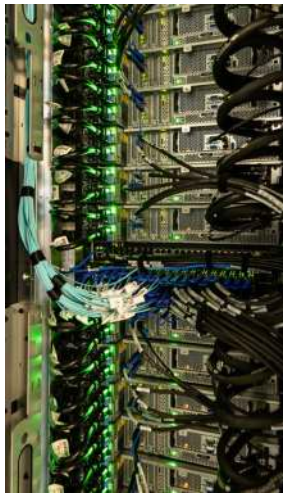
Node Stats

- Faster I/O - up to 5.6x more I/O bandwidth than x86 servers
- GPUs: 2-6 NVIDIA® Tesla® V100 GPUs with NVLink
- CPUs: 2x POWER9 CPUs
- Simplest AI architecture: Share RAM across CPUs & GPUs
- PCIe Gen4 2x faster vs PCIe Gen3 in x86

https://apps.kaonadn.net/4882011/IBMPower8/index.html#AC922;C1272

Summit Infrastructure

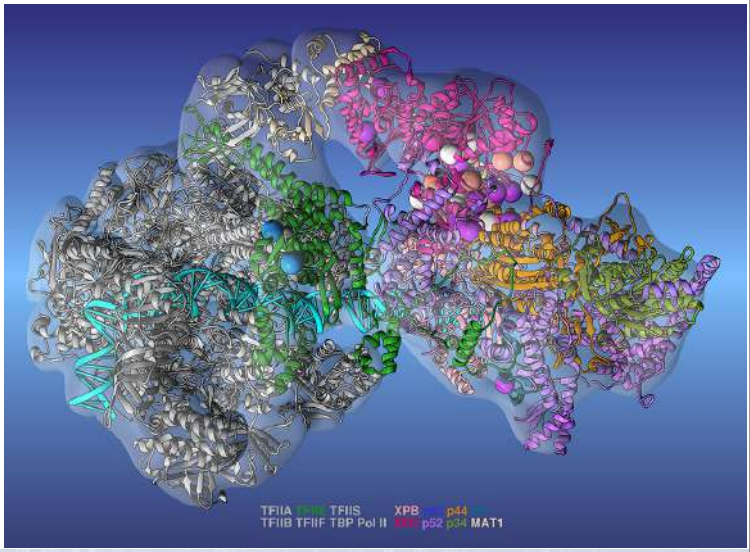




R&D Highlight

- Accurately identify clusters of gene mutations (spheres), which helps study the emergence of various genetic diseases
- From months to days using CryoEM & NAMD

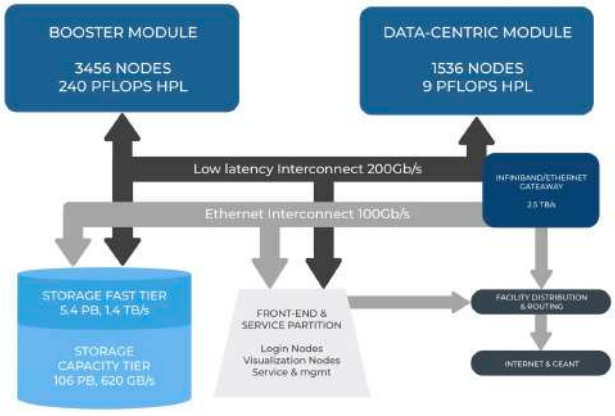
https://www.olcf.ornl.gov/2019/05/20/summit-charts-a-course-to-uncover-the-origins-of-genetic-diseases/



4 - Leonardo - BullSequana XH200

- Site: EuroHPC/CINECA
- Familia de sisteme: Atos
- Procesor: Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64GB
- OS: Linux
- Arhitectura: Cluster
- Aplicatii: Cercetare
- Anul instalarii: 2022
- Numar de core-uri: 1,824,768
- Rmax(PFlops): **238**,7
- Rpeak(PFlops): **304**,5
- HPCG(PFlops): **3,**11
- Memorie: **3,1** PB
- Consum: **7.404** kW
- Interconectare: Quad-Rail NVidia HDR 100 Infiniband

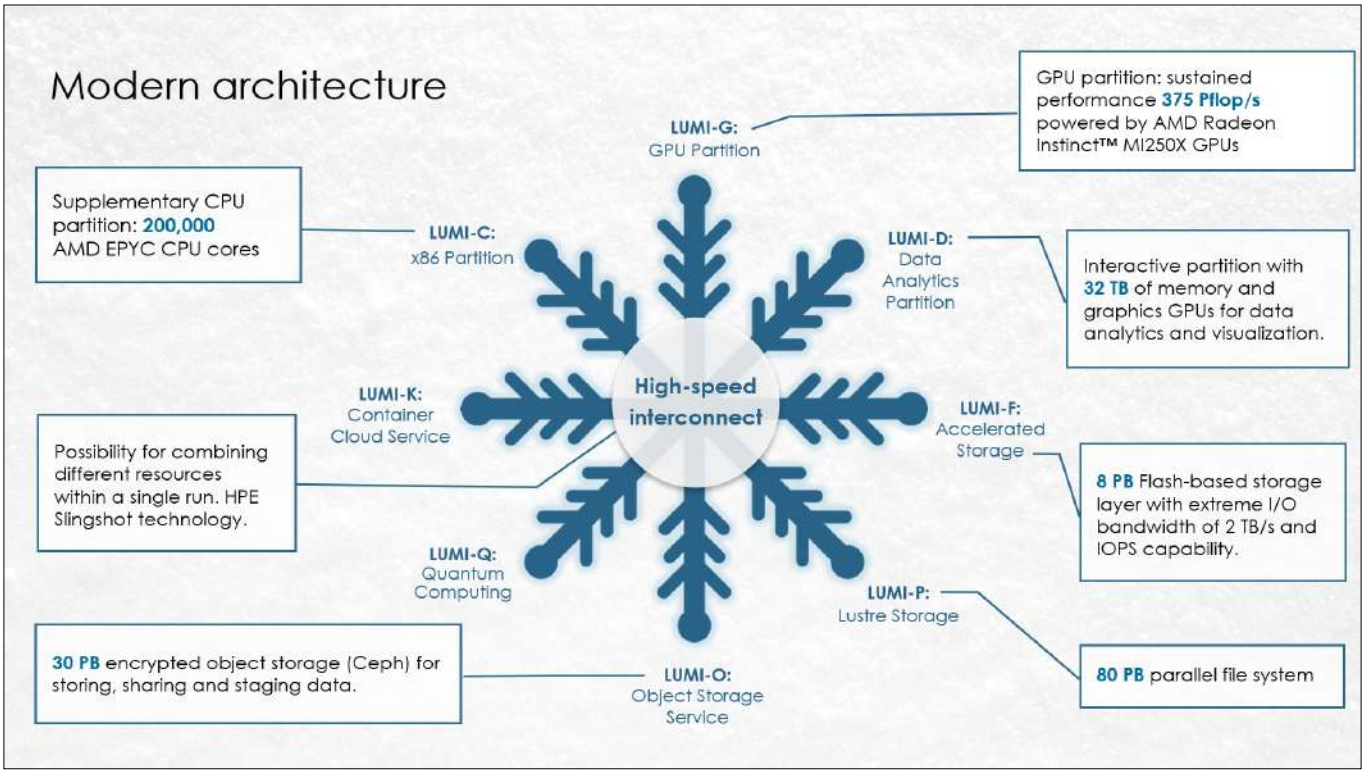
Leonardo Node Infrastructure

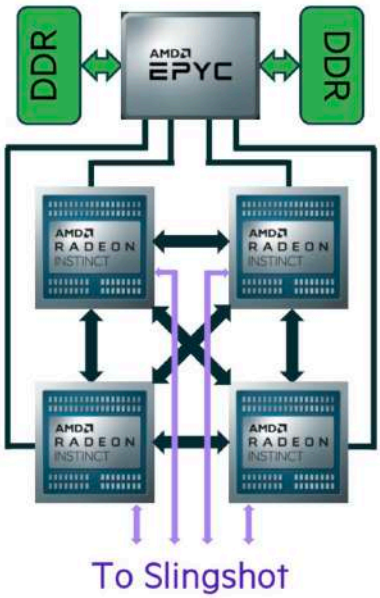


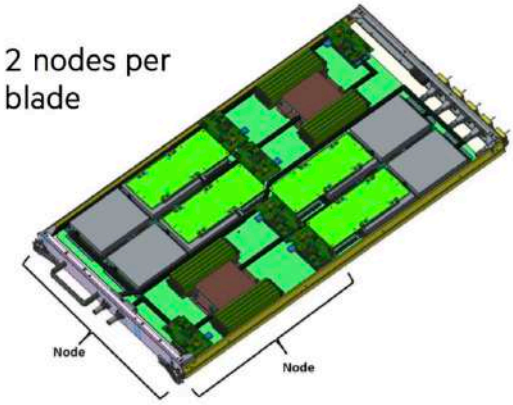
- 3456 booster nodes
  - Intel Xeon 8358 32 cores & 4XNvidia A100 64GB HBM2
- 1536 data-centric nodes
  - Intel Saphire Rapids 2x56 cores / 48x32 GB DDR5 4.8GHz
- 16 visualization nodes
  - 2 x Icelake 32cores 2.4GHz / 3 NVIDIA V100, RAM 16x32 GB

3 - Lumi - HPE Cray EX235a

- Site: EuroHPC/CSC
- Familia de sisteme: HPE Cray EX235a
- Procesor: AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X
- OS: HPE Cray OS
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2023
- Numar de core-uri: 2,220,288
- Rmax(PFlops): **309**,1
- Rpeak(PFlops): **428**,7
- HPCG(PFlops): **3,**4
- Consum: **6.015** kW
- Interconectare: Slingshot-11



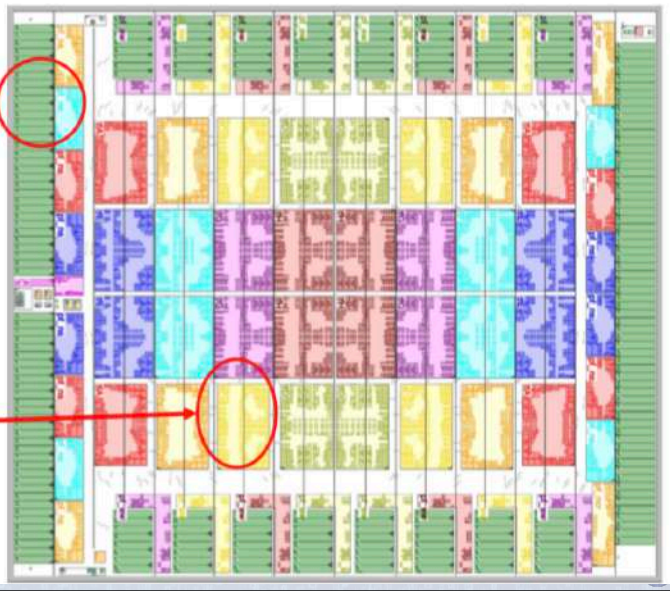




Node Architecture

Slingshot 11

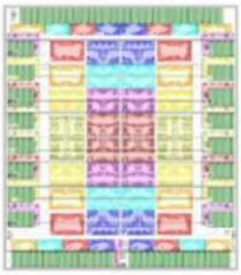
- TSMC 16nm FF
- 64 ports x 200 Gbps/dir
  - PAM4 56G
- ~250W
- Tiled architecture:
  - 32 peripheral function blocks
    - Network SerDes, MAC/PCS/LLR
    - Ethernet Lookup functions
  - 32 tile blocks
    - All other port functionality
  - Management Block (MB)
- First silicon Sept '18, Production Q4'19



Slingshot Interconnect

Rosetta Switch

- Multiple QoS levels
- Aggressive adaptive routing
- Advanced congestion control
- Very low average and tail latency
- High performance multicast and reduction



64 ports x 200 Gbps

SS-10 (100Gb)
Injection: ~14 TB/s
Bisection: ~24 TB/s

SS-11 (200Gb)
Injection: ~28 TB/s
Bisection: ~24 TB/s

Mellanox ConnectX NIC

Slingshot 10

- HPE Cray MPI stack
- Ethernet functionality
- RDMA offload

Cassini NIC

Slingshot 11

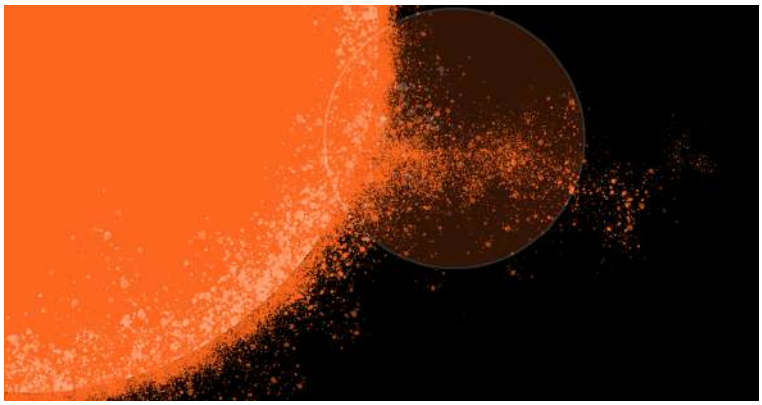
- MPI hardware tag matching
- MPI progress engine
- One-sided operations
- Collectives
- 2X injection bandwidth

Slingshot 11

- Liquid or air cooled 1U form-factor
- Up to 8 blades / chassis & up to 64 blades / cabinet
- 100-200 Gb/s in a TOR pattern
- 1.2 B packets / s (600M / direction)

Lumi R&D Highlight

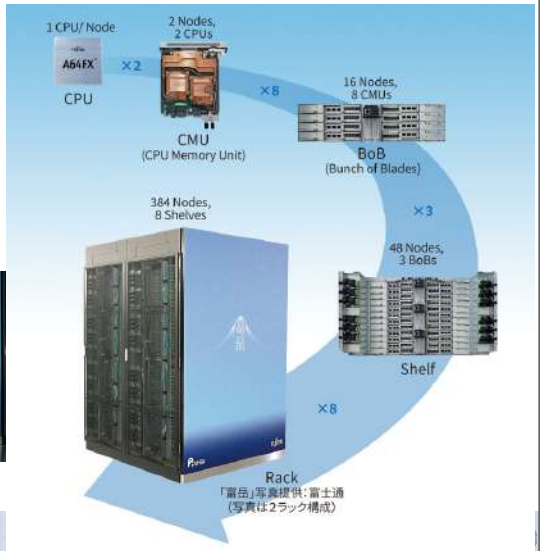
- Information from under the Sun's surface about the emergence of solar storms
  - Predict solar storms



https://www.lumi-supercomputer.eu/researchers-discovered-information-from-under-the-suns-surface-about-the-emergence-of-solar-storms-using-the-lumi-supercomputer/

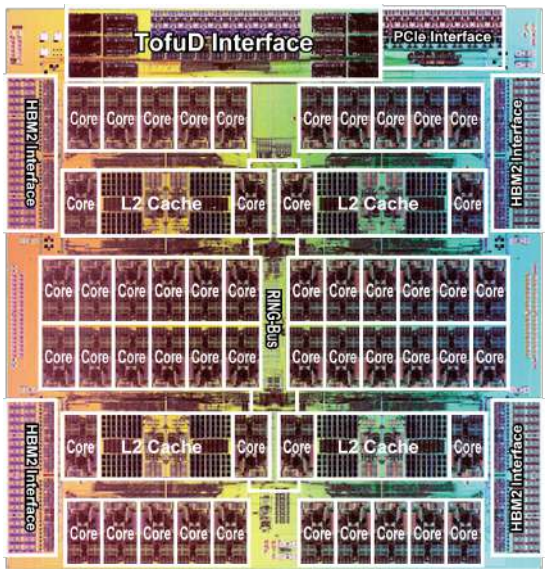
2 Fugaku K Computer Update

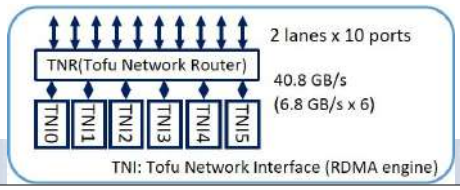
- Site: RIKEN Advanced Institute for Computational Science
- Familia de sisteme: Fujistu – SPARC64
- Procesor: A64FX 48C 2.2GHz
- OS: Linux
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2020
- Numar de core-uri: 7.630.848
- Rmax(PFlops): **442**
- Rpeak(PFlops): **537**
- HPCG(PFlops): **16**
- Memorie: **5,087** TB
- Consum: **29.899** kW
- Interconectare: Custom – Tofu



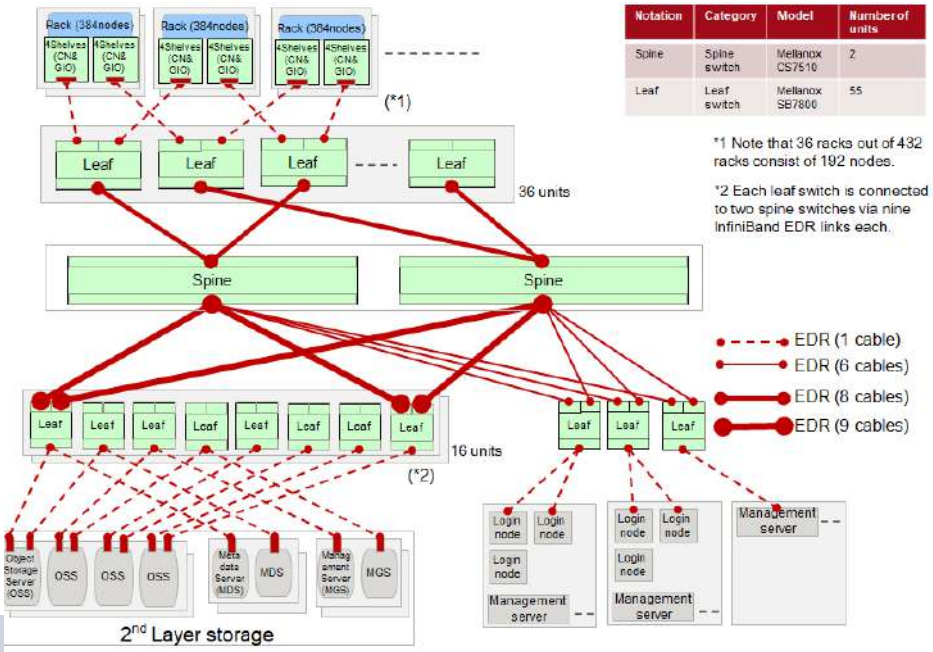
Node & Interconnect Stats

- 48+2 cores @2-2.2GHz 7nm FinFET
- Cache:
  - L1D/core: 64 KiB, 4way, 256 GB/s (load), 128 GB/s (store)
  - L2/CMG: 8 MiB, 16way
  - L2/node: 4 TB/s (load), 2 TB/s (store)
  - L2/core: 128 GB/s (load), 64 GB/s (store)
- Memory 1024 GB/s





I/O Networking



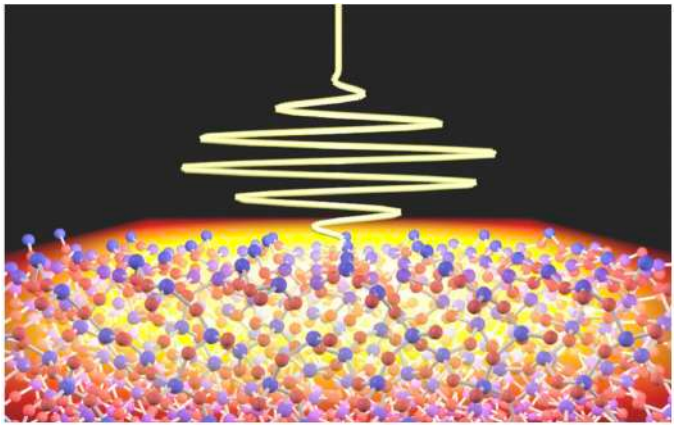
Notation	Category	Model	Number of units
Spine	Spine switch	Mellanox CS7510	2
Leaf	Leaf switch	Mellanox SB7800	55

*1 Note that 36 racks out of 432 racks consist of 192 nodes.

*2 Each leaf switch is connected to two spine switches via nine InfiniBand EDR links each.

Fugaku R&D Highlights

- COVID-19 Projects
  - Drug candidates
  - Pandemic Phenomena
- Model interactions between matter and light at the atomic scale

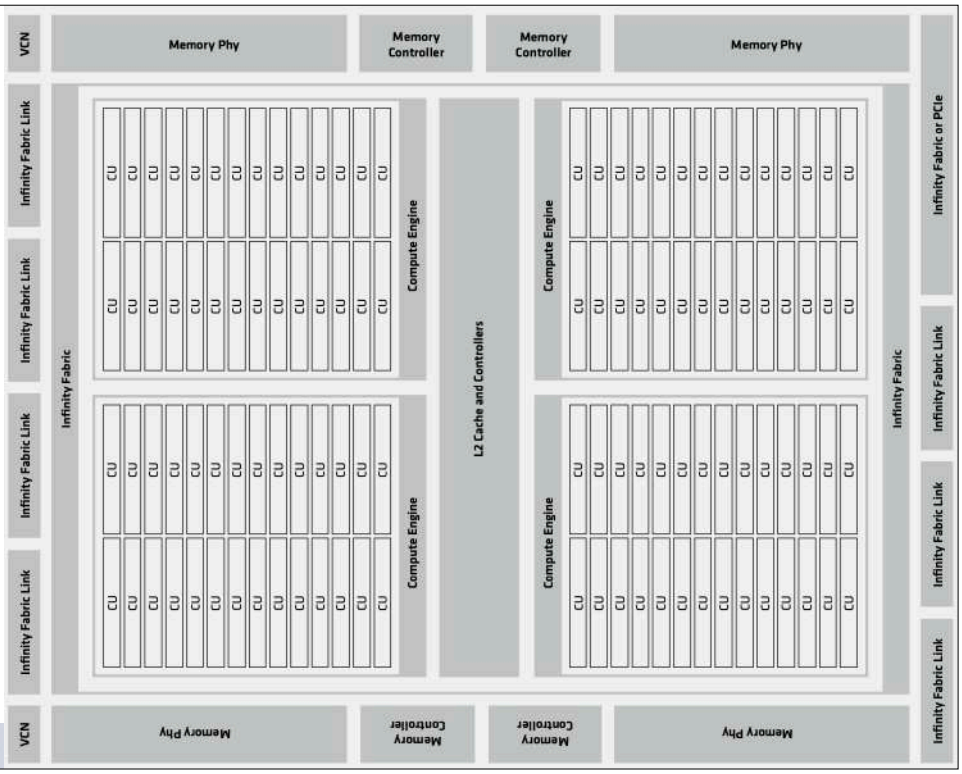


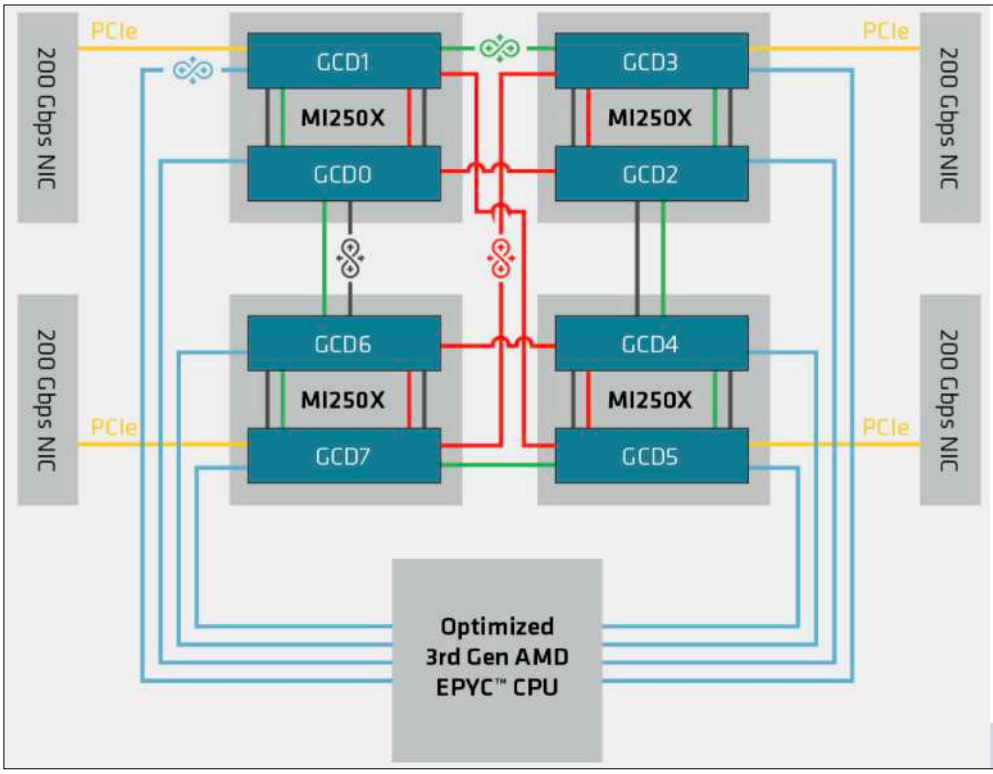
https://www.r-ccs.riken.jp/en/fugaku/research/

1 - Frontier - HPE Cray EX235a

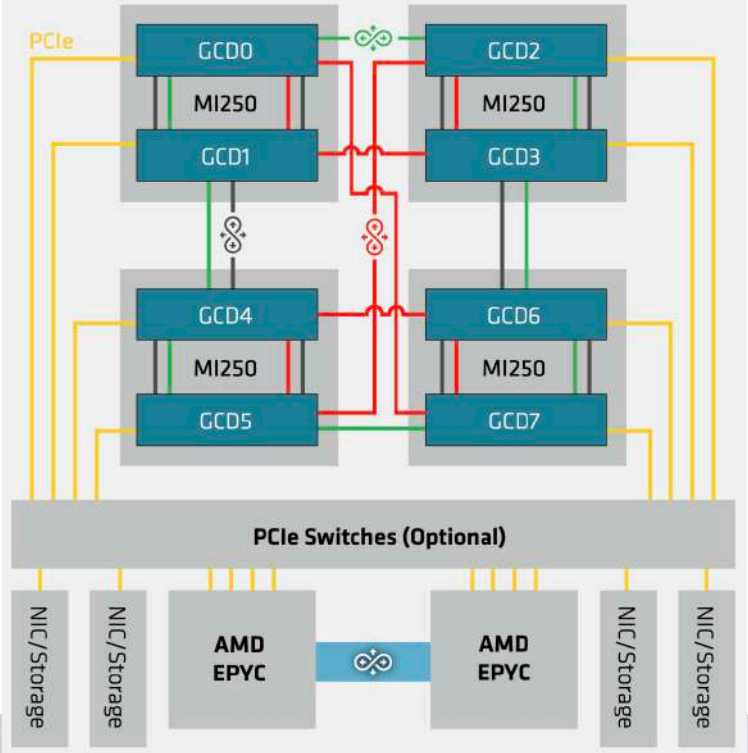
- Site: DOE/SC/Oak Ridge National Laboratory
- Familia de sisteme: HPE Cray EX235a
- Procesor: AMD Optimized 3rd Gen EPYC 64C 2GHz, AMD Instinct MI250X
- OS: HPE Cray OS
- Arhitectura: MPP
- Aplicatii: Cercetare
- Anul instalarii: 2021
- Numar de core-uri: 8,699,904
- Rmax(**EFlops**): **1**,194
- Rpeak(**EFlops**): **1**,679
- HPCG(PFlops): **14,**05
- Memorie: **3,1** PB
- Consum: **22.703** kW
- Interconectare: Slingshot-11

AMD Instinct MI250X



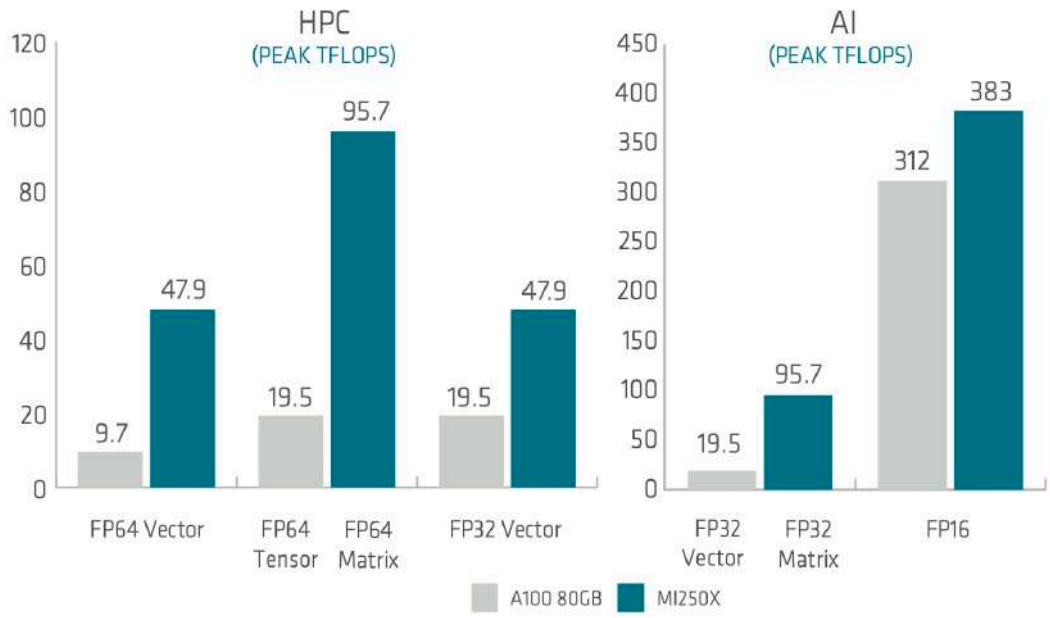


AMD Instinct MI250X HPC Node



AMD Instinct MI250X HPC/ML Node

Performance	MI210	MI250	MI250X
CDNA2 Graphics Compute Die (GCD)	1	2	2
Compute Units	104 CU	208CU	220CU
Stream processors	6,656	13,312	14,080
Matrix Cores	416	832	880
Peak FP64/FP32 Vector	22.6 TF	45.3 TF	47.9 TF
Peak FP64/FP32 Matrix	45.3 TF	90.5 TF	95.7 TF
Peak FP16/BF16	181.0 TF	362.1 TF	383.0 TF
Peak INT4/INT8	181.0 TOPS	362.1 TOPS	383.0 TOPS
Memory			
Memory Size	64GB HBM2e	128GB HBM2e	128GB HBM2e
Memory Interface	4,096 bits	8,192 bits	8,192 bits
Memory Clock	1.6GHz	1.6GHz	1.6GHz
Memory Bandwidth	up to 1.6 TB/sec	up to 3.2TB/sec2	up to 3.2TB/sec2
Scalability			
Infinity Fabric Links	up to 3	up to 6	up to 8
xGMI Bridge Card Configuration	Yes (Dual | Quad Hives)	NA	NA
Coherency Enabled	No	No	Yes



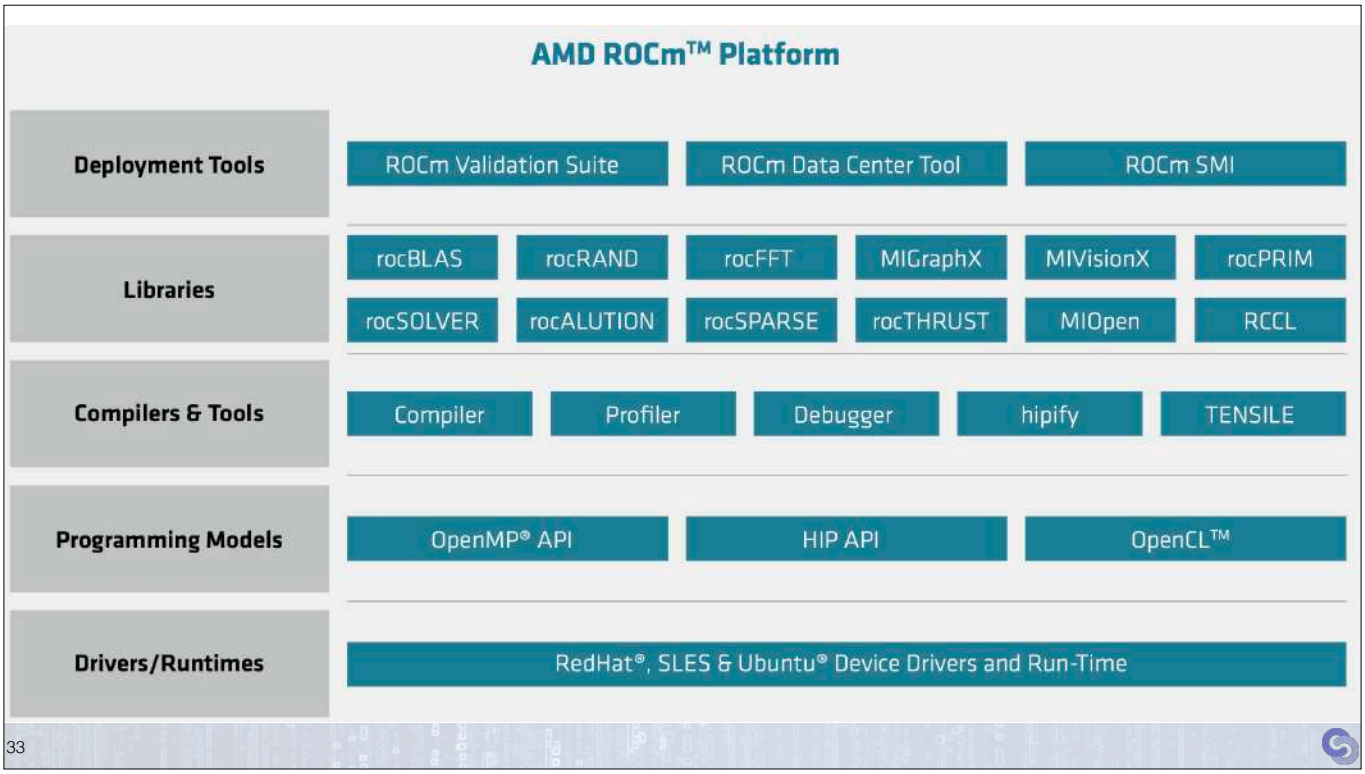
Graph 1: Peak TFLOPS across range of mixed-precision Compute[1]

Original

```
float vxi = 0.0f, vyi = 0.0f, vzi = 0.0f;
for (int j = threadIdx.x; j < count1; j += blockDim.x) {
  float dx = xx1[j] - xxi;
  float dy = yy1[j] - yyi;
  float dz = zz1[j] - zzi;
  float dist2 = dx*dx + dy*dy + dz*dz;
  if (dist2 < fsrrmax2) {
    float rtemp = (dist2 + rsm2)*(dist2 + rsm2)*(dist2 + rsm2);
    float f_over_r = massi*mass1[j]*(1.0f/sqrt(rtemp) -
        (ma0 + dist2*(ma1 + dist2*(ma2 + dist2*(ma3 +
        dist2*(ma4 + dist2*ma5))))));
    vxi += fcoeff*f_over_r*dx;
    vyi += fcoeff*f_over_r*dy;
    vzi += fcoeff*f_over_r*dz;
  }
}
```

Modified to use Packed FMA32

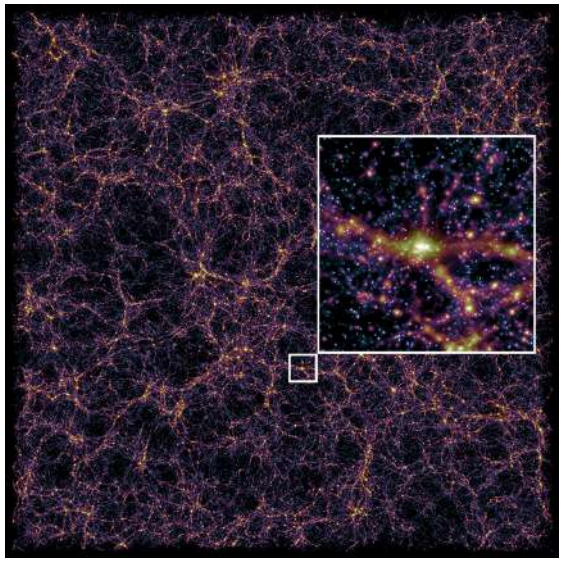
```
float vxi = 0.0f, vyi = 0.0f, vzi = 0.0f;
  for (int j = threadIdx.x; j < count1; j += 2*blockDim.x) {
    float2 dx = {xx1[j] - xxi, xx1[j + blockDim.x] - xxi};
    float2 dy = {yy1[j] - yyi, yy1[j + blockDim.x] - yyi};
    float2 dz = {zz1[j] - zzi, zz1[j + blockDim.x] - zzi};
    float2 dist2 = dx*dx + dy*dy + dz*dz;
    bool check[2] = {dist2.x < fsrrmax2, dist2.y < fsrrmax2};
    if (check[0] || check[1]) {
      float2 rtemp = (dist2 + rsm2)*(dist2 + rsm2)*(dist2 + rsm2);
      float2 mass1_2 = {mass1[j],mass1[j + blockDim.x]};
      float2 sqrt_rtemp = {sqrtf(rtemp.x),sqrtf(rtemp.y)};
      float2 f_over_r = massi*mass1_2*(1.0f/sqrt_rtemp -
          (ma0 + dist2*(ma1 + dist2*(ma2 + dist2*(ma3 +
          dist2*(ma4 + dist2*ma5))))));
      float2 vxi_tmp = fcoeff*f_over_r*dx;
      float2 vyi_tmp = fcoeff*f_over_r*dy;
      float2 vzi_tmp = fcoeff*f_over_r*dz;
      vxi += check[0] ? vxi_tmp.x : 0.0f;
      vxi += check[1] ? vxi_tmp.y : 0.0f;
      vyi += check[0] ? vyi_tmp.x : 0.0f;
      vyi += check[1] ? vyi_tmp.y : 0.0f;
      vzi += check[0] ? vzi_tmp.x : 0.0f;
      vzi += check[1] ? vzi_tmp.y : 0.0f;
    }
  }
```



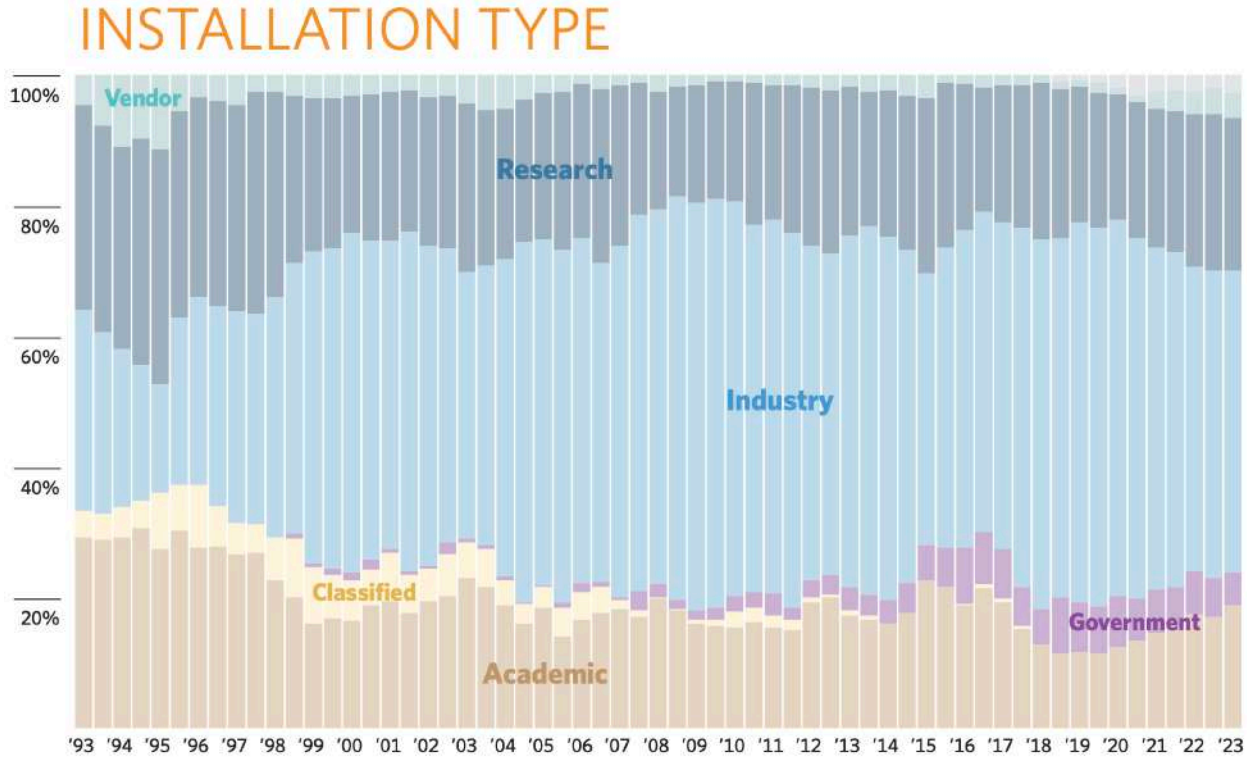
Frontier R&D Highlight

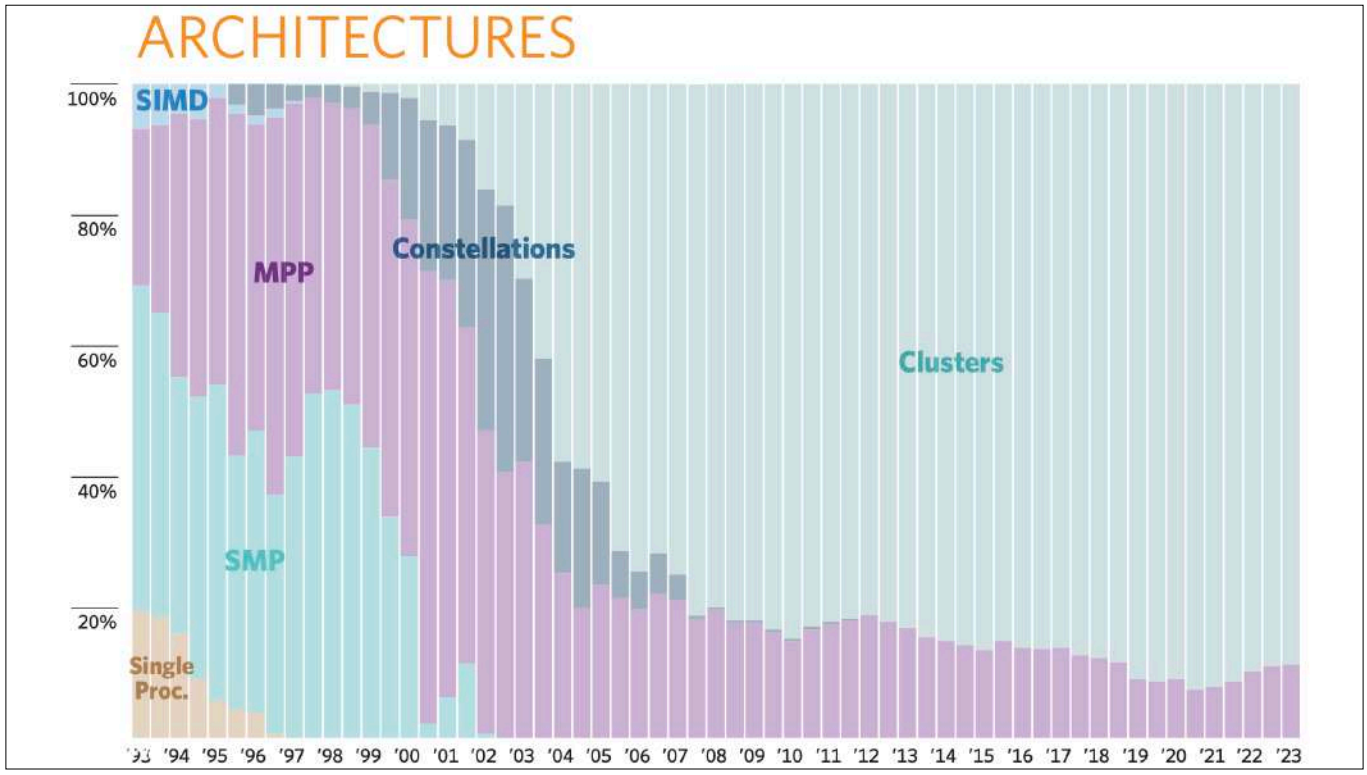
- More detailed Universe
- HACC (Hardware/Hybrid Accelerated Cosmology Code)
- Port from NVidia GP-GPUs to AMD Instinct

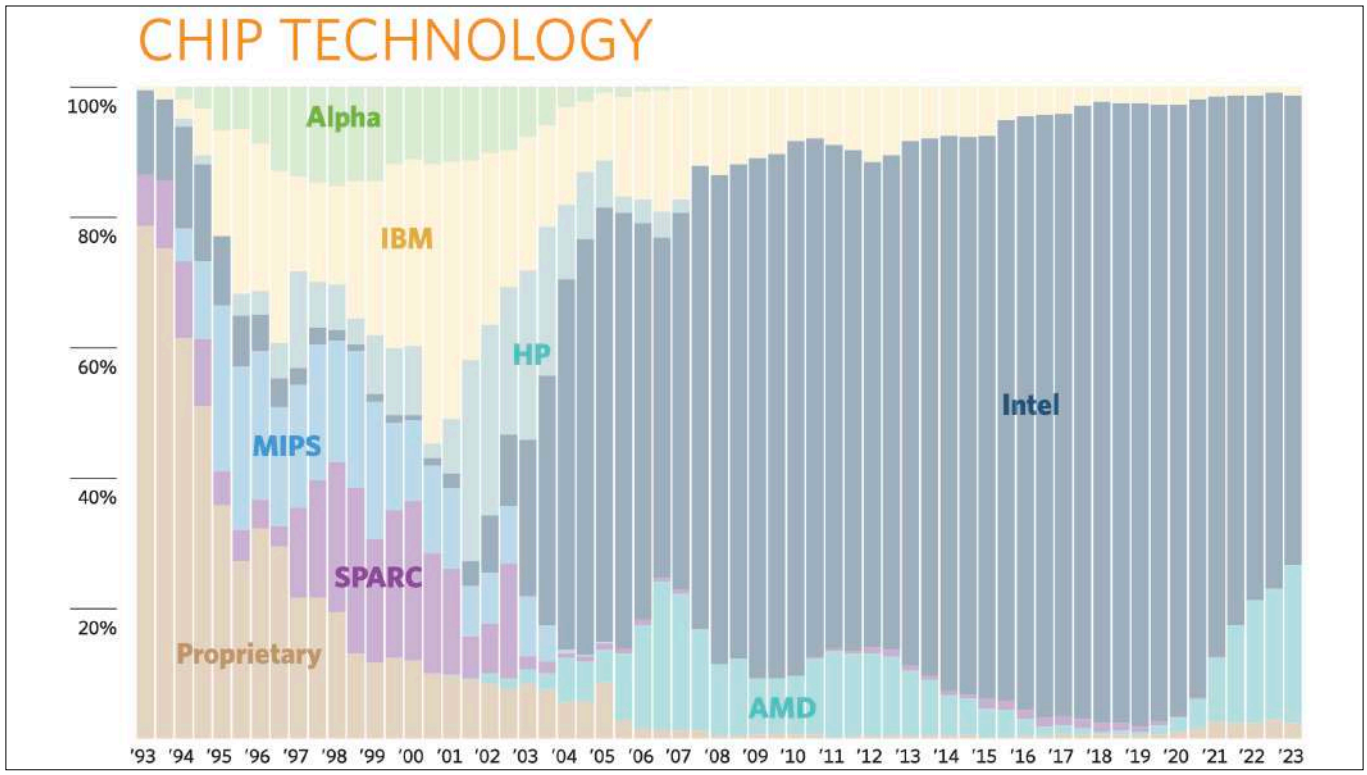
https://www.olcf.ornl.gov/2023/04/26/simulating-a-more-detailed-universe-with-frontier/



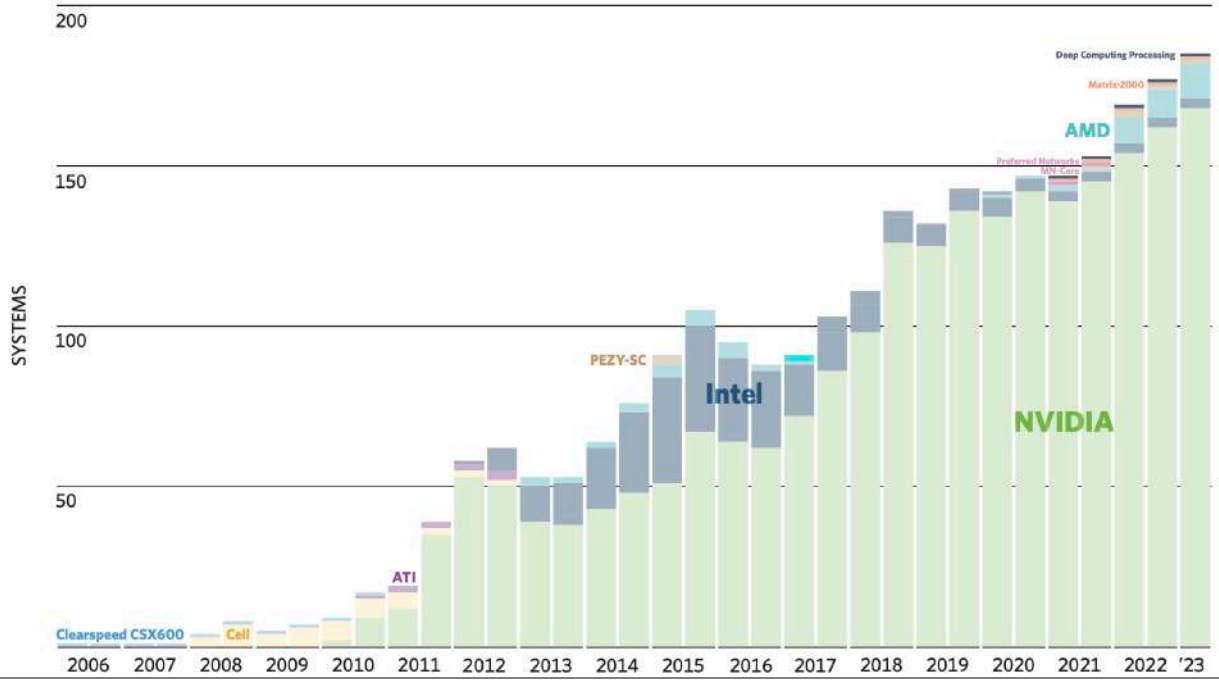
Concluzii Top 500




ARCHITECTURES
100%
80%
60%
40%
20%
SIMD
MPP
Constellations
Clusters
SMP
Single Proc.
'93 '94 '95 '96 '97 '98 '99 '00 '01 '02 '03 '04 '05 '06 '07 '08 '09 '10 '11 '12 '13 '14 '15 '16 '17 '18 '19 '20 '21 '22 '23


CHIP TECHNOLOGY
100%
80%
60%
40%
20%
Alpha
IBM
HP
Intel
MIPS
SPARC
Proprietary
AMD
'93 '94 '95 '96 '97 '98 '99 '00 '01 '02 '03 '04 '05 '06 '07 '08 '09 '10 '11 '12 '13 '14 '15 '16 '17 '18 '19 '20 '21 '22 '23

ACCELERATORS/CO-PROCESSORS



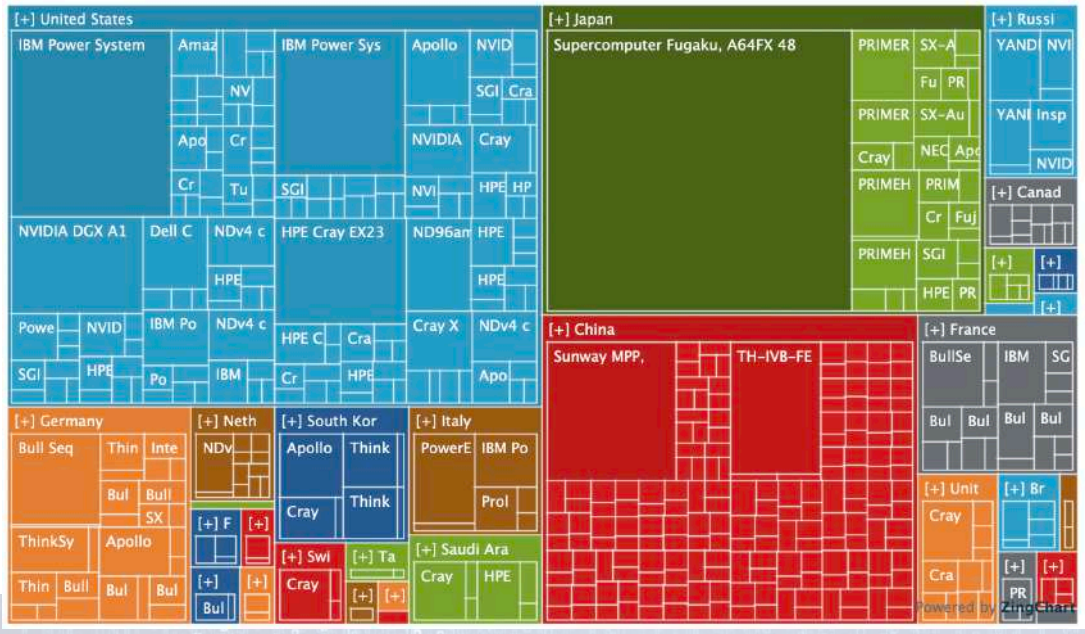
Vendors

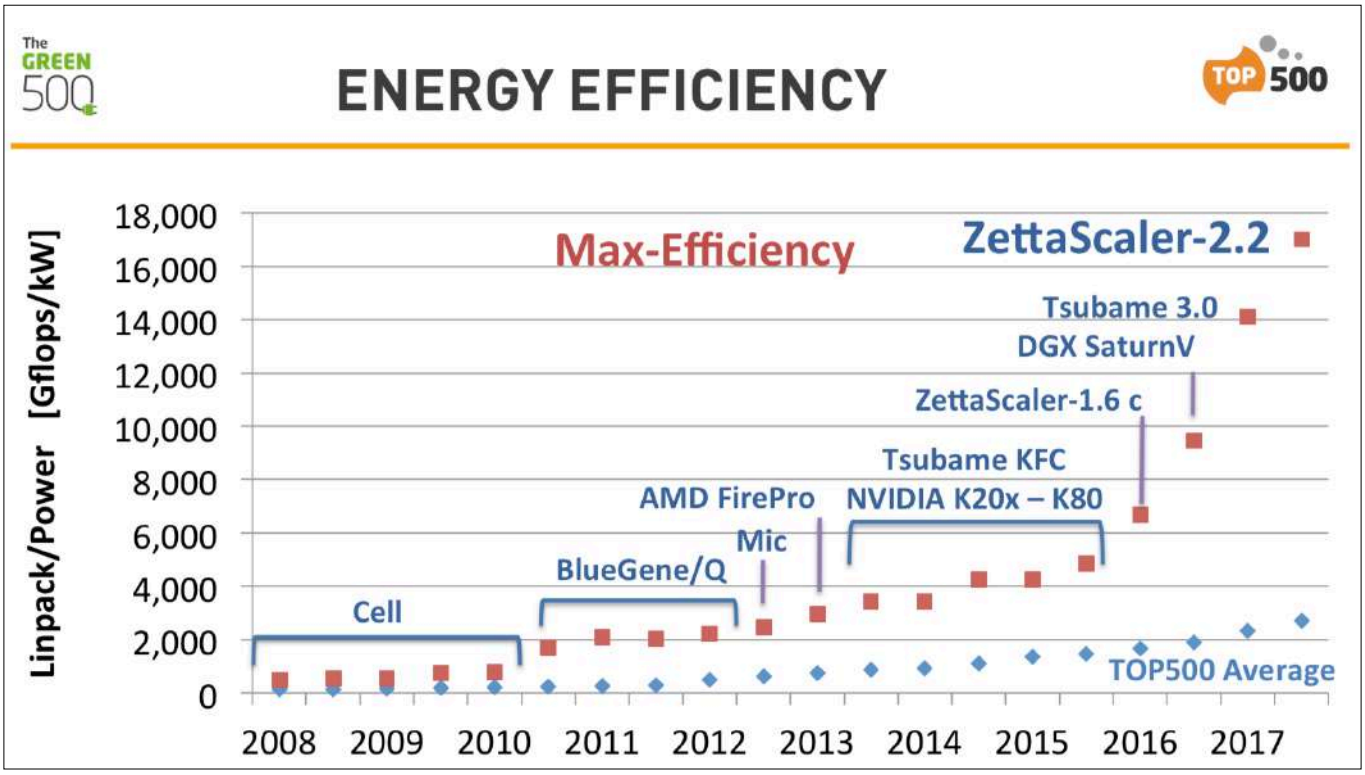
		Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
1	Lenovo	173	34.6	330,547	610,259	12,339,312
2	Sugon	71	14.2	119,286	301,610	5,182,664
3	Inspur	66	13.2	102,121	219,012	2,666,744
4	HPE	35	7	122,353	179,809	3,174,784
5	Cray/HPE	35	7	162,476	249,470	6,029,800
6	Atos	23	4.6	64,828	103,337	2,117,592
7	Fujitsu	13	2.6	60,538	96,398	1,589,640
8	IBM	13	2.6	216,919	287,902	6,005,272
9	Penguin Computing	11	2.2	20,034	25,248	614,136
10	Dell EMC	11	2.2	52,309	89,404	1,276,988

Interconnect

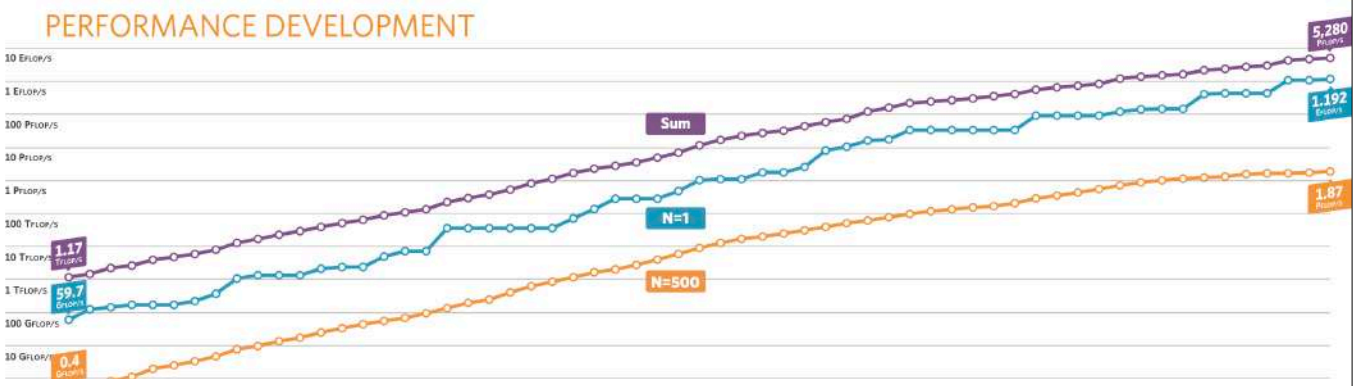
		Count	System Share (%)	Rmax (TFlops)	Rpeak (TFlops)	Cores
1	Gigabit Ethernet	259	51.8	430,575	897,359	16,913,772
2	Infiniband	140	28	662,896	1,013,372	15,946,634
3	Omnipath	50	10	181,500	285,493	4,704,264
4	Custom Interconnect	45	9	356,933	524,811	24,991,220
5	Proprietary Network	5	1	13,009	16,406	508,496
6	Myrinet	1	0.2	1,975	6,595	89,600

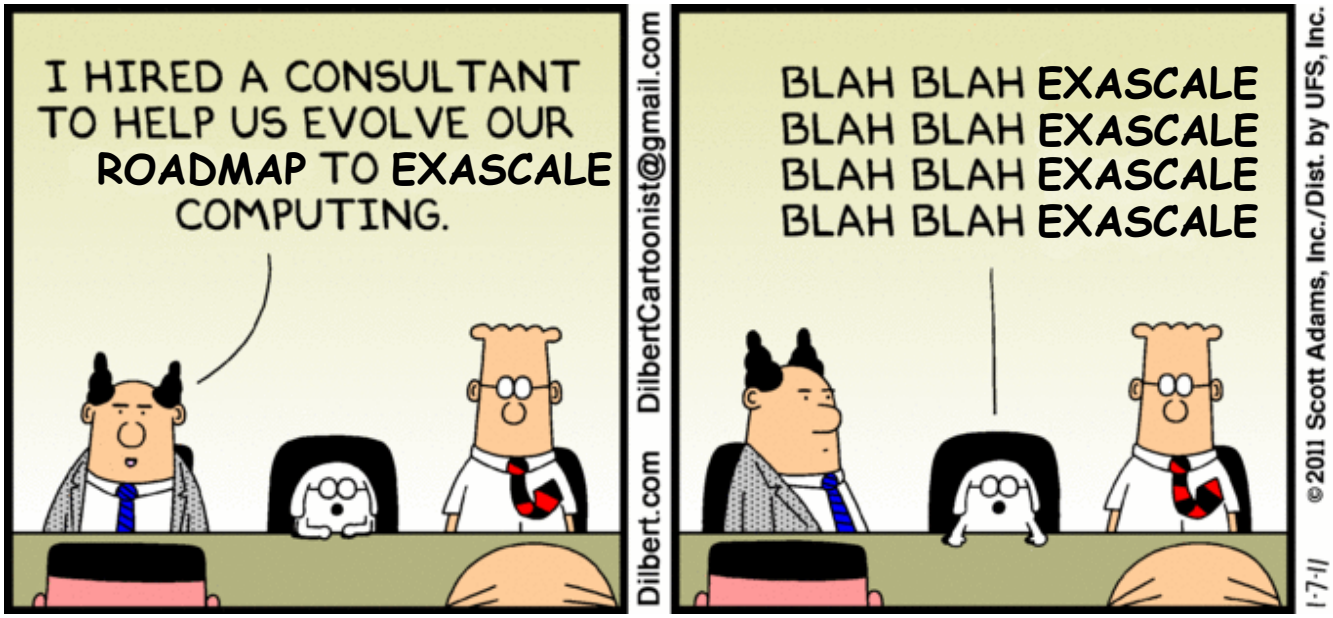
Instalari @Tari





Performanta in timp…





Feedback @ASC

Ce am invatat in aces curs?

- Top 5
- Concluzii Top 500

Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro

Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 14
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest

In cursul anterior

- Top 10
- Concluzii Top 500

Cuprins

- Scurta recapitulare a cursului
- Examen si Feedback

Scurta recapitulare a cursului

Un Scurt Sumar al Cursului

- Intro & Recapitulare
- Nivelul structural de descriere a sitemelor numerice - PMS
- Ierarhia de memorii, localitatea datelor, cache
- Comutatoare ierarhice & neierarhice
- Structuri de calcul cu prelucrare paralela – taxonomii
- Structuri SIMD
- Intercalarea perfecta si conectarea inversa
- Arhitecturi omogene/eterogene: CPU / GP-GPU

Un Scurt Sumar al Cursului (2)

- Arhitecturi MIMD – Caracteristici
- Arhitectura Cm* - comunicare intra- si inter-cluster
- Interconectarea in sisteme cu resurse multiple
- Retele de comutare (Ierarhice & Delta)
- Debugging, profiling, analiza de performanta
- Analiza de corectitudine a codurilor paralele
- Top 500, Benchmarking & the current Top 10

Bibliografie

- Structura si Arhitectura Sistemelor Numerice; T. Moisa, N. Tapus 1999
- Introduction to Parallel Computing: Design & Analysis of Algorithms; V. Kumar, A. Grama, A. Gupta, G. Karypis; Addison Wesley; 2nd Edition 2003
- The Sourcebook of Parallel Computing; J. Dongarra, I. Foster, W. Grapp, K. Kennedy; Morgan Kaufmann 2002
- Computer Organization & Design – The Hardware Software Interface; D.A. Patterson, J. Hennesy; Morgan Kaufmann; 5th Edition 2014
- Computer Architecture: A Quantitative Approach; J. Hennesy, D.A. Patterson; Morgan Kaufmann; 4th Edition 2007
- Introduction to Parallel Processing; Algorithms and Architectures; Behrooz Parhami; Kluwer 2002
- Techniques for Optimizing Applications: High Performance Computing; Rajat P. Garg, Ilya Sharapov; Sun 2001.
- Practical Computing on the Cell Broadband Engine; Sandeep Koranne; Springer 2009.
- Inside the Machine; Jon Stokes; No Starch Press 2006.
- www.top500.org

Notarea la ASC

- Laborator: 5/5.5
  - Trei teme: 4.5:
    - Python: 1.5
    - Optimizare C: 1.5
    - GP-GPU: 1.5
  - Activitate la laborator: 0.5
  - Prezentare iff act == 0.5
  - Bonusuri:
    - Max 20% @teme
- Examen: 5
  - Min > 2.5 lab
  - CA/CB/CC: Oral || Scris

Examen ASC

- Examenul scris are durata maxima de 1h și include doar subiecte de teorie (nota între 0-10)
- Nota == Teorie*0.5 + Laborator
- Intrarea in examen si promovarea nu poate avea loc decat daca nota finala de la laborator este **strict mai mare ca 2.5**
- Toate seriile pot opta pentru examen oral
- Examen + restante
  - CA - 6.06 de la 11AM (TBC)
  - CB - 12.06 de la 11AM (TBC)
  - CC - 12.06 de la 8AM (TBC)

What next?

- Q&A?
- Next time:
  - Examen
  - Last but not least: Feedback